

PGS. TS. TRẦN VĂN LONG (chủ biên)
TS. NGUYỄN MẠNH HÙNG, TS. ĐẶNG THỊ MAI

BÀI GIẢNG HỌC MÁY THỐNG KÊ

Trí tuệ nhân tạo
(AI)

Học máy
(Machine Learning)

Học sâu
(Deep Learning)

AI Tạo sinh
(Generative AI)

PGS. TS. TRẦN VĂN LONG (chủ biên)
TS. NGUYỄN MẠNH HÙNG, TS. ĐẶNG THỊ MAI

BÀI GIẢNG

HỌC MÁY THỐNG KÊ

TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
HÀ NỘI - 2025

Lời nói đầu

Học máy là gì? Một thuật toán machine learning là một thuật toán có khả năng học tập từ dữ liệu. Một chương trình máy tính được gọi là “học tập” từ *kinh nghiệm* E để hoàn thành *nhiệm vụ* T với hiệu quả được đo bằng *phép đánh giá* P , nếu hiệu quả của nó khi thực hiện nhiệm vụ T , khi được đánh giá bởi P , cải thiện theo kinh nghiệm E .

Trong học máy thường chia thành 3 loại học máy cơ bản: Học máy có giám sát, học máy không có giám sát, và học tăng cường. Gần đây một lĩnh vực mới được phát triển mạnh mẽ là học sâu (deep learning) là một phần trong học máy.

Học máy có giám sát là một thuật toán học máy để học ra một hàm mà ánh xạ tương ứng mỗi giá trị đầu vào (input) với một giá trị đầu ra (output) dựa trên bộ dữ liệu (đầu vào-đầu ra). Thuật toán được gọi là học máy có giám sát bởi vì trong quá trình học được dạy bởi dữ liệu output (xác định được đúng sai). Quá trình học sẽ dừng khi đạt được mức độ đánh giá nhất định.

Học máy không có giám sát là thuật toán học máy được sử dụng để rút ra các kết luận từ dữ liệu chỉ gồm có các dữ liệu đầu vào mà không bao gồm dữ liệu đầu ra. Các thuật toán học máy không có giám sát thông dụng là các thuật toán phân cụm (cluster analysis) được sử dụng để khám phá dữ liệu nhằm tìm các nhóm dữ liệu.

Bài giảng học máy thống kê được thiết kế giảng dạy cho sinh viên hệ Đại học của ngành Toán ứng dụng, chuyên ngành Toán tin ứng dụng. Bài giảng viết chi tiết từ các khái niệm, các mô hình toán học, và lập trình chạy mô phỏng thuật toán trên ngôn ngữ Python.

Bài giảng được chia làm 6 chương: Chương 1 và Chương 2 do TS. Nguyễn Mạnh Hùng biên soạn. Chương 3 và Chương 4 do TS. Đặng Thị Mai biên soạn. Chương 5 và Chương 6 do PGS. TS. Trần Văn Long biên soạn và làm chủ biên bài giảng.

Các dữ liệu và code chương trình tham khảo tại trang web:

<https://github.com/tvlongams/HOCMAY>

Hà Nội, 12/2025

Các tác giả

Mục lục

Lời nói đầu	i
Mục lục	v
Danh sách hình vẽ	vii
Danh sách bảng	ix
Chương 1 HỒI QUY TUYẾN TÍNH	1
1.1 Hồi quy tuyến tính đơn	2
1.2 Mô hình hồi quy tuyến tính đa biến	16
1.3 Một số mô hình hồi quy có hiệu chỉnh	17
1.3.1 Hồi quy Ridge	18
1.3.2 Hồi quy Lasso	22
1.4 Kiểm tra sự phụ thuộc tuyến tính	28
1.5 Bài tập	32
1.6 Thực hành	34
Chương 2 PHÂN LOẠI	37
2.1 Một số khái niệm cơ bản	37
2.2 Hồi quy logistic	40
2.2.1 Phân loại nhị phân bằng hồi quy logistic	40
2.2.2 Phân loại đa lớp bằng hồi quy logistic	50
2.3 Một số mô hình phân loại Bayes	56
2.3.1 Phương pháp LDA	56
2.3.2 Phương pháp QDA	58
2.3.3 Naive Bayes	58
2.4 Bài tập	63
2.5 Thực hành	64
Chương 3 MÔ HÌNH CÂY	67
3.1 Mô hình cây hồi quy	68

3.2	Mô hình cây phân loại	75
3.3	Bagging, Random Forest, Boosting	84
3.3.1	Phương pháp Bagging	84
3.3.2	Phương pháp Random Forest	85
3.3.3	Phương pháp Boosting	85
3.4	Bài tập	86
3.5	Thực hành	87
Chương 4	MÁY VÉC-TƠ HỖ TRỢ	89
4.1	Siêu phẳng tách và lề	89
4.1.1	Giới thiệu về máy véc-tơ hỗ trợ	89
4.2	Tối ưu lề lớn nhất	93
4.3	Biên mềm	97
4.4	Hàm nhân	98
4.5	Máy véc-tơ hỗ trợ đa lớp	99
4.5.1	Phương pháp OVA	99
4.5.2	Phương pháp OVO	100
4.6	Bài tập	117
4.7	Thực hành	118
Chương 5	PHÂN CỤM DỮ LIỆU	119
5.1	Khái niệm phân cụm dữ liệu	119
5.1.1	Khái niệm phân cụm dữ liệu	119
5.1.2	Biến đổi dữ liệu	121
5.1.3	Khoảng cách	124
5.1.4	Đánh giá cụm dữ liệu	125
5.2	Phân cụm dữ liệu bằng phân vùng	127
5.2.1	Thuật toán k -means	127
5.2.2	Thực hành với thư viện <code>sklearn.cluster.KMeans</code>	129
5.2.3	Một số cải tiến thuật toán k -means	132
5.3	Phân cụm bằng phân tầng	133
5.3.1	Phương pháp gộp phân tầng	133
5.3.2	Phương pháp chia phân tầng	137
5.3.3	Một số phương pháp khác phân cụm bằng phân tầng	139
5.3.4	Thực hành với thư viện <code>scipy.cluster.hierarchy</code>	140
5.4	Bài tập	145

5.5	Thực hành	146
Chương 6	MẠNG NEURON	149
6.1	Mô hình mạng neuron	149
6.2	Mô hình mạng neuron	153
6.2.1	Mạng neuron Perceptron	153
6.2.2	Mạng neuron đơn tầng	155
6.2.3	Mạng neuron đa tầng	157
6.3	Thuật toán lan truyền ngược	160
6.4	Thực hành với thư viện <code>sklearn</code> và <code>pytorch</code>	166
6.4.1	Mạng hồi quy đơn tầng với <code>sklearn</code>	166
6.4.2	Mạng phân loại đơn tầng với <code>sklearn</code>	167
6.4.3	Mạng hồi quy đa tầng với <code>sklearn</code>	170
6.4.4	Mạng phân loại đa tầng với <code>sklearn</code>	171
6.4.5	Mạng hồi quy đơn tầng với <code>pytorch</code>	173
6.4.6	Mạng phân loại đơn tầng với <code>pytorch</code>	175
6.4.7	Mạng hồi quy đa tầng với <code>pytorch</code>	178
6.4.8	Mạng phân loại đa tầng với <code>pytorch</code>	181
6.5	Bài tập	184
6.6	Thực hành	186
	Tài liệu tham khảo	189

Danh sách hình vẽ

2.1	Phân loại Bayes với 2 lớp, đường nét đứt biểu diễn biên phân loại lý thuyết, đường liền nét biểu diễn biên phân loại LDA từ dữ liệu huấn luyện	57
5.1	Dữ liệu DNA: Ma trận biểu hiện 6830 genes (cột) và 64 mẫu (hàng)	121
6.1	Mạng thần kinh sinh học	150
6.2	Mạng thần kinh nhân tạo	150
6.3	Mạng neuron đơn tầng	157
6.4	Mạng neuron đa tầng	159
6.5	Sơ đồ tiến của mạng neuron.	160
6.6	Sơ đồ lan truyền ngược tính đạo hàm.	161
6.7	Đồ thị tính hàm nhiều biến	162
6.8	Đồ thị tính đạo hàm của hàm nhiều biến	163
6.9	Đồ thị tính mô hình hồi quy	163
6.10	Đồ thị tính mô hình hồi quy	164
6.11	Đồ thị tính mô hình phân loại	165

Danh sách bảng

5.1	Các phép biến đổi dữ liệu	122
6.1	Bảng chân trị cho hàm AND	154
6.2	Bảng chân trị cho hàm OR	154
6.3	Bảng chân trị cho hàm NOT	155
6.4	Bảng chân trị cho hàm XOR	155

Chương 1

HỒI QUY TUYẾN TÍNH

Trong chương này chúng ta nghiên cứu về mô hình hồi quy trong học máy có giám sát, trong đó ta học một hàm f biến dữ liệu đầu vào x thành giá trị đầu ra y . Hàm f xác định bởi tham số θ dưới dạng

$$y = f(x; \theta),$$

được gọi là hàm hồi quy. Chương trình học máy xác định tham số θ tối ưu để sai số xấp xỉ là bé nhất.

Bài toán hồi quy là bài toán xác định mối liên hệ giữa các biến độc lập (independent variables) và biến đầu ra nhận giá trị thực liên tục gọi là biến phụ thuộc (dependent variable).

- Ví dụ 1.1.**
1. Xây dựng một hàm dự báo giá nhà: Biến đầu vào là vị trí của căn nhà, diện tích, số phòng, số tầng, và một số yếu tố tiện ích của căn nhà. Biến đầu ra là giá trị của căn nhà.
 2. Xây dựng mô hình dự báo giá trị của một xe ô tô cũ: Biến đầu vào là các thuộc tính của một chiếc xe như hãng xe, năm sản xuất, động cơ, và một số thuộc tính ảnh hưởng đến giá trị của xe. Biến đầu ra là giá trị của xe.
 3. Trong các mô hình xe ô tô tự hành ta cần xác định góc đánh lái theo thời gian thực dựa vào các biến đầu vào được xác định từ các sensor và camera của xe thu được.

Trong chương này, chúng ta xét một số mô hình hồi quy:

- Hồi quy tuyến tính đơn

$$y = f(x, \theta) = b + wx$$

- Hồi quy tuyến tính đa biến

$$y = f(x, \theta) = b + w_1 x_1 + w_2 x_2 + \cdots + w_p x_p.$$

- Hồi quy đa thức

$$y = f(x, \theta) = b + w_1 x + w_2 x^2 + \cdots + w_p x^p.$$

Giả sử ta có tập dữ liệu huấn luyện mô hình $(x_i, y_i), i = 1, \dots, n$ với x_i là các dữ liệu cho biến đầu vào x và y_i là dữ liệu cho biến đầu ra y . Tham số θ được xác định bằng phương pháp bình phương tối thiểu, nghĩa là ta tìm θ để tổng bình phương các sai số là bé nhất. Với mỗi điểm dữ liệu (x_i, y_i) sai số được xác định là

$$e_i = y_i - f(x_i, \theta),$$

và trung bình bình phương của sai số, còn gọi là hàm tổn thất, được xác định bởi

$$E = E(\theta) = \frac{1}{n} \sum_{i=1}^n [y_i - f(x_i, \theta)]^2. \quad (1.1)$$

Ta ước lượng tham số $\hat{\theta}$ để hàm tổn thất $E(\theta)$ đạt giá trị nhỏ nhất, nghĩa là

$$\hat{\theta} = \operatorname{argmin}_{\theta} E(\theta).$$

1.1 Hồi quy tuyến tính đơn

Xét dữ liệu $x_i, i = 1, \dots, n$ là dữ liệu một chiều cho biến độc lập x và $y_i, i = 1, \dots, n$ là dữ liệu một chiều cho biến phụ thuộc y . Mô hình hồi quy tuyến tính đơn xác định mối liên hệ giữa x và y cho bởi công thức

$$y = b + wx. \quad (1.2)$$

Ta xác định tham số b, w để

$$E = \frac{1}{n} \sum_{i=1}^n [y_i - (b + wx_i)]^2 \longrightarrow \min.$$

Ta tìm các tham số thỏa mãn hệ phương trình

$$\begin{cases} \frac{\partial E}{\partial b} = -\frac{2}{n} \sum_{i=1}^n (y_i - (b + wx_i)) & = 0 \\ \frac{\partial E}{\partial w} = -\frac{2}{n} \sum_{i=1}^n (y_i - (b + wx_i)) x_i & = 0 \end{cases} \quad (1.3)$$

Rút gọn hệ trên ta được phương trình chính tắc

$$\begin{cases} nb + \sum_{i=1}^n x_i w &= \sum_{i=1}^n y_i, \\ \sum_{i=1}^n x_i b + \sum_{i=1}^n x_i^2 w &= \sum_{i=1}^n x_i y_i. \end{cases} \quad (1.4)$$

Hệ phương trình trên có nghiệm

$$\begin{cases} w &= \frac{n \sum_{i=1}^n x_i y_i - \sum_{i=1}^n x_i \cdot \sum_{i=1}^n y_i}{n \sum_{i=1}^n x_i^2 - \left(\sum_{i=1}^n x_i \right)^2} \\ b &= \frac{1}{n} \sum_{i=1}^n y_i - w \frac{1}{n} \sum_{i=1}^n x_i = \bar{y} - w \bar{x}. \end{cases}$$

Xét ví dụ minh họa với tập dữ liệu **Advertising** bao gồm chi phí dành cho quảng cáo và doanh số bán sản phẩm ở $n = 200$ thị trường khác nhau. Mục đích của chúng ta là ước lượng các tham số của mô hình hồi quy tuyến tính (1.2) với x là chi phí quảng cáo trên ti vi và y là doanh số bán hàng. Kết quả tính toán dưới đây cho thấy:

$$y = 0,2605 + 0,5218x.$$

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
[2]: df=pd.read_csv('Data/Advertising.csv')
```

```
[3]: df.head()
```

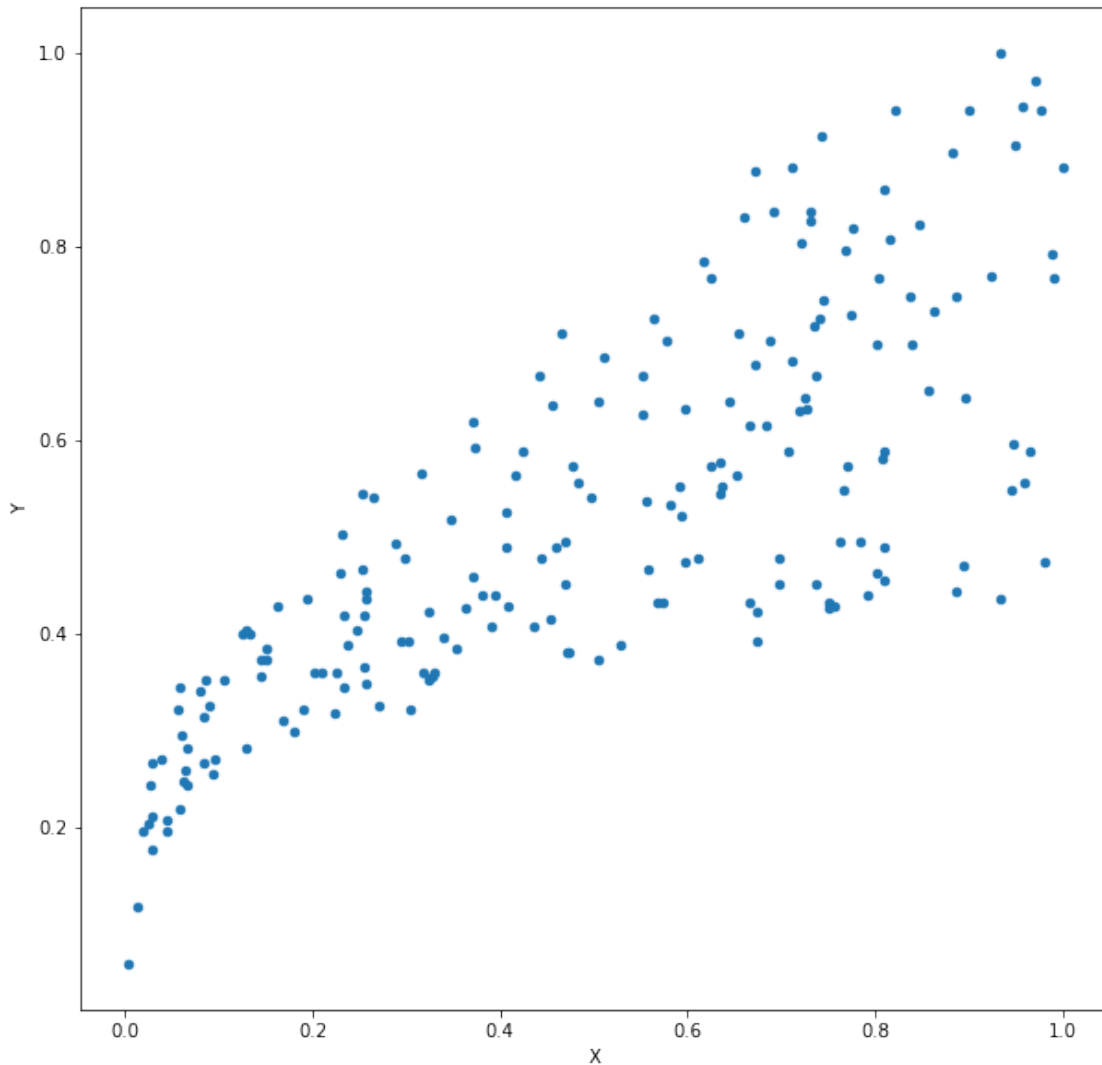
```
[3]:      TV  Radio  Newspaper  Sales
0  230.1   37.8      69.2    22.1
1   44.5   39.3      45.1    10.4
2   17.2   45.9      69.3     9.3
3  151.5   41.3      58.5    18.5
4  180.8   10.8      58.4    12.9
```

```
[4]: x=df['TV'].values
#x=df['Newspaper'].values
y=df['Sales'].values
```

```
[5]: df['X']=x/x.max()
df['Y']=y/y.max()
```

```
x=df['X'].values  
y=df['Y'].values
```

```
[6]: df.plot.scatter(x='X',y='Y',figsize=(10,10))  
      #plt.figure(figsize=(10,10))  
      #plt.scatter(x_scale,y_scale)  
      plt.show()
```



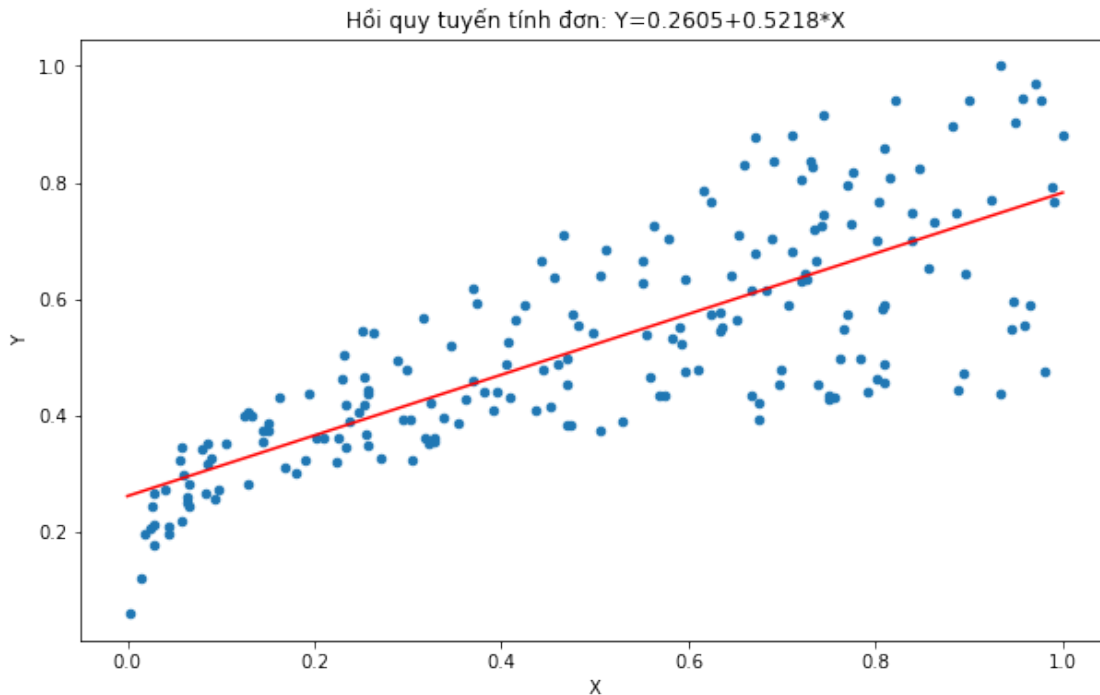
```
[7]: def min_least_squares(x,y):  
      x_bar=x.mean()  
      y_bar=y.mean()  
      x1=x-x_bar  
      y1=y-y_bar  
      w=sum(x1*y1)/sum(x1*x1)
```

```
b=y_bar-w*x_bar
return [b,w]
```

```
[8]: b,w=min_least_squares(x,y)
print(b.round(4),w.round(4))
```

```
0.2605 0.5218
```

```
[9]: ax1=df.plot.scatter(x='X',y='Y',figsize=(10,6))
x2=np.linspace(0,1,100)
y2=b+w*x2
plt.plot(x2,y2,c='red')
plt.title('Hồi quy tuyến tính đơn: Y=0.2605+0.5218*X')
plt.show()
```



Hệ số xác định R^2 có thể sử dụng để đo lường mức độ phù hợp của mô hình hồi quy với dữ liệu, được tính bởi công thức sau đây:

$$R^2 = \frac{TSS - RSS}{TSS} = 1 - \frac{RSS}{TSS}.$$

Đại lượng

$$RSS = \sum_{i=1}^n (y_i - \hat{y})^2$$

đo biến thiên của dữ liệu không giải thích được bằng mô hình hồi quy và

$$TSS = \sum_{i=1}^n (y_i - \bar{y})^2$$

đo biến thiên tổng thể của dữ liệu. Do đó, R^2 xác định tỉ số biến thiên của biến y có thể giải thích được khi sử dụng biến x . Giá trị của R^2 gần 1 nghĩa là phần lớn biến thiên của dữ liệu được giải thích bởi mô hình hồi quy. Giá trị của R^2 gần 0 nghĩa là mô hình hồi quy không giải thích được nhiều biến thiên của dữ liệu. Điều này xảy ra khi mô hình hồi quy tuyến tính không đúng, hoặc phương sai σ^2 của thành phần nhiễu ngẫu nhiên cao, hoặc có thể cả hai.

Trong ví dụ với dữ liệu **Advertising**, hệ số xác định $R^2 = 0,6119$ thể hiện rằng có khoảng 2/3 biến thiên của doanh số (sales) được giải thích bằng cách hồi quy tuyến tính chi phí quảng cáo TV.

```
[10]: def R2(y_pred, y):
        y_bar=np.mean(y)
        RSS=np.sum((y-y_pred)**2)
        TSS=np.sum((y-y_bar)**2)
        R2=1-RSS/TSS
        return R2
```

```
[11]: y_pred=b+w*x
        R2=R2(y_pred,y)
        print('R2=', R2.round(4))
```

R2= 0.6119

Tối ưu tham số bằng phương gradient descent: Các hệ số của hàm hồi quy tuyến tính được tính bằng cách cực tiểu hóa hàm mất mát. Trong trường hợp tổng quát, người ta dùng phương pháp tối ưu theo độ dốc (gradient descent) của hàm nhiều biến $f(x) \rightarrow \min$ bằng cách lặp lại

$$x_{k+1} = x_k - \eta \nabla f(x_k), \quad (1.5)$$

với η là tốc độ học (learning rate). Khi đó giá trị của hàm $f(x)$ giảm dần

$$f(x_0) \geq f(x_1) \geq f(x_2) \geq \dots,$$

và khi đó dãy $\{x_k\}$ hội tụ đến điểm cực tiểu địa phương của hàm $f(x)$.

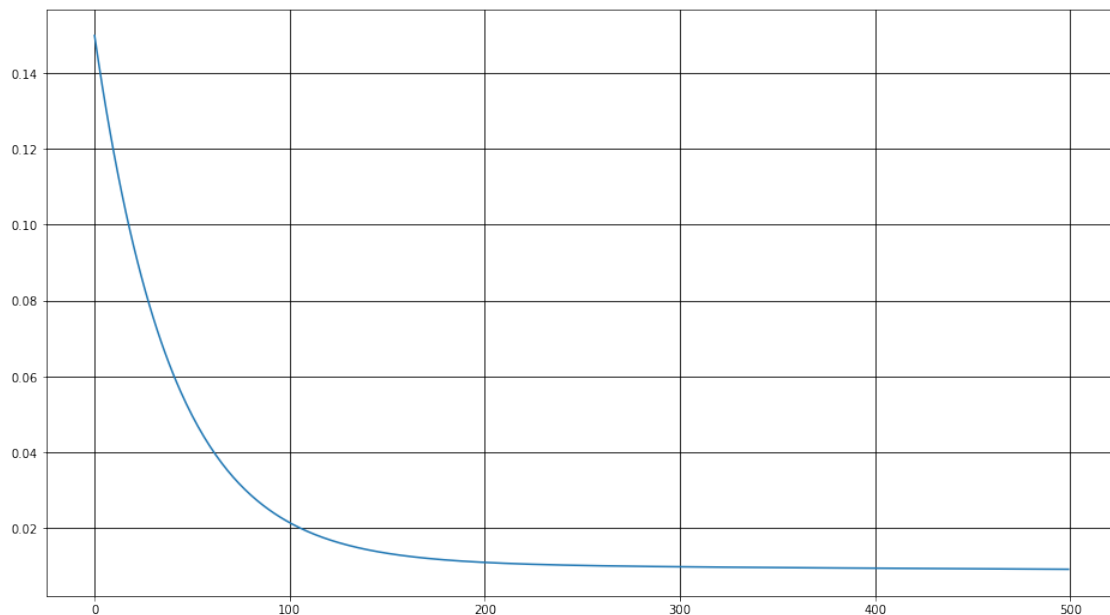
```
[12]: def gradient(b, w, x, y):  
    y_pred=b+w*x  
    error=y-y_pred  
    db=-np.mean(error)  
    dw=-np.mean(x*error)  
    return [db, dw]
```

```
[13]: def optimize(x, y, num_epochs=1000):  
    b0=0  
    w0=0  
    lr=0.01  
    loss=[]  
    for i in range(num_epochs):  
        grad=gradient(b0, w0, x, y)  
        b1=b0-lr*grad[0]  
        w1=w0-lr*grad[1]  
        error=np.mean((y-(b1+w1*x))**2)/2  
        loss.append(error)  
        epsilon=np.abs(b1-b0)+np.abs(w1-w0)  
        if epsilon<1e-6:  
            break;  
        else:  
            b0=b1  
            w0=w1  
    return [b1, w1, loss]
```

Xét tập dữ liệu về chi phí quảng cáo TV và doanh số bán hàng, hàm hồi quy dạng tuyến tính. Thực hiện thuật toán tối ưu theo độ dốc, đồ thị của hàm mất mát cho thấy hàm mất mát giảm nhanh trong 100 bước lặp đầu tiên, sau đó hội tụ về một giá trị cố định.

```
[14]: num_epochs=500  
b, w, loss=optimize(x, y, num_epochs)
```

```
[15]: plt.figure(figsize=(16, 9))  
plt.grid('True', color='k')  
plt.plot(loss)  
plt.show()
```

Thư viện pytorch Ta có thể sử dụng thư viện pytorch để tự động tính đạo hàm và tối ưu tham số. Thuật toán gradient descent được thực hiện qua các bước sau đây:

Bước 1: Khởi tạo giá trị ban đầu cho các tham số.

Bước 2: Lan truyền thuận (tính giá trị dự báo).

Bước 3: Tính hàm mất mát.

Bước 4: Lan truyền ngược (tính gradient).

Bước 5: Cập nhật các tham số và lặp lại từ Bước 2.

```
[16]: import torch
```

```
[17]: X=torch.from_numpy(x)
      Y=torch.from_numpy(y)
```

```
[18]: def forward(x):
      y=b+w*x
      return y
```

```
[19]: def criterion(y_pred, y):
      return torch.mean((y_pred - y)**2)
```

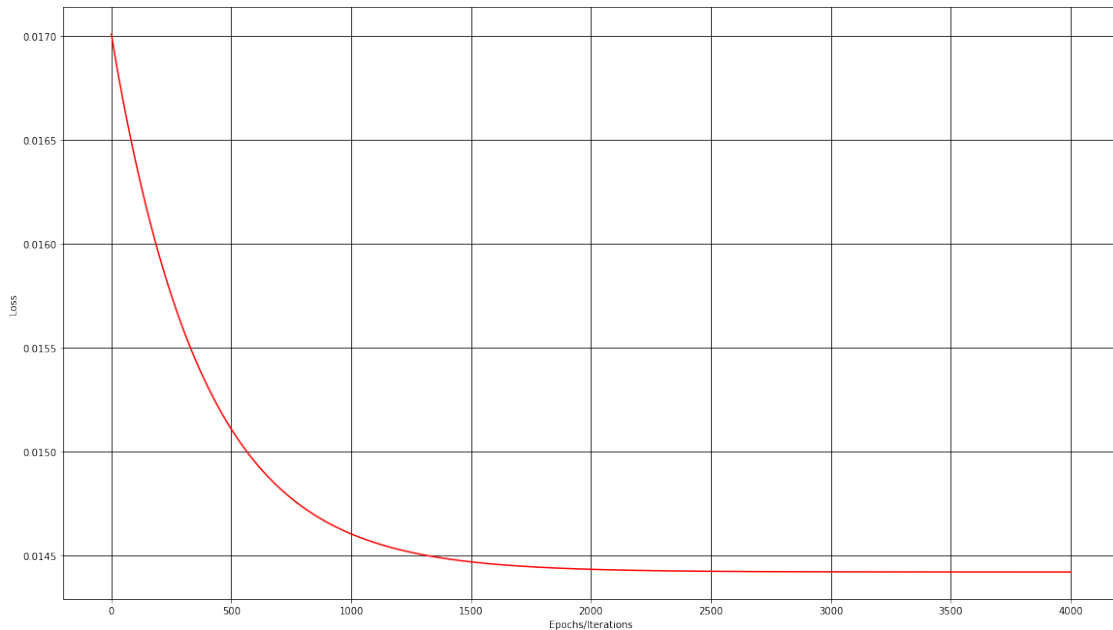
```
[20]: w = torch.tensor(0.0, requires_grad=True)
      b = torch.tensor(0.0, requires_grad=True)
```

```
[21]: lr = 0.01
      loss_list = []
      epochs = 4001

      for epoch in range(epochs):
          # making predictions with forward pass
          Y_pred = forward(X)
          # calculating the loss between original and predicted data
          -points
          loss = criterion(Y_pred, Y)
          # storing the calculated loss in a list
          loss_list.append(loss.item())
          # backward pass for computing the gradients of the loss w.r.
          -t to learnable parameters
          loss.backward()
          # updateing the parameters after each iteration
          w.data = w.data - lr*w.grad.data
          b.data = b.data - lr*b.grad.data
          # zeroing gradients after each iteration
          w.grad.data.zero_()
          b.grad.data.zero_()
          # printing the values for understanding
          if(epoch%1000==0):
              print('{}, \t{}, \t{}, \t{}'.format(epoch, b.item(), w.
              -item(), loss.item()))

          # Plotting the loss after each iteration
          plt.figure(figsize=(16,9))
          plt.plot(loss_list, 'r')
          plt.tight_layout()
          plt.grid('True', color='k')
          plt.xlabel("Epochs/Iterations")
          plt.ylabel("Loss")
          plt.show()
```

0,	0.35081732273101807,	0.3464164733886719,	0.01700811075434008
1000,	0.28513678908348083,	0.4754031300544739,	0.014603859123419001
2000,	0.26704180240631104,	0.5094680786132812,	0.014433663670062057
3000,	0.26221901178359985,	0.5185473561286926,	0.014421573305071711
4000,	0.26093390583992004,	0.5209668874740601,	0.01442071447477441



Phương pháp Stochastic Gradient Descent (SGD) Thuật toán SGD là một biến thể của thuật toán Gradient Descent, thay vì tính gradient trên toàn bộ tập dữ liệu (batch), ta chỉ dùng một mẫu nhỏ (mini-batch) để cập nhật tham số ở mỗi bước. Các bước của thuật toán SGD như sau:

Bước 1: Khởi tạo giá trị ban đầu cho các tham số.

Bước 2: Xáo trộn (shuffle) dữ liệu huấn luyện

Bước 3: Chia dữ liệu thành các mẫu nhỏ (mini-batch)

Bước 4: Với mỗi mini-batch, thực hiện

- Lan truyền thuận (tính giá trị dự báo).
- Tính hàm mất mát.
- Lan truyền ngược (tính gradient).
- Cập nhật các tham số.

Việc chia dữ liệu thành các mini-batch có thể được thực hiện bằng đối tượng **Dataloader** trong **PyTorch**. Tham số *batch_size* cho phép người dùng định nghĩa kích thước của mini-batch. Người ta thường chọn *batch_size* bằng lũy thừa của 2, ví dụ như 16, 32, 64, 128. Việc chọn cỡ cho mini-batch có thể gây ảnh hưởng đến quá trình hội tụ của hàm mất mát như thể hiện trong các ví dụ dưới đây.

```
[22]: from torch.utils.data import DataLoader
      from torch.utils.data import TensorDataset
```

```
[23]: dataset = TensorDataset(X, Y)
```

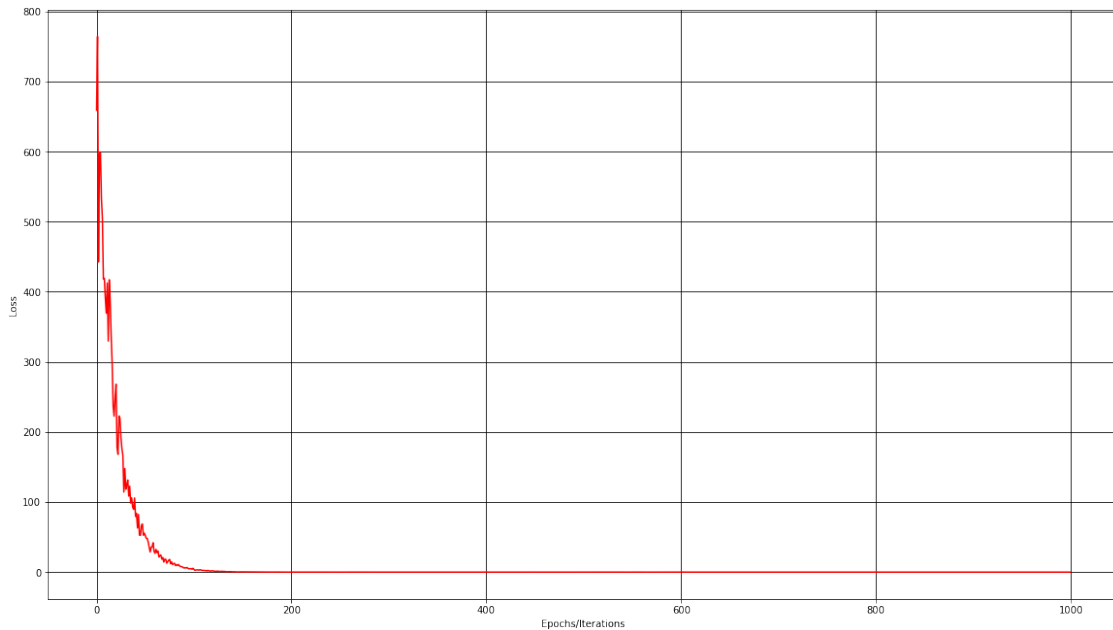
```
[24]: batch_size = 3
      train_loader = DataLoader(dataset, batch_size=batch_size,
                               shuffle=True)
      w = torch.tensor(-10.0, requires_grad=True)
      b = torch.tensor(-20.0, requires_grad=True)

      lr = 0.01
      loss_list = []
      epochs = 201

      for epoch in range(epochs):
          for x,y in train_loader:
              y_pred = forward(x)
              loss = criterion(y_pred, y)
              loss_list.append(loss.item())
              loss.backward()
              w.data = w.data - lr * w.grad.data
              b.data = b.data - lr * b.grad.data
              w.grad.data.zero_()
              b.grad.data.zero_()
              if (epoch%100==0):
                  print('{} , \t{} , \t{} , \t{}'.format(epoch, b.item(),
                  w.item(), loss.item()))
```

```
0,      -3.4971442222595215,    -1.1530405282974243,    24.39557412907397
100,     0.26722511649131775,    0.5253843665122986,    0.008049826379571528
200,     0.2624262869358063,    0.5232833027839661,    0.004279837364852177
```

```
[25]: # Plotting the loss after each iteration
      plt.figure(figsize=(16,9))
      plt.plot(loss_list[:1000], 'r')
      plt.tight_layout()
      plt.grid('True', color='k')
      plt.xlabel("Epochs/Iterations")
      plt.ylabel("Loss")
      plt.savefig('batch_SGD.png')
      plt.show()
```



```
[26]: batch_size = len(dataset)
train_loader = DataLoader(dataset, batch_size=batch_size,
    shuffle=True)

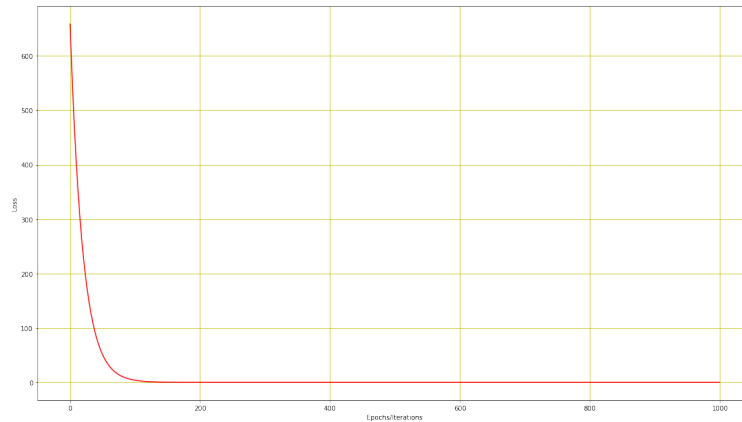
w = torch.tensor(-10.0, requires_grad=True)
b = torch.tensor(-20.0, requires_grad=True)

lr = 0.01
loss_list = []
epochs = 5001

for epoch in range(epochs):
    for x,y in train_loader:
        y_pred = forward(x)
        loss = criterion(y_pred, y)
        loss_list.append(loss.item())
        loss.backward()
        w.data = w.data - lr* w.grad.data
        b.data = b.data - lr* b.grad.data
        w.grad.data.zero_()
        b.grad.data.zero_()
    if(epoch%1000==0):
        print('{} \t{} \t{} \t{}'.format(epoch, b.item(),
    w.item(), loss.item()))
```

0,	-19.490394592285156,	-9.729619979858398,	658.502276077417
1000,	0.23396438360214233,	0.5717384815216064,	0.014632074662211525
2000,	0.2534030079841614,	0.5351439714431763,	0.014435667220107602
3000,	0.25858384370803833,	0.5253907442092896,	0.014421715583971926
4000,	0.2599642872810364,	0.5227919220924377,	0.0144207246111365
5000,	0.2603309154510498,	0.522100567817688,	0.014420654206210621

```
[27]: # Plotting the loss after each iteration
plt.figure(figsize=(16,9))
plt.plot(loss_list[:1000], 'r')
plt.tight_layout()
plt.grid('True', color='y')
plt.xlabel("Epochs/Iterations")
plt.ylabel("Loss")
plt.savefig('dataset_SGD.png')
plt.show()
```



```
[28]: batch_size = 1
train_loader = DataLoader(dataset, batch_size=batch_size,
    shuffle=True)

w = torch.tensor(-10.0, requires_grad=True)
b = torch.tensor(-20.0, requires_grad=True)

lr = 0.001
loss_list = []
epochs = 501

for epoch in range(epochs):
    for x,y in train_loader:
        y_pred = forward(x)
        loss = criterion(y_pred, y)
```

```

        loss_list.append(loss.item())
        loss.backward()
        w.data = w.data - lr*w.grad.data
        b.data = b.data - lr*b.grad.data
        w.grad.data.zero_()
        b.grad.data.zero_()
    if (epoch%100==0):
        print('{} \t{} \t{} \t{}'.format(epoch, b.item(),
-w.item(), loss.item()))

```

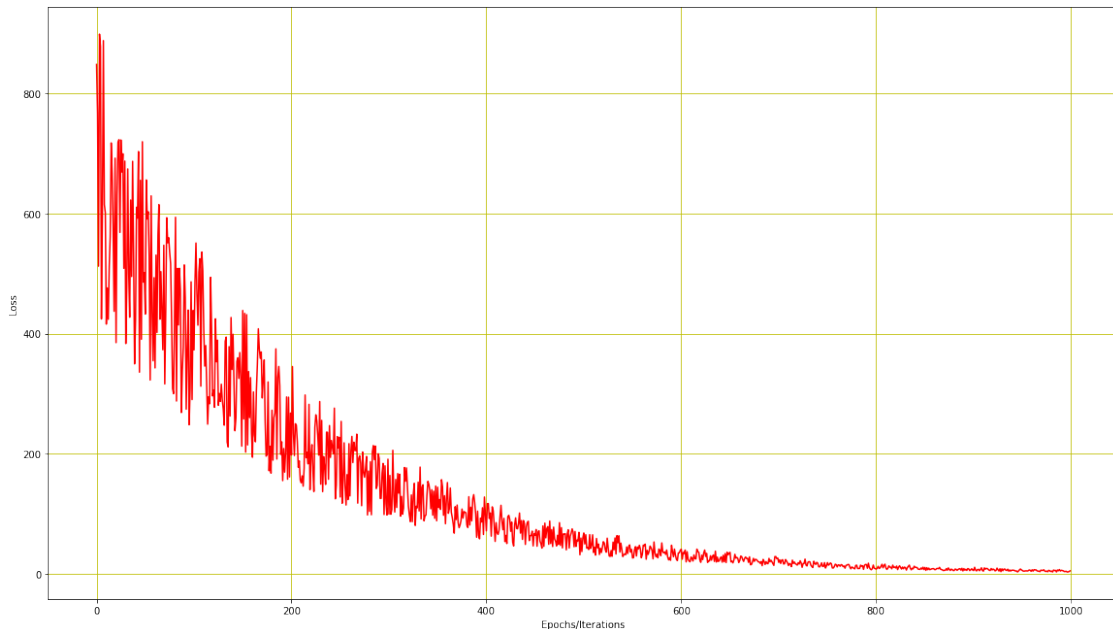
0,	-11.995521545410156,	-5.740663528442383,	267.7295032077621
100,	0.25354310870170593,	0.5351884365081787,	0.00010927409970363888
200,	0.2602687478065491,	0.5228720307350159,	0.0004274639305389082
300,	0.2607899308204651,	0.5220794081687927,	0.0014411322785351209
400,	0.25988784432411194,	0.5214386582374573,	0.0029943554163584275
500,	0.26007840037345886,	0.5216882824897766,	0.0004414454275949094

[29]: *# Plotting the loss after each iteration*

```

plt.figure(figsize=(16,9))
plt.plot(loss_list[:1000], 'r')
plt.tight_layout()
plt.grid('True', color='y')
plt.xlabel("Epochs/Iterations")
plt.ylabel("Loss")
plt.savefig('SGD.png')
plt.show()

```



Mô hình Linear của PyTorch Việc xây dựng mô hình hồi quy tuyến tính đơn có thể thực hiện một cách đơn giản nhờ lớp *Linear* của PyTorch. Để cài đặt mô hình, ta sử dụng câu lệnh `torch.nn.Linear(1,1)`.

```
[30]: x=df['X'].values.reshape(-1,1)
      y=df['Y'].values.reshape(-1,1)

[31]: from torch.autograd import Variable

[32]: X_data = Variable(torch.from_numpy(x).float())
      Y_data = Variable(torch.from_numpy(y).float())

[33]: class LinearRegressionModel(torch.nn.Module):
      def __init__(self):
          super(LinearRegressionModel, self).__init__()
          self.linear = torch.nn.Linear(1, 1)

      def forward(self, x):
          y_pred = self.linear(x)
          return y_pred

# our model
our_model = LinearRegressionModel()
criterion = torch.nn.MSELoss(size_average = False)
optimizer = torch.optim.SGD(our_model.parameters(), lr = 0.001)

for epoch in range(201):
    y_pred = our_model(X_data)
    loss = criterion(y_pred, Y_data)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    if(epoch%50==0):
        print('epoch: {},weight: {}, bias: {}, loss: {}'.format(
            epoch,
            our_model.state_dict()['linear.weight'].item(),
            our_model.state_dict()['linear.bias'].item(),
            loss.item()))

epoch: 0,weight: 0.803718, bias: 0.38061, loss: 62.08174
epoch: 50,weight: 0.56641, bias: 0.236792, loss: 2.91963
epoch: 100,weight: 0.53352, bias: 0.25426, loss: 2.88656
epoch: 150,weight: 0.52490, bias: 0.25884, loss: 2.88429
epoch: 200,weight: 0.52264, bias: 0.26004, loss: 2.88414
```



```
[34]: new_var = Variable(torch.Tensor([[0.5]]))
      pred_y = our_model(new_var)
      print("predict (after training)", new_var.item(),
            our_model(new_var).item())
```

```
predict (after training) 0.5 0.5213648080825806
```

Sử dụng thư viện sklearn: Mô hình tương tự cũng được cài đặt trong nhiều thư viện khác như *sklearn*, *statsmodels*, ...

```
[35]: from sklearn.linear_model import LinearRegression
```

```
[36]: reg=LinearRegression()
      reg.fit(x.reshape(-1,1),y)
      print(reg.intercept_, reg.coef_)
      print(reg.score(x.reshape(-1,1),y))
```

```
0.2604664277454703 [0.52184667]
0.611875050850071
```

1.2 Mô hình hồi quy tuyến tính đa biến

Trong phần này chúng ta nghiên cứu về hàm hồi quy đa biến

$$y = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_p x_p + \varepsilon, \quad (1.6)$$

trong đó $x_1, x_2, \dots, x_p$ là p biến độc lập và y là biến phụ thuộc với n giá trị quan sát trong tập huấn luyện. Ta ký hiệu các ma trận

$$y = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, X = \begin{pmatrix} 1 & x_{11} & x_{12} & \cdots & x_{1p} \\ 1 & x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & x_{n1} & x_{n2} & \cdots & x_{np} \end{pmatrix}, w = \begin{pmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_p \end{pmatrix}. \quad (1.7)$$

Để thuận tiện ta ký hiệu $x_{i,0} = 1$. Véc tơ tham số w được xác định bằng phương pháp bình phương tối thiểu, nghĩa là ta sẽ tìm w để sai số bình phương trung bình đạt giá trị nhỏ nhất

$$E = \frac{1}{n} \sum_{i=1}^n \left(y_i - \sum_{j=0}^p x_{ij} w_j \right)^2 \rightarrow \min. \quad (1.8)$$

Đạo hàm của E đối với w tính bởi công thức

$$\frac{\partial E}{\partial w} = -\frac{2}{n} X^T (y - Xw) = -\frac{2}{n} \begin{pmatrix} \sum_{i=1}^n \left(y_i - \sum_{j=0}^p x_{ij} w_j \right) \\ \sum_{i=1}^n x_{i1} \left(y_i - \sum_{j=0}^p x_{ij} w_j \right) \\ \vdots \\ \sum_{i=1}^n x_{ip} \left(y_i - \sum_{j=0}^p x_{ij} w_j \right) \end{pmatrix} = 0. \quad (1.9)$$

Từ đó dẫn đến hệ phương trình chuẩn tắc để tìm các hệ số của mô hình hồi quy tuyến tính:

$$(X^T X)w = X^T y. \quad (1.10)$$

Nếu ma trận $X^T X$ khả nghịch, các hệ số w được tính bởi công thức

$$w = (X^T X)^{-1} X^T y. \quad (1.11)$$

1.3 Một số mô hình hồi quy có hiệu chỉnh

Mô hình hồi quy tuyến tính có lợi thế trong việc diễn giải mối quan hệ giữa các biến và cũng tương đối hiệu quả so với các mô hình phi tuyến trong nhiều bài toán thực tế. Trong phần tiếp theo này, chúng ta sẽ xem xét việc cải tiến mô hình tuyến tính, bằng cách thay thế phương pháp khớp bình phương tối thiểu thông thường bằng một phương pháp khớp khác. Việc thay thế này có thể mang lại độ chính xác dự báo tốt hơn.

Trong trường hợp mối quan hệ giữa các biến độc lập và phụ thuộc là xấp xỉ tuyến tính, ước lượng bình phương tối thiểu có sự thiên lệch thấp (low bias). Nếu $n \gg p$ (số quan sát lớn hơn nhiều số lượng biến) thì ước lượng bình phương tối thiểu cũng có phương sai thấp (low variance), và do đó mô hình sẽ hoạt động tốt với dữ liệu kiểm tra. Tuy nhiên, nếu n không lớn hơn nhiều so với p , hoặc thậm chí $p > n$, thì ước lượng bình phương tối thiểu có phương sai lớn, dẫn đến hiện tượng quá khớp (over fitting) và kết quả dự báo kém. Bằng cách ràng buộc hoặc thu nhỏ các hệ số ước lượng có thể làm giảm đáng kể phương sai mà không làm tăng nhiều sự thiên lệch. Điều này giúp làm cải thiện độ chính xác của dự báo bởi mô hình. Có hai kỹ thuật hiệu chỉnh được sử dụng phổ biến để thu hẹp các hệ số hồi quy về 0 là *hồi quy Ridge* và *hồi quy Lasso*, sẽ được trình bày trong mục này.

1.3.1 Hồi quy Ridge

Trong hồi quy Ridge, ta thêm số hạng hiệu chỉnh

$$\lambda \|w\|^2 = \lambda(w_0^2 + w_1^2 + \dots + w_p^2)$$

với hệ số $\lambda > 0$ vào hàm sai số E , nghĩa là ta sẽ cực tiểu hóa hàm E có dạng

$$E = \frac{1}{n} \sum_{i=1}^n \left(y_i - \sum_{j=0}^p x_{ij} w_j \right)^2 + \lambda \|w\|^2 \rightarrow \min. \quad (1.12)$$

Khi đó, phương trình chuẩn tắc trở thành

$$(X^T X + n\lambda I)w = X^T y,$$

với ma trận $X^T X + n\lambda I$ luôn khả nghịch và các tham số hồi quy w được xác định bởi công thức

$$w = (X^T X + n\lambda I)^{-1} X^T y. \quad (1.13)$$

Trong ví dụ minh họa dưới đây, chúng ta sẽ dự báo mức lương (Salary) của các cầu thủ bóng chày dựa trên số liệu thống kê liên quan đến thành tích của các cầu thủ trong mùa thi đấu trước. Dữ liệu được cho trong file *Hitters.csv*. Kết quả tính toán cho thấy các hệ số của mô hình hồi quy giảm dần về 0 khi λ tiến đến vô cùng. Hệ số hiệu chỉnh $\lambda = 141,42$ thì sai số bình phương trung bình đạt giá trị nhỏ nhất. Nếu sử dụng hệ số hiệu chỉnh này, hệ số xác định tính được đối với tập dữ liệu huấn luyện và tập dữ liệu kiểm tra tương ứng là 51,75% và 17,68%.

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import scale
from sklearn import model_selection
from sklearn.linear_model import Ridge, RidgeCV
from sklearn.model_selection import train_test_split
from sklearn.model_selection import KFold, cross_val_score
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler
%matplotlib inline
plt.style.use('seaborn-white')
```

```
[2]: df = pd.read_csv('Data/Hitters.csv', index_col=0).dropna()
      dummies = pd.get_dummies(df[['League', 'Division',
      ↳'NewLeague']])
      y = df.Salary
      X_ = df.drop(['Salary', 'League', 'Division', 'NewLeague'],
      ↳axis=1).astype('float64')
      X = pd.concat([X_, dummies[['League_N', 'Division_W',
      ↳'NewLeague_N']]], axis=1)
      X_train, X_test, y_train, y_test = train_test_split(X, y,
      ↳test_size=0.2, random_state=42)
      list_numerical=X_train.columns[:16]
      scaler = StandardScaler().fit(X_train[list_numerical])
      X_train[list_numerical] = scaler.
      ↳transform(X_train[list_numerical])
      X_test[list_numerical] = scaler.
      ↳transform(X_test[list_numerical])
```

```
[3]: alphas = np.linspace(0.01,500,100)
      ridge = Ridge()
      coefs = []
      for a in alphas:
          ridge.set_params(alpha=a)
          ridge.fit(X_train, y_train)
          coefs.append(ridge.coef_)
      plt.figure(figsize=(16,9))
      ax = plt.gca()
      ax.plot(alphas, coefs)
      ax.set_xscale('log')
      plt.axis('tight')
      plt.xlabel('alpha')
      plt.ylabel('Standardized Coefficients')
      plt.title('Ridge coefficients as a function of alpha')
      plt.show()
```

```
[4]: kf = KFold(n_splits=5, random_state=42, shuffle=True)
      train_index=[]
      validation_index=[]
      for i, (t_index, v_index) in enumerate(kf.
      ↳split(X_train,y_train)):
          train_index.append(t_index)
          validation_index.append(v_index)
```

```
[5]: alphas = np.linspace(0.01, 500, 100)
```

```
[6]: def Test_MSE(x_train, y_train, x_test, y_test, a):
      model=Ridge(alpha=a)
      model.fit(x_train, y_train)
      mse=mean_squared_error(model.predict(x_test), y_test)
      return mse
```

```
[7]: def V_MSE(X_train, y_train, train_index, validation_index, a):
      mse=[]
      for i in range(len(train_index)):
          t_index= train_index[i]
          v_index=validation_index[i]
          error=Test_MSE(X_train.iloc[t_index], y_train.
              -iloc[t_index],
                          X_train.iloc[v_index], y_train.
              -iloc[v_index], a)
          mse.append(error)
      return mse
```

```
[8]: loss=[]
      for a in alphas:
          mse=V_MSE(X_train, y_train, train_index, validation_index, a)
          loss.append(mse)
```

```
[9]: mse_alpha=np.array(loss).reshape(100, 5).mean(axis=1)
      best_alpha=alphas[np.argmin(mse_alpha)]
      print('best alpha: ', best_alpha.round(2))
```

```
best alpha: 141.42
```

```
[10]: ridge_best = Ridge(alpha=best_alpha)
      ridge_best.fit(X_train, y_train)
      print('R squared training set', round(ridge_best.score(X_train,
          -y_train)*100, 2))
      print('R squared test set', round(ridge_best.score(X_test,
          -y_test)*100, 2))
      print('Mean squared error training
          -set', mean_squared_error(y_train, ridge_best.predict(X_train)).
          -round(2))
      print('Mean squared error test set', mean_squared_error(y_test,
          -ridge_best.predict(X_test)).round(2))
```

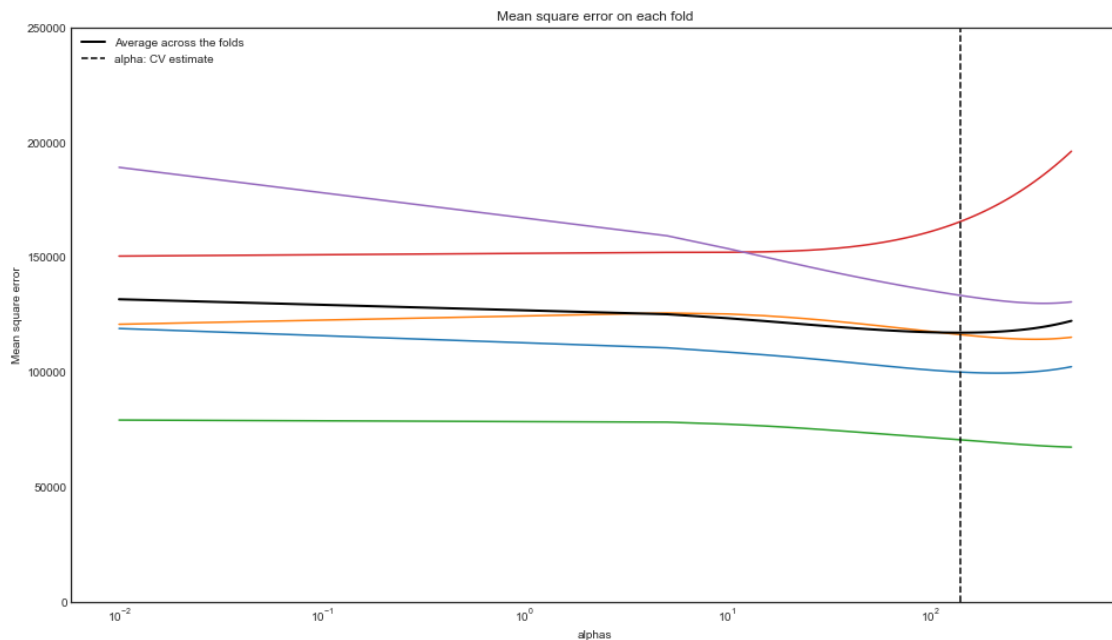
R squared training set 51.75

R squared test set 17.68

Mean squared error training set 100331.0

Mean squared error test set 148899.2

```
[11]: plt.figure(figsize=(16,9))
plt.semilogx(alphas,loss)
plt.plot(
    alphas ,
    np.array(loss).reshape(100,5).mean(axis=1),"k",
    label="Average across the folds",
    linewidth=2,
)
plt.axvline(best_alpha, linestyle="--", color="k", label="alpha:
-CV estimate")
plt.legend()
plt.xlabel("alphas")
plt.ylabel("Mean square error")
plt.title("Mean square error on each fold")
plt.axis("tight")
ymin, ymax = 0, 250000
plt.ylim(ymin, ymax)
plt.show()
```



1.3.2 Hồi quy Lasso

Trong hàm hồi quy Lasso ta thêm số hạng hiệu chỉnh

$$\lambda \|w\|_1 = \lambda(|w_0| + |w_1| + \cdots + |w_p|),$$

với hệ số $\lambda > 0$ vào hàm sai số E , nghĩa là ta đi cực tiểu hóa hàm

$$E = \frac{1}{2n} \sum_{i=1}^n \left(y_i - \sum_{j=0}^p x_{ij} w_j \right)^2 + \lambda \|w\|_1 \rightarrow \min. \quad (1.14)$$

Để tối ưu hàm trên ta cần sử dụng phương pháp tối ưu theo tọa độ. Để thuận tiện ta viết hàm mục tiêu dưới dạng

$$E = \frac{1}{2n} \|y - Xw\|^2 + \lambda \|w\|_1 = \frac{1}{2n} \|y - \sum_{j=0}^p X_j w_j\|^2 + \lambda \|w\|_1, \quad (1.15)$$

trong đó $X_0, X_1, \dots, X_p$ là các cột của ma trận X .

Ta xét trường hợp đơn giản $X^T X = I$. Khai triển hàm mục tiêu E dưới dạng hàm bậc hai dạng

$$E = \frac{1}{2n} y^T y + \frac{1}{2n} w^T w - \frac{1}{n} \sum_{j=0}^p (y^T X_j) w_j + \lambda \left(\sum_{j=0}^p |w_j| \right). \quad (1.16)$$

Lấy đạo hàm riêng của E theo biến w_j , chú ý hàm $|w_j|$ không khả vi tại $w_j = 0$, nên ta có 3 trường hợp sau

- Nếu $w_j > 0$,

$$\frac{\partial E}{\partial w_j} = \frac{1}{n} w_j - \frac{1}{n} y^T X_j + \lambda = 0 \Rightarrow w_j^{lasso} = y^T X_j - n\lambda.$$

- Nếu $w_j < 0$,

$$\frac{\partial E}{\partial w_j} = \frac{1}{n} w_j - \frac{1}{n} y^T X_j - \lambda = 0 \Rightarrow w_j^{lasso} = y^T X_j + n\lambda.$$

- Nếu $w_j = 0$, khi đó ta đặt

$$w_j^{lasso} = 0.$$

Vậy tối ưu theo tọa độ của hệ số w_j được tính theo công thức

$$w_j^{lasso} = \begin{cases} y^T X_j - n\lambda & \text{nếu } y^T X_j > n\lambda; \\ y^T X_j + n\lambda & \text{nếu } y^T X_j < -n\lambda; \\ 0 & \text{nếu } -n\lambda \leq y^T X_j \leq n\lambda. \end{cases} \quad (1.17)$$

Ta đặt hàm ngưỡng mềm

$$S(x, \lambda) = \begin{cases} x - \lambda & \text{nếu } x > \lambda; \\ x + \lambda & \text{nếu } x < -\lambda; \\ 0 & \text{nếu } -\lambda \leq x \leq \lambda. \end{cases}$$

Khi đó

$$w_j^{lasso} = S(y^T X_j, n\lambda).$$

Các hệ số của hàm hồi quy sẽ quy về 0 nếu hệ số đủ nhỏ. Do đó hàm hồi quy lasso còn được gọi là hàm hồi quy thưa.

Bây giờ ta xét trường hợp tổng quát, hàm mục tiêu được viết lại dưới dạng

$$E = \frac{1}{2n} y^T y + \frac{1}{2n} w^T (X^T X) w - \frac{1}{n} \sum_{j=0}^p (y^T X_j) w_j + \lambda \left(\sum_{j=0}^p |w_j| \right). \quad (1.18)$$

Lấy đạo hàm đối với biến w_j ta được

$$\begin{aligned} \frac{\partial E}{\partial w_j} &= \frac{1}{n} X^T X w - \frac{1}{n} y^T X_j + \lambda \operatorname{sgn}(w_j) \\ &= \frac{1}{n} \left(X_j^T X_j w_j + X_{0:j-1}^T X_{0:j-1} w_{0:j-1} + X_{j+1:n}^T X_{j+1:n} w_{j+1:n} - y^T X_j \right) + \lambda \operatorname{sgn}(w_j). \end{aligned}$$

Giải phương trình $\frac{\partial E}{\partial w_j} = 0$ ta được

$$w_j^{lasso} = \frac{1}{X_j^T X_j} S \left(y^T X_j - X_{0:j-1}^T X_{0:j-1} w_{0:j-1} - X_{j+1:n}^T X_{j+1:n} w_{j+1:n}, n\lambda \right). \quad (1.19)$$

Thuật toán lặp tìm các hệ số hồi quy lasso như sau:

Thuật toán 1.1 Tối ưu hoá hệ số hồi quy lasso.

Data: $(x_i, y_i), i = 1, 2, \dots, n$.

Result: $w_0, w_1, \dots, w_p$

Khởi tạo $w^{(0)}$;

repeat

for $j \leftarrow 0$ **to** p **do**

$$\left| \begin{array}{l} w_j^{(k+1)} = \frac{1}{X_j^T X_j} S \left(y^T X_j - X_{0:j-1}^T X_{0:j-1} w_{0:j-1}^{(k+1)} - X_{j+1:n}^T X_{j+1:n} w_{j+1:n}^{(k)} \right) \end{array} \right.$$

end

until $\|w^{(k+1)} - w^{(k)}\| < \varepsilon$;

Trở lại xét ví dụ dự báo bằng *hồi quy lasso* với dữ liệu *Hitters.csv*. Tương tự hồi quy Ridge, các hệ số hồi quy cũng tiến dần về 0 khi hệ số λ tiến đến vô cùng. Hệ số hiệu chỉnh λ tốt nhất, được xác định bằng phương pháp *xác thực chéo (Cross-Validation)*, là 10,25. Khi đó, hệ số xác định đối với tập dữ liệu huấn luyện và tập dữ liệu kiểm tra tương ứng bằng 54,5% và 19,8%.

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import Lasso, LassoCV
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
%matplotlib inline
plt.style.use('seaborn-white')
```

```
[2]: df = pd.read_csv('Data/Hitters.csv', index_col=0).dropna()
df.index.name = 'Player'
df.head()
```

```
[2]:
```

	AtBat	Hits	HmRun	Runs	RBI	Walks	Years	CAtBat	CHits	\
Player										
-Alan Ashby	315	81	7	24	38	39	14	3449	835	
-Alvin Davis	479	130	18	66	72	76	3	1624	457	
-Andre Dawson	496	141	20	65	78	37	11	5628	1575	
-Andres Galarraga	321	87	10	39	42	30	2	396	101	
-Alfredo Griffin	594	169	4	74	51	35	11	4408	1133	

	CHmRun	CRuns	CRBI	CWalks	League	Division	PutOuts	\
Player								
-Alan Ashby	69	321	414	375	N	W	632	
-Alvin Davis	63	224	266	263	A	W	880	
-Andre Dawson	225	828	838	354	N	E	200	
-Andres Galarraga	12	48	46	33	N	E	805	
-Alfredo Griffin	19	501	336	194	A	W	282	

	Assists	Errors	Salary	NewLeague
Player				
-Alan Ashby	43	10	475.0	N
-Alvin Davis	82	14	480.0	A
-Andre Dawson	11	3	500.0	N
-Andres Galarraga	40	4	91.5	N
-Alfredo Griffin	421	25	750.0	A

```
[3]: dummies = pd.get_dummies(df[['League', 'Division',
    -'NewLeague']])
#dummies.info()
print(dummies.head())
```

```
League_A  League_N  Division_E  Division_W  NewLeague_A  NewLeague_N
0         1         0         1         0         1
```

1	0	0	1	1	0
0	1	1	0	0	1
0	1	1	0	0	1
1	0	0	1	1	0

```
[4]: y = df.Salary
X_ = df.drop(['Salary', 'League', 'Division', 'NewLeague'],
             -axis=1).astype('float64')
X = pd.concat([X_, dummies[['League_N', 'Division_W',
                           -'NewLeague_N']]], axis=1)
```

```
[5]: X_train, X_test, y_train, y_test = train_test_split(X, y,
             -test_size=0.2, random_state=42)
```

```
[6]: list_numerical=X_train.columns[:16]
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler().fit(X_train[list_numerical])
X_train[list_numerical] = scaler.
             -transform(X_train[list_numerical])
X_test[list_numerical] = scaler.
             -transform(X_test[list_numerical])
```

```
[7]: reg = Lasso(alpha=1)
reg.fit(X_train, y_train)
print('R squared training set', round(reg.score(X_train,
             -y_train)*100, 2))
print('R squared test set', round(reg.score(X_test, y_test)*100,
             -2))
```

R squared training set 58.62

R squared test set 30.22

```
[8]: # Training data
pred_train = reg.predict(X_train)
mse_train = mean_squared_error(y_train, pred_train)
print('MSE training set', round(mse_train, 2))

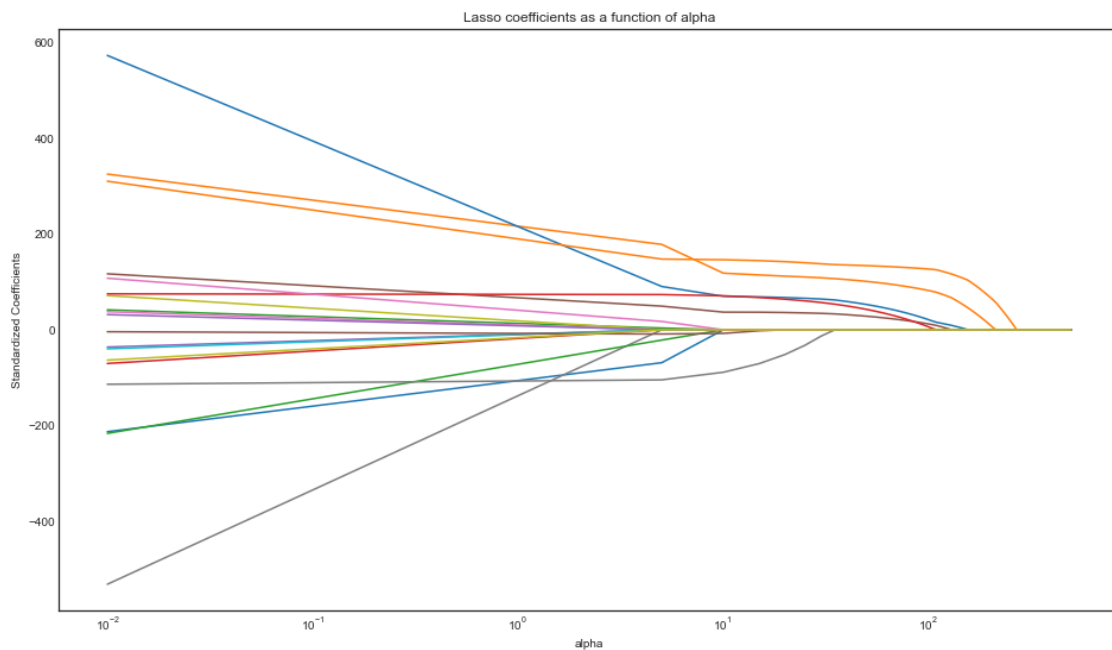
# Test data
pred = reg.predict(X_test)
mse_test = mean_squared_error(y_test, pred)
print('MSE test set', round(mse_test, 2))
```

```
[9]: alphas = np.linspace(0.01, 500, 100)
lasso = Lasso(max_iter=10000)
coefs = []

for a in alphas:
    lasso.set_params(alpha=a)
    lasso.fit(X_train, y_train)
    coefs.append(lasso.coef_)

plt.figure(figsize=(16, 9))
ax = plt.gca()

ax.plot(alphas, coefs)
ax.set_xscale('log')
plt.axis('tight')
plt.xlabel('alpha')
plt.ylabel('Standardized Coefficients')
plt.title('Lasso coefficients as a function of alpha')
plt.show()
```



```
[10]: model = LassoCV(cv=5, random_state=0, max_iter=10000)
      model.fit(X_train, y_train)
      model.alpha_
```

```
[10]: 10.250469583914377
```

```
[11]: lasso_best = Lasso(alpha=model.alpha_)
      lasso_best.fit(X_train, y_train)
      print('R squared training set', round(lasso_best.score(X_train,
      -y_train)*100, 2))
      print('R squared test set', round(lasso_best.score(X_test,
      -y_test)*100, 2))
```

```
R squared training set 54.5
```

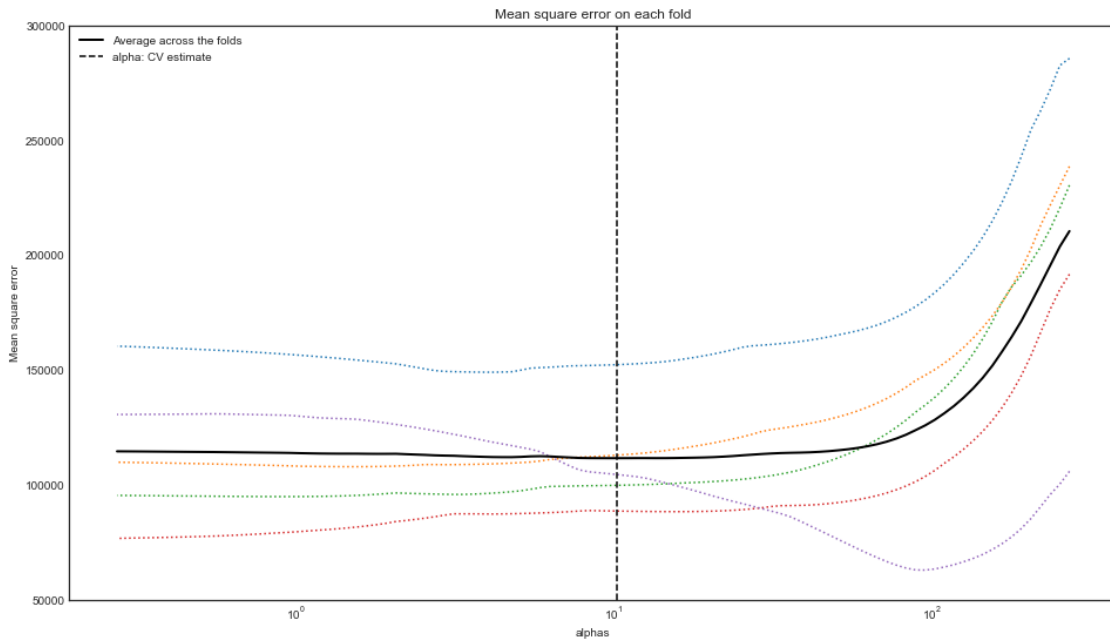
```
R squared test set 19.8
```

```
[12]: print('mean squared error training set:',
      -mean_squared_error(y_train, lasso_best.predict(X_train)))
      print('mean squared error test set:', mean_squared_error(y_test,
      -lasso_best.predict(X_test)))
```

```
mean squared error training set: 94613.03119763568
```

```
mean squared error test set: 145067.03639338588
```

```
[13]: plt.figure(figsize=(16,9))
      plt.semilogx(model.alphas_, model.mse_path_, ":")
      plt.plot(
          model.alphas_ ,
          model.mse_path_.mean(axis=-1), "k",
          label="Average across the folds",
          linewidth=2,
      )
      plt.axvline(model.alpha_, linestyle="--", color="k",
      -label="alpha: CV estimate")
      plt.legend()
      plt.xlabel("alphas")
      plt.ylabel("Mean square error")
      plt.title("Mean square error on each fold")
      plt.axis("tight")
      ymin, ymax = 50000, 300000
      plt.ylim(ymin, ymax);
```



1.4 Kiểm tra sự phụ thuộc tuyến tính

Hệ số phóng đại phương sai (VIF – variance inflation factor) có thể sử dụng để đo lường mức độ trùng lặp/dư thừa của từng biến độc lập khi có mặt các biến độc lập khác:

$$VIF(X_j) = \frac{1}{1 - R_{X_j|X_{-j}}^2} \quad (1.20)$$

trong đó $R_{X_j|X_{-j}}^2$ là hệ số xác định khi biến thứ j được coi là biến phản hồi, và các biến còn lại ($p - 1$ biến) được coi là biến độc lập trong ma trận $X \in R^{N \times p}$ (biến y không được sử dụng khi tính VIF). Giá trị VIF càng lớn thì biến độc lập đó càng được giải thích tốt bởi các biến độc lập khác, có nghĩa là biến thứ j là trùng lặp hay dư thừa.

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.stats.outliers_influence import
    variance_inflation_factor as VIF
%matplotlib inline
plt.style.use('seaborn-white')
```

```
[2]: df=pd.read_csv('Data/Credit.csv').drop(['Unnamed: 0'],axis=1)
df.head()
```

```
[2]:
```

	Income	Limit	Rating	Cards	Age	Education	Gender	Student	Married	\
0	14.891	3606	283	2	34	11	Male	No	Yes	
1	106.025	6645	483	3	82	15	Female	Yes	Yes	
2	104.593	7075	514	4	71	11	Male	No	No	
3	148.924	9504	681	3	36	11	Female	No	No	
4	55.882	4897	357	2	68	16	Male	No	Yes	

	Ethnicity	Balance
0	Caucasian	333
1	Asian	903
2	Asian	580
3	Asian	964
4	Caucasian	331

```
[3]: dummies = pd.get_dummies(df[['Gender', 'Student',
    _'Married', 'Ethnicity']])
dummies.head()
```

```
[3]:
```

	Gender_Male	Gender_Female	Student_No	Student_Yes	Married_No	\
0	1	0	1	0	0	
1	0	1	0	1	0	
2	1	0	1	0	1	
3	0	1	1	0	1	
4	1	0	1	0	0	

	Married_Yes	Ethnicity_African American	Ethnicity_Asian	Ethnicity_Caucasian
0	1	0	0	1
1	1	0	1	0
2	0	0	1	0
3	0	0	1	0
4	1	0	0	1

```
[5]: X=df.drop(['Gender', 'Student',
    _'Married', 'Ethnicity', 'Balance'], axis=1)
X = pd.concat([X_, dummies[['Gender_Female', 'Student_Yes',
    _'Married_Yes', 'Ethnicity_African
    _American', 'Ethnicity_Asian']]], axis=1)
X.head()
```

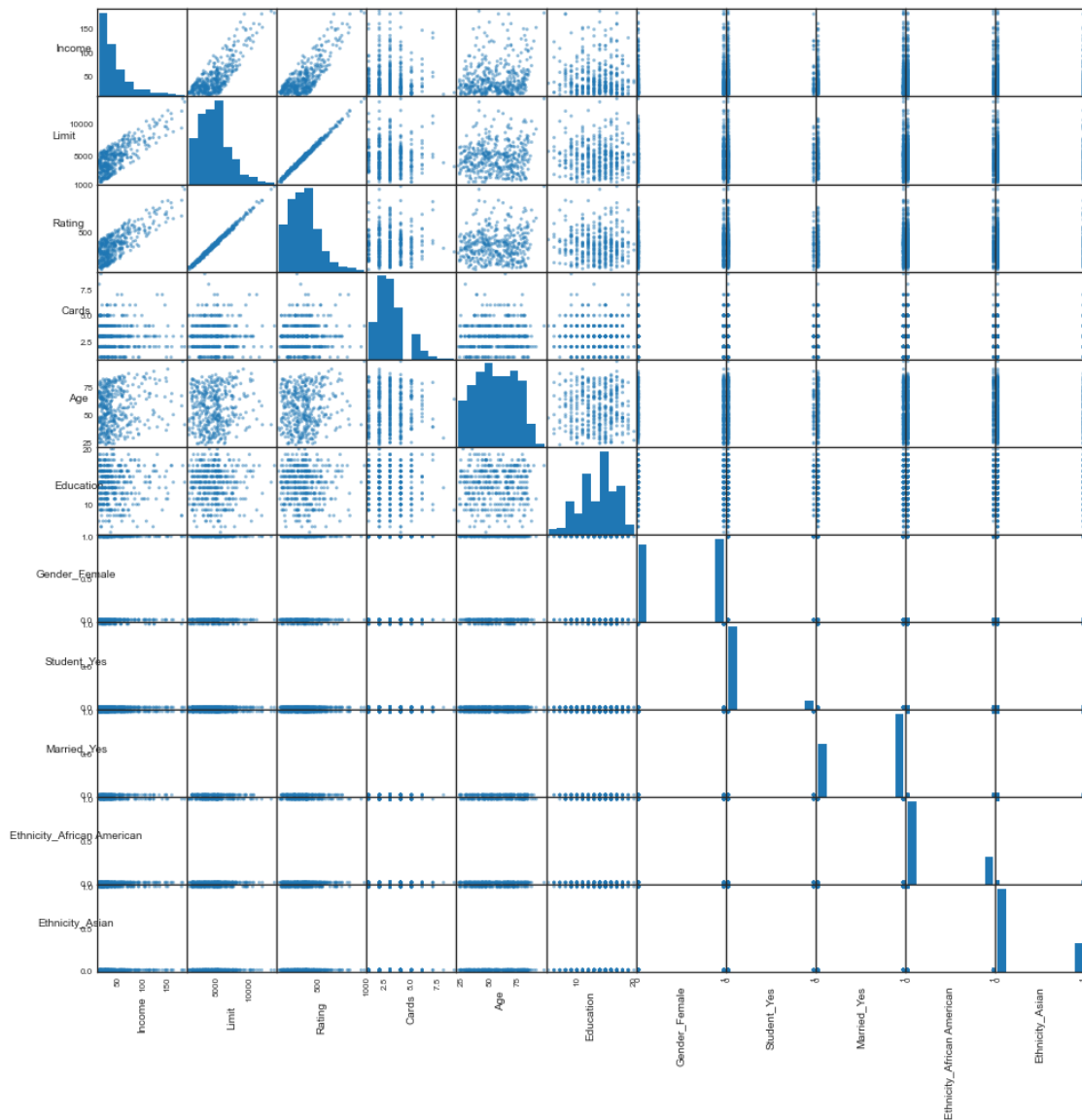
```
[5]:
```

	Income	Limit	Rating	Cards	Age	Education	Gender_Female	Student_Yes	\
0	14.891	3606	283	2	34	11	0	0	
1	106.025	6645	483	3	82	15	1	1	
2	104.593	7075	514	4	71	11	0	0	
3	148.924	9504	681	3	36	11	1	0	
4	55.882	4897	357	2	68	16	0	0	

	Married_Yes	Ethnicity_African American	Ethnicity_Asian
0	1	0	0
1	1	0	1
2	0	0	1
3	0	0	1
4	1	0	0

```
[6]: plt.figure()
axes=pd.plotting.scatter_matrix(X,figsize=(16,16))
for ax in axes.flatten():
```

```
ax.xaxis.label.set_rotation(90)
ax.yaxis.label.set_rotation(0)
plt.show()
```



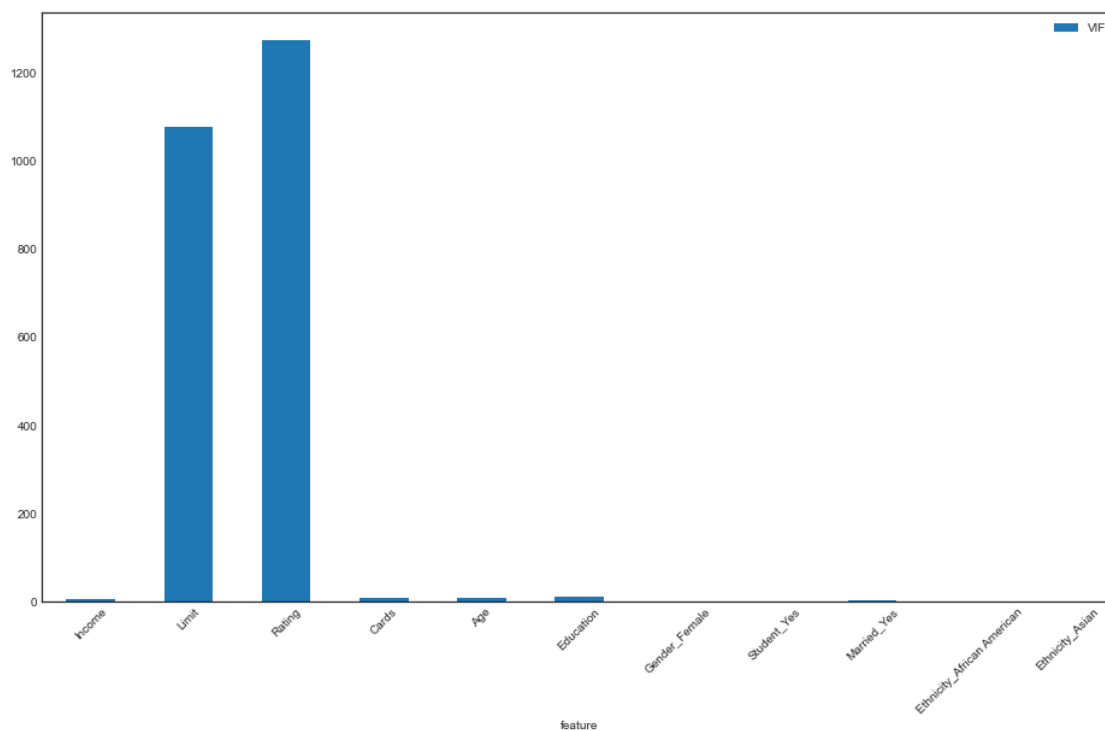
```
[7]: vif_data = pd.DataFrame()
vif_data["feature"] = X.columns
vif_data["VIF"] = [VIF(X.values, i) for i in range(len(X.
    _columns))]
```

```
[8]: vif_data.head(11)
```

```
[8]:
```

	feature	VIF
0	Income	7.210894
1	Limit	1076.918485
2	Rating	1273.228966
3	Cards	8.178001
4	Age	9.710669
5	Education	11.787414
6	Gender_Female	2.042636
7	Student_Yes	1.146115
8	Married_Yes	2.646457
9	Ethnicity_African American	1.500733
10	Ethnicity_Asian	1.510362

```
[9]: plt.figure()  
ax = vif_data.plot.bar(x='feature', y='VIF',  
                        rot=45, figsize=(16, 9))  
plt.show()
```



1.5 Bài tập

Bài 1.1. Giả sử chúng ta có một bộ dữ liệu về nhân viên mới ra trường tại một công ty công nghệ, với năm biến dự đoán (predictor): X_1 = GPA (Điểm trung bình tích lũy, thang điểm 4.0); X_2 = Kinh nghiệm (Số tháng kinh nghiệm thực tập có liên quan); X_3 = Giới tính (Biến giả: 1 cho Nữ và 0 cho Nam); X_4 = Tương tác GPA $\times$ Giới tính (GPA $\times$ X_3); X_5 = Tương tác Kinh nghiệm $\times$ Giới tính (Kinh nghiệm $\times$ X_3). Biến phản hồi (Y) là Mức lương khởi điểm hàng năm (tính bằng nghìn đô la). Chúng ta sử dụng phương pháp bình phương tối thiểu (least squares) để ước tính mô hình và thu được các hệ số sau:

$$\hat{\beta}_0 = 60, \quad \hat{\beta}_1 = 5, \quad \hat{\beta}_2 = 1.2, \quad \hat{\beta}_3 = -10, \quad \hat{\beta}_4 = 4, \quad \hat{\beta}_5 = -0.5$$

(a) Câu trả lời nào là đúng, và giải thích tại sao?

- Với giá trị GPA và Kinh nghiệm cố định, trung bình, nam giới kiếm được nhiều tiền hơn nữ giới.
- Với giá trị GPA và Kinh nghiệm cố định, trung bình, nữ giới kiếm được nhiều tiền hơn nam giới.
- Với giá trị Kinh nghiệm cố định, nữ giới kiếm được nhiều tiền hơn nam giới với điều kiện GPA đủ thấp.
- Với giá trị Kinh nghiệm cố định, nữ giới kiếm được nhiều tiền hơn nam giới với điều kiện GPA đủ cao.

(b) Dự đoán mức lương khởi điểm hàng năm của một nhân viên Nữ với GPA là 3.5 và Kinh nghiệm 6 tháng.

(c) Nhận định sau đây Đúng hay Sai: Hệ số tương tác Kinh nghiệm $\times$ Giới tính ($\hat{\beta}_5 = -0.5$) cho thấy rằng, đối với nam giới, mỗi tháng kinh nghiệm tăng thêm sẽ dẫn đến mức tăng lương nhỏ hơn so với nữ giới. Giải thích câu trả lời của bạn.

Bài 1.2. Giả sử chúng ta đang thực hiện hồi quy tuyến tính đơn:

$$Y = \beta_0 + \beta_1 X + \epsilon.$$

Giá trị dự báo có dạng:

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i,$$

trong đó, các hệ số $\hat{\beta}_0$ và $\hat{\beta}_1$ được ước lượng bằng phương pháp OLS:

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}, \quad \hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x},$$

với $\bar{x}$ và $\bar{y}$ là giá trị trung bình của X và Y . Chứng minh rằng có thể viết $\hat{y}_i$ dưới dạng tổ hợp tuyến tính của các giá trị phản hồi y_i :

$$\hat{y}_i = \sum_{j=1}^n h_{ij} y_j$$

và hãy tìm ra biểu thức cho hệ số h_{ij} .

Bài 1.3. Bài tập này liên quan đến việc sử dụng hồi quy tuyến tính đa biến trên tập dữ liệu `Auto`.

- (a) Tạo ma trận biểu đồ phân tán (scatterplot matrix) bao gồm tất cả các biến trong tập dữ liệu.
- (b) Tính ma trận tương quan giữa các biến bằng cách sử dụng phương thức `DataFrame.corr()`.
- (c) Sử dụng hàm `sm.OLS()` để thực hiện hồi quy tuyến tính đa biến với `mpg` là biến phản hồi và tất cả các biến khác, ngoại trừ `name`, là các biến dự đoán. Sử dụng hàm `summarize()` để in kết quả. Nhận xét về kết quả đầu ra bằng cách trả lời các câu hỏi:
 - Có mối quan hệ nào giữa các biến dự đoán và biến phản hồi không? Sử dụng hàm `anova_lm()` từ `statsmodels` để trả lời câu hỏi này.
 - Những biến dự đoán nào có mối quan hệ có ý nghĩa thống kê đối với biến phản hồi?
 - Hệ số của biến `year` cho thấy điều gì?
- (d) Tạo một số biểu đồ chẩn đoán (diagnostic plots) của mô hình hồi quy tuyến tính và cho nhận xét. Các biểu đồ phần dư (residual plots) có cho thấy bất kỳ điểm ngoại lai (outliers) nào lớn bất thường không?
- (e) Thêm vào mô hình các tương tác (interactions) giữa các biến dự đoán. Tương tác nào có ý nghĩa thống kê?
- (g) Thử một vài phép biến đổi (transformations) dữ liệu khác nhau, ví dụ $\log(X)$, $\sqrt{X}$, X^2 . Nhận xét về kết quả thu được.

1.6 Thực hành

Thực hành 1.1. Tập dữ liệu **Auto** bao gồm các thông tin về mức tiêu hao nhiên liệu, công suất động cơ và các thông tin khác về xe ô tô, được cung cấp trong **ISLP package**. Bài tập này liên quan đến việc sử dụng hồi quy tuyến tính đơn trên tập dữ liệu Auto.

- (a) Sử dụng hàm `sm.OLS()` để thực hiện hồi quy tuyến tính đơn với *mpg* (mức tiêu hao nhiên liệu) là biến phản hồi và *horsepower* (công suất động cơ) là biến dự đoán. Sử dụng hàm `summarize()` để in kết quả. Nhận xét về kết quả đầu ra bằng cách trả lời các câu hỏi:
- Có mối quan hệ nào giữa biến dự đoán và biến phản hồi không? Mối quan hệ mạnh đến mức nào và dương hay âm?
 - Giá trị dự đoán của biến *mpg* là bao nhiêu khi *horsepower* bằng 98? Khoảng tin cậy 95% tương ứng là bao nhiêu?
- (b) Vẽ đồ thị dữ liệu biến phản hồi và biến dự đoán. Vẽ đường hồi quy bình phương tối thiểu.

Thực hành 1.2. Thu thập dữ liệu ($n = 200$ quan sát) với một biến dự đoán X và một biến phản hồi Y . Sau đó khớp (fit) hai mô hình hồi quy sau với dữ liệu:

- Hồi quy Tuyến tính (Linear): $\hat{Y}_{\text{Lin}} = \hat{\beta}_0 + \hat{\beta}_1 X$,
- Hồi quy Bậc hai (Quadratic): $\hat{Y}_{\text{Quad}} = \hat{\beta}_0 + \hat{\beta}_1 X + \hat{\beta}_2 X^2$.

- (a) Giả sử mối quan hệ thực sự giữa X và Y là tuyến tính, tức là $Y = \beta_0 + \beta_1 X + \epsilon$. Hãy so sánh tổng bình phương phần dư (RSS) trên tập huấn luyện của hai mô hình. Chúng ta sẽ kỳ vọng mô hình nào có RSS thấp hơn, kỳ vọng bằng nhau, hay không đủ thông tin để kết luận? Giải thích câu trả lời.
- (b) Trả lời câu (a) nhưng sử dụng RSS trên tập kiểm tra thay vì trên tập huấn luyện.
- (c) Giả sử mối quan hệ thực sự giữa X và Y là phi tuyến. Hãy so sánh RSS trên tập huấn luyện cho hai mô hình. Chúng ta sẽ kỳ vọng mô hình nào có RSS thấp hơn, kỳ vọng bằng nhau, hay không đủ thông tin để kết luận? Giải thích câu trả lời.
- (d) Trả lời câu (c) nhưng sử dụng RSS trên tập kiểm tra thay vì trên tập huấn luyện.

Thực hành 1.3. Bài tập này liên quan đến việc tạo dữ liệu mô phỏng và khớp mô hình hồi quy tuyến tính đơn với dữ liệu đó. Để đảm bảo kết quả nhất quán, sử dụng bộ tạo số ngẫu nhiên mặc định với seed được đặt thành 1 trước khi bắt đầu.

- (a) Sử dụng phương thức *normal()*, tạo một vectơ x chứa 100 quan sát được lấy từ phân phối chuẩn $N(0, 1)$, là thể hiện của thuộc tính X .
- (b) Sử dụng phương thức *normal()*, tạo một vectơ ϵ chứa 100 quan sát được lấy từ một phân phối chuẩn $N(0, 0.25)$.
- (c) Sử dụng x và ϵ , tạo một vectơ y theo mô hình:

$$Y = -1 + 0.5X + \epsilon \quad (1.21)$$

Độ dài của vectơ y là bao nhiêu? Các giá trị β_0 và β_1 trong mô hình tuyến tính này là bao nhiêu?

- (d) Vẽ biểu đồ phân tán (scatterplot) hiển thị mối quan hệ giữa x và y . Nhận xét về những gì quan sát được.
- (e) Xây dựng mô hình hồi quy tuyến tính thực nghiệm để dự đoán y theo x . So sánh $\hat{\beta}_0$ và $\hat{\beta}_1$ với β_0 và β_1 .
- (g) Vẽ đường hồi quy tuyến tính thực nghiệm và đường hồi quy tổng thể (với màu khác) trên biểu đồ phân tán ở câu (d). Sử dụng phương thức *legend()* để tạo nhãn giải thích phù hợp.
- (h) Giữ nguyên mô hình (1.21). Tăng/giảm nhiễu bằng cách tăng/giảm phương sai của phân phối chuẩn được sử dụng để tạo ϵ . Sau đó lặp lại các bước từ (a) đến (g). Nhận xét về kết quả thu được.

Chương 2

PHÂN LOẠI

Mô hình hồi quy tuyến tính thảo luận trong Chương 1 giả định rằng biến phản hồi y là *biến định lượng* (*quantitative variable*). Tuy nhiên, trong nhiều trường hợp biến phản hồi lại là *biến định tính* (*qualitative variable*) hay còn gọi là *biến phân loại* (*categorical variable*). Trong chương này, chúng ta nghiên cứu các phương pháp dự báo các phản hồi định tính, được gọi là quá trình **phân loại** (**classification**). Các phương pháp phân loại thường hoạt động dựa trên việc dự đoán xác suất để một quan sát thuộc về mỗi lớp của biến định tính, sau đó mới đưa ra quyết định phân loại. Theo nghĩa này, chúng cũng hoạt động giống như các phương pháp hồi quy.

Có rất nhiều mô hình/kỹ thuật phân loại khác nhau (*classifiers*) có thể được sử dụng để dự đoán một phản hồi định tính. Chương này sẽ trình bày một số mô hình phân loại được sử dụng rộng rãi như: *hồi quy logistic*, *phân tích biệt số tuyến tính* (*linear discriminant analysis*), *phân tích biệt số bậc hai* (*quadratic discriminant analysis*).

2.1 Một số khái niệm cơ bản

Ma trận lỗi (Confusion matrix) Giả sử ta xét bài toán phân loại gồm có 2 lớp là lớp P (positive) còn gọi là lớp dương tính và lớp N (negative) còn gọi là lớp âm tính và tổng số là $T = N + P$. Xét một mô hình phân loại (dự báo) với hai khả năng phân loại là lớp P hoặc lớp N .

		Thực tế		Total
		Positive	Negative	
Dự đoán	Positive	TP	FP	$TP + FP$
	Negative	FN	TN	$FN + TN$
Total		$P = TP + FN$	$N = FP + TN$	$T = P + N$

Một số thuật ngữ:

TP (True Positive): là số mẫu dự đoán thuộc lớp P thực tế là dương tính (dự đoán đúng).

FP (False Positive): là số mẫu dự đoán thuộc lớp N thực tế là dương tính (dự đoán sai).

FN (False Negative): là số mẫu dự đoán thuộc lớp P thực tế là âm tính (dự đoán sai)

TN (True Negative): là số mẫu dự đoán thuộc lớp N thực tế là âm tính (dự đoán đúng).

Để đánh giá hiệu quả của một mô hình phân loại ta dựa vào các đại lượng sau:

Độ chuẩn xác (Accuracy) Tỷ lệ phân loại đúng (tỷ lệ phân loại đúng lớp P và lớp N)

$$Acc = \frac{TP + TN}{T} = \frac{TP + TN}{TP + FP + TN + FN}.$$

Độ nhạy (Sensitivity hoặc Recall hoặc True Positive Rate) Tỷ lệ dương tính thực (tỷ lệ phân loại lớp P)

$$TPR = \frac{TP}{P} = \frac{TP}{TP + FN}.$$

Độ chính xác (Precision hoặc Position Predictive Value) Tỷ lệ dương tính dự báo đúng

$$PPV = \frac{TP}{TP + FP}.$$

Giá trị F_1 (F_1 score) F_1 là trung bình điều hoà giữa Độ chính xác và Độ nhạy.

$$F_1 = \frac{2}{\frac{1}{Recall} + \frac{1}{Precision}}.$$

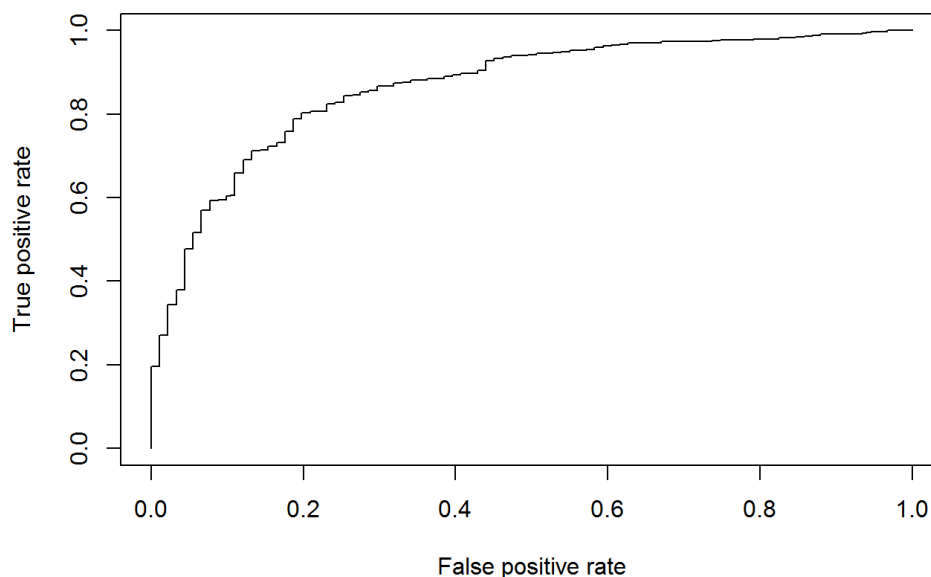
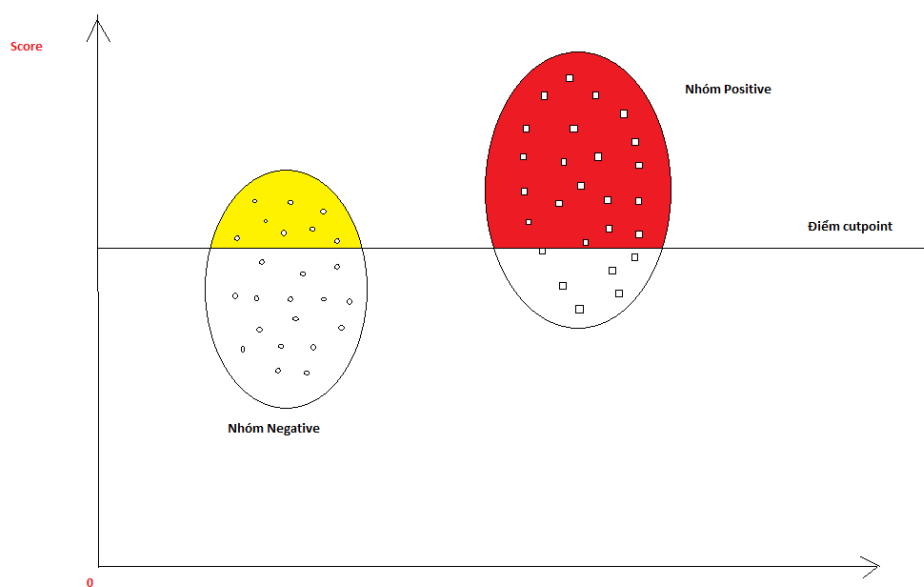
Độ đặc hiệu (Specificity hoặc True Negative Rate) Tỷ lệ âm tính thực (tỷ lệ phân loại lớp N)

$$TNR = \frac{TN}{N} = \frac{TN}{TN + FP}.$$

Tỷ lệ dương tính giả (FPR-False Positive Rate)

$$FPR = \frac{FP}{N} = \frac{FP}{TN + FP} = 1 - \frac{TN}{TN + FP} = 1 - TNR.$$

Đường cong ROC (Receiver Operating Characteristic) Đường cong ROC là đường biểu thị mối liên hệ giữa tỷ lệ dương tính giả FPR trên trục hoành và tỷ lệ dương tính thật TPR trên trục tung. Trong các loại mô hình phân loại thường phụ thuộc vào một giá trị gọi là ngưỡng (threshold hoặc cutpoint), khi giá trị của ngưỡng thay đổi thì bảng ma trận lỗi thay đổi theo như hình minh họa dưới đây.



Khi giá trị của điểm cắt (cutpoint) thay đổi thì tỷ lệ dương tính giả FPR và tỷ lệ dương tính thật TPR thay đổi và cho ta một đường cong gọi là đường cong ROC. Đường cong ROC phản ánh mức độ chính xác của mô hình. Mô hình tốt nhất là có tỷ lệ dương tính giả $FPR = 0$ và tỷ lệ dương tính thật $TPR = 1$. Do đó mô hình tốt nhất tương ứng với điểm gần điểm $(FPR, TPR) = (0, 1)$ trên đồ thị của đường cong ROC.

2.2 Hồi quy logistic

2.2.1 Phân loại nhị phân bằng hồi quy logistic

Giả sử ta xét bài toán phân loại dữ liệu gồm có n quan sát, mỗi quan sát gồm p biến, $x_1, x_2, \dots, x_n \in R^p$ và phân loại bởi các biến tương ứng $y_1, y_2, \dots, y_n \in \{0, 1\}$ sao cho cực tiểu hoá xác suất phân loại sai.

Trong phần này ta ký hiệu mỗi véc-tơ x (gồm p thành phần) viết dạng hàng và véc-tơ tham số β (gồm p thành phần) viết dạng cột, ta xét mô hình phân loại logistic dạng

$$P(y = 0|x) = \frac{e^{\beta_0 + x\beta}}{1 + e^{\beta_0 + x\beta}},$$

$$P(y = 1|x) = \frac{1}{1 + e^{\beta_0 + x\beta}}.$$

Hàm phân loại trên được viết lại dưới dạng

$$P(y|x) = \left(\frac{1}{1 + e^{\beta_0 + x\beta}} \right)^y \left(1 - \frac{1}{1 + e^{\beta_0 + x\beta}} \right)^{1-y}, \quad y \in \{0, 1\}. \quad (2.1)$$

Để xác định các tham số β_0, β ta sử dụng phương pháp hàm hợp lý cực đại (maximum likelihood function). *Hàm hợp lý* được xác định bằng tích các hàm xác suất của dữ liệu huấn luyện cho bởi công thức dưới đây:

$$L(\beta_0, \beta) = \prod_{i=1}^n P(y_i|x_i) = \prod_{i=1}^n \left(\frac{1}{1 + e^{\beta_0 + x_i\beta}} \right)^{y_i} \left(1 - \frac{1}{1 + e^{\beta_0 + x_i\beta}} \right)^{1-y_i}. \quad (2.2)$$

Để thuận tiện ta đặt $p_i = P(y_i = 1|x_i) = \frac{1}{1 + e^{\beta_0 + x_i\beta}}$ và xét hàm *log-hợp lý*:

$$l(\beta_0, \beta) = \log L(\beta_0, \beta) = \sum_{i=1}^n \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]. \quad (2.3)$$

Bộ tham số β_0, β cực đại hóa hàm hợp lý $L(\beta_0, \beta)$ cũng đồng thời cực đại hóa hàm log-hợp lý $l(\beta_0, \beta)$. Trong thực hành, việc ước lượng β_0, β thường được thực hiện thông qua việc

cực tiểu hoá một hàm mục tiêu, còn gọi là *hàm mất mát entropy chéo*:

$$\ell(\beta_0, \beta) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]. \quad (2.4)$$

Để tìm cực tiểu của hàm mục tiêu $\ell(\beta_0, \beta)$, ta phải giải các phương trình đạo hàm riêng của hàm $\ell(\beta_0, \beta)$ bằng 0. Do đó, ta có thể áp dụng phương pháp lặp Newton-Raphson như được cho trong công thức dưới đây:

$$(\beta_0, \beta) \leftarrow (\beta_0, \beta) - \left(\nabla^2 \ell(\beta_0, \beta) \right)^{-1} \nabla \ell(\beta_0, \beta), \quad (2.5)$$

trong đó $\nabla f(v) = \left(\frac{\partial f}{\partial v_i} \right)_{(p+1) \times 1}$ và $\nabla^2 f(v) = \left(\frac{\partial^2 f}{\partial v_i \partial v_j} \right)_{(p+1) \times (p+1)}$.

Chúng ta sẽ minh họa hồi quy logistic với tập dữ liệu *Default.csv*.

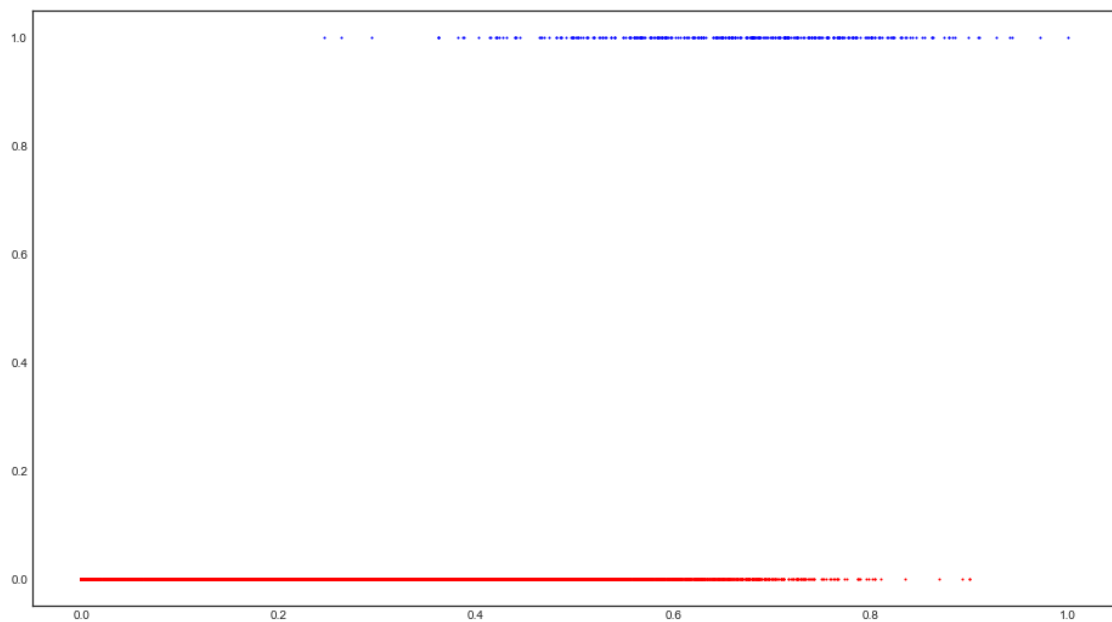
```
[1]: # %load ../standard_import.txt
import pandas as pd
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report,
    _precision_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import plot_confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
%matplotlib inline
plt.style.use('seaborn-white')
```

```
[2]: df = pd.read_csv('Data/Default.csv')
df['student2'] = df.student.factorize()[0]
df['default2'] = df.default.factorize()[0]
df=df.drop(['default', 'student'], axis=1)
df.head(3)
```

```
[2]:      balance      income  student2  default2
0    729.526495  44361.625074         0         0
1    817.180407  12106.134700         1         0
2   1073.549164   31767.138947         0         0
```

```
[3]: x=df['balance']  
x=x/x.max()  
y=df['default2']
```

```
[4]: plt.figure(figsize=(16,9))  
id0=y==0  
id1=y==1  
plt.scatter(x[id0],y[id0],c='red',s=1)  
plt.scatter(x[id1],y[id1],c='blue',s=1)  
plt.show()
```



```
[5]: x=x.values.reshape(-1,1)  
x.shape
```

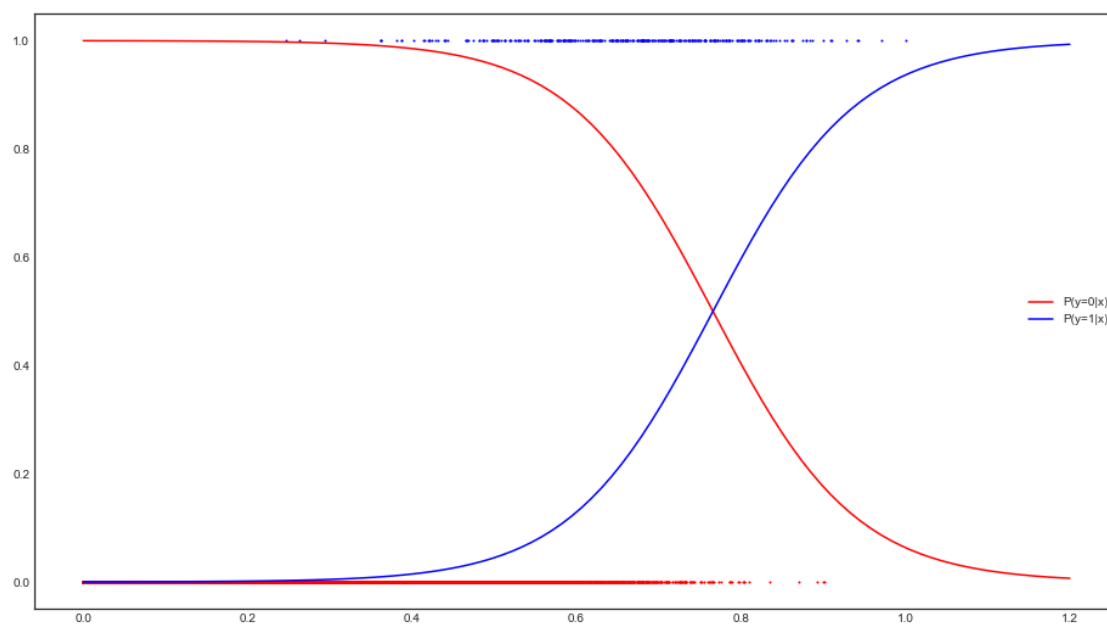
```
[5]: (10000, 1)
```

```
[6]: model = LogisticRegression()  
model.fit(x, y)  
print(model.intercept_)  
print(model.coef_[0])
```

```
[-8.80979582]
```

```
[11.50181457]
```

```
[7]: plt.figure(figsize=(16,9))
      id0=y==0
      id1=y==1
      plt.scatter(x[id0],y[id0],c='red',s=1)
      plt.scatter(x[id1],y[id1],c='blue',s=1)
      xx=np.linspace(0,1.2,100).reshape(-1,1)
      yy=1/(1+np.exp(model.intercept_+model.coef_[0][0]*xx))
      plt.plot(xx,yy,c='red',label='P(y=0|x)')
      plt.plot(xx,1-yy,c='blue',label='P(y=1|x)')
      plt.legend(loc='best')
      plt.show()
```



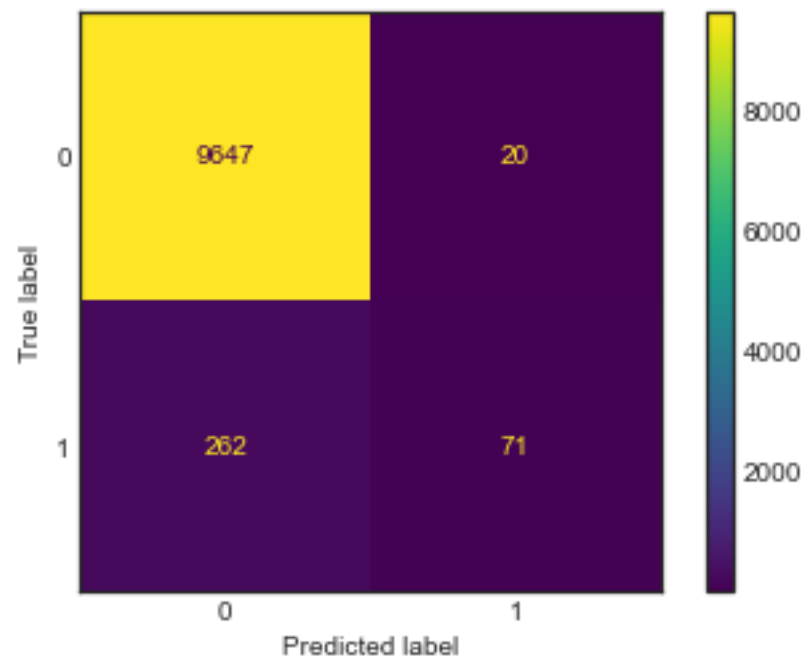
```
[8]: y_pred=model.predict(x)
```

```
[9]: confusion_matrix(y, y_pred)
```

```
[9]: array([[9647,   20],
           [ 262,   71]], dtype=int64)
```

```
[10]: plt.figure(figsize=(8,8))
       ConfusionMatrixDisplay.from_estimator(model,x,y)
       plt.show()
```

<Figure size 576x576 with 0 Axes>



```
[11]: pred_prob = model.predict_proba(x)
```

```
[12]: from sklearn.metrics import roc_curve

fpr, tpr, thresh = roc_curve(y, pred_prob[:,1], pos_label=1)
```

```
[13]: from sklearn.metrics import roc_auc_score
# auc scores
auc_score = roc_auc_score(y, pred_prob[:,1])
auc_score
```

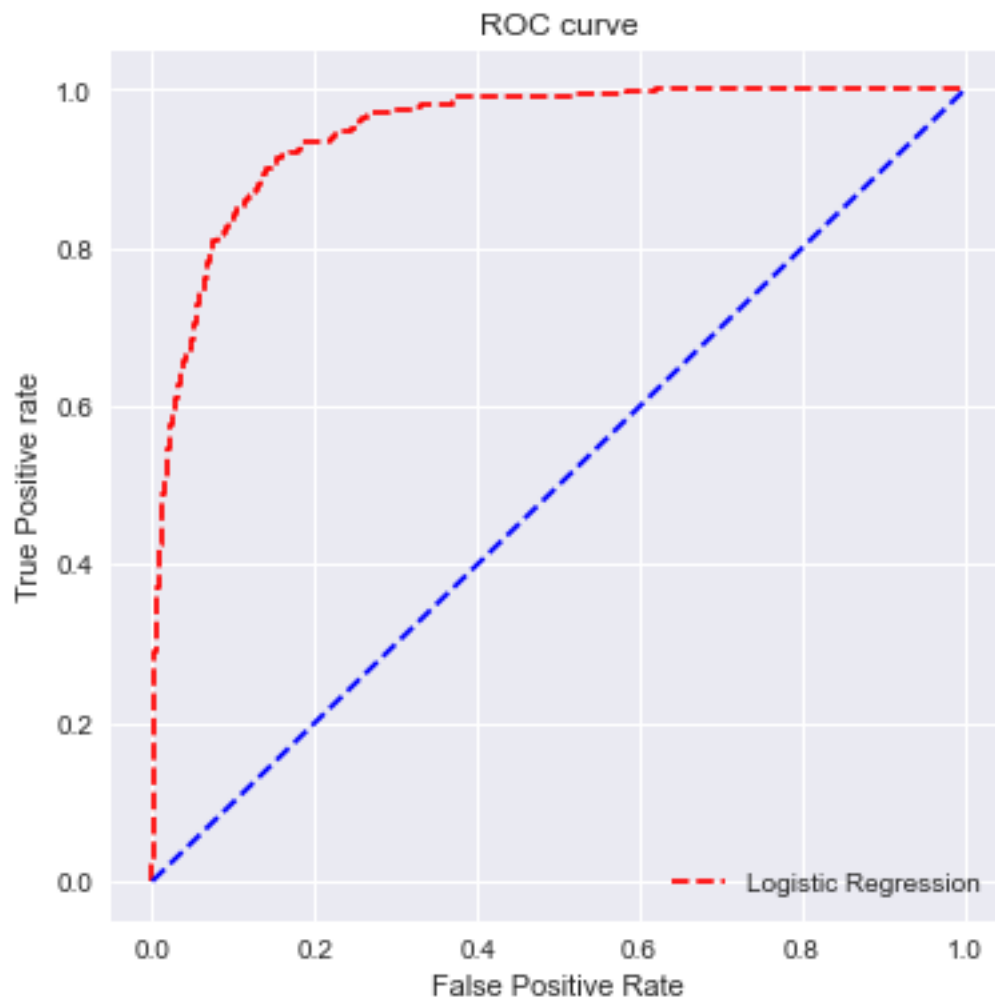
```
[13]: 0.9479784946837808
```

```
[14]: # roc curve for tpr = fpr
random_probs = [0 for i in range(len(y))]
p_fpr, p_tpr, _ = roc_curve(y, random_probs, pos_label=1)
```

```
[15]: # matplotlib
import matplotlib.pyplot as plt
plt.style.use('seaborn')
plt.figure(figsize=(6,6))
# plot roc curves
plt.plot(fpr, tpr, linestyle='--', color='red', label='Logistic
-Regression')
```

```
plt.plot(p_fpr, p_tpr, linestyle='--', color='blue')
# title
plt.title('ROC curve')
# x label
plt.xlabel('False Positive Rate')
# y label
plt.ylabel('True Positive rate')

plt.legend(loc='best')
plt.savefig('ROC', dpi=300)
plt.show();
```



```
[16]: pred_prob = model.predict_proba(x)[: ,1]
```

```
[17]: cut_off=np.linspace(0,1,11)
      cut_off
```

```
[17]: array([0. , 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1. ])
```

```
[18]: CM=[]
      for cutpoint in cut_off:
          y_pred=pred_prob>=cutpoint
          cm=confusion_matrix(y, y_pred)
          CM.append(cm.flatten())
```

```
[19]: df=pd.DataFrame(CM,columns=['TP','FN','FP','TN'])
      df
```

```
[19]:
```

	TP	FN	FP	TN
0	0	9667	0	333
1	9089	578	85	248
2	9436	231	141	192
3	9577	90	195	138
4	9625	42	233	100
5	9647	20	262	71
6	9658	9	293	40
7	9663	4	313	20
8	9664	3	325	8
9	9667	0	331	2
10	9667	0	333	0

```
[20]: df['TPR']=df['TP']/(df['TP']+df['FN'])
      df['FPR']=df['FP']/(df['FP']+df['TN'])
```

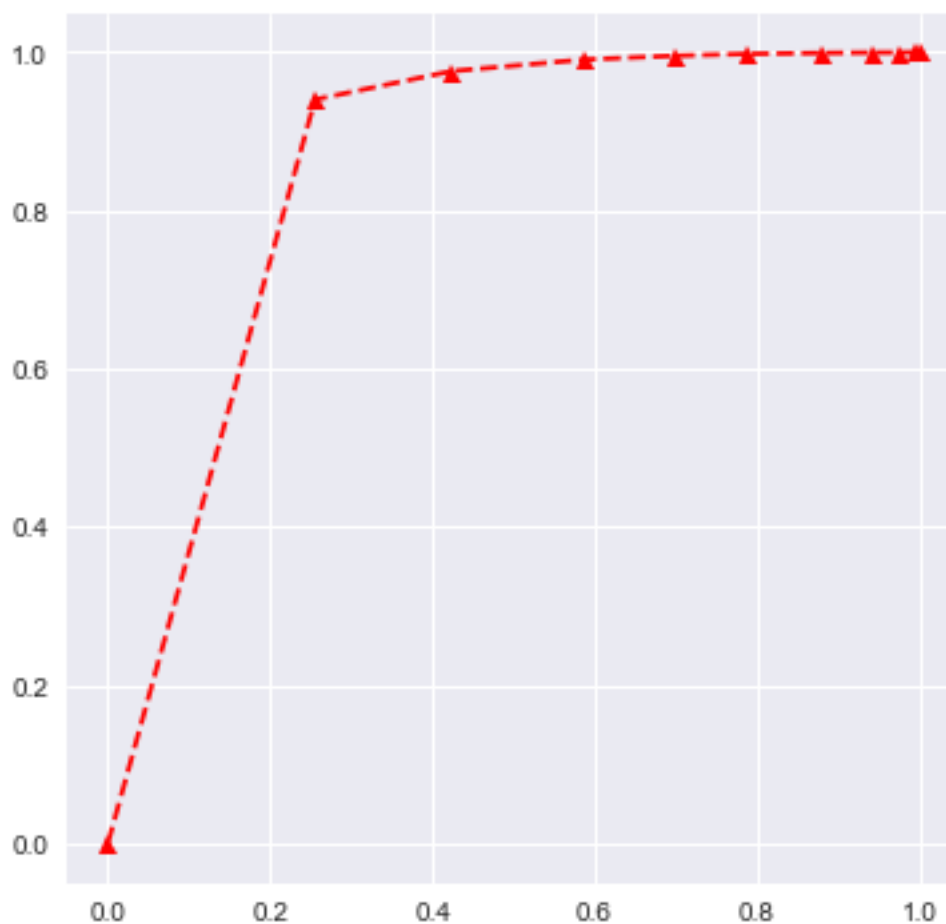
```
[21]: df
```

```
[21]:
```

	TP	FN	FP	TN	TPR	FPR
0	0	9667	0	333	0.000000	0.000000
1	9089	578	85	248	0.940209	0.255255
2	9436	231	141	192	0.976104	0.423423
3	9577	90	195	138	0.990690	0.585586
4	9625	42	233	100	0.995655	0.699700
5	9647	20	262	71	0.997931	0.786787
6	9658	9	293	40	0.999069	0.879880
7	9663	4	313	20	0.999586	0.939940
8	9664	3	325	8	0.999690	0.975976
9	9667	0	331	2	1.000000	0.993994

```
10  9667      0  333      0  1.000000  1.000000
```

```
[22]: plt.figure(figsize=(6,6))
plt.plot(df['FPR'],df['TPR'],marker='^',c='red',linestyle='--')
plt.show()
```



```
[23]: from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```
[24]: df = pd.read_csv('Data/Default.csv')
df['student2'] = df.student.factorize()[0]
df['default2'] = df.default.factorize()[0]
df=df.drop(['default','student'],axis=1)
df.head(3)
```

```
[24]:      balance      income  student2  default2
0    729.526495  44361.625074         0         0
```



```

1    817.180407    12106.134700         1         0
2   1073.549164    31767.138947         0         0

```

```

[25]: X=df[['balance','income','student2']].values
      y=df['default2'].values
      print(X.shape,y.shape)

```

```

(10000, 3) (10000,)

```

```

[26]: X_train, X_test, y_train, y_test = train_test_split(X, y,
      -test_size=0.3, random_state=42)

```

```

[27]: model = LogisticRegression().fit(X_train, y_train)

```

```

[28]: print(model.intercept_,model.coef_)

```

```

[-2.92273069] [[ 4.10824216e-03 -1.35591924e-04 -3.80773860e+00]]

```

```

[29]: print(model.score(X_train,y_train).round(4),model.
      -score(X_test,y_test).round(4))

```

```

0.9664 0.9683

```

```

[31]: p_test=model.predict_proba(X_test)[:,-1]
      p_train=model.predict_proba(X_train)[:,-1]

```

```

[32]: cut_off=np.linspace(0,1,11)
      CM_test=[]
      CM_train=[]
      for cutpoint in cut_off:
          y_test_pred=p_test>=cutpoint
          y_train_pred=p_train>=cutpoint
          cm=confusion_matrix(y_test, y_test_pred)
          CM_test.append(cm.flatten())
          cm=confusion_matrix(y_train, y_train_pred)
          CM_train.append(cm.flatten())

```

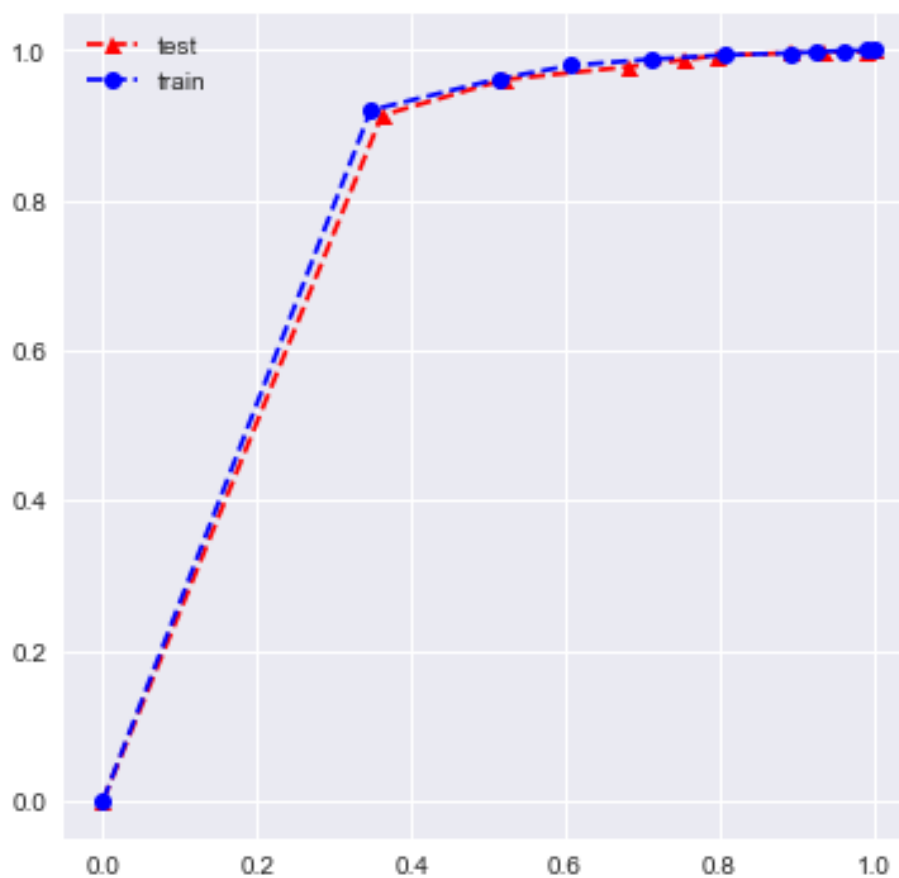
```

[33]: df_test=pd.DataFrame(CM_test,columns=['TP','FN','FP','TN'])
      df_train=pd.DataFrame(CM_train,columns=['TP','FN','FP','TN'])

```

```
[34]: df_test['TPR'] = df_test['TP'] / (df_test['TP'] + df_test['FN'])  
df_test['FPR'] = df_test['FP'] / (df_test['FP'] + df_test['TN'])  
df_train['TPR'] = df_train['TP'] / (df_train['TP'] + df_train['FN'])  
df_train['FPR'] = df_train['FP'] / (df_train['FP'] + df_train['TN'])
```

```
[35]: plt.figure(figsize=(6, 6))  
plt.  
    _plot(df_test['FPR'], df_test['TPR'], marker='^', c='red', linestyle='--',  
    _', label='test')  
plt.  
    _plot(df_train['FPR'], df_train['TPR'], marker='o', c='blue', linestyle='--',  
    _', label='train')  
plt.legend()  
plt.show()
```



2.2.2 Phân loại đa lớp bằng hồi quy logistic

Bây giờ chúng ta xét bài toán phân loại dữ liệu gồm có n quan sát trong không gian p -chiều $x_1, x_2, \dots, x_n$ và phân loại bởi các biến tương ứng $y_1, y_2, \dots, y_n \in \{1, 2, \dots, K\}$ với số lớp $K > 2$.

Ký hiệu $\beta_{k0}, k = 1, 2, \dots, K$ là các hệ số chặn (intercept) và các hệ số độ dốc $\beta_k, k = 1, 2, \dots, K$. Mô hình *hồi quy logistic đa lớp* (multinomial logistic regression) có dạng:

$$P(y = k|x) = \frac{e^{\beta_{k0} + x\beta_k}}{\sum_{j=1}^K e^{\beta_{j0} + x\beta_j}}, k = 1, 2, \dots, K. \quad (2.6)$$

Ta định nghĩa hàm softmax như sau:

$$\text{softmax}(z_1, z_2, \dots, z_K) = \left(\frac{e^{z_1}}{\sum_{j=1}^K e^{z_j}}, \frac{e^{z_2}}{\sum_{j=1}^K e^{z_j}}, \dots, \frac{e^{z_K}}{\sum_{j=1}^K e^{z_j}} \right). \quad (2.7)$$

Đặt $z_k = \beta_{k0} + x\beta_k, k = 1, 2, \dots, K$, hàm xác suất dự đoán trên các lớp có dạng

$$P(y|x) = \text{softmax}(z_1, z_2, \dots, z_K). \quad (2.8)$$

Khi đó mẫu dữ liệu x sẽ được phân loại vào lớp k nếu $P(y = k|x) = \max\{P(y|x)\}$, nghĩa là

$$\text{class}(y) = \text{argmax}_k \{P(y = k|x)\}. \quad (2.9)$$

Các tham số β_{k0}, β_k được ước lượng bằng phương pháp hàm hợp lý cực đại. Tương tự như trong hồi quy logistic nhị phân, ta véc-tơ hoá các biến phân lớp

$$\begin{aligned} y = 1 &\Rightarrow y = (1, 0, \dots, 0), \\ y = 2 &\Rightarrow y = (0, 1, \dots, 0), \\ &\vdots \\ y = K &\Rightarrow y = (0, 0, \dots, 1). \end{aligned}$$

Nói cách khác, khi biến $y = k$, ta biểu diễn bằng véc-tơ $y = (y_1, y_2, \dots, y_K)$ với thành phần $y_k = 1$ và các thành phần $y_i = 0, i \neq k$.

Để thuận tiện cho tính toán, ta cực tiểu hoá hàm mục tiêu (được gọi là hàm mất mát entropy chéo):

$$\ell(\beta_0, \beta_1, \dots, \beta_K) = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{ik} \log(p_{ik}), \quad (2.10)$$

trong đó

$$p_{ik} = P(y = k|x_i) = \frac{e^{\beta_{k0} + x_i \beta_k}}{\sum_{j=1}^K e^{\beta_{j0} + x_i \beta_j}}, k = 1, 2, \dots, K \quad (2.11)$$

là các xác suất dự đoán mẫu x_i thuộc lớp k . Việc cực tiểu hóa hàm mất mát entropy chéo có thể thực hiện bằng thuật toán Gradient Descent.

Xét bài toán phân loại bằng hồi quy logistic với tập dữ liệu *Iris dataset*.

```
[1]: import pandas as pd
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report,
    precision_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import plot_confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
from sklearn.model_selection import train_test_split
%matplotlib inline
plt.style.use('seaborn-white')
```

```
[2]: from sklearn.datasets import load_iris, load_wine
X=load_iris().data
y=load_iris().target
```

```
[3]: # Add noisy features to make the problem harder
random_state = np.random.RandomState(0)
n_samples, n_features = X.shape
for i in range(n_samples):
    for j in range(n_features):
        X[i, j]=X[i, j]+random_state.randn()
```

```
[4]: X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)
```

```
[5]: model = LogisticRegression(multi_class='multinomial')
model.fit(X_train,y_train)
b=model.intercept_
```

```
w=model.coef_  
print(b)  
print(w)
```

```
[ 2.98071951  2.02311176 -5.00383127]  
[[ 0.14570155  0.84011942 -1.91622336 -0.40896651]  
 [-0.19665999 -0.46918763  0.45009462  0.08723277]  
 [ 0.05095844 -0.37093179  1.46612875  0.32173374]]
```

```
[6]: x_test=X_test[0]  
z=np.dot(x_test,w.T)+b  
print(z)
```

```
[-4.03447243  1.81212653  2.22234591]
```

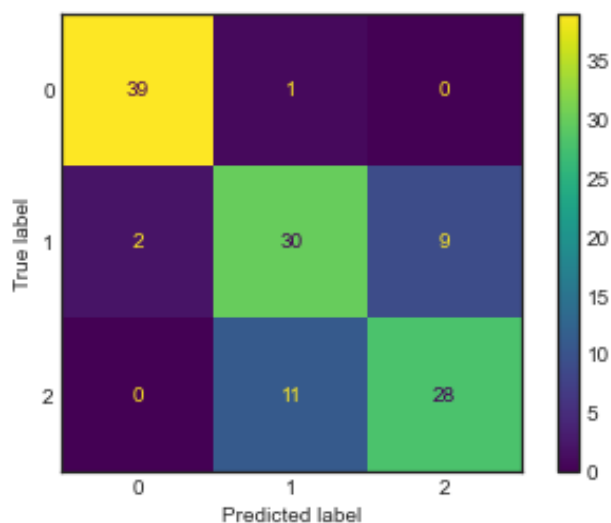
```
[7]: proba=np.exp(z)/np.sum(np.exp(z))  
print(proba)
```

```
[0.00115126 0.39840033 0.60044841]
```

```
[8]: y_train_pred=model.predict(X_train)  
print(confusion_matrix(y_train,y_train_pred))
```

```
[[39  1  0]  
 [ 2 30  9]  
 [ 0 11 28]]
```

```
[9]: plt.figure(figsize=(8,8))  
ConfusionMatrixDisplay.from_estimator(model,X_train,y_train)  
plt.show()
```



```
[10]: print(classification_report(y_train,y_train_pred))
```

	precision	recall	f1-score	support
0	0.95	0.97	0.96	40
1	0.71	0.73	0.72	41
2	0.76	0.72	0.74	39
accuracy			0.81	120
macro avg	0.81	0.81	0.81	120
weighted avg	0.81	0.81	0.81	120

```
[11]: from sklearn.metrics import roc_curve, auc
      from sklearn.preprocessing import label_binarize
```

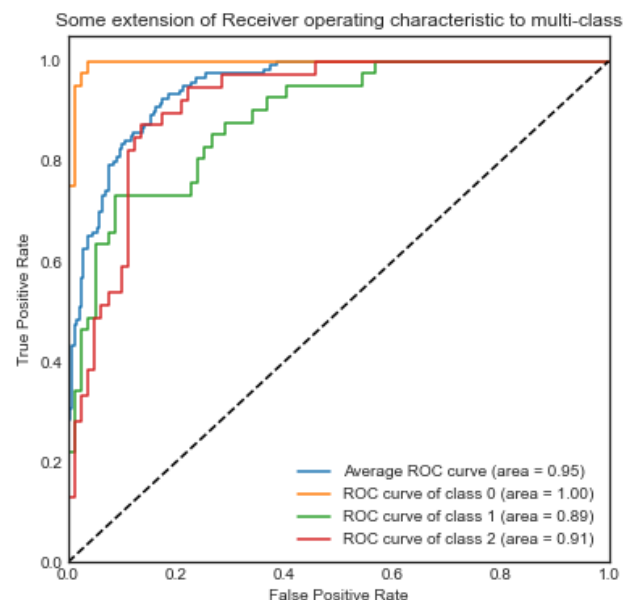
```
[12]: y_train_pred = model.predict_proba(X_train)
      y_train_binarize=label_binarize(y_train, classes=[0, 1, 2])
```

```
[13]: fpr = dict()
      tpr = dict()
      roc_auc = dict()
      for i in range(3):
          fpr[i], tpr[i], _ = roc_curve(y_train_binarize[:, i],
          -y_train_pred[:, i])
          roc_auc[i] = auc(fpr[i], tpr[i])
      fpr["micro"], tpr["micro"], _ = roc_curve(y_train_binarize.ravel(),
      -y_train_pred.ravel())
      roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])
```

```
[14]: # Plot ROC curve
plt.figure(figsize=(6,6))
plt.plot(fpr["micro"], tpr["micro"],
         label='Average ROC curve (area = {0:0.2f})'
         ''.format(roc_auc["micro"]))

for i in range(3):
    plt.plot(fpr[i], tpr[i], label='ROC curve of class {0} (area
    _= {1:0.2f})'
            ''.format(i, roc_auc[i]))

plt.plot([0, 1], [0, 1], 'k--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Some extension of Receiver operating characteristic
-to multi-class')
plt.legend(loc="lower right")
plt.show()
```

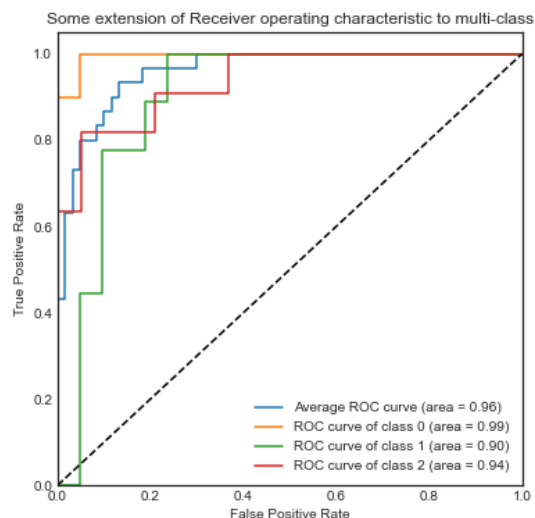


```
[15]: y_test_pred = model.predict_proba(X_test)
y_test_binarize=label_binarize(y_test, classes=[0, 1, 2])
#y_train_binarize
```

```
[16]: fpr = dict()
      tpr = dict()
      roc_auc = dict()
      for i in range(3):
          fpr[i], tpr[i], _ = roc_curve(y_test_binarize[:, i],
                                         -y_test_pred[:, i])
          roc_auc[i] = auc(fpr[i], tpr[i])
      fpr["micro"], tpr["micro"], _ = roc_curve(y_test_binarize.
                                         -ravel(), y_test_pred.ravel())
      roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])
```

```
[17]: plt.figure(figsize=(6,6))
      plt.plot(fpr["micro"], tpr["micro"],
                label='Average ROC curve (area = {0:0.2f})'
                ''.format(roc_auc["micro"]))
      for i in range(3):
          plt.plot(fpr[i], tpr[i], label='ROC curve of class {0} (area
          -= {1:0.2f})'
                  ''.format(i, roc_auc[i]))

      plt.plot([0, 1], [0, 1], 'k--')
      plt.xlim([0.0, 1.0])
      plt.ylim([0.0, 1.05])
      plt.xlabel('False Positive Rate')
      plt.ylabel('True Positive Rate')
      plt.title('Some extension of Receiver operating characteristic
      -to multi-class')
      plt.legend(loc="lower right")
      plt.show()
```



2.3 Một số mô hình phân loại Bayes

Trong phần trước ta sử dụng hàm $P(Y = k|X)$ để phân loại các lớp. Trong phần này chúng ta xem xét một số mô hình phân loại sử dụng công thức Bayes

$$P(B|A) = \frac{P(B)P(A|B)}{P(A)}. \quad (2.12)$$

Giả sử ta phân loại dữ liệu X thành K lớp với $K \geq 2$. Ta gọi π_k là xác suất tiên nghiệm (prior probability) để chọn được một dữ liệu thuộc lớp k , trong đó

$$\pi_k \geq 0, \sum_{k=1}^K \pi_k = 1. \quad (2.13)$$

Khi đó ta có công thức

$$p_k(x) = P(Y = k|X = x) = \frac{P(Y = k)P(X = x|Y = k)}{P(X)} = \frac{\pi_k P(X = x|Y = k)}{\sum_{l=1}^K \pi_l P(X = x|Y = l)}. \quad (2.14)$$

Công thức trên biểu diễn xác suất hậu nghiệm (posterior probability) để quan sát $X = x$ thuộc lớp thứ k .

Để tính xác suất $p_k(x)$, ta có thể ước lượng π_k và $f_k(x) = P(X = x|Y = k)$ rồi thay vào (2.14). Nói chung, có thể dễ dàng ước lượng π_k , nhưng sẽ khó khăn hơn để ước lượng $f_k(x)$. Người ta thường đưa ra một số giả định đơn giản hóa, dẫn đến việc sử dụng các phương pháp được trình bày sau đây để ước lượng $f_k(x)$.

2.3.1 Phương pháp LDA

Phương pháp LDA (Linear Discriminant Analysis) là phương pháp phân loại phân biệt tuyến tính. Phương pháp LDA giả sử các lớp có phân phối chuẩn nhiều chiều có cùng ma trận hiệp phương sai Σ và có trung bình của lớp thứ k là μ_k .

Trường hợp số chiều $p = 1$: Trong không gian một chiều, hàm mật độ của phân phối chuẩn có dạng

$$f_k(x) = \frac{1}{\sqrt{2\pi}\sigma_k} \exp\left(-\frac{1}{2\sigma_k^2}(x - \mu_k)^2\right). \quad (2.15)$$

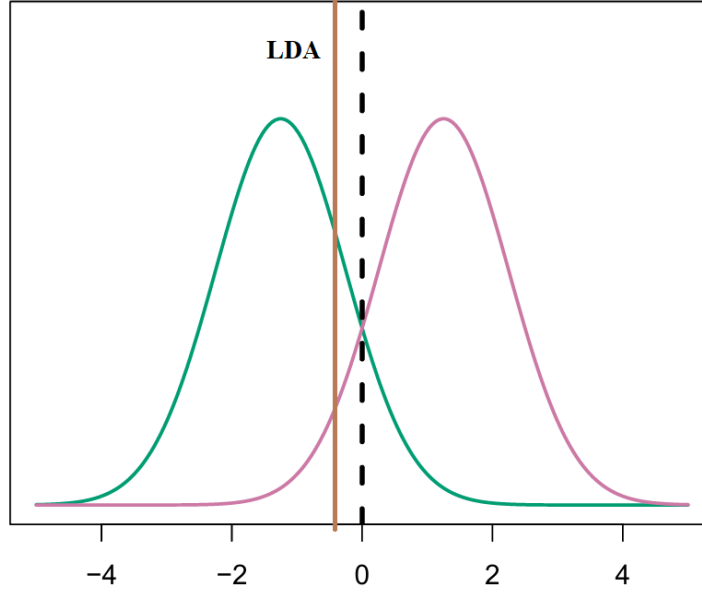
Thêm giả thiết $\sigma_1^2 = \dots = \sigma_K^2 = \sigma^2$, ta tính được

$$p_k(x) = \frac{\pi_k \exp\left(-\frac{1}{2\sigma^2}(x - \mu_k)^2\right)}{\sum_{l=1}^K \pi_l \exp\left(-\frac{1}{2\sigma^2}(x - \mu_l)^2\right)}. \quad (2.16)$$

Quan sát $X = x$ sẽ được phân loại vào lớp có xác suất (2.16) lớn nhất. Bằng cách lấy logarit 2 vế của (2.16) và sắp xếp lại các số hạng, dễ dàng suy ra rằng quan sát $X = x$ được phân loại vào lớp có đại lượng

$$\delta_k(x) = x \frac{\mu_k}{\sigma^2} - \frac{\mu_k^2}{2\sigma^2} + \log(\pi_k) \quad (2.17)$$

đạt giá trị lớn nhất.



Hình 2.1: Phân loại Bayes với 2 lớp, đường nét đứt biểu diễn biên phân loại lý thuyết, đường liền nét biểu diễn biên phân loại LDA từ dữ liệu huấn luyện

Trường hợp số chiều $p > 1$: Hàm mật độ xác suất của lớp thứ k trong không gian p chiều có dạng

$$f_k(x) = \frac{1}{(2\pi)^{p/2}(\det \Sigma)^{1/2}} \exp\left(-\frac{1}{2}(x - \mu_k)^T \Sigma^{-1}(x - \mu_k)\right). \quad (2.18)$$

Quan sát $X = x$ phân loại vào lớp có xác suất cao nhất $P(Y = k|X = x)$. Lấy logarit của hàm trên và bỏ phần chung của các số hạng đi, ta suy ra rằng quan sát $X = x$ được phân loại vào lớp thứ k nếu hàm phân loại $\delta_k(x)$ lớn nhất với

$$\delta_k(x) = x^T \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^T \Sigma^{-1} \mu_k + \log(\pi_k). \quad (2.19)$$

Hàm phân loại được xác định bởi công thức

$$class(x) = \operatorname{argmax}_k \{\delta_k(x)\}. \quad (2.20)$$

2.3.2 Phương pháp QDA

Phương pháp QDA (Quadratic Discriminant Analysis) là phương pháp phân loại phân biệt bậc 2. Khác với LDA, QDA giả thiết rằng mỗi lớp có ma trận hiệp phương sai riêng. Do đó, giả sử lớp thứ k có phân phối chuẩn nhiều chiều $N(\mu_k, \Sigma_k)$. Khi đó hàm phân loại $\delta_k(x)$ xác định bởi công thức

$$\begin{aligned}\delta_k(x) &= -\frac{1}{2}(x - \mu_k)^T \Sigma_k^{-1} (x - \mu_k) - \frac{1}{2} \log(\det \Sigma_k) + \log \pi_k. \\ &= -\frac{1}{2} x^T \Sigma_k^{-1} x + x^T \Sigma_k \mu_k - \frac{1}{2} \mu_k^T \Sigma_k^{-1} \mu_k - \frac{1}{2} \log(\det \Sigma_k) + \log \pi_k.\end{aligned}$$

Hàm phân loại được xác định bởi công thức

$$class(x) = \operatorname{argmax}_k \{\delta_k(x)\}. \quad (2.21)$$

2.3.3 Naive Bayes

Trong phần trên ta sử dụng các mô hình phân phối chuẩn để ước lượng cho hàm mật độ của các lớp. Phương pháp phân loại Naive Bayes dựa trên giả thiết các biến của dữ liệu là độc lập, như vậy hàm mật độ $f_k(x) = P(X = x | Y = k)$ được phân tích thành tích các hàm mật độ thành phần

$$f_k(x) = f_{k1}(x_1) \cdot f_{k2}(x_2) \cdots f_{kp}(x_p), \quad (2.22)$$

trong đó $f_{kj}(x_j)$ là hàm mật độ của biến dự đoán X_j trong lớp thứ k .

Hàm $f_{kj}(x_j)$ là các hàm mật độ xác suất 1 chiều hay còn gọi là các hàm mật độ biên duyên, có thể ước lượng dựa trên dữ liệu huấn luyện $x_{1j}, x_{2j}, \dots, x_{n_j}$ như sau:

- Nếu X_j là biến định lượng (quantitative), ta có thể giả sử $X_j | Y = k$ có phân phối chuẩn $N(\mu_{jk}, \sigma_{jk}^2)$, hoặc ước lượng $f_{kj}(x_j)$ bằng hàm mật độ nhân (kernel density estimator).
- Nếu biến X_j là biến định tính (qualitative) thì ta ước lượng $f_{kj}(x_j)$ bằng tỷ lệ các quan sát đối với biến X_j trong lớp thứ k . Cụ thể, nếu $X_j \in \{1, 2, \dots, K\}$ và lớp ℓ có n_ℓ quan sát, được chia thành $n_{\ell 1}, \dots, n_{\ell K}$ quan sát cho các giá trị của X_j . Khi đó hàm mật độ xác suất $f_{kj}(x_j)$ có giá trị được ước lượng bởi các tỷ lệ $\frac{n_{\ell 1}}{n_\ell}, \dots, \frac{n_{\ell K}}{n_\ell}$.

Do đó phương pháp phân loại Naive Bayes có thể được áp dụng đối với các bài toán có cả biến định lượng và biến định tính. Một số mô hình Naive Bayes thường được sử dụng: MultinomialNB, BernoulliNB, và GaussianNB.

```
[1]: import pandas as pd
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.discriminant_analysis import
    _LinearDiscriminantAnalysis
from sklearn.discriminant_analysis import
    _QuadraticDiscriminantAnalysis
from sklearn.metrics import classification_report,
    _precision_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import plot_confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
%matplotlib inline
plt.style.use('seaborn-white')
```

```
[2]: df = pd.read_csv('Data/Default.csv')
df['student2'] = df.student.factorize()[0]
df['default2'] = df.default.factorize()[0]
df=df.drop(['default', 'student'],axis=1)
df.head()
```

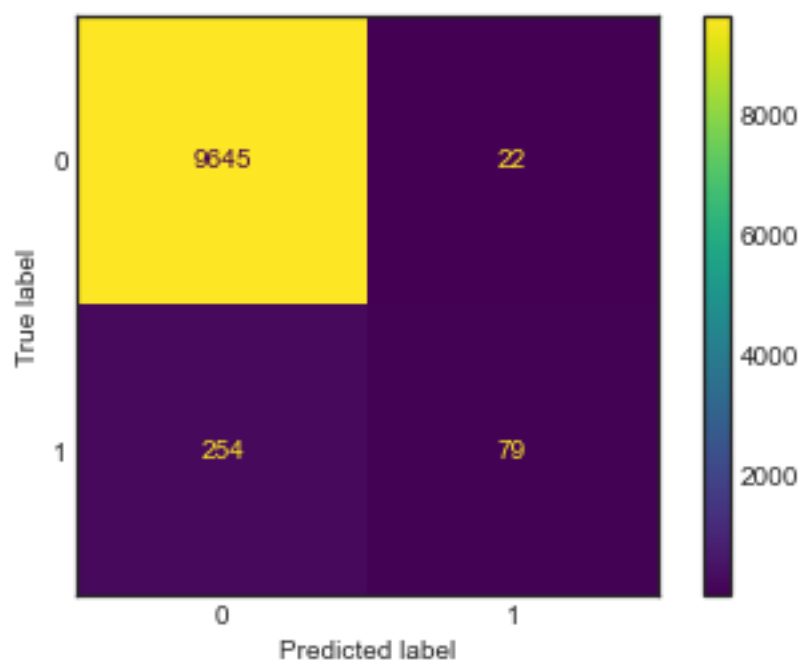
```
[2]:
```

	balance	income	student2	default2
0	729.526495	44361.625074	0	0
1	817.180407	12106.134700	1	0
2	1073.549164	31767.138947	0	0
3	529.250605	35704.493935	0	0
4	785.655883	38463.495879	0	0

```
[3]: X = df[['balance', 'income', 'student2']].values
y = df.default2.values
```

```
[4]: lda = LinearDiscriminantAnalysis(solver='svd')
lda.fit(X, y)
y_pred = lda.predict(X)
```

```
[5]: plt.figure(figsize=(8,8))
ConfusionMatrixDisplay.from_estimator(lda,X,y)
plt.show()
```



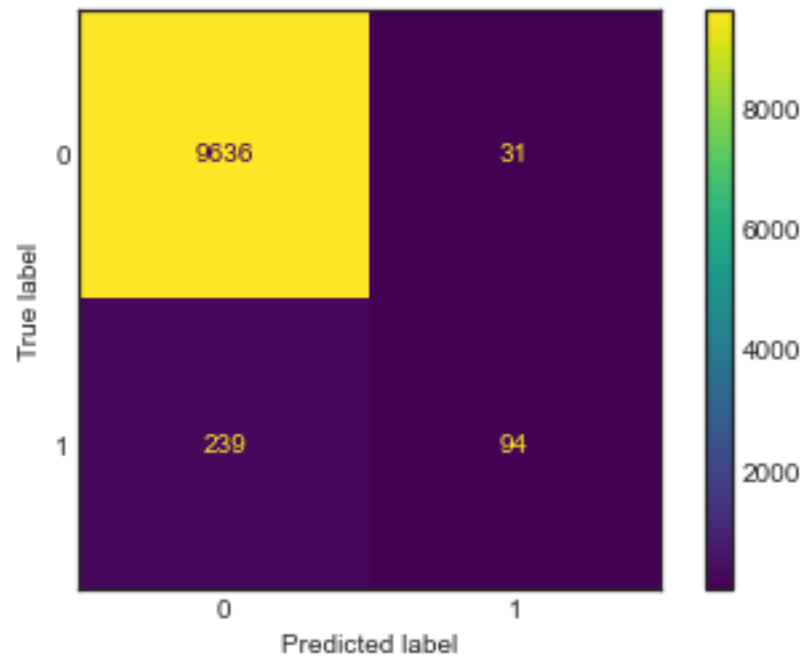
```
[6]: print(classification_report(y, y_pred, target_names=['No',
    -'Yes']))
```

	precision	recall	f1-score	support
No	0.97	1.00	0.99	9667
Yes	0.78	0.24	0.36	333
accuracy			0.97	10000
macro avg	0.88	0.62	0.67	10000
weighted avg	0.97	0.97	0.97	10000

```
[7]: qda = QuadraticDiscriminantAnalysis()
qda.fit(X, y)
pred = qda.predict(X)
```

```
[8]: plt.figure(figsize=(8,8))
ConfusionMatrixDisplay.from_estimator(qda,X,y)
```

```
plt.show()
```



```
[9]: print(classification_report(y, y_pred, target_names=['No',
    _'Yes']))
```

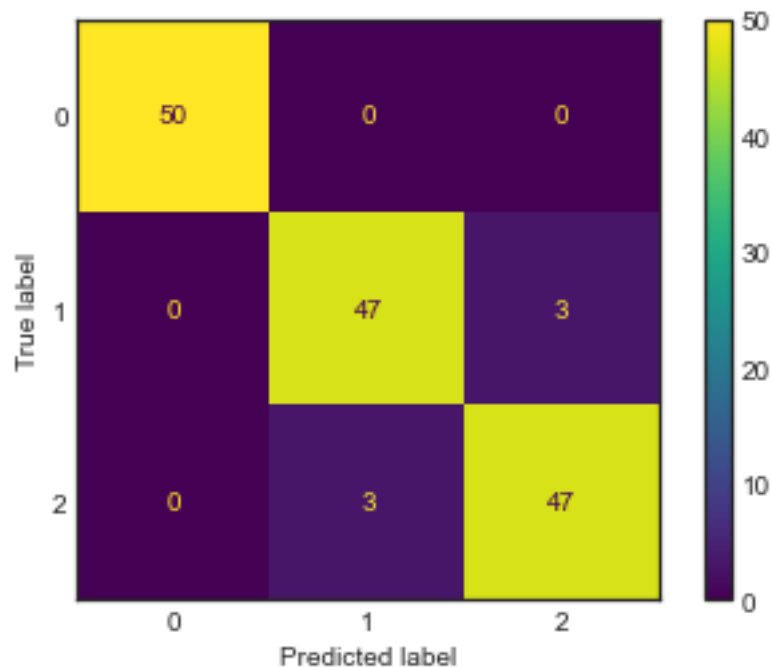
	precision	recall	f1-score	support
No	0.97	1.00	0.99	9667
Yes	0.78	0.24	0.36	333
accuracy			0.97	10000
macro avg	0.88	0.62	0.67	10000
weighted avg	0.97	0.97	0.97	10000

```
[10]: from sklearn.datasets import load_iris
    from sklearn.naive_bayes import GaussianNB
```

```
[11]: X, y = load_iris(return_X_y=True)
```

```
[12]: gnb = GaussianNB()
      gnb.fit(X, y)
      y_pred = gnb.predict(X)
```

```
[13]: plt.figure(figsize=(8,8))
      ConfusionMatrixDisplay.from_estimator(gnb,X,y)
      plt.show()
```



```
[14]: print(classification_report(y, y_pred, target_names=['0',
      _'1', '2']))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	50
1	0.94	0.94	0.94	50
2	0.94	0.94	0.94	50
accuracy			0.96	150
macro avg	0.96	0.96	0.96	150
weighted avg	0.96	0.96	0.96	150

2.4 Bài tập

Bài 2.1. Xét bài toán phân loại Bayes với số biến dự đoán $p = 1$. Chứng minh rằng lớp có mật độ xác suất (2.16) lớn nhất chính là lớp có hàm phân loại LDA (2.17) lớn nhất.

Bài 2.2. Xét mô hình QDA trong trường hợp biến (đặc trưng) $p = 1$. Giả sử có K lớp, và nếu một quan sát thuộc về lớp thứ k thì X sẽ tuân theo phân phối chuẩn một chiều, $X \sim N(\mu_k, \sigma_k^2)$. Chứng minh rằng trong trường hợp này, hàm phân loại Bayes không phải là tuyến tính. Hãy lập luận rằng thực tế nó là một hàm bậc hai.

Bài 2.3. Khảo sát sự khác biệt giữa LDA và QDA.

- Trong trường hợp đường biên phân loại Bayes là tuyến tính, mô hình LDA hay QDA sẽ có hiệu suất tốt hơn trên tập huấn luyện? Trên tập kiểm tra?
- Trong trường hợp đường biên phân loại Bayes là phi tuyến, mô hình LDA hay QDA sẽ có hiệu suất tốt hơn trên tập huấn luyện? Trên tập kiểm tra?
- Khi kích thước mẫu n tăng lên, độ chính xác dự báo trên tập kiểm tra của QDA so với LDA sẽ cải thiện, giảm đi, hay không thay đổi? Giải thích lý do.
- Phát biểu sau đây là Đúng hay Sai: "Ngay cả khi đường biên phân loại Bayes là tuyến tính, việc sử dụng QDA vẫn có thể mang lại tỷ lệ lỗi trên tập kiểm tra thấp hơn so với LDA, vì QDA có đủ độ linh hoạt để mô hình hóa một đường biên tuyến tính." Hãy lập luận cho câu trả lời.

Bài 2.4. Thu thập dữ liệu của một nhóm sinh viên trong một lớp học thống kê với các biến số: X_1 = số giờ học, X_2 = điểm trung bình (GPA), và Y = đạt điểm A môn học (biến phân loại). Người ta sử dụng một mô hình phân loại hồi quy logistic:

$$P(Y = A | x_1, x_2) = \frac{e^{\beta_0 + \beta_1 x_1 + \beta_2 x_2}}{1 + e^{\beta_0 + \beta_1 x_1 + \beta_2 x_2}}$$

và thu được các hệ số ước lượng: $\hat{\beta}_0 = -6$, $\hat{\beta}_1 = 0.05$, $\hat{\beta}_2 = 1$.

- Hãy ước tính xác suất để một sinh viên học trong 40 giờ và có điểm GPA là 3.5, nhận được điểm A trong lớp học này.
- Sinh viên ở câu (a) cần phải học bao nhiêu giờ để có 50% khả năng nhận được điểm A trong lớp học?

2.5 Thực hành

Thực hành 2.1. Chúng ta muốn dự báo liệu một cổ phiếu nhất định sẽ chi trả cổ tức trong năm nay hay không dựa trên biến X là lợi nhuận (đơn vị: %) của năm trước. Theo quan sát một số lượng lớn các công ty, chúng ta thấy rằng:

- Giá trị trung bình của X đối với các công ty có trả cổ tức là $\mu_1 = 10$.
- Giá trị trung bình của X đối với các công ty không trả cổ tức là $\mu_2 = 0$.
- Phương sai của X cho cả hai nhóm công ty này là $\sigma^2 = 36$.
- Cuối cùng, có 80% tổng số các công ty thực hiện trả cổ tức.

Giả thiết rằng X tuân theo phân phối chuẩn, hãy ước lượng xác suất để một công ty sẽ trả cổ tức trong năm nay nếu biết rằng lợi nhuận của nó năm trước là $X = 4$.

Gợi ý: Sử dụng công thức Bayes và hàm mật độ của biến chuẩn.

Thực hành 2.2. Sử dụng bộ dữ liệu về giao dịch tài chính **Weekly** nằm trong gói thư viện **ISLP** để trả lời các câu hỏi sau đây:

- Hãy tóm tắt và vẽ các biểu đồ biểu diễn dữ liệu Weekly. Nhận xét về bất kỳ quy luật nào xuất hiện.
- Thực hiện hồi quy logistic với biến *Direction* (Chiều hướng thị trường) là biến phản hồi và năm biến *Lag* (độ trễ) cùng với biến *Volume* (Khối lượng giao dịch) làm các biến dự báo. Sử dụng hàm `summary()` để in kết quả. Sự xuất hiện của biến dự báo nào có ý nghĩa thống kê?
- Tính toán ma trận nhầm lẫn (confusion matrix) và tỉ lệ dự đoán đúng tổng thể. Hãy giải thích ma trận nhầm lẫn cho biết điều gì về các loại sai lầm mà mô hình hồi quy logistic đã mắc phải.
- Bây giờ, hãy khớp mô hình hồi quy logistic bằng cách sử dụng dữ liệu huấn luyện trong giai đoạn từ năm 1990 đến 2008, với *Lag2* là biến dự báo duy nhất. Tính toán ma trận nhầm lẫn và tỉ lệ dự đoán đúng tổng thể cho dữ liệu test (dữ liệu từ năm 2009 và 2010).
- Lặp lại câu (d) bằng phương pháp LDA, QDA và Naive Bayes.
- Phương pháp nào mang lại kết quả tốt nhất với tập dữ liệu này?

- (h) Thử nghiệm với các tổ hợp biến dự báo khác nhau, bao gồm cả các phép biến đổi biến số và các tương tác (interaction) có thể có, cho mỗi phương pháp trên. Báo cáo các biến số, phương pháp và ma trận nhầm lẫn tương ứng mang lại kết quả tốt nhất trên dữ liệu kiểm tra.

Thực hành 2.3. Sử dụng bộ dữ liệu về nhà ở **Boston** trong thư viện **ISLP**, hãy khớp các mô hình phân loại để dự báo liệu một vùng ngoại ô cụ thể có tỷ lệ tội phạm nằm trên hay dưới mức trung vị (median). Thử nghiệm các mô hình hồi quy logistic, LDA và Naive Bayes bằng cách sử dụng các tập hợp con khác nhau của các biến dự báo. Hãy mô tả những kết quả tìm được.

Gợi ý: tự tạo biến phản hồi bằng cách sử dụng các biến có sẵn trong bộ dữ liệu Boston.

Chương 3

MÔ HÌNH CÂY

Phương pháp học máy bằng mô hình cây là phương pháp phân chia không gian dữ liệu thành các miền nhỏ hơn (thông thường là các hình chữ nhật) và được áp dụng cho cả bài toán hồi quy và bài toán phân loại.

Giả sử không gian dữ liệu huấn luyện gồm n quan sát $x_1, x_2, \dots, x_n$ trong không gian p chiều với các nhãn tương ứng $y_1, y_2, \dots, y_n$. Giả sử miền của không gian dữ liệu là R được phân hoạch thành M miền con $R_1, R_2, \dots, R_M$ không giao nhau và phủ kín miền R . Đối với miền R_m thì biến dự báo là giá trị hằng và ký hiệu và c_m . Khi đó hàm dự báo được xác định bởi công thức

$$f(x) = \sum_{i=1}^M c_m I(x \in R_m), \quad (3.1)$$

trong đó hàm chỉ tiêu $I(x \in R_m)$ bằng 1 nếu $x \in R_m$ và bằng 0 nếu $x \notin R_m$.

Đối với bài toán hồi quy thì giá trị c_m được tính bằng trung bình cộng của tất cả các giá trị của dữ liệu trong miền R_m .

Đối với bài toán phân loại thì giá trị c_m được xác định bằng phương pháp bầu chọn lớn nhất (major vote) nghĩa là quan sát sẽ được phân loại theo nhóm có trọng số cao nhất trong miền đó. Như vậy mỗi miền sẽ được phân loại đúng một nhóm. Vì tập các quy tắc phân tách được sử dụng để phân chia không gian các biến dự báo có thể được tóm tắt bằng một cây, những cách tiếp cận này được gọi là phương pháp cây quyết định (decision tree).

Có nhiều thuật toán xây dựng mô hình cây để tự động phân chia miền dữ liệu dưới dạng phân tầng.

3.1 Mô hình cây hồi quy

Trong phần này ta xét bài toán hồi quy bằng mô hình cây (Regression Tree). Quá trình xây dựng cây hồi quy có hai bước cơ bản:

- Chúng ta chia không gian biến dự báo, nghĩa là tập các giá trị có của $x_1, x_2, \dots, x_n$ thành M vùng tách biệt, không chồng lấn lên nhau $R_1, R_2, \dots, R_M$.
- Với mỗi quan sát rơi vào vùng R_m nào đó, chúng ta đưa ra một giá trị dự báo như nhau, đó là giá trị trung bình các giá trị của biến đáp ứng (y) cho các quan sát huấn luyện trong R_m .

Chẳng hạn, giả sử ở Bước 1 chúng ta có 2 vùng R_1 và R_2 với các giá trị trong mỗi vùng tương ứng là 100 và 200. Một quan sát x , nếu $x \in R_1$ thì giá trị dự báo là 100, nếu $x \in R_2$ thì giá trị dự báo là 200. Như vậy Bước 2 ta có thể tính ngay được giá trị dự báo.

Xây dựng cây Bây giờ chúng ta sẽ phân tích chi tiết Bước 1 ở trên, nghĩa là cách xây dựng các vùng $R_1, R_2, \dots, R_M$. Các vùng R_m có thể có hình dạng bất kỳ. Tuy nhiên chúng ta sẽ chia không gian các biến dự báo thành các hình hộp chữ nhật (hyperbox) để cho đơn giản và dễ diễn giải mô hình dự báo thu được. Mục tiêu là tìm các hộp $R_m, m = 1, 2, \dots, M$ là cực tiểu hoá sai số được cho bởi công thức

$$RSS = \sum_{m=1}^M \sum_{x_i \in R_m} (y_i - c_m)^2. \quad (3.2)$$

Tuy nhiên việc tính toán sẽ không khả thi khi xét mỗi phân hoạch có thể có của không gian đặc trưng ở trong M hộp. Vì lý do này, chúng ta sử dụng phương pháp tiếp cận tham lam (greedy) và từ trên xuống được gọi là tách nhị phân đệ quy (recursive binary splitting). Nó được gọi là từ trên xuống bởi vì nó bắt đầu từ đỉnh cây (chứa mọi dữ liệu huấn luyện) rồi lần lượt tách không gian biến dự báo ra, mỗi lần tách được chỉ ra bằng hai nhánh đi hướng xuống. Phương pháp này gọi là tham lam bởi vì ở mỗi bước của quá trình xây dựng cây, điểm tách tốt nhất được thực hiện tại mỗi bước, thay vì đánh giá tổng thể.

Ta xuất phát từ tập huấn luyện với tất cả các quan sát. Để thực hiện tách nhị phân đệ quy, trước hết ta gọi miền hiện tại cần phân chia là miền R và cần phân chia theo thuộc tính thứ j thành hai miền nhỏ hơn R_1 và R_2 mà dẫn đến khả năng thu giảm sai số RSS là nhiều nhất. Nghĩa là chúng ta cần xem xét tất cả các thuộc tính của dữ liệu $j = 1, 2, \dots, p$

và tất cả các giá trị có thể có của điểm cắt cho mỗi biến dự báo rồi chọn biến dự báo và điểm cắt sao cho cây thu được có sai số RSS thấp nhất.

Chi tiết hơn, ta cần có một điểm cắt s theo thuộc tính thứ j để

$$R_1(j, s) = \{X : X_j \leq s\}, \quad R_2(j, s) = \{X : X_j > s\}. \quad (3.3)$$

Do đó ta cần tìm các giá trị j, s để

$$\min_{j,s} \left[\min_{c_1} \sum_{i:x_i \in R_1} (y_i - c_1)^2 + \min_{c_2} \sum_{i:x_i \in R_2} (y_i - c_2)^2 \right]. \quad (3.4)$$

Các giá trị c_1, c_2 dễ dàng tính được bằng giá trị trung bình

$$c_1 = \frac{1}{|R_1|} \sum_{i:x_i \in R_1} y_i,$$

$$c_2 = \frac{1}{|R_2|} \sum_{i:x_i \in R_2} y_i.$$

Các thuộc tính j được duyệt lần lượt theo các thuộc tính của dữ liệu và giá trị s được duyệt lần lượt theo các giá trị của dữ liệu tương ứng với chiều thứ j và khi đó ta xác định j và s .

Tiếp theo chúng ta lặp lại quá trình này, tìm biến dự báo tốt nhất và điểm cắt tốt nhất để tách tiếp dữ liệu nhằm cực tiểu hoá RSS cho mỗi vùng thu được. Tuy nhiên ở lần này, thay vì tách toàn bộ không gian biến dự báo, chúng ta chỉ tách một vùng trong hai cùng đã xác định trước đó. Giả sử bây giờ chúng ta có ba vùng. Lần này chúng ta lại tìm cách tách một trong ba vùng nhằm cực tiểu hoá sai số RSS . Quá trình này tiếp tục cho đến khi đạt tới được một tiêu chí dừng nào đó; chẳng hạn chúng ta có thể tiếp tục cho đến khi không còn vùng nào có chứa quá 10 quan sát.

Tỉa cây Quá trình được mô tả trong bước xây dựng cây sinh ra các dự báo tốt trên tập huấn luyện nhưng có khả năng quá khớp dữ liệu, dẫn đến hiệu năng kém cho tập kiểm tra. Điều này là do cây thu được có thể quá phức tạp chẳng hạn như độ sâu cây lớn hay số nút là lớn. Một cây nhỏ với ít điểm tách hơn (số vùng ít đi) có thể sai số RSS cao hơn và diễn giải tốt hơn.

Một chiến lược thường được sử dụng là để một cây T_0 phát triển rất lớn, rồi cắt tỉa nó lại để được một cây con. Kỹ thuật cắt tỉa được gọi là cắt tỉa đường nổi yếu nhất. Thay vì xét mỗi cây con có thể có, chúng ta xét một dãy các cây, được xác định bằng một tham số tình chỉnh không âm λ . Với mỗi giá trị của λ có một cây con T của cây T_0 sao cho

$$\sum_{m=1}^{|T|} \sum_{x_i \in R_m} (y_i - c_m)^2 + \lambda |T|$$

nhỏ đến mức có thể. Ở đây $|T|$ là số nút lá của cây T .

Chi tiết hơn, giả sử ta xây dựng mô hình bằng mô hình cây T có số nút lá bằng $M = |T|$. Với mỗi miền R_m ta ký hiệu

$$\begin{aligned} N_m &= \#\{x \in R_m\}, \\ c_m &= \frac{1}{N_m} \sum_{i: x_i \in R_m} y_i, \\ Q_m(T) &= \frac{1}{N_m} \sum_{i: x_i \in R_m} (y_i - c_m)^2. \end{aligned}$$

Ta cần xây dựng cây T để cực tiểu hoá hàm mục tiêu

$$C(T, \lambda) = \sum_{m=1}^{|T|} N_m Q_m(T) + \lambda \cdot |T|, \quad (3.5)$$

trong đó hệ số $\lambda \geq 0$ là hệ số hiệu chỉnh.

Đoạn code này thiết lập môi trường và cung cấp các thư viện cần thiết để làm việc với mô hình cây quyết định.

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pydot
from IPython.display import Image
from sklearn.model_selection import train_test_split,
    _cross_val_score
from six import StringIO
from sklearn.tree import DecisionTreeRegressor,
    _DecisionTreeClassifier, export_graphviz
from sklearn.metrics import mean_squared_error, confusion_matrix,
    _classification_report
%matplotlib inline
plt.style.use('seaborn-white')
```

Hàm này tạo ra hình ảnh của các mô hình cây bằng cách sử dụng Pydot.

```
[2]: # This function creates images of tree models using pydot
def print_tree(estimator, features, class_names=None,
    _filled=True):
    tree = estimator
    names = features
```

```

color = filled
classn = class_names

dot_data = StringIO()
export_graphviz(estimator, out_file=dot_data,
feature_names=features, class_names=classn, filled=filled)
graph = pydot.graph_from_dot_data(dot_data.getvalue())
return graph

```

Dòng lệnh sau đọc dữ liệu từ file Hitters.csv trong thư mục Data, xóa các hàng chứa giá trị thiếu, kết quả lưu vào biến df.

```
[3]: df = pd.read_csv('Data/Hitters.csv').dropna()
```

Đoạn code sau là bước tiền xử lý dữ liệu trước khi huấn luyện. Đọc dữ liệu từ file CSV, đặt cột đầu tiên làm index, xóa các hàng bị thiếu. Tạo biến giả (one-hot encoding) cho các cột phân loại. Xác định biến mục tiêu (Salary) để dự đoán. Chọn hai đặc trưng dạng số (Years, Hits) làm biến đầu vào ban đầu.

```
[4]: df = pd.read_csv('Data/Hitters.csv', index_col=0).dropna()
dummies = pd.get_dummies(df[['League', 'Division',
-'NewLeague']])
y = df.Salary
X=df[['Years', 'Hits']]

```

Tạo mô hình cây, với số nút là bằng 3:

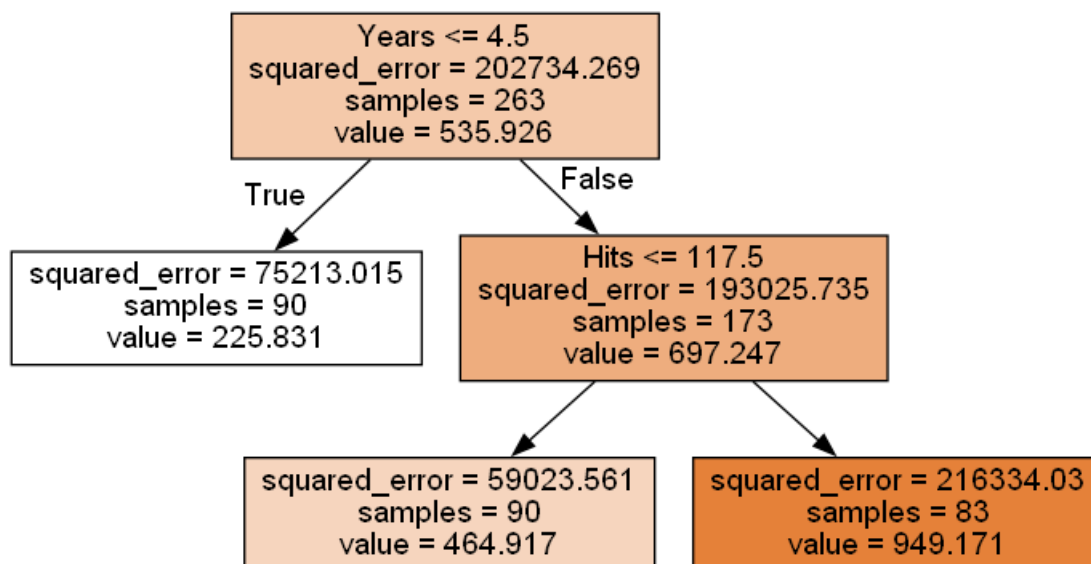
```
[5]: regr = DecisionTreeRegressor(max_leaf_nodes=3)
regr.fit(X, y)
```

```
[5]: DecisionTreeRegressor(max_leaf_nodes=3)
```

Sử dụng hàm print_tree để chuyển mô hình cây hồi quy thành sơ đồ cây:

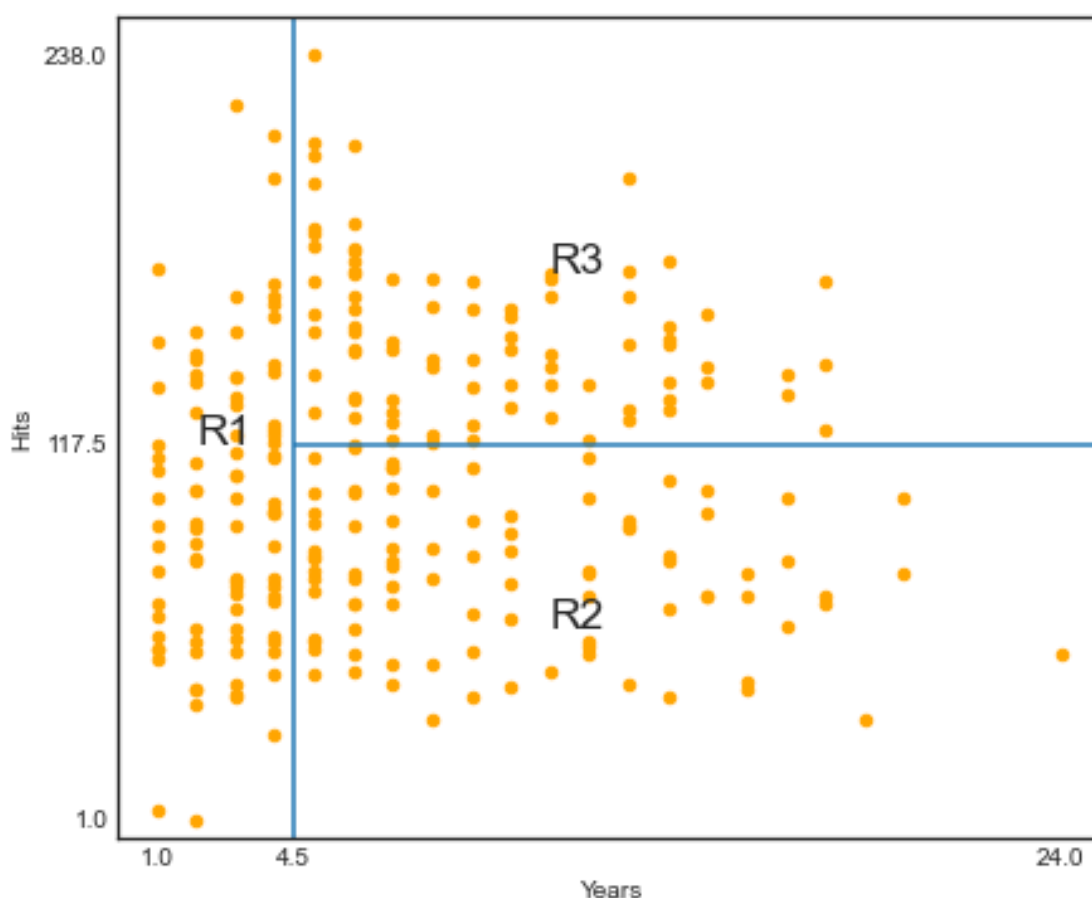
```
[6]: graph, = print_tree(regr, features=['Years', 'Hits'])
Image(graph.create_png())
```

```
[6]:
```

Trong ví dụ này, bộ dữ liệu có 263 phần tử, điều kiện chia nút là $\text{Year} \leq 4.5$. Sai số trung bình phương của toàn bộ dữ liệu bằng 202734.269, giá trị trung bình bằng 697.247. Ta thấy ở nhánh trái, nếu đúng là $\text{Year} \leq 4.5$ thì giá trị trung bình bằng 225.831. Ở nhánh phải, $\text{Year} > 4.5$ thì tiếp tục chia theo Hits. Ở nhánh trái, nếu $\text{Hits} \leq 117.5$ và $\text{Year} > 4.5$ thì mức thu nhập trung bình (value) bằng 464.917. Ở nhánh phải, nếu $\text{Hits} > 117.5$ và $\text{Year} > 4.5$ thì mức thu nhập trung bình (value) cao nhất nhóm bằng 949.171.

```
[7]: df.plot('Years', 'Hits', kind='scatter', color='orange',
           figsize=(7, 6))
plt.xlim(0, 25)
plt.ylim(ymin=-5)
plt.xticks([1, 4.5, 24])
plt.yticks([1, 117.5, 238])
plt.vlines(4.5, ymin=-5, ymax=250)
plt.hlines(117.5, xmin=4.5, xmax=25)
plt.annotate('R1', xy=(2, 117.5), fontsize='xx-large')
plt.annotate('R2', xy=(11, 60), fontsize='xx-large')
plt.annotate('R3', xy=(11, 170), fontsize='xx-large');
```



Hình ảnh minh họa cây hồi quy chia dữ liệu dựa trên 2 biến là trục X (Years) và trục Y (Hits). R1 là vùng những người chơi có năm kinh nghiệm nhỏ hơn hoặc bằng 4.5. R2 là vùng những người chơi có năm kinh nghiệm lớn hơn 4.5 nhưng Hits lại nhỏ hơn hoặc bằng 117.5. R3 là vùng những người chơi có năm kinh nghiệm lớn hơn 4.5 và Hits lớn hơn 117.5. Hình này cho thấy cây hồi quy không dự đoán theo đường thẳng liên tục như hồi quy tuyến tính, mà chia dữ liệu thành các ô R1, R2, R3 rồi gán giá trị trung bình cho từng ô.

```
[8]: regr.score(X, y)
```

```
[8]: 0.43665573627391885
```

Độ chính xác của mô hình cây hồi quy xấp xỉ bằng 0.4366
Để rõ hơn về mô hình cây hồi quy, ta xét ví dụ sau. Code được minh họa qua các lệnh dưới đây.
Đọc file Boston.csv nằm trong thư mục Data.

```
[9]: boston_df = pd.read_csv('Data/Boston.csv')
      #boston_df.info()
```

Đoạn code sau tiền xử lý dữ liệu, trong đó X là biến đầu vào, Y là biến đầu ra. X_train, Y_train là dữ liệu đặc trưng và nhãn cho tập huấn luyện, còn X_test và Y_test là dữ liệu đặc trưng và nhãn cho tập kiểm tra. Chia dữ liệu thành 2 phần theo tỉ lệ 1:1 (tỉ lệ giữa tập huấn luyện và tập kiểm tra) thông qua hàm train_test_split.

```
[10]: X = boston_df.drop('medv', axis=1)
      y = boston_df.medv

      X_train, X_test, y_train, y_test = train_test_split(X, y,
      -test_size=0.5, random_state=0)
```

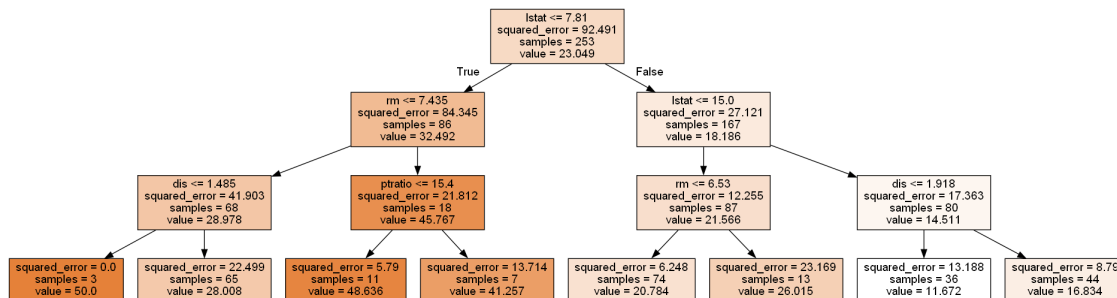
Đoạn code sau khởi tạo mô hình cây hồi quy với độ sâu tối đa bằng 3. Huấn luyện mô hình bằng dữ liệu của tập train và dự đoán kết quả thông qua tập test.

```
[11]: # Pruning not supported. Choosing max depth 3)
      regr2 = DecisionTreeRegressor(max_depth=3)
      regr2.fit(X_train, y_train)
      pred = regr2.predict(X_test)
```

Vẽ và hiển thị cây hồi quy thông qua lệnh sau:

```
[12]: graph, = print_tree(regr2, features=X.columns)
      Image(graph.create_png())
```

[12]:



Qua kết quả của sơ đồ cây ở trên ta thấy, mẫu dữ liệu tại nút gốc bằng 259, điều kiện chia nhánh tại nút gốc là $lstat \leq 7.81$, giá trị trung bình của mẫu dữ liệu tại nút gốc bằng 23.049. Theo kết quả phân nhánh thì ta có thể rút ra kết luận rằng:

Nhóm nhà giá cao khi tỉ lệ dân số nghèo thấp và số lượng phòng trung bình (rm) cao. Nhóm nhà giá trung bình khi tỉ lệ dân số nghèo trung bình (lstat). Nhóm nhà giá thấp

khi tỉ lệ dân số nghèo cao. Hay có thể nói cây hồi quy đã tìm được luật chia nhóm đó là: Nhà nhiều phòng và dân số nghèo ít thì giá cao. Nhà ở khu dân số nghèo nhiều thì giá thấp.

```
[13]: regr2.score(X_train, y_train)
```

```
[13]: 0.8612338526838175
```

Độ đo của hệ số xác định R^2 trên tập huấn luyện xấp xỉ bằng 0.8612.

```
[14]: regr2.score(X_test, y_test)
```

```
[14]: 0.6567545227706459
```

Độ đo của hệ số xác định R^2 trên tập kiểm tra xấp xỉ bằng 0.6568.

3.2 Mô hình cây phân loại

Một cây phân loại (classification tree) rất giống với cây hồi quy, ngoại trừ nó được dùng để dự đoán một biến dự báo định tính thay vì định lượng. Với cây phân loại, chúng ta dự đoán cho một quan sát thuộc về lớp xảy ra phổ biến nhất của các quan sát huấn luyện trong một vùng chứa quan sát. Khi diễn giải các kết quả của cây phân lớp, chúng ta thường quan tâm không chỉ là dự đoán lớp tương ứng với một vùng nút cụ thể mà còn quan tâm đến tỷ lệ lớp trong số các quan sát huấn luyện nằm trong vùng đó.

Nhiệm vụ xây dựng một cây phân lớp hoàn toàn tương tự như nhiệm vụ xây dựng cây hồi quy. Cũng giống như trong bối cảnh cây hồi quy, chúng ta dùng kỹ thuật phân tách nhị phân đệ quy để xây dựng cây phân lớp. Đối với tiêu chí xây dựng hàm mục tiêu tách nhị phân là tỷ suất sai số phân lớp. Tỷ suất sai số phân lớp đơn giản là tỷ lệ các quan sát huấn luyện trong vùng đó nhưng không thuộc về lớp phổ biến nhất:

$$E = 1 - \max_k p_{mk},$$

trong đó p_{mk} biểu diễn cho phần quan sát huấn luyện trong vùng thứ m có thuộc về lớp thứ k .

Chi tiết cách tính tỷ lệ phân loại, với miền R_m có N_m quan sát, ta ký hiệu

$$p_{mk} = \frac{1}{N_m} \sum_{i: x_i \in R_m} I(y_i = k), \quad (3.6)$$

là tỷ lệ của quan sát nhóm k trong miền R_m . Miền R_m được phân loại thành nhóm

$$k(m) = \operatorname{argmax}_k p_{mk}.$$

Ta xác định độ thuần nhất của miền R_m bằng các phương pháp sau:

- Chỉ số Gini (Gini index) là một độ đo về độ thuần nhất của nút, giá trị nhỏ chỉ ra rằng nút chứa chủ yếu các quan sát của cùng một lớp. Công thức tính chỉ số Gini như sau:

$$Q_m(T) = \sum_{k \neq \ell} p_{mk} p_{m\ell} = \sum_{k=1}^K p_{mk}(1 - p_{mk}),$$

- Entropy chéo là một chỉ số thay cho chỉ số Gini, Entropy sẽ nhận giá trị gần 0 nếu tất cả các p_{mk} đều gần 0 hoặc gần 1:

$$Q_m(T) = - \sum_{k=1}^K p_{mk} \log p_{mk}.$$

Khi xây dựng một cây phân lớp, thường người ta dùng chỉ số Gini hoặc entropy để đánh giá chất lượng của một điểm tách cụ thể, bởi vì hai cách tiếp cận này dễ phát hiện tính thuần nhất nút hơn tỷ suất sai số phân lớp.

Ta cần xây dựng cây T để cực tiểu hoá hàm mục tiêu

$$C(T, \lambda) = \sum_{m=1}^{|T|} N_m Q_m(T) + \lambda \cdot |T|. \quad (3.7)$$

Đoạn code sau thiết lập môi trường và import các thư viện cần thiết để làm việc với mô hình cây quyết định.

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import pydot
from IPython.display import Image
from sklearn.model_selection import train_test_split,
    _cross_val_score
from six import StringIO
from sklearn.tree import DecisionTreeRegressor,
    _DecisionTreeClassifier, export_graphviz
from sklearn.metrics import confusion_matrix,
    _classification_report
%matplotlib inline
plt.style.use('seaborn-white')
```

Hàm này tạo ra hình ảnh của các mô hình cây bằng cách sử dụng Pydot.

```
[2]: # This function creates images of tree models using pydot
def print_tree(estimator, features, class_names=None,
               filled=True):
    tree = estimator
    names = features
    color = filled
    classn = class_names

    dot_data = StringIO()
    export_graphviz(estimator, out_file=dot_data,
                    feature_names=features, class_names=classn, filled=filled)
    graph = pydot.graph_from_dot_data(dot_data.getvalue())
    return graph
```

Đoạn code sau sẽ đọc dữ liệu từ file Heart.csv, và xử lý dữ liệu: loại bỏ cột thừa, loại bỏ các hàng bị thiếu giá trị. Hiển thị 5 dòng đầu tiên của tập dữ liệu sau khi đã được xử lý.

```
[3]: df2 = pd.read_csv('Data/Heart.csv').drop('Unnamed: 0', axis=1).
      dropna()
      df2.head()
```

Dưới đây là thông tin về 5 bệnh nhân trong bộ dữ liệu huấn luyện mô hình dự đoán nguy cơ mắc bệnh tim của bệnh nhân với các thông tin như: tuổi, giới tính, kiểu đau ngực, huyết áp, cholesterol, đường huyết lúc đói (Fbs), kết quả điện tâm đồ (RestECG), nhịp tim tối đa khi quá sức (MaxHR), đau ngực khi quá sức (ExAng), mức độ chênh ST khi quá sức so với lúc nghỉ (Oldpeak), độ dốc của đoạn ST khi quá sức (Slope), số lượng mạch vành chính bị cản trở nhìn thấy qua chụp Xquang (Ca), kết quả mô phỏng lưu lượng máu đến tim (Thal), có bệnh tim hay không (AHD).

```
[3]:   Age  Sex  ChestPain  RestBP  Chol  Fbs  RestECG  MaxHR
      ExAng  Oldpeak  \
0   63    1      typical    145    233    1         2    150
      0      2.3
1   67    1  asymptomatic    160    286    0         2    108
      1      1.5
2   67    1  asymptomatic    120    229    0         2    129
      1      2.6
3   37    1   nonanginal    130    250    0         0    187
      0      3.5
```

4	41	0	nontypical	130	204	0	2	172
→	0	1.4						

	Slope	Ca	Thal	AHD
0	3	0.0	fixed	No
1	2	3.0	normal	Yes
2	2	2.0	reversable	Yes
3	3	0.0	normal	No
4	1	0.0	normal	No

Đoạn code sau tiền xử lý dữ liệu, đó là chuyển 2 cột dạng chuỗi phân loại (ChestPain, Thal) thành dạng số.

```
[4]: df2.ChestPain = pd.factorize(df2.ChestPain)[0]
df2.Thal = pd.factorize(df2.Thal)[0]
df2.head()
```

Hiển thị thông tin về 5 bệnh nhân sau khi đã tiền xử lý dữ liệu.

```
[4]:
```

	Age	Sex	ChestPain	RestBP	Chol	Fbs	RestECG	MaxHR	ExAng
→	Oldpeak	\							
0	63	1	0	145	233	1	2	150	0
→	2.3								
1	67	1	1	160	286	0	2	108	1
→	1.5								
2	67	1	1	120	229	0	2	129	1
→	2.6								
3	37	1	2	130	250	0	0	187	0
→	3.5								
4	41	0	3	130	204	0	2	172	0
→	1.4								

	Slope	Ca	Thal	AHD
0	3	0.0	0	No
1	2	3.0	1	Yes
2	2	2.0	2	Yes
3	3	0.0	1	No
4	1	0.0	1	No

Dữ liệu đầu vào cho mô hình học máy hay tập các đặc trưng kí hiệu là X2, biến mục tiêu hay nhãn của mô hình kí hiệu là Y2. Trong đó dữ liệu đầu vào được xử lý từ dữ liệu gốc bằng cách bỏ đi cột AHD, Y2 là biến nhận giá trị 0 hoặc 1 sau khi chuyển đổi các giá

trị trong cột AHD.

```
[5]: X2 = df2.drop('AHD', axis=1)
      y2 = pd.factorize(df2.AHD)[0]
```

Khởi tạo cây quyết định, không giới hạn độ sâu, giới hạn số nút lá tối đa trong cây bằng 6 và khi tách tại mỗi nút thuật toán chỉ được phép chọn tối đa 3 đặc trưng, ta sử dụng dòng lệnh sau:

```
[6]: clf = DecisionTreeClassifier(max_depth=None, max_leaf_nodes=6,
      -max_features=3)
      clf.fit(X2, y2)
```

Dòng lệnh sau tạo một cây quyết định, trong đó mỗi lần tách nhánh chỉ xét tối đa 3 đặc trưng và toàn bộ cây chỉ có nhiều nhất 6 nút lá.

```
[6]: DecisionTreeClassifier(max_features=3, max_leaf_nodes=6)
```

Trả về độ chính xác của mô hình (accuracy), ta dùng dòng lệnh sau:

```
[7]: clf.score(X2, y2)
```

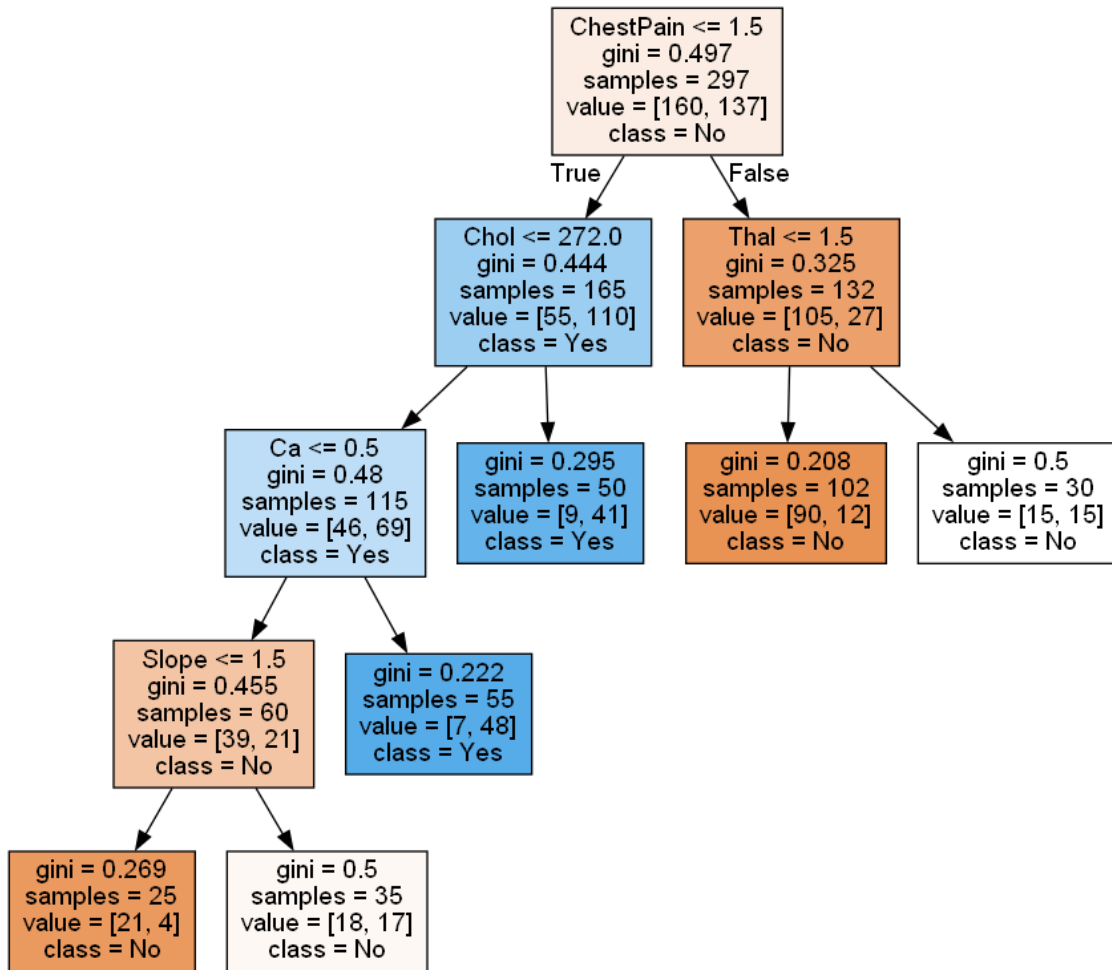
Kết quả độ chính xác thu được:

```
[7]: 0.7845117845117845
```

Đoạn code sau dùng để trực quan hóa cây quyết định clf. Cây sẽ được vẽ với nhãn đặc trưng và nhãn lớp ("No", "Yes"), sau đó hiển thị trực tiếp dưới dạng ảnh PNG trong môi trường notebook.

```
[8]: graph2, = print_tree(clf, features=X2.columns,
      -class_names=['No', 'Yes'])
      Image(graph2.create_png())
```

```
[8]:
```

Theo kết quả trên ta có, kích thước mẫu dữ liệu bằng 297, 160 mẫu thuộc lớp No và 137 mẫu thuộc lớp Yes. Nhãn dự đoán của nút này là No. Mức độ hỗn loạn tại nút này bằng 0.497, có thể thấy sự phân bố dữ liệu giữa 2 lớp là khá đều nhau. Tại nhánh trái, khi $\text{ChestPain} \leq 1.5$ có 165 mẫu dữ liệu, cây chia nhánh theo điều kiện $\text{Chol} \leq 272.0$ và tiếp tục chia theo điều kiện $\text{Ca} \leq 0.5$. Như vậy có thể thấy cây quyết định trên phân loại bệnh tim dựa trên các biến: ChestPain, Chol, Ca, Slope, Thal.

Ta xét ví dụ ứng dụng tiếp theo cho thuật toán Cây quyết định, bài toán bán ghế ô tô cho trẻ em ở các cửa hàng.

```
[9]: df3 = pd.read_csv('Data/Carseats.csv').drop('Unnamed: 0',
        ↳axis=1)
```

```
df3.head()
```

```
[9]:   Sales  CompPrice  Income  Advertising  Population  Price
     └ShelveLoc  Age  \
0    9.50         138      73           11          276     120
     └ Bad    42
1   11.22         111      48           16          260      83
     └Good    65
2   10.06         113      35           10          269      80
     └Medium   59
3    7.40         117     100            4          466      97
     └Medium   55
4    4.15         141      64            3          340     128
     └ Bad    38

      Education  Urban   US
0             17   Yes  Yes
1             10   Yes  Yes
2             12   Yes  Yes
3             14   Yes  Yes
4             13   Yes   No
```

Quá trình tiền xử lý dữ liệu: tạo biến mục tiêu nhị phân High từ biến Sales, mã hóa biến phân loại ShelveLoc, chuyển đổi các biến nhị phân "Yes"/"No" của các nhân (Urban, US) thành số 0/1.

```
[10]: df3['High'] = df3.Sales.map(lambda x: 1 if x>8 else 0)
      df3.ShelveLoc = pd.factorize(df3.ShelveLoc)[0]

      df3.Urban = df3.Urban.map({'No':0, 'Yes':1})
      df3.US = df3.US.map({'No':0, 'Yes':1})
```

```
[11]: df3.head()
```

Hiển thị thông tin của 5 cửa hàng sau khi đã tiền xử lý dữ liệu:

```
[11]:   Sales  CompPrice  Income  Advertising  Population  Price
     └ShelveLoc  Age  \
0    9.50         138      73           11          276     120
     └    0    42
1   11.22         111      48           16          260      83
     └    1    65
```

2	10.06		113	35		10	269	80
	2	59						
3	7.40		117	100		4	466	97
	2	55						
4	4.15		141	64		3	340	128
	0	38						

	Education	Urban	US	High
0	17	1	1	1
1	10	1	1	1
2	12	1	1	1
3	14	1	1	0
4	13	1	0	0

Đoạn code sau gán x là các đặc trưng đầu vào, y là nhãn đầu ra, chia mẫu dữ liệu giữa tập huấn luyện và tập kiểm tra theo tỉ lệ 1:1. Mục đích là huấn luyện mô hình trên X_{train} , Y_{train} , sau đó đánh giá mô hình trên X_{test} , Y_{test} .

```
[12]: X = df3.drop(['Sales', 'High'], axis=1)
      y = df3.High

      X_train, X_test, y_train, y_test = train_test_split(X, y,
      -test_size=0.5, random_state=0)
```

Để tạo một cây quyết định, giới hạn độ sâu của cây bằng 4, sau đó huấn luyện cây trên toàn bộ dữ liệu (x, y) ta dùng dòng lệnh sau:

```
[13]: clf = DecisionTreeClassifier(max_depth=4)
      clf.fit(X, y)
```

```
[13]: DecisionTreeClassifier(max_depth=4)
```

Kết quả bảng ma trận độ đo đánh giá hiệu quả của mô hình được in ra bởi dòng lệnh sau:

```
[14]: print(classification_report(y, clf.predict(X)))
```

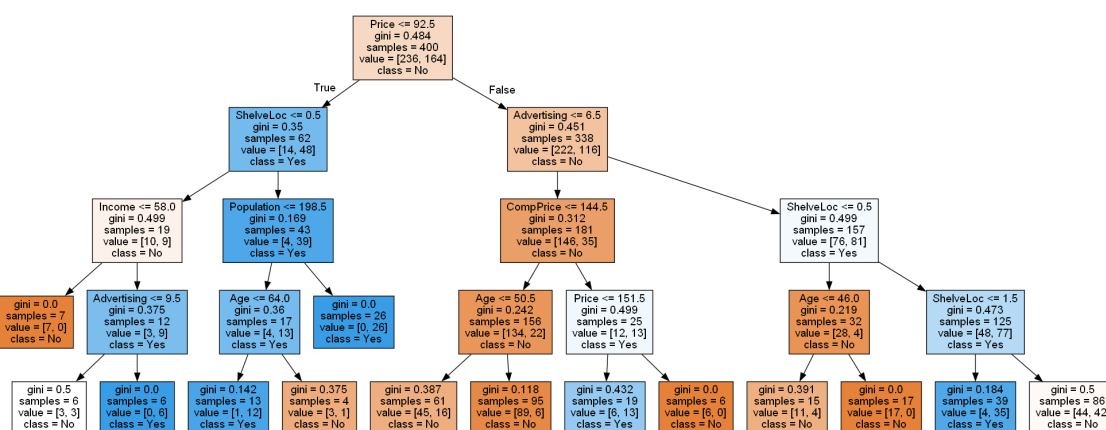
	precision	recall	f1-score	support
0	0.76	0.95	0.84	236
1	0.89	0.56	0.69	164

accuracy			0.79	400
macro avg	0.83	0.76	0.77	400
weighted avg	0.81	0.79	0.78	400

Đoạn code sau dùng để trực quan hóa cây quyết định clf với các nút, điều kiện phân nhánh và các nhãn ở lá. Cây sẽ được vẽ với nhãn đặc trưng và nhãn lớp (“No”, “Yes”), sau đó hiển thị trực tiếp dưới dạng ảnh PNG trong môi trường notebook.

```
[15]: graph3, = print_tree(clf, features=X.columns, class_names=['No',
    -'Yes' ])
    Image(graph3.create_png())
```

[15]:



Theo kết quả trên ta có, kích thước mẫu dữ liệu bằng 400, trong đó có 236 mẫu thuộc lớp No và 164 mẫu thuộc lớp Yes. Nhãn dự đoán của nút này là No. Mức độ hỗn loạn tại nút này bằng 0.484, có thể thấy sự phân bố dữ liệu giữa 2 lớp là khá đều nhau.

Đoạn code sau tạo và hiển thị ma trận nhầm lẫn (confusion matrix) dưới dạng DataFrame với nhãn rõ ràng, giúp đánh giá độ chính xác dự đoán của mô hình phân loại.

```
[16]: y_pred=clf.predict(X)
    cm = pd.DataFrame(confusion_matrix(y, y_pred).T, index=['No',
    -'Yes'], columns=['No', 'Yes'])
    cm.index.name = 'Predicted'
    cm.columns.name = 'True'
    cm
```

Kết quả thu được:

[16]:

True	No	Yes
Predicted		
No	225	72
Yes	11	92

3.3 Bagging, Random Forest, Boosting

3.3.1 Phương pháp Bagging

Phương pháp Bagging (viết tắt của bootstrap Aggregation) là một kỹ thuật thuộc nhóm học kết hợp (ensemble learning) nhằm giảm phương sai của một phương pháp dự báo thống kê.

Bagging hoạt động dựa trên ý tưởng rằng việc trung bình cộng các kết quả dự báo từ nhiều mô hình được huấn luyện trên các tập dữ liệu khác nhau sẽ giúp giảm phương sai. Tuy nhiên, trong thực tế chúng ta thường không có sẵn nhiều tập dữ liệu huấn luyện độc lập từ cùng một quần thể, vì vậy Bagging sử dụng kỹ thuật Bootstrap để tạo ra các tập dữ liệu giả lập từ một tập huấn luyện duy nhất.

Quy trình xây dựng mô hình Bagging (đặc biệt phổ biến với cây quyết định) bao gồm:

- **Lấy mẫu Bootstrap:** Từ tập dữ liệu huấn luyện gốc có quy mô N , thực hiện lấy mẫu có hoàn lại (with replacement) B lần để tạo ra B tập dữ liệu bootstrap khác nhau. Trung bình mỗi tập bootstrap chứa khoảng $2/3$ số quan sát độc nhất của tập gốc.
- **Huấn luyện song song:** Xây dựng một mô hình dự báo (như cây hồi quy hoặc cây phân loại) trên mỗi tập dữ liệu bootstrap này. Các mô hình này thường được để phát triển sâu và không cắt tỉa để đảm bảo độ chệch (bias) thấp nhưng phương sai (variance) cao.
- **Tổng hợp kết quả (Aggregation):**
 - Đối với bài toán hồi quy: Dự báo cuối cùng là trung bình cộng kết quả từ tất cả B mô hình:

$$\hat{f}_{bag}(x) = \frac{1}{B} \sum_{i=1}^B \hat{f}_b(x).$$

- Đối với bài toán phân loại: Dự báo cuối cùng được thực hiện bằng cách lấy phiếu bầu đa số (majority vote) từ B kết quả phân loại của các mô hình con.

3.3.2 Phương pháp Random Forest

Phương pháp Random Forest (Rừng ngẫu nhiên) là một kỹ thuật học kết hợp (ensemble learning) mạnh mẽ, được xây dựng bằng cách kết hợp một tập hợp lớn các cây quyết định để tạo ra một mô hình dự báo duy nhất. Đây được coi là một sự sửa đổi đáng kể của phương pháp Bagging nhằm giảm phương sai của mô hình dự báo.

Ý tưởng chính của Random Forest là xây dựng một bộ sưu tập các cây quyết định có tương quan thấp và sau đó lấy trung bình kết quả của chúng. Việc trung bình cộng các biến có tương quan cao không giúp giảm phương sai nhiều. Random Forest giải quyết vấn đề này bằng cách xáo trộn các biến dự báo tại mỗi bước chia nhánh.

Quy trình xây dựng một Random Forest bao gồm các bước sau:

- **Lấy mẫu Bootstrap:** Với mỗi cây b từ 1 đến B , rút một mẫu bootstrap từ dữ liệu huấn luyện.
- **Xây dựng cây với tập con biến ngẫu nhiên:** Tại mỗi nút của cây, thay vì xét toàn bộ p biến dự báo, thuật toán chỉ chọn ngẫu nhiên một tập con gồm m biến với $(m \leq p)$ làm ứng viên để thực hiện chia nhánh.
- **Không cắt tỉa:** Các cây thường được để phát triển đến mức tối đa (không cắt tỉa) để đảm bảo độ chệch thấp.
- **Tổng hợp:** - Hồi quy: Dự báo bằng cách lấy trung bình cộng kết quả từ tất cả các cây. - Phân loại: Dự báo bằng cách biểu quyết đa số (majority vote) từ các lớp được dự đoán bởi mỗi cây.

3.3.3 Phương pháp Boosting

Khác với các phương pháp như Bagging hay Random Forest xây dựng các mô hình độc lập, Boosting xây dựng các mô hình con một cách tuần tự. Ý tưởng cốt lõi là kết hợp đầu ra của nhiều bộ học "yếu" (weak learners) — thường là các cây quyết định nhỏ — để tạo thành một "ủy ban" mạnh mẽ có khả năng dự báo vượt trội.

AdaBoost (Adaptive Boosting): Đây là thuật toán Boosting phổ biến nhất cho bài toán phân loại nhị phân. Nó hoạt động bằng cách áp dụng trọng số lên các quan sát huấn luyện. Tại mỗi bước, những quan sát bị phân loại sai bởi mô hình trước đó sẽ được tăng trọng số, buộc mô hình tiếp theo phải tập trung vào việc xử lý các trường hợp khó này. Kết quả cuối cùng là tổng có trọng số của các bộ phân loại con.

Gradient Boosting: Phương pháp này mở rộng Boosting cho các bài toán hồi quy và các hàm mất mát khác nhau,. Thay vì thay đổi trọng số dữ liệu, nó khớp mỗi mô hình mới vào phần dư (residuals) hoặc gradient của hàm mất mát từ mô hình hiện tại,. Quá trình này giúp mô hình "học chậm" (learn slowly), từ đó đạt được hiệu suất tốt hơn bằng cách tích lũy dần các cải tiến nhỏ.

Về mặt lý thuyết, Boosting có thể được xem là một phương pháp khớp mô hình cộng tính (additive model) bằng cách sử dụng kỹ thuật mô hình hóa cộng tính từng giai đoạn (forward stagewise additive modeling). Trong đó, AdaBoost thực chất đang tối ưu hóa hàm mất mát mũ (exponential loss), trong khi Gradient Boosting thường tối ưu hóa độ lệch (deviance) của phân phối nhị thức hoặc đa thức.

3.4 Bài tập

Bài 3.1. Xét 1 nút của cây phân loại chứa 20 mẫu dữ liệu thuộc 3 lớp, trong đó lớp A chứa 8 mẫu, lớp B chứa 7 mẫu, lớp C chứa 5 mẫu.

a) Tính gini index và entropy của mỗi nút.

b) Giả sử chia nút này thành 2 nút con dựa trên thuộc tính, nút trái $A = 5, B = 2, C = 1$, nút phải $A = 3, B = 5, C = 14$. Tính gini index của từng nút con, gini sau khi chia nút, và gini gain.

Bài 3.2. Một cửa hàng muốn quyết định có mở cửa hàng vào ban đêm hay không dựa trên các thuộc tính: Thời tiết, Ngày, Khách, Nhân. Dữ liệu thu được của 8 ngày như sau:

STT	Thời tiết	Ngày	Khách	Nhân
1	Nắng	Cuối tuần	Đông	Có
2	Nắng	Cuối tuần	Ít	Có
3	Nắng	Thường	Đông	Không
4	Nắng	Thường	Ít	Không
5	Mưa	Cuối tuần	Đông	Có
6	Mưa	Cuối tuần	Ít	Không
7	Mưa	Thường	Đông	Không
8	Mưa	Thường	Ít	Không

a) Tính Entropy của dữ liệu.

b) Tính Information Gain cho từng thuộc tính.

c) Xây dựng cây quyết định theo ID3.

Bài 3.3. Cho tập dữ liệu huấn luyện gồm 8 mẫu, 2 thuộc tính liên tục và biến mục tiêu liên tục như sau:

STT	Diện tích $x_1 (m^2)$	Số phòng x_2	Giá nhà y - tỷ đồng
1	40	1	1.2
2	50	2	1.5
3	60	2	1.8
4	70	3	2.5
5	80	3	2.8
6	90	4	3.2
7	100	4	3.6
8	110	5	4.0

a) Chọn thuộc tính x_1 (Diện tích) để thực hiện lần phân chia đầu tiên tại ngưỡng $x_1 \leq 75$. Hãy tính giá trị dự báo ($\hat{y}$) cho hai nút lá mới được tạo ra.

b) Hãy tính tổng bình phương sai số cho điểm chia này.

$$RSS = \sum_{i \in \text{Trái}} (y_i - \hat{y}_l)^2 + \sum_{i \in \text{Phải}} (y_i - \hat{y}_r)^2.$$

c) Giữa việc chia theo x_1 (Diện tích) và x_2 (Số phòng), thuộc tính nào có khả năng làm giảm RSS nhiều hơn ở bước đầu tiên? Tại sao?

3.5 Thực hành

Thực hành 3.1. Thu thập bộ dữ liệu mẫu $n = 50$ về diện tích $X(m^2)$ và giá nhà Y (tỷ đồng).

a) Tính giá trị trung bình Y và SSE khi không chia nhánh.

b) Chọn các điểm chia tiềm năng và vẽ cây hồi quy 3 tầng.

c) Huấn luyện cây hồi quy bằng Python (sklearn) và dự đoán giá nhà cho diện tích $50m^2$ và $100m^2$.

Thực hành 3.2. Hãy viết các hàm sau bằng ngôn ngữ Python, trong đó mỗi đầu vào y là một véc-tơ:

a) `sq_loss`: với đầu vào là véc-tơ y , hàm trả về tổng bình phương các sai khác giữa mỗi phần tử và giá trị trung bình cộng.

b) `mis_match`: với đầu vào là véc-tơ y , hàm trả về số lượng phần tử không trùng khớp giữa mỗi phần tử và giá trị mode (giá trị xuất hiện nhiều nhất).

Thực hành 3.3. Thu thập bộ dữ liệu mẫu $n = 30$ về chi phí quảng cáo X (triệu đồng/ngày) và doanh thu Y (triệu đồng/ngày).

- a) Nếu không chia nhánh hãy tính tổng giá trị trung bình Y và SSE.
- b) Chọn các điểm chia tiềm năng và vẽ cây hồi quy 3 tầng.
- c) Huấn luyện cây hồi quy bằng Python (sklearn) và dự đoán doanh thu cho chi phí quảng cáo 5(triệu) và 20(triệu).

Thực hành 3.4. Tải bộ dữ liệu Iris từ sklearn.datasets.

- a) In ra số lượng mẫu, số lượng thuộc tính.
- b) In ra 5 dòng dữ liệu đầu tiên
- c) Cho biết số mẫu của mỗi lớp.
- d) Chia dữ liệu giữa tập huấn luyện và tập kiểm tra theo tỉ lệ 7:3. Huấn luyện dữ liệu với mô hình Decision tree, dự đoán trên tập kiểm tra và tính các độ đo đánh giá hiệu quả của mô hình như là: Accuracy, Precision, Recall, F1- score.

Chương 4

MÁY VÉC-TƠ HỖ TRỢ

Chương này tập trung vào máy véc-tơ hỗ trợ (SVM-Support Vector Machine), một phương pháp phân loại mạnh mẽ được phát triển trong cộng đồng khoa học máy tính vào những năm 1990 và ngày càng trở nên phổ biến nhờ hiệu suất tốt trong nhiều bối cảnh khác nhau.

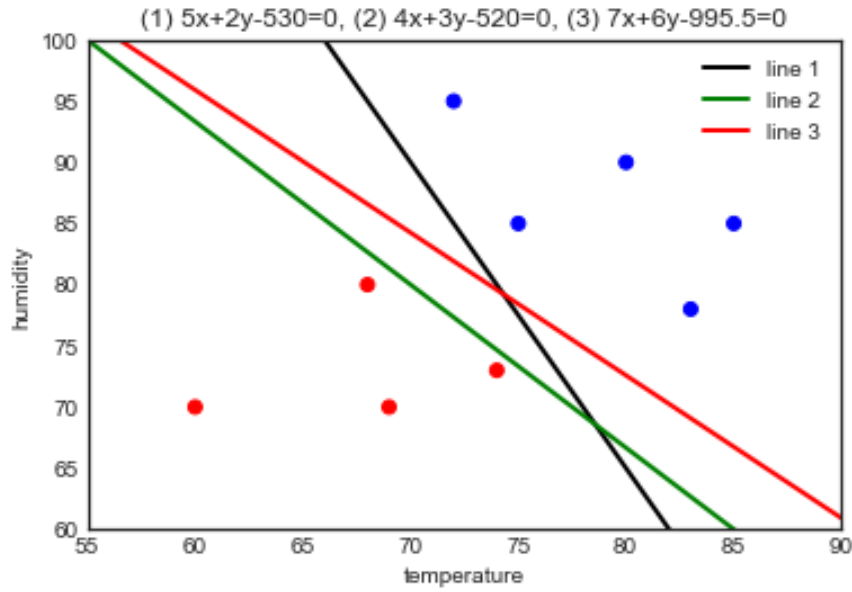
4.1 Siêu phẳng tách và lề

4.1.1 Giới thiệu về máy véc-tơ hỗ trợ

Ta xét bài toán phân loại dựa vào điều kiện thời tiết để đưa ra quyết định có tham gia thể thao hay không. Dữ liệu quá khứ được cho bằng bảng dưới đây:

	temperature	humidity	play
0	85	85	no
1	60	70	yes
2	80	90	no
3	72	95	no
4	68	80	yes
5	74	73	yes
6	69	70	yes
7	75	85	no
8	83	78	no

Dữ liệu trên gồm 2 nhóm, để thuận tiện ta ký hiệu nhóm 'no' tương ứng với -1 và nhóm 'yes' tương ứng với 1 .



Ta vẽ đồ thị của hai yếu tố trên đồ thị và ta tìm mặt phẳng tách hai nhóm dữ liệu trên. Có nhiều đường thẳng tách hai nhóm dữ liệu trên. Ta gọi lề của mặt phẳng tách là khoảng cách nhỏ nhất từ các điểm đến đường thẳng tách hai nhóm. Đường thẳng tách có lề lớn nhất là đường thẳng tối ưu chia 2 nhóm dữ liệu trên. Các điểm có khoảng cách tới đường thẳng tối ưu bằng với lề lớn nhất được gọi là các véc-tơ hỗ trợ.

Siêu phẳng Một siêu phẳng trong không gian $\mathbb{R}^p$ là một không gian con affine có số chiều $p - 1$ và phương trình có dạng

$$\alpha_0 + \alpha_1 x_1 + \alpha_2 x_2 + \cdots + \alpha_p x_p = 0, \quad (4.1)$$

trong đó α_i là các số thực. Siêu phẳng chia không gian thành hai miền ước với giá trị của biểu thức > 0 hoặc < 0 .

Trong không gian hai chiều ($p = 2$) thì siêu phẳng là một đường thẳng, trong không gian ba chiều ($p = 3$) thì siêu phẳng là một mặt phẳng.

Khoảng cách Khoảng cách từ một điểm $x' = (x'_1, x'_2, \dots, x'_p)$ đến siêu phẳng được tính bởi công thức

$$d = \frac{|\alpha_0 + \alpha_1 x'_1 + \cdots + \alpha_p x'_p|}{\sqrt{\alpha_1^2 + \cdots + \alpha_p^2}}. \quad (4.2)$$

Ta ký hiệu $\alpha = (\alpha_1, \dots, \alpha_p)$, $x' = (x'_1, \dots, x'_p)$ thì khoảng cách được viết lại thành

$$d = \frac{|\alpha_0 + \alpha \cdot x'|}{\|\alpha\|}.$$

Siêu phẳng tách Giả sử ta có n điểm quan sát $x_1, x_2, \dots, x_n$ trong không gian p -chiều và được chia thành hai loại tương ứng với các nhãn $y_1, y_2, \dots, y_n \in \{-1, 1\}$. Một siêu phẳng

$$\alpha_0 + \alpha_1 x_1 + \alpha_2 x_2 + \dots + \alpha_p x_p = 0$$

được gọi là **siêu phẳng tách** nếu

1. Với mỗi quan sát x_i tương ứng với nhãn $y_i = -1$ thoả mãn:

$$\alpha_0 + \alpha \cdot x_i < 0.$$

2. Với mỗi quan sát x_i tương ứng với nhãn $y_i = 1$ thoả mãn:

$$\alpha_0 + \alpha \cdot x_i > 0.$$

Lề (margin) Là khoảng cách ngắn nhất giữa đường ranh giới quyết định và các điểm dữ liệu gần nhất của các lớp. Lề bằng 2 lần khoảng cách ngắn nhất từ các điểm đến siêu phẳng tách.

Siêu phẳng tách tối ưu Siêu phẳng tách đối với dữ liệu có lề lớn nhất được gọi là siêu phẳng tách tối ưu.

Véc-tơ hỗ trợ Những điểm dữ liệu nằm ngay sát mép của đường ranh giới được gọi là véc-tơ hỗ trợ (support vectors). Ta cũng có thể xem các điểm có khoảng cách đến siêu phẳng tách bằng $\frac{1}{2}$ giá trị là các véc-tơ hỗ trợ.

Máy véc-tơ hỗ trợ Siêu phẳng tách tối ưu đối với dữ liệu được gọi là máy véc-tơ hỗ trợ.

Ví dụ 4.1. Xét dữ liệu có 2 điểm $A(x_1 = (2, 1), y_1 = 1), B(x_2 = (4, 3), y_2 = -1)$. Tìm đường thẳng tách tối ưu và lề đối với hai điểm trên.

Giải:

Giả sử phương trình của siêu phẳng tách có dạng:

$$w_1 x_1 + w_2 x_2 - b = 0.$$

Đường thẳng tách tối ưu đối với 2 điểm chính là phương trình đường trung trực của hai điểm đã cho. Phương trình đường trung trực của hai điểm $A(2, 1), B(4, 3)$ là phương trình

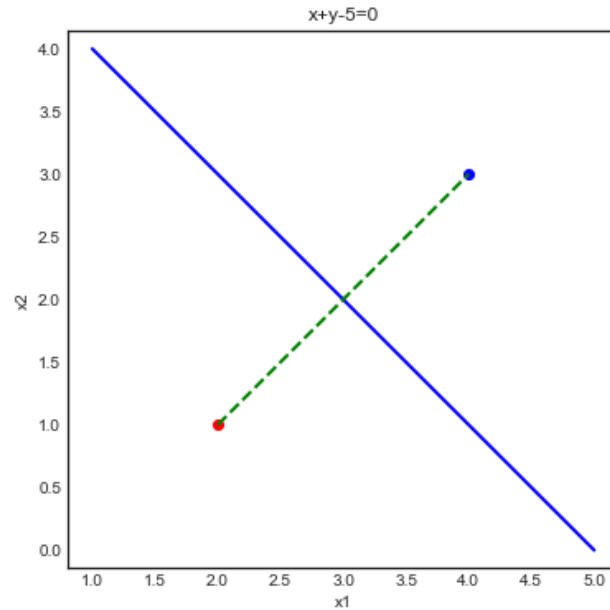
$$(4 - 2)\left(x_1 - \frac{2 + 4}{2}\right) + (3 - 1)\left(x_2 - \frac{1 + 3}{2}\right) = 0 \text{ hay } x_1 + x_2 - 5 = 0.$$

Vậy ta có $w_1 = 1, w_2 = 1, b = 5$ và phương trình siêu phẳng tối ưu là

$$x_1 + x_2 - 5 = 0.$$

Giá trị lề (margin) là khoảng cách giữa hai điểm A và B ,

$$\text{margin} = \sqrt{(4-2)^2 + (3-1)^2} = 2\sqrt{2}.$$



Ví dụ 4.2. Xét dữ liệu có 3 điểm $A(x_1 = (2, 2), y_1 = 1), B(x_2 = (4, 5), y_2 = -1), C(x_3 = (7, 4), y_3 = -1)$. Tìm mặt đường thẳng tách tối ưu và lề đối với ba điểm trên.

Giải:

Đối với 3 điểm A lớp $y = 1$ và B, C thuộc lớp $y = -1$ nên giá trị lề là khoảng cách nhỏ nhất từ điểm A đến 2 điểm B, C . Do đó giá trị lề là

$$\text{margin} = \min\{AB, AC\} = \min\{\sqrt{13}, \sqrt{29}\} = \sqrt{13}.$$

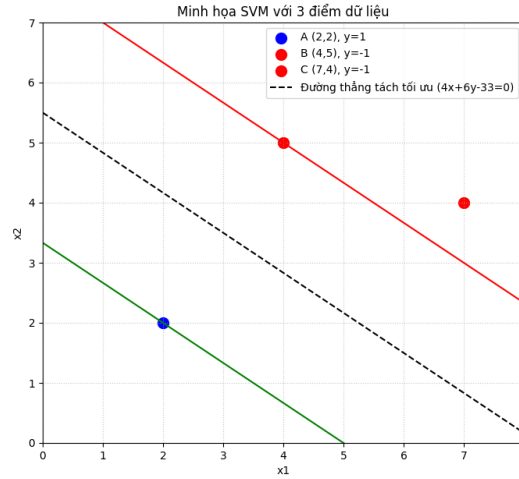
Đường thẳng tách tối ưu là đường thẳng trung trực của đoạn AB có phương trình là

$$(4-2)\left(x_1 - \frac{2+4}{2}\right) + (5-2)\left(x_2 - \frac{2+5}{2}\right) = 0 \text{ hay } 2x_1 + 3x_2 - \frac{33}{2} = 0.$$

Đường thẳng tách tối ưu có phương trình:

$$4x_1 + 6x_2 - 33 = 0.$$

Ta thấy các điểm A, B là các véc-tơ hỗ trợ, điểm C không là véc-tơ hỗ trợ.



4.2 Tối ưu lồi lớn nhất

Giả sử rằng dữ liệu $(x_i, y_i), i = 1, 2, \dots, n$ có siêu phẳng tách. Giả sử phương trình của siêu phẳng tách có dạng

$$wx - b = 0. \quad (4.3)$$

Do có siêu phẳng tách, ta có thể chọn 2 siêu phẳng song song tách hai nhóm dữ liệu với nhau mà khoảng cách giữa chúng là lớn nhất có thể. Khi đó lề của dữ liệu đối với mặt phẳng tách bằng một khoảng cách giữa hai mặt phẳng song song ở trên. Ta có thể mô tả hai phương trình dưới dạng

$$wx - b = +1$$

$$wx - b = -1$$

Đối với các điểm x_i "phía trên" có nhãn $y_i = 1$ thỏa mãn bất phương trình

$$wx_i - b \geq 1, \text{ nếu } y_i = 1.$$

Đối với các điểm x_i "phía dưới" có nhãn $y_i = -1$ thỏa mãn bất phương trình

$$wx_i - b \leq -1, \text{ nếu } y_i = -1.$$

Hai điều kiện trên được viết gộp thành

$$(wx_i - b)y_i \geq 1, i = 1, 2, \dots, n.$$

Khoảng cách giữa hai siêu phẳng trên là $\frac{2}{\|w\|}$ nên giá trị lẻ bằng $\frac{2}{\|w\|}$. Vậy để giá trị lẻ lớn nhất tương đương với giá trị $\|w\|$ nhỏ nhất và cũng tương đương với hàm

$$\frac{1}{2}\|w\|^2$$

đạt giá trị nhỏ nhất.

Vậy dẫn đến bài toán tối ưu tìm các tham số w, b thỏa mãn

$$\frac{1}{2}\|w\|^2 \rightarrow \min \quad (4.4)$$

$$(wx_i - b)y_i \geq 1, i = 1, 2, \dots, n. \quad (4.5)$$

Các điều kiện Karush-Kuhn-Tucker (KKT) là một tập hợp các điều kiện cần để một điểm là nghiệm cực trị cục bộ của bài toán tối ưu hóa phi tuyến với ràng buộc đẳng thức và bất đẳng thức. Chúng được sử dụng rộng rãi trong lý thuyết tối ưu hóa, đặc biệt trong các thuật toán như Support Vector Machines (SVM), lập trình tuyến tính/phi tuyến, và các mô hình học máy liên quan đến tối ưu hóa có ràng buộc.

Điều kiện KKT Xét bài toán tối ưu hóa sau:

$$\min_{\mathbf{x}} f(\mathbf{x})$$

với các ràng buộc:

- Bất đẳng thức: $g_i(\mathbf{x}) \leq 0$ cho $i = 1, \dots, m$
- Đẳng thức: $h_j(\mathbf{x}) = 0$ cho $j = 1, \dots, p$

Giả sử f, g_i, h_j là các hàm khả vi liên tục.

Giả sử $\mathbf{x}^*$ là một điểm cực tiểu cục bộ. Các điều kiện sau được thỏa mãn:

1. Đạo hàm của hàm Lagrangian bằng không.

$$\nabla f(\mathbf{x}^*) + \sum_{i=1}^m \lambda_i \nabla g_i(\mathbf{x}^*) + \sum_{j=1}^p \mu_j \nabla h_j(\mathbf{x}^*) = 0$$

2. Điểm $\mathbf{x}^*$ thỏa mãn tất cả ràng buộc.

$$g_i(\mathbf{x}^*) \leq 0 \quad \forall i, \quad h_j(\mathbf{x}^*) = 0 \quad \forall j$$

3. Các nhân tử Lagrange cho bất đẳng thức không âm.

$$\lambda_i \geq 0 \quad \forall i$$

4. Đối với mỗi ràng buộc bất đẳng thức, hoặc ràng buộc hoạt động (bằng 0) hoặc nhân tử Lagrange bằng 0.

$$\lambda_i g_i(\mathbf{x}^*) = 0 \quad \forall i$$

Áp dụng điều kiện vào bài toán tối ưu (4.4) $f(w) = \frac{1}{2} \|w\|^2$ với các ràng buộc bất đẳng thức (4.5) $g_i(w) = 1 - (wx_i - b)y_i \leq 0, i = 1, 2, \dots, n$ với hàm Lagrange

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 + \sum_{i=1}^n \alpha_i [1 - (wx_i - b)y_i]$$

ta có các điều kiện

$$\begin{aligned} w + \sum_{i=1}^n \alpha_i (-x_i y_i) &= 0, \\ \sum_{i=1}^n \alpha_i y_i &= 0, \\ g_i(w) = 1 - (wx_i - b)y_i &\leq 0, i = 1, 2, \dots, n, \\ \alpha_i &\geq 0, i = 1, 2, \dots, n. \end{aligned}$$

Thay $w = \sum_{i=1}^n \alpha_i (x_i y_i)$ và $\sum_{i=1}^n \alpha_i y_i = 0$ vào nhân tử Lagrange ta có hàm mục tiêu

$$\begin{aligned} \Phi(\alpha) &= \frac{1}{2} \|w\|^2 + \sum_{i=1}^n \alpha_i - w \sum_{i=1}^n \alpha_i x_i y_i \\ &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \|w\|^2 \\ &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j). \end{aligned}$$

Lời giải của bài toán tối ưu (4.4), (4.5) trên được xác định thông qua bài toán tối ưu sau:

Tìm véc-tơ $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_n) \geq 0$ thỏa mãn ràng buộc

$$\sum_{i=1}^n \alpha_i y_i = 0, \tag{4.6}$$

sao cho

$$\Phi(\alpha) = \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j (x_i \cdot x_j) \rightarrow \max. \quad (4.7)$$

Khi đó giá trị của w tính theo công thức

$$w = \sum_{i=1}^n \alpha_i y_i x_i.$$

Thực tế tổng trên chỉ cần tính đối với $\alpha_i > 0$ là các véc-tơ hỗ trợ nghĩa là tại các điểm này có $y_i(w x_i - b) = 1$, do $y_i = \pm 1$ nên $w x_i = b + y_i$.

Đối với các điểm thuộc lớp dương $y_i = 1$ thì giá trị $w \cdot x_i$ nhỏ nhất (gần biên nhất) là $b + y_i = b + 1$. Đối với các điểm thuộc lớp âm $y_i = -1$ thì giá trị $w \cdot x_i$ lớn nhất (gần biên nhất) là $b + y_i = b - 1$. Ta lấy giá trị trung bình của hai biên này (siêu phẳng tách nằm chính giữa) ta có công thức tính giá trị của b như sau:

$$b = \frac{\min_{i:y_i=1}(w \cdot x_i) + \max_{i:y_i=-1}(w \cdot x_i)}{2}.$$

Từ công thức tính các giá trị của w, b

$$w = \sum_{i=1}^n \alpha_i y_i x_i, \quad (4.8)$$

$$b = \frac{1}{2} \left(\min_{i:y_i=1} (w \cdot x_i) + \max_{i:y_i=-1} (w \cdot x_i) \right), \quad (4.9)$$

hàm phân loại được xác định bởi công thức

$$\text{class}(x) = \text{sign}(wx - b) = \begin{cases} 1 & \text{nếu } wx - b > 0, \\ -1 & \text{nếu } wx - b < 0. \end{cases} \quad (4.10)$$

Ví dụ 4.3. Xét dữ liệu có 2 điểm $A(x_1 = (2, 1), y_1 = 1), B(x_2 = (4, 3), y_2 = -1)$. Tìm hàm phân loại tối ưu đối với dữ liệu trên.

Giải:

Từ hàm mục tiêu (4.7) thay vào ta thu được biểu thức đối với $\alpha = (\alpha_1, \alpha_2)$ như sau:

$$\Phi(\alpha) = \alpha_1 + \alpha_2 - \frac{1}{2} (5\alpha_1^2 - 22\alpha_1\alpha_2 + 25\alpha_2^2) \rightarrow \max,$$

với ràng buộc (4.6) là

$$\alpha_1 - \alpha_2 = 0, \alpha_1 \geq 0, \alpha_2 \geq 0.$$

Giải bài toán trên bằng phương pháp đạo hàm ta tính được $\alpha_1 = \frac{1}{4}, \alpha_2 = \frac{1}{4}$ và từ đó tính giá trị

$$w = \left(-\frac{1}{2}, -\frac{1}{2}\right), b = -\frac{5}{2}.$$

Chi tiết công thức tính w, b như sau:

$$w = \alpha_1 y_1 x_1 + \alpha_2 y_2 x_2 = \frac{1}{4} \cdot 1 \cdot (2, 1) + \frac{1}{4} \cdot (-1) \cdot (4, 3) = \left(-\frac{1}{2}, -\frac{1}{2}\right), \quad (4.11)$$

$$b = \frac{\min_{y=1} \langle w, x_i \rangle + \max_{y=-1} \langle w, x_i \rangle}{2} = \frac{-3/2 - 7/2}{2} = -\frac{5}{2}. \quad (4.12)$$

Hàm phân loại là hàm dấu của hàm

$$f(x_1, x_2) = -\frac{1}{2}x_1 - \frac{1}{2}x_2 + \frac{5}{2}.$$

Ví dụ 4.4. Xét dữ liệu có 3 điểm $A(x_1 = (2, 2), y_1 = 1), B(x_2 = (4, 5), y_2 = -1), C(x_3 = (7, 4), y_3 = -1)$. Tìm đường thẳng tách tối ưu hai lớp trên.

Giải:

Hàm mục tiêu đối với véc-tơ $\alpha = (\alpha_1, \alpha_2, \alpha_3)$ như sau:

$$\Phi(\alpha) = (\alpha_1 + \alpha_2 + \alpha_3) - \frac{1}{2} \left(8\alpha_1^2 + 41\alpha_2^2 + 65\alpha_3^2 - 36\alpha_1\alpha_2 - 44\alpha_1\alpha_3 + 96\alpha_2\alpha_3 \right),$$

với ràng buộc $\alpha_1 - \alpha_2 + \alpha_3 = 0, \alpha_i \geq 0, i = 1, 2, 3$.

Lời giải bài toán tối ưu trên ta được $\alpha = \left(\frac{2}{13}, \frac{2}{13}, 0\right)$. Ta tính được giá trị

$$w = \left(\frac{-4}{13}, \frac{-6}{13}\right), \quad b = \frac{33}{13}.$$

Hàm phân loại

$$f(x_1, x_2) = \frac{-4}{13}x_1 - \frac{6}{13}x_2 + \frac{33}{13}.$$

Đường phẳng tách tối ưu là: $4x_1 + 6x_2 - 33 = 0$.

4.3 Biên mềm

Phương pháp máy véc-tơ hỗ trợ cho ta hàm phân loại tốt khi hai nhóm dữ liệu được tách biệt tuyến tính tốt. Tuy nhiên, có nhiều bài toán trong thực tế không có siêu phẳng nào tách biệt hai lớp mà chỉ có siêu phẳng tách hầu hết các điểm. Khi đó, các điểm được

tách biệt bởi một siêu phẳng thì có một số ít các điểm hoặc bị phân loại sai hoặc phân loại đúng nhưng rất gần siêu phẳng. Do đó chúng ta cần tìm siêu phẳng hạn chế hai loại trên. Để làm điều đó ta đưa vào một biến phụ gọi là biến *slack* ξ_i .

1. Nếu $\xi_i = 0$ thì biến x_i phân loại đúng và "xa" đường biên.
2. Nếu $0 < \xi_i < 1$ thì biến x_i phân loại đúng nhưng rất "gần" đường biên.
3. Nếu $\xi_i \geq 1$ thì biến x_i phân loại sai.

Tổng các sai số $\sum_{i=1}^n \xi_i$ được gọi là sai số mềm và ta cần hạn chế nó hay cực tiểu hoá. Đối với các ràng buộc của bài toán thì điều kiện

$$y_i(wx_i - b) \geq 1$$

được gọi là các ràng buộc cứng, nghĩa là nó đúng đối với các điểm x_i có độ hy sinh $\xi_i = 0$. Đối với các điểm có sự hy sinh $\xi_i > 0$ thì điều kiện cứng thay bởi điều kiện mềm

$$y_i(wx_i - b) \geq 1 - \xi_i.$$

Vậy dẫn đến bài toán biên mềm tối ưu tìm các tham số w, b thoả mãn

$$\begin{aligned} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i &\rightarrow \min \\ (wx_i - b)y_i &\geq 1 - \xi_i, i = 1, 2, \dots, n, \\ \xi_i &\geq 0, i = 1, 2, \dots, n. \end{aligned}$$

4.4 Hàm nhân

Đối với một số bài toán mà không thoả mãn điều kiện tách biệt tuyến tính trong không gian dữ liệu p chiều thì ta sử dụng một ánh xạ $\Phi: \mathbb{R}^p \rightarrow \mathbb{R}^M$ biến từ không gian có số chiều thấp lên không gian có số chiều cao hơn ($M > p$) mà dữ liệu trên không gian đó tách được tuyến tính. Tích vô hướng trên không gian đặc trưng $\mathbb{R}^M$ được gọi là hàm nhân, cụ thể hàm nhân $K(x, y)$ được xác định bởi công thức:

$$K(x, y) = \langle \Phi(x), \Phi(y) \rangle. \quad (4.13)$$

Ví dụ 4.5. Hàm nhân đa thức thuần nhất

Một số hàm nhân thông dụng thường được sử dụng:

1. Hàm nhân đa thức thuần nhất $K(x, y) = \langle x, y \rangle^d$,
2. Hàm nhân đa thức không thuần nhất $K(x, y) = \left(\langle x, y \rangle + \theta \right)^d$,
3. Hàm nhân Gauss (RBF) $K(x, y) = e^{\|x-y\|^2/2\sigma^2}$,
4. Hàm nhân Laplace $K(x, y) = e^{\|x-y\|/\sigma}$,
5. Hàm nhân sigmoid $K(x, y) = \tanh(\alpha \langle x, y \rangle + c)$.

Khi đó ta giải bài toán phân loại bằng phương pháp hàm nhân máy véc-tơ hỗ trợ

Tìm véc-tơ $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_n) \geq 0$ thoả mãn ràng buộc

$$\sum_{i=1}^n \alpha_i y_i = 0, \quad (4.14)$$

sao cho

$$\sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j K(x_i, x_j) \rightarrow \max. \quad (4.15)$$

Khi đó giá trị của w, b tính theo công thức

$$w = \sum_{i=1}^n \alpha_i y_i x_i, \quad (4.16)$$

$$b = \frac{1}{2} \left(\min_{i:y_i=1} K(w, x_i) + \max_{i:y_i=-1} K(w, x_i) \right). \quad (4.17)$$

Khi đó hàm phân lớp được xác định bởi công thức

$$f(x) = \sum_{i=1}^n \alpha_i y_i K(x_i, x) - b. \quad (4.18)$$

4.5 Máy véc-tơ hỗ trợ đa lớp

Đối với bài toán phân loại đa lớp, nghĩa là các biến phân loại $y_i \in \{1, 2, \dots, K\}$ với $K > 2$ ta sử dụng máy véc-tơ hỗ trợ theo hai cách OVA "One-vs-All" và OVO "One-vs-One".

4.5.1 Phương pháp OVA

Giả sử ta có K lớp. Ta xây dựng K bài toán phân loại như sau: Trước hết ta gán nhóm đầu tiên với nhãn +1 và các nhóm còn lại với nhãn -1 và ta có hàm phân loại $f_1(x)$. Tiếp theo, ta gán nhóm thứ hai bằng nhãn +1 và các nhóm còn lại với nhãn -1 và ta được

hàm phân loại $f_2(x)$. Tiếp tục như thế đến bài toán thứ K , ta gán nhóm thứ K bằng nhãn +1 và các nhóm còn lại bằng nhãn -1 và ta được hàm phân loại $f_K(x)$.

Hàm phân loại của biến x mà hàm phân loại đạt giá trị lớn nhất, nghĩa là

$$\text{class}(x) = \operatorname{argmax}_k f_k(x). \quad (4.19)$$

4.5.2 Phương pháp OVO

Đối với phương pháp OVO ta xây dựng phương pháp phân loại cho đôi một hai lớp riêng biệt. Như vậy nếu có K nhóm thì ta cần xây dựng $C_K^2 = K(K-1)/2$ bài toán phân loại bằng máy véc-tơ hỗ trợ. Khi đó biến x được phân loại dựa vào phương pháp bầu chọn đa số (major vote).

Để chuẩn bị toàn bộ thư viện cần thiết cho phân tích dữ liệu và xây dựng mô hình phân loại (SVM, LDA), đồng thời cài đặt style trực quan để vẽ biểu đồ trong Jupyter Notebook ta sử dụng đoạn code sau.

```
[1]: # %load ../standard_import.txt
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import label_binarize
from sklearn.model_selection import train_test_split,
    GridSearchCV
from sklearn.svm import SVC, LinearSVC
from sklearn.discriminant_analysis import
    LinearDiscriminantAnalysis
from sklearn.metrics import confusion_matrix, roc_curve, auc,
    classification_report

%matplotlib inline
plt.style.use('seaborn-white')
```

Đoạn code sau định nghĩa hàm `plot_svc` để trực quan hóa mô hình SVM 2 chiều: vẽ nền phân tách theo dự đoán của mô hình, hiển thị dữ liệu gốc, đánh dấu các điểm quan trọng trong việc xác định biên dữ liệu.

```
[2]: def plot_svc(svc, X, y, h=0.02, pad=0.25):
    x_min, x_max = X[:, 0].min()-pad, X[:, 0].max()+pad
    y_min, y_max = X[:, 1].min()-pad, X[:, 1].max()+pad
```

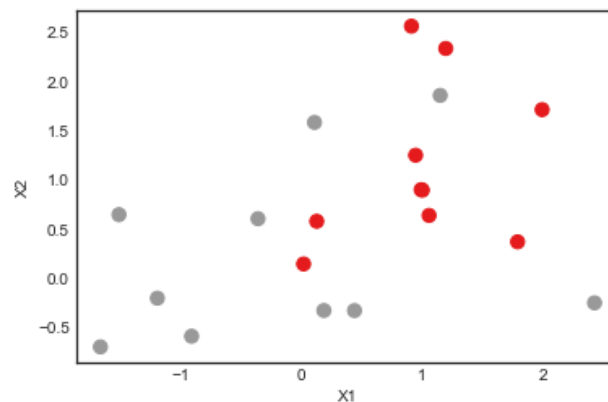
```

xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.
-arange(y_min, y_max, h))
Z = svc.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z, cmap=plt.cm.Set1, alpha=0.2)

plt.scatter(X[:,0], X[:,1], s=70, c=y, cmap=plt.cm.Set1)
# Support vectors indicated in plot by vertical lines
sv = svc.support_vectors_
plt.scatter(sv[:,0], sv[:,1], c='k', marker='|', s=100,
-linewidths=1)
plt.xlim(x_min, x_max)
plt.ylim(y_min, y_max)
plt.xlabel('X1')
plt.ylabel('X2')
plt.show()
#print('Number of support vectors: ', svc.support_.size)

```

Đoạn code sau tạo ra 20 điểm dữ liệu ngẫu nhiên trong không gian 2 chiều, chia thành 2 lớp (+1 và -1). Các điểm thuộc lớp (-1) được dịch chuyển để tách biệt khỏi lớp (+1). Dữ liệu được biểu diễn trên đồ thị scatter để quan sát sự phân bố giữa hai lớp.



```

[3]: # Generating random data: 20 observations of 2 features and
-divide into two classes.
np.random.seed(5)
X = np.random.randn(20,2)
y = np.repeat([1,-1], 10)

X[y == -1] = X[y == -1] +1
plt.scatter(X[:,0], X[:,1], s=70, c=y, cmap=plt.cm.Set1)
plt.xlabel('X1')

```

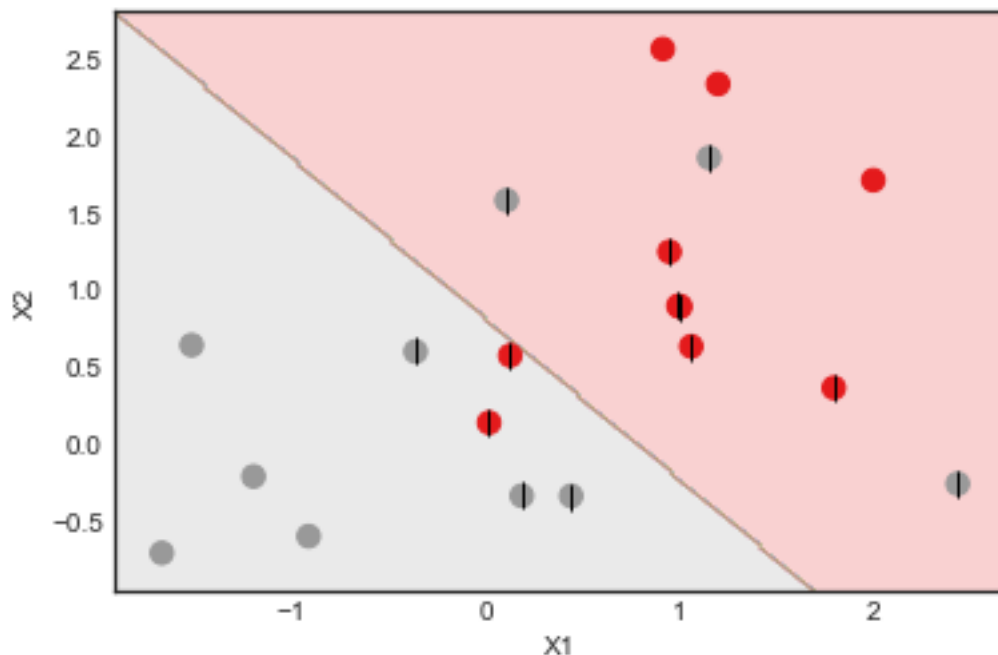
```
plt.ylabel('X2');
```

Hình trên mô tả kết quả của dữ liệu. Trục x1 biểu diễn giá trị đặc trưng thứ nhất, trục x2 biểu diễn giá trị đặc trưng thứ 2. Các chấm đỏ thuộc lớp (+1), các chấm xám thuộc lớp (-1). Ta thấy các điểm màu đỏ tập trung nhiều hơn ở vùng phía trên bên phải, các điểm màu xám nằm chủ yếu ở gần gốc tọa độ hoặc bên trái, phía dưới trục hoành.

Đoạn code sau, tạo một mô hình SVC từ thư viện sklearn.svm, huấn luyện mô hình trên tập dữ liệu đã sinh ra, và sau đó trực quan hóa đường thẳng phân tách cùng các máy véc tơ hỗ trợ. Cụ thể, hàm `svc.fit(X, y)` dùng để huấn luyện mô hình SVC với dữ liệu: X là ma trận đặc trưng (20 điểm, mỗi điểm có 2 đặc trưng), y là nhãn lớp (+1 và -1). Hàm `plot_svc(svc, X, y)` dùng để: vẽ đường biên quyết định của SVC, vẽ các đường biên margin, đánh dấu các điểm dữ liệu quan trọng nằm gần biên.

```
[4]: # Support Vector Classifier with linear kernel.
svc = SVC(C= 1.0, kernel='linear')
svc.fit(X, y)

plot_svc(svc, X, y)
```



Hình vẽ trên minh họa cách SVM tuyến tính phân loại hai lớp dữ liệu. Đường chéo là ranh giới quyết định tách hai lớp (+1) và (-1), các điểm có viền đậm là support vectors,

đây là các điểm nằm gần hoặc nằm trên biên margin, chúng là những điểm quan trọng nhất để xác định vị trí đường biên phân chia. Nếu loại bỏ các điểm khác không phải support vectors, đường biên sẽ không thay đổi.

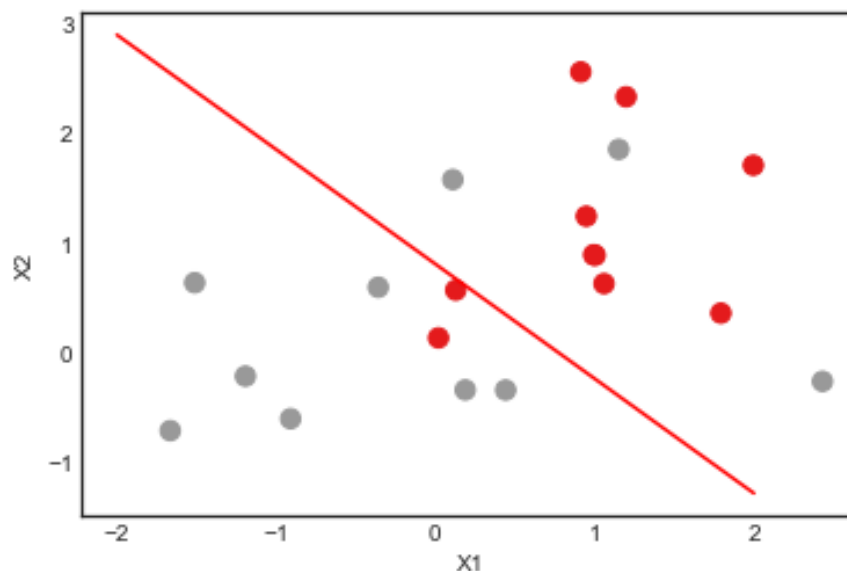
```
[5]: w=svc.coef_[0]
     print(w)
```

Véc-tơ w biểu diễn hướng pháp tuyến với siêu phẳng phân tách dữ liệu là: $w = [-0.73273926 - 0.70326681]$

```
[6]: b=svc.intercept_[0]
     print(b)
```

Giá trị hằng số chệch b hay tham số dịch chuyển siêu phẳng trong không gian nhiều chiều là $b = 0.5729441$.

Hình sau minh họa dữ liệu 2D gồm hai lớp đỏ và xám. Đường thẳng đỏ là ranh giới phân chia mà SVM tìm được, các điểm màu xám nằm chủ yếu bên trái và dưới đường ranh giới, các điểm màu đỏ nằm chủ yếu ở phía trên.



Đoạn code sau vẽ dữ liệu huấn luyện trong không gian 2 chiều và vẽ thêm đường thẳng phân chia (decision boundary) của SVM tuyến tính dựa vào vector trọng số w và hệ số chặn b .

```
[7]: # w1x1+w2x2+b=0
     xx=np.linspace(-2,2,50)
     yy=-(w[0]*xx+b)/w[1]
```



```
plt.plot(xx,yy,c='red')

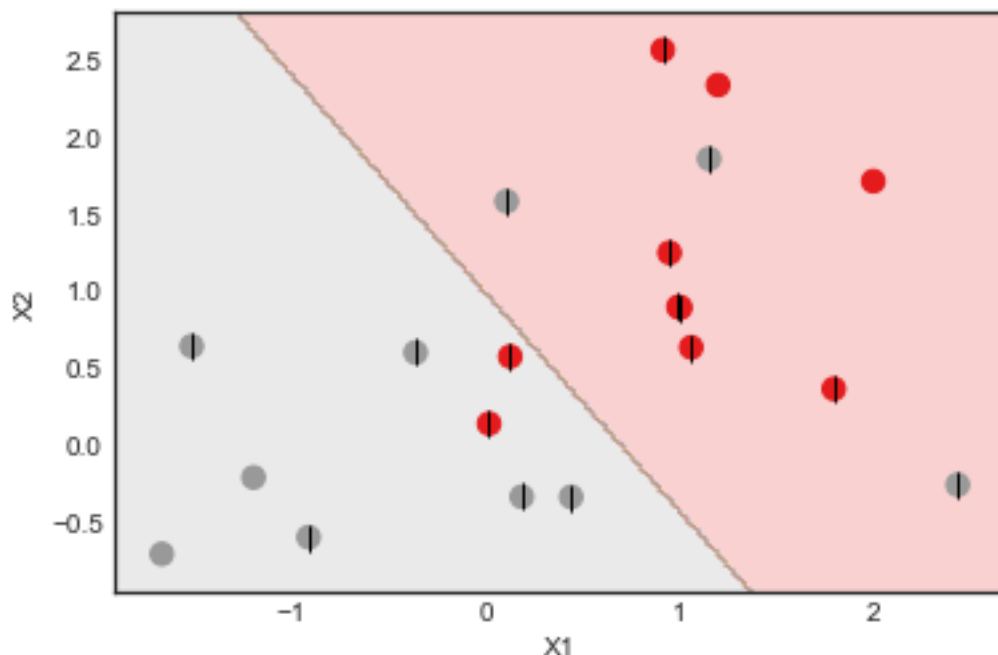
plt.scatter(X[:,0], X[:,1], s=70, c=y, cmap=plt.cm.Set1)
plt.xlabel('X1')
plt.ylabel('X2');
```

Để huấn luyện một mô hình SVM tuyến tính với $C=0.1$ và trực quan hóa kết quả bằng cách vẽ dữ liệu, đường phân chia, margin, và các support vectors ta sử dụng các dòng lệnh sau:

```
[8]: # When using a smaller cost parameter (C=0.1) the margin is
      # wider, resulting in more support vectors.
svc2 = SVC(C=0.1, kernel='linear')
svc2.fit(X, y)
plot_svc(svc2, X, y)
```

Hàm `svc2.fit(X, y)` huấn luyện mô hình SVM trên dữ liệu huấn luyện, trong đó X là ma trận đặc trưng, y là nhãn lớp.

Hàm `plot_svc(svc2, X, y)` dùng để vẽ dữ liệu và đường biên phân loại của SVM. Các điểm dữ liệu được hiển thị theo lớp màu khác nhau, các điểm support vectors được đánh dấu đặc biệt. Hình vẽ mẫu dữ liệu thu được dưới đây:



Đoạn code sau dùng `GridSearchCV` để tìm giá trị C tối ưu cho mô hình SVM tuyến tính trong tập $\{0.001, 0.01, 0.1, 1, 5, 10, 100\}$. Với mỗi giá trị C , mô hình sẽ được đánh giá bằng

10-fold cross-validation (Tức là dữ liệu được chia làm 10 phần, mỗi lần 9 phần để huấn luyện và 1 phần để kiểm tra, lặp lại 10 lần). Theo cách đó giúp đánh giá mô hình ổn định hơn. Sau đó giá trị C được chọn dựa vào độ chính xác nào tốt nhất.

```
[9]: # Select the optimal C parameter by cross-validation
tuned_parameters = [{'C': [0.001, 0.01, 0.1, 1, 5, 10, 100]}]
clf = GridSearchCV(SVC(kernel='linear'), tuned_parameters,
                  cv=10, scoring='accuracy', return_train_score=True)
clf.fit(X, y)
#clf.cv_results_
```

```
[9]: GridSearchCV(cv=10, estimator=SVC(kernel='linear'),
                  param_grid=[{'C': [0.001, 0.01, 0.1, 1, 5, 10,
                  -100]}],
                  return_train_score=True, scoring='accuracy')
```

Dòng lệnh sau có tác dụng trả về bộ tham số tối ưu mà GridSearchCV đã tìm được sau khi thử nghiệm toàn bộ lưới tham số.

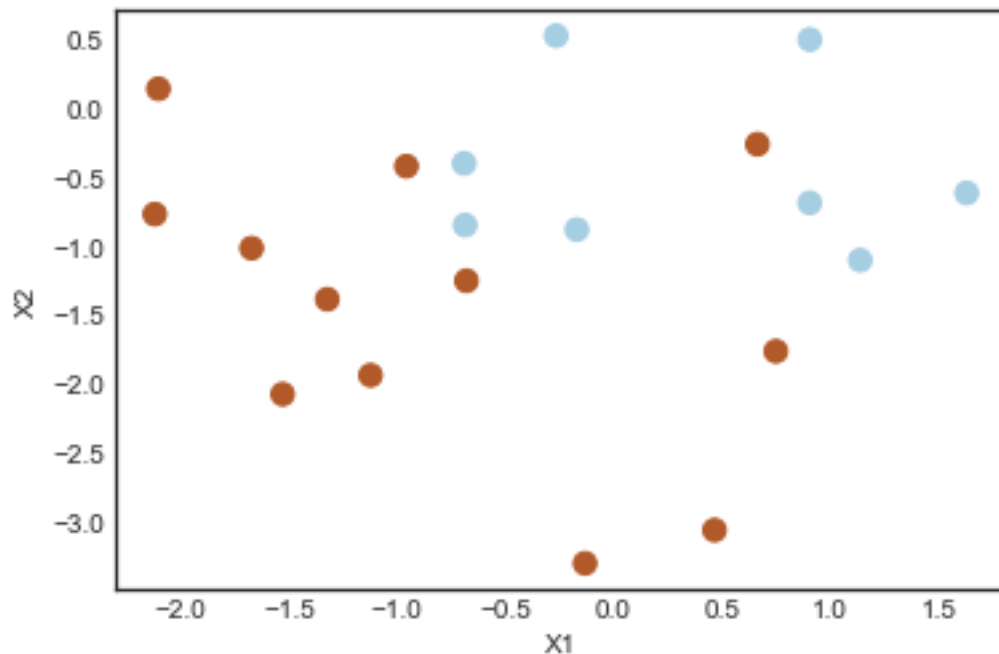
```
[10]: # 0.001 is best according to GridSearchCV.
clf.best_params_
```

Tham số tối ưu nhận được là:

```
[10]: {'C': 0.001}
```

Ta xét thêm một minh họa nữa về sử dụng Cây quyết định để phân loại dữ liệu. Đoạn code sau giúp sinh ra ma trận X_{test} kích thước 20×2 . Mỗi hàng là một điểm dữ liệu gồm 2 đặc trưng, các giá trị được lấy từ phân phối chuẩn chuẩn $N(0,1)$. Sau đó, sinh ngẫu nhiên nhãn cho 20 điểm dữ liệu. Mỗi nhãn có giá trị (+1) hoặc (-1), dữ liệu được chia thành 2 lớp. Dịch chuyển 1 lớp để tách biệt, sau cùng là vẽ biểu đồ phân tán để trực quan hóa dữ liệu.

```
[11]: # Generating test data
np.random.seed(1)
X_test = np.random.randn(20, 2)
y_test = np.random.choice([-1, 1], 20)
X_test[y_test == 1] = X_test[y_test == 1] - 1
plt.scatter(X_test[:, 0], X_test[:, 1], s=70, c=y_test, cmap=plt.
            -cm.Paired)
plt.xlabel('X1')
plt.ylabel('X2');
```



Hình trên minh họa dữ liệu vừa được sinh ra, các điểm màu đỏ phân bố phía dưới bên trái, các điểm màu xanh tập trung bên phải và phía trên.

Đoạn code sau dùng mô hình SVM là svc2 với tham số $C=0.1$ để dự đoán nhãn cho tập dữ liệu kiểm tra X_{test} , sau đó hiển thị bảng ma trận nhầm lẫn (confusion matrix) dưới dạng bảng để đánh giá độ chính xác phân loại cho từng lớp.

```
[12]: # svc2 : C = 0.1
y_pred = svc2.predict(X_test)
pd.DataFrame(confusion_matrix(y_test, y_pred), index=svc.
             _classes_, columns=svc.classes_)
```

Ma trận nhầm lẫn thu được:

```
[12]:      -1    1
-1      2    6
1       0   12
```

Tuy nhiên ta có thể huấn luyện một mô hình SVM tuyến tính với biên rộng là $C=0.001$. Sử dụng tham số đó để dự đoán nhãn trên tập kiểm tra, in ra ma trận nhầm lẫn.

```
[13]: svc3 = SVC(C=0.001, kernel='linear')
svc3.fit(X, y)
```

```
# svc3 : C = 0.001
y_pred = svc3.predict(X_test)
pd.DataFrame(confusion_matrix(y_test, y_pred), index=svc3.
             _classes_, columns=svc3.classes_)
# The misclassification is the same
```

Bảng ma trận nhầm lẫn với $C = 0.001$.

```
[13]:      -1    1
      -1    2    6
       1    0   12
```

Kết quả cho thấy số lượng lỗi phân loại không thay đổi so với mô hình trước khi $C=0.1$. Điều này có thể thấy rằng trong bộ dữ liệu này, việc thay đổi C chưa ảnh hưởng rõ rệt đến hiệu quả phân loại nhân của lớp.

Ngoài ra ta có thể điều chỉnh lại dữ liệu để các điểm thuộc hai lớp không bị chồng lấn, và có thể tìm được một đường thẳng phân chia rõ ràng giữa chúng. Đoạn code sau giúp dịch chuyển dữ liệu lớp (+1) để làm cho hai lớp dữ liệu phân tách tuyến tính, sau đó vẽ biểu đồ scatter để trực quan hóa dữ liệu.

```
[14]: # Changing the test data so that the classes are really
      _seperable with a hyperplane.
X_test[y_test == 1] = X_test[y_test == 1] -1
plt.scatter(X_test[:,0], X_test[:,1], s=70, c=y_test, cmap=plt.
            _cm.Paired)
plt.xlabel('X1')
plt.ylabel('X2');
```

Để tạo một SVM tuyến tính với $C=10$, sau đó huấn luyện mô hình trên bộ dữ liệu X_{test} , y_{test} , ta dùng dòng lệnh sau:

```
[15]: svc4 = SVC(C=10.0, kernel='linear')
      svc4.fit(X_test, y_test)
```

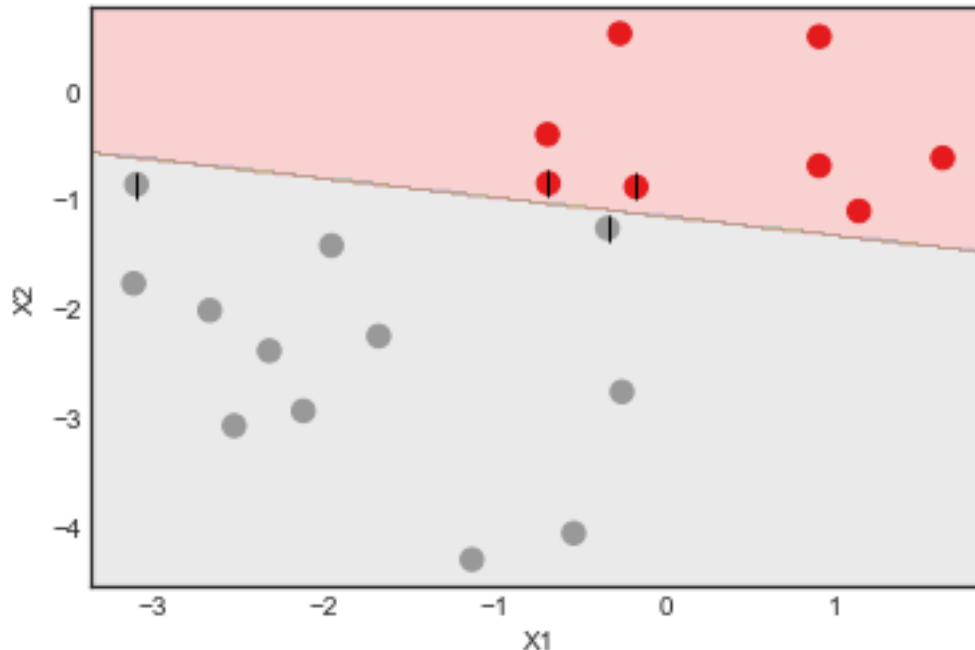
Lệnh khởi tạo một mô hình SVM trong thư viện scikit-learn với tham số $C = 10$:

```
[15]: SVC(C=10.0, kernel='linear')
```

Để vẽ các điểm dữ liệu trong X_{test} với màu sắc theo y_{test} , vẽ thêm các đường biên và đánh dấu các support vectors bằng vòng tròn hay ký hiệu đặc biệt ta dùng dòng lệnh sau:

```
[16]: plot_svc(svc4, X_test, y_test)
```

Hình ảnh của dữ liệu thu được:



Để khởi tạo một mô hình SVM tuyến tính với $C=1$ và huấn luyện mô hình trên tập dữ liệu $X_{\text{test}}, y_{\text{test}}$ ta sử dụng đoạn code sau:

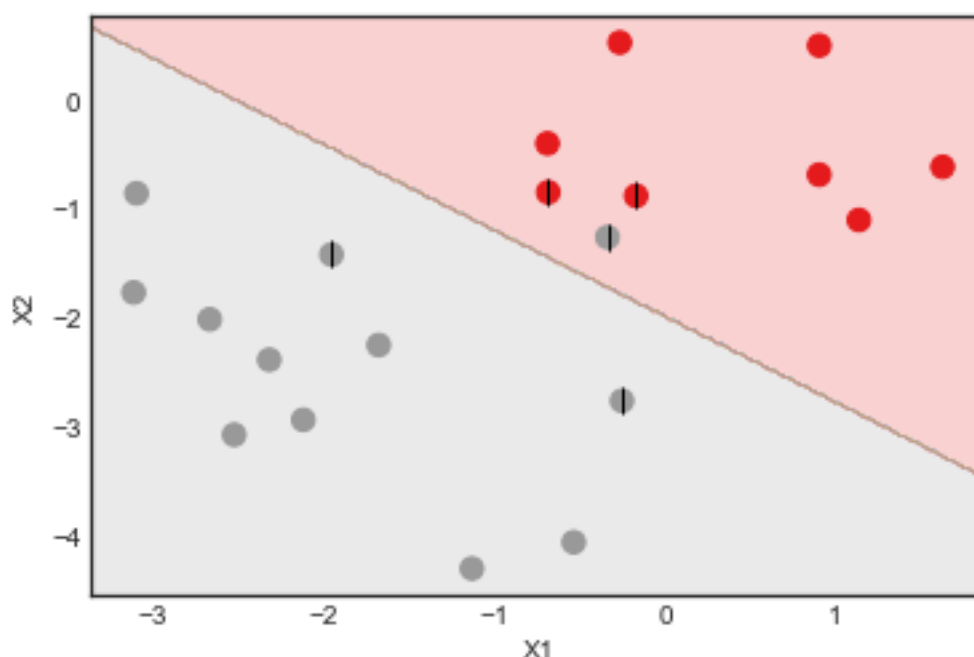
```
[17]: # Increase the margin. Now there is one misclassification:
      -increased bias, lower variance.
svc5 = SVC(C=1, kernel='linear')
svc5.fit(X_test, y_test)
```

Tạo ra một mô hình SVM tuyến tính, với tham số điều chuẩn $C=1$ để cân bằng giữa việc có biên rộng và số lỗi phân loại cho phép, với $C = 1$ đây là mức tham số trung bình. Ta sử dụng dòng code sau:

```
[17]: SVC(C=1, kernel='linear')
```

Hàm `plot_svc(svc5, X_test, y_test)` dưới đây để vẽ hình minh họa kết quả phân loại của mô hình SVM tuyến tính `svc5` trên tập dữ liệu $X_{\text{test}}, y_{\text{test}}$, gồm các điểm dữ liệu, đường siêu phẳng, margins và support vectors. Trong đó, điểm dữ liệu được vẽ màu khác nhau cho mỗi lớp. Có đường chia lớp và support vectors được đánh dấu.

```
[18]: plot_svc(svc5, X_test, y_test)
```

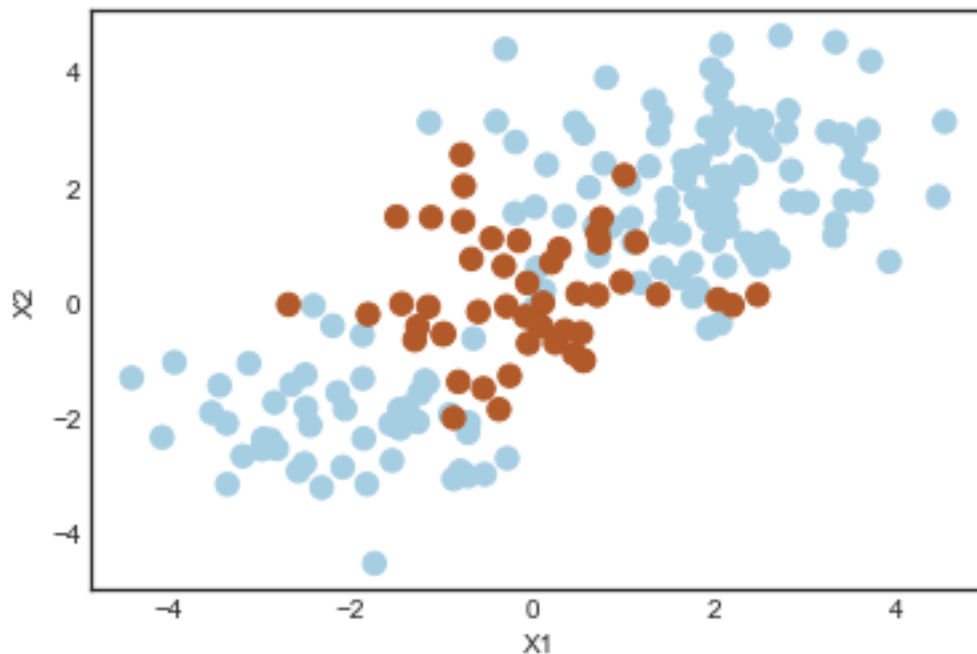


Hình trên minh họa dữ liệu được phân chia theo hai lớp. Các điểm véc-tơ hỗ trợ là những điểm được đánh dấu.

Ví dụ sau đây ứng dụng thuật toán Cây quyết định để phân loại mẫu dữ liệu. Sinh dữ liệu ngẫu nhiên, ma trận kích thước 200x2, mỗi phần tử có phân phối Chuẩn tắc $N(0,1)$. Sau đó dịch chuyển để tạo thành cụm, có 150 điểm thuộc lớp (-1) và 50 điểm thuộc lớp 1. Chia tập dữ liệu tập huấn luyện và tập kiểm tra theo tỉ lệ 1:1. Vẽ các điểm dữ liệu với hai màu khác nhau theo các lớp.

```
[19]: # Generating test data
np.random.seed(8)
X = np.random.randn(200, 2)
X[:100] = X[:100] + 2
X[101:150] = X[101:150] - 2
y = np.concatenate([np.repeat(-1, 150), np.repeat(1, 50)])
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size=0.5, random_state=2)
plt.scatter(X[:, 0], X[:, 1], s=70, c=y, cmap=plt.cm.Paired)
plt.xlabel('X1')
plt.ylabel('X2');
```

Ta thu được hình ảnh mẫu dữ liệu thuộc hai lớp được sinh ngẫu nhiên sau đây:



Đoạn code sau xây dựng và huấn luyện một mô hình SVM phi tuyến với kernel RBF để phân loại dữ liệu. Với $C=1.0$ và $\gamma=1$ đây là tham số khá cao giúp cho biên có thể uốn cong nhiều, mô hình sẽ tạo ranh giới phân chia phức tạp, uốn cong nhiều để bao quanh các cụm dữ liệu.

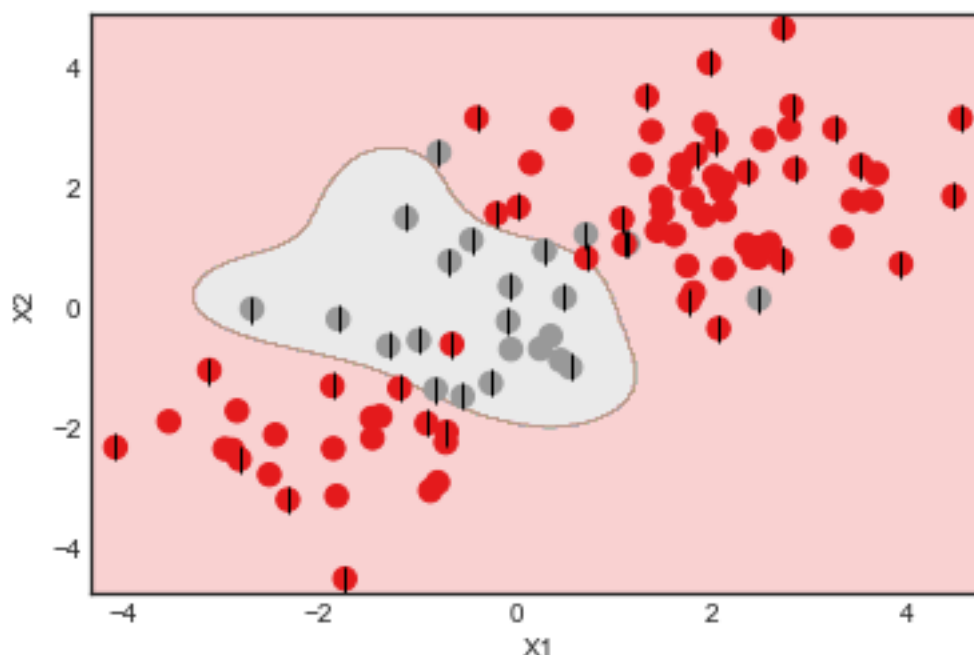
```
[20]: svm = SVC(C=1.0, kernel='rbf', gamma=1)
      svm.fit(X_train, y_train)
```

Ta có lệnh gọi hàm khởi tạo mô hình SVM từ thư viện scikit-learn ở dưới đây trong đó $\gamma = 1$ là tham số quan trọng khi dùng kernel RBF (radial basis function). Hệ số γ xác định ảnh hưởng của từng điểm dữ liệu huấn luyện lên ranh giới quyết định. Khi γ càng lớn thì mô hình chỉ xem xét vùng lân cận hẹp quanh mỗi điểm dữ liệu. Khi γ nhỏ thì mô hình xem xét ảnh hưởng rộng hơn của các điểm dữ liệu.

```
[20]: SVC(gamma=1)
```

Để vẽ đường phân cách của SVM cùng với dữ liệu huấn luyện, giúp trực quan hóa mô hình đã học để phân loại ta dùng dòng lệnh sau:

```
[21]: plot_svc(svm, X_train, y_train)
```



Hình trên minh họa cách SVM với kernel RBF và $\gamma=1$ trong phân loại dữ liệu. Ta thấy vùng đỏ là mô hình dự đoán lớp đỏ, vùng xám là mô hình dự đoán lớp xám. Ranh giới cong phân chia hai lớp được tạo ra dựa trên các support vectors. Tuy mô hình phân loại được phần lớn dữ liệu, nhưng vẫn có một số điểm bị phân loại nhầm.

```
[22]: # Increasing C parameter, allowing more flexibility
svm2 = SVC(C=100, kernel='rbf', gamma=1.0)
svm2.fit(X_train, y_train)
```

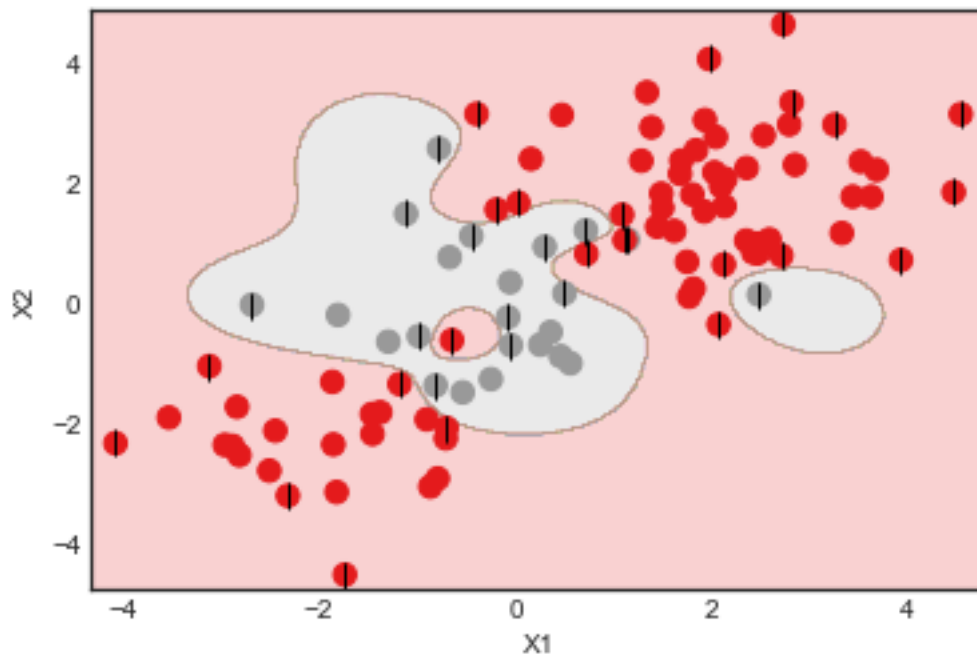
Dòng lệnh sau tạo một mô hình SVM với kernel RBF, $\gamma = 1.0$, và $C = 100$, nghĩa là mô hình sẽ có độ linh hoạt rất cao, nhằm phân loại hầu như tất cả điểm huấn luyện chính xác, tuy nhiên sẽ có rủi ro overfitting nghĩa là có khả năng giảm lỗi trên tập huấn luyện nhưng có thể mất khả năng tổng quát trên tập kiểm tra.

```
[22]: SVC(C=100, gamma=1.0)
```

Để vẽ các điểm dữ liệu huấn luyện màu theo nhãn, vẽ vùng quyết định và tô nền theo phân loại mà mô hình dự đoán, đồng thời vẽ đường cong ranh giới phân tách các lớp và các support vectors được đánh dấu đặc biệt ta dùng lệnh sau:

```
[23]: plot_svc(svm2, X_train, y_train)
```

Thu được hình minh họa dữ liệu được phân tách theo hai lớp:



Để tìm cặp tham số tốt nhất (C , γ) cho mô hình SVM với kernel RBF, đoạn code sau thử nghiệm tất cả 25 tổ hợp tham số. Đánh giá qua 10-fold cross-validation, nghĩa là chia dữ liệu huấn luyện thành 10 phần, mỗi lần huấn luyện trên 9 phần và kiểm thử trên 1 phần. Khi đó ta sẽ thu được cặp (C , γ) cho độ chính xác cao nhất.

```
[24]: # Set the parameters by cross-validation
tuned_parameters = [{'C': [0.01, 0.1, 1, 10, 100],
                        'gamma': [0.5, 1, 2, 3, 4]}]
clf = GridSearchCV(SVC(kernel='rbf'), tuned_parameters, cv=10,
                  scoring='accuracy', return_train_score=True)
clf.fit(X_train, y_train)
#clf.cv_results_
```

Đoạn code sau dùng GridSearchCV để tìm bộ tham số C và γ tốt nhất cho SVC (kernel RBF). Nó sẽ thử 25 tổ hợp tham số, đánh giá bằng cross-validation 10 folds, chọn bộ nào cho accuracy cao nhất, đồng thời lưu lại cả điểm train/test để phân tích độ phù hợp mô hình.

```
[24]: GridSearchCV(cv=10, estimator=SVC(),
                  param_grid=[{'C': [0.01, 0.1, 1, 10, 100],
                                'gamma': [0.5, 1, 2, 3, 4]}],
                  return_train_score=True, scoring='accuracy')
```

```
[25]: clf.best_params_
```

Trả về bộ tham số tối ưu mà GridSearch tìm được. Ở đây, SVM hoạt động tốt nhất khi:

```
[25]: {'C': 10, 'gamma': 0.5}
```

```
[26]: confusion_matrix(y_test, clf.best_estimator_.predict(X_test))
```

Dòng lệnh trên dùng để đánh giá hiệu suất phân loại của mô hình SVM tối ưu trên tập kiểm tra, kết quả thu được là ma trận nhầm lẫn sau đây:

```
[26]: array([[66,  7],
          [ 6, 21]], dtype=int64)
```

Độ chính xác trên tập test của mô hình SVM là:

```
[27]: # 15% of test observations misclassified
      clf.best_estimator_.score(X_test, y_test)
```

```
[27]: 0.87
```

Để khởi tạo một mô hình SVM với kernel RBF ($C = 1$, $\gamma = 2$) rồi huấn luyện nó trên tập dữ liệu X_{train} , y_{train} ta sử dụng câu lệnh sau. Mô hình này sẽ tìm ra một đường biên phi tuyến để phân loại dữ liệu.

```
[28]: svm3 = SVC(C=1, kernel='rbf', gamma=2)
      svm3.fit(X_train, y_train)
```

```
[28]: SVC(C=1, gamma=2)
```

Đoạn code sau tạo một mô hình SVM với kernel RBF, $C=1$, $\gamma=50$. Đây là mô hình khá linh hoạt, có khả năng phân chia dữ liệu huấn luyện tốt, nhưng dễ gặp hiện tượng quá khớp.

```
[29]: # More flexible model
      svm4 = SVC(C=1, kernel='rbf', gamma=50)
      svm4.fit(X_train, y_train)
```

```
[29]: SVC(C=1, gamma=50)
```

Ta so sánh hiệu năng của 2 mô hình SVM với cùng kernel RBF nhưng khác tham số γ ($\gamma = 1$ và $\gamma = 50$) trên cả tập huấn luyện và tập kiểm tra thông qua đường cong ROC và diện tích dưới đường cong (AUC).

```
[30]: y_train_score3 = svm3.decision_function(X_train)
y_train_score4 = svm4.decision_function(X_train)

false_pos_rate3, true_pos_rate3, _ = roc_curve(y_train,
        -y_train_score3)
roc_auc3 = auc(false_pos_rate3, true_pos_rate3)

false_pos_rate4, true_pos_rate4, _ = roc_curve(y_train,
        -y_train_score4)
roc_auc4 = auc(false_pos_rate4, true_pos_rate4)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
ax1.plot(false_pos_rate3, true_pos_rate3, label='SVM $\gamma =
        -1$ ROC curve (area = %0.2f)' % roc_auc3, color='b')
ax1.plot(false_pos_rate4, true_pos_rate4, label='SVM $\gamma =
        -50$ ROC curve (area = %0.2f)' % roc_auc4, color='r')
ax1.set_title('Training Data')

y_test_score3 = svm3.decision_function(X_test)
y_test_score4 = svm4.decision_function(X_test)

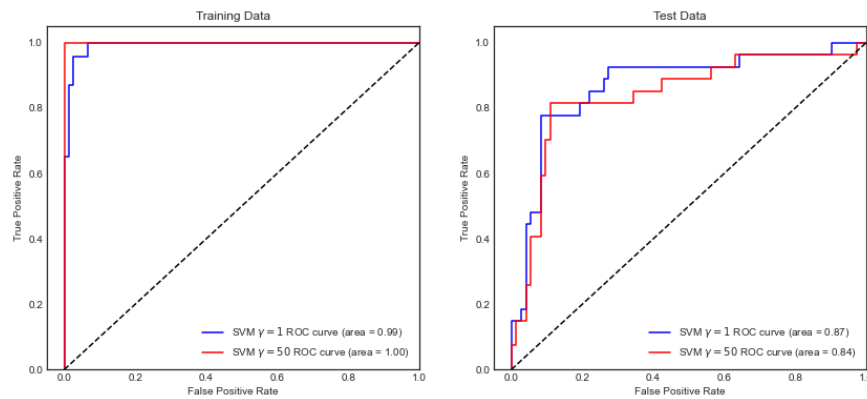
false_pos_rate3, true_pos_rate3, _ = roc_curve(y_test,
        -y_test_score3)
roc_auc3 = auc(false_pos_rate3, true_pos_rate3)

false_pos_rate4, true_pos_rate4, _ = roc_curve(y_test,
        -y_test_score4)
roc_auc4 = auc(false_pos_rate4, true_pos_rate4)

ax2.plot(false_pos_rate3, true_pos_rate3, label='SVM $\gamma =
        -1$ ROC curve (area = %0.2f)' % roc_auc3, color='b')
ax2.plot(false_pos_rate4, true_pos_rate4, label='SVM $\gamma =
        -50$ ROC curve (area = %0.2f)' % roc_auc4, color='r')
ax2.set_title('Test Data')

for ax in fig.axes:
    ax.plot([0, 1], [0, 1], 'k--')
    ax.set_xlim([-0.05, 1.0])
    ax.set_ylim([0.0, 1.05])
    ax.set_xlabel('False Positive Rate')
    ax.set_ylabel('True Positive Rate')
    ax.legend(loc="lower right")
```

Kết quả thu được ở hình dưới đây:



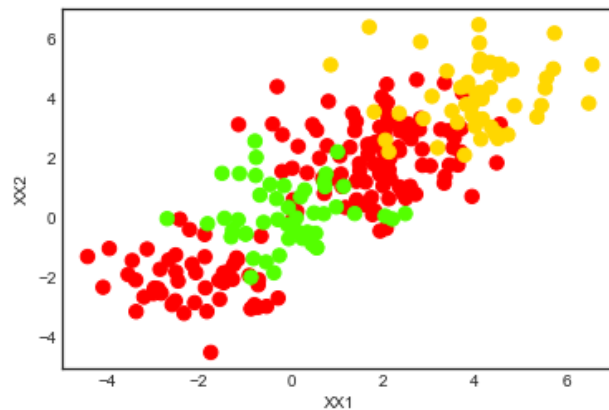
Hình bên trái là đồ thị của dữ liệu huấn luyện, hình bên phải là đồ thị của dữ liệu kiểm tra. Đường màu xanh là mô hình SVM với $\gamma = 1$, đường màu đỏ là mô hình SVM với $\gamma = 50$.

Trên tập huấn luyện, ta thấy mô hình SVM với $\gamma = 1$ ta có $AUC \approx 0.99$ có thể nói mô hình phân loại gần như hoàn hảo, mô hình SVM với $\gamma = 50$ có $AUC = 1.00$ như vậy mô hình phân loại tuyệt đối trên tập huấn luyện. Trên tập kiểm tra, mô hình SVM $\gamma = 1$ có $AUC \approx 0.87$, mô hình SVM với $\gamma = 50$ có $AUC \approx 0.84$. Như vậy, mô hình $\gamma = 1$ hoạt động tốt hơn mô hình SVM khi $\gamma = 50$ trên tập kiểm tra. Vậy mô hình $\gamma = 50$ tuy hoàn hảo trên tập huấn luyện nhưng kém hơn khi áp dụng vào dữ liệu mới dẫn tới hiện tượng quá khớp.

```
[31]: # Adding a third class of observations
np.random.seed(8)
XX = np.vstack([X, np.random.randn(50, 2)])
yy = np.hstack([y, np.repeat(0, 50)])
XX[yy == 0] = XX[yy == 0] + 4

plt.scatter(XX[:, 0], XX[:, 1], s=70, c=yy, cmap=plt.cm.prism)
plt.xlabel('XX1')
plt.ylabel('XX2');
```

Ta mở rộng dữ liệu 2 lớp ban đầu thành 3 lớp, trong đó lớp mới (class = 0) gồm 50 điểm Gaussian, được dịch chuyển ra xa để tránh chồng lấn với hai lớp ban đầu. Sau đó trực quan hóa dữ liệu 3 lớp bằng scatter plot.

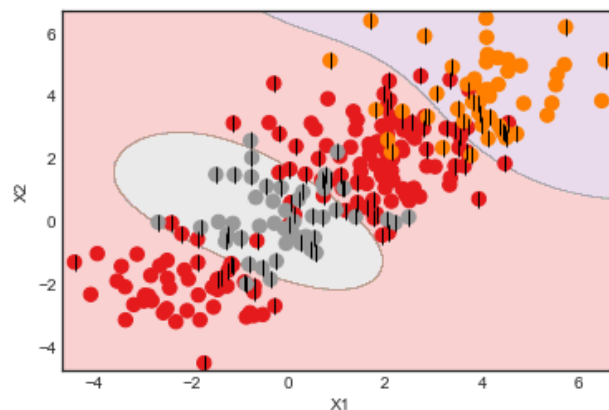


Đoạn code sau khởi tạo một mô hình SVM phi tuyến (kernel RBF, C=1) và huấn luyện nó trên tập dữ liệu 3 lớp (XX, yy).

```
[32]: svm5 = SVC(C=1, kernel='rbf')
      #svm5 = SVC(C=1, kernel='linear')
      svm5.fit(XX, yy)
```

Sử dụng lệnh `plot_svc(svm5, XX, yy)` để vẽ đường biên phân chia và vùng phân loại của mô hình SVM (svm5) trên tập dữ liệu ba lớp XX, yy. Nó cho ta cái nhìn trực quan về cách mô hình phân loại trong không gian 2 chiều.

```
[33]: plot_svc(svm5, XX, yy)
```



Hình trên là kết quả trực quan hóa của SVM với kernel RBF trên dữ liệu 3 lớp. Nhóm đỏ: phần lớn được phân loại đúng trong vùng hồng, nhưng có một số điểm đỏ nằm trong vùng xám hoặc tím là dự đoán sai. Nhóm xám: tập trung ở trung tâm (trái, giữa),

được phân cách tương đối rõ với nhóm đỏ. Nhóm cam: nằm ở phía trên bên phải, được phân cách bằng đường cong RBF kernel khỏi nhóm đỏ.

Trong đó, các điểm tròn là dữ liệu gốc, màu nền là vùng mà mô hình SVM dự đoán thuộc về một lớp cụ thể. Đường cong ngăn cách vùng là biên phân chia các lớp. Các đường thẳng nhỏ trên các điểm tròn thể hiện support vectors là những điểm dữ liệu quan trọng được SVM chọn ra để xây dựng đường biên.

4.6 Bài tập

Bài 4.1. Xét dữ liệu có 2 điểm $A(x_1 = (2, 1), y_1 = 1), B(x_2 = (5, 4), y_2 = -1)$. Tìm đường thẳng tối ưu và lề tách hai điểm trên.

Bài 4.2. Xét dữ liệu có 2 điểm $A(x_1 = (1, 2), y_1 = 1), B(x_2 = (3, 6), y_2 = -1), C(x_3 = (7, 5), y_3 = -1)$. Tìm đường thẳng tối ưu và lề tách hai điểm trên. Xác định các véc-tơ hỗ trợ.

Bài 4.3. Xét tập dữ liệu huấn luyện gồm 3 lớp C_1, C_2, C_3 với các điểm trong mặt phẳng: Lớp $C_1 = \{A_1(1, 1), A_2(2, 1)\}$, $C_2 = \{B_1(4, 4), B_2(5, 4)\}$, $C_3 = \{C_1(1, 5), C_2(2, 6)\}$.

- Xây dựng SVM đa lớp theo phương pháp OVO
- Viết phương trình các siêu phẳng phân tách
- Điểm $x(3, 3)$ thuộc lớp nào.

Bài 4.4. Cho tập dữ liệu nhỏ gồm các quan sát sau: lớp $C_1 = \{(3, 4), (2, 2), (4, 4), (1, 4)\}$, lớp $C_2 = \{(2, 1), (4, 3), (4, 1)\}$.

- Hãy vẽ các điểm này trên đồ thị và vẽ tay một siêu phẳng phân tách hoàn hảo hai lớp này.
- Xác định Lề (Margin) và xác định siêu phẳng tối ưu cho tập dữ liệu trên.
- Xác định các điểm là vectơ hỗ trợ.

Bài 4.5. Bài toán tối ưu cho bộ phân loại véc-tơ hỗ trợ dẫn đến bài toán tối ưu

$$\min_{w, b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i,$$

với các ràng buộc $\xi_i \geq 0, y_i(x_i^T w - b) \geq 1 - \xi_i$.

- Chứng minh bài toán trên tương đương với bài toán tối ưu

$$\min_{w, b} \sum_{i=1}^n \left[1 - y_i f(x_i) \right]_+ + \lambda \frac{\|w\|^2}{2},$$

trong đó $f(x) = x^T w - b$, $\lambda = \frac{1}{C}$ và $z_+ = \max\{0, z\}$.

b) Sử dụng điều kiện KKT của bài toán tối ưu SVM. Chứng minh nếu quan sát x_i nằm ngoài lề và phân loại đúng ($y_i f(x_i) > 1$) thì hệ số α_i tương ứng bằng 0.

4.7 Thực hành

Thực hành 4.1. Sử dụng bộ dữ liệu Iris 3 lớp hoa (Setosa, Versicolor, Virginica). Chia bộ dữ liệu giữa tập huấn luyện và tập kiểm tra theo tỉ lệ 70% và 30%, sau đó chuẩn hóa dữ liệu.

- a) Huấn luyện SVM với kernel RBF.
- b) Đánh giá hiệu quả của mô hình dựa trên các độ đo như là: Accuracy, Recall, F1-score.
- c) Thử với một số tham số khác nhau của giá trị $C = 0.01, C = 0.05, C = 0.1$.

Thực hành 4.2. Sử dụng bộ dữ liệu Digits. Chia bộ dữ liệu giữa tập huấn luyện và tập kiểm tra theo tỉ lệ 70% và 30%, sau đó chuẩn hóa dữ liệu.

- a) Huấn luyện SVM đa lớp với OVO.
- b) Đánh giá hiệu quả của mô hình dựa trên các độ đo như là: Accuracy, recall, $F1 - score$.
- c) Thử với một số tham số khác nhau của giá trị $C = 0.01, C = 1, C = 10$.

Thực hành 4.3. Sinh dữ liệu dạng two moons.

- a) Huấn luyện dữ liệu SVM với Linear kernel, RBF kernel.
- b) Đánh giá hiệu quả của mô hình dựa trên các độ đo như là: Accuracy, Recall, F1-score.
- c) Thử với một số tham số khác nhau của giá trị $C = 0.01, C = 1, C = 10$.

Chương 5

PHÂN CỤM DỮ LIỆU

Phân cụm dữ liệu là phương pháp học máy không có giám sát với mục tiêu là chia tập các đối tượng của dữ liệu thành các nhóm đối tượng tương đồng. Các thuật toán phân cụm dữ liệu được ứng dụng rộng rãi trong nhiều lĩnh vực như nhận dạng, trí tuệ nhân tạo, công nghệ thông tin, xử lý ảnh, y học, tâm lý học và marketing.

Trong chương này, chúng ta nghiên cứu một số khái niệm về phân cụm dữ liệu, phân tích các thuật toán phân cụm bằng phân vùng (k -means và fuzzy k -means), thuật toán phân cụm phân tầng (phương pháp gộp và phương pháp chia), và giới thiệu một số ứng dụng của phương pháp phân cụm dữ liệu (phân tích gene, phân tích chuỗi thời gian, phân cụm khách hàng).

5.1 Khái niệm phân cụm dữ liệu

Phân cụm (clustering hay cluster analysis) là việc nhóm các đối tượng có tính chất tương đồng vào cùng một nhóm (được gọi là cluster) và nó không tương đồng với các nhóm khác theo một nghĩa nào đó.

Phân cụm dữ liệu là một việc chính trong phân tích khám phá dữ liệu mà được ứng dụng trong nhiều lĩnh vực như học máy, nhận dạng, phân tích ảnh, tin y sinh, đồ họa máy tính v.v..

5.1.1 Khái niệm phân cụm dữ liệu

Một tập hợp các đối tượng (còn gọi là điểm, quan sát, bản ghi) và mỗi đối tượng có các biến (còn gọi là thuộc tính, đặc trưng). Trong khuôn khổ bài giảng ta mô tả tập dữ liệu D gồm n đối tượng dạng tập hợp $D = \{x_1, x_2, \dots, x_n\}$, trong đó mỗi đối tượng có p

thuộc tính và ký hiệu bởi véc-tơ hàng $x_i = (x_{i1}, x_{i2}, \dots, x_{ip})$ với x_{ij} là giá trị của đối tượng x_i đối với thuộc tính j .

Định nghĩa 5.1 (Phân cụm dữ liệu). Phân cụm dữ liệu là chia tập dữ liệu D thành K cụm dữ liệu $C_1, C_2, \dots, C_K$ mỗi cụm là một tập con của tập D , các tập con này đôi một rời nhau ($C_i \cap C_j = \emptyset, i \neq j$) và hợp của chúng bằng D ($D = C_1 \cup C_2 \cup \dots \cup C_K$). Các đối tượng trong mỗi cụm dữ liệu C_k là tương đồng với nhau và các đối tượng trong các cụm dữ liệu khác nhau không tương đồng với nhau.

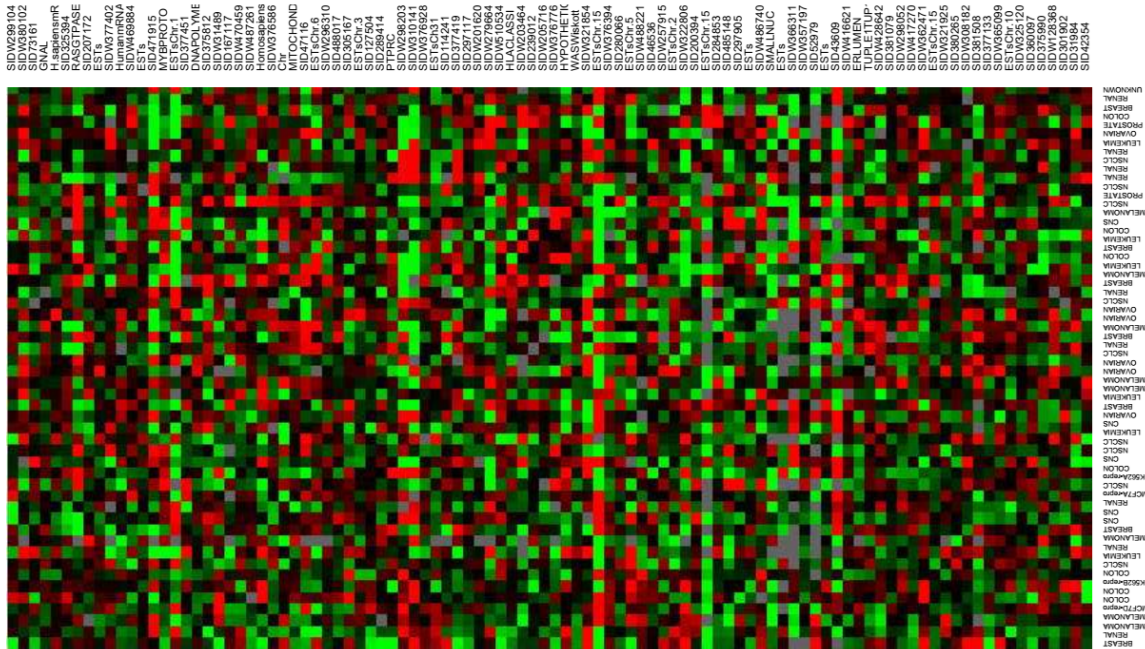
Ví dụ 5.1 (Phân cụm gene). DNA là từ viết tắt của Axit Deoxyribo Nucleic, và là vật liệu cơ bản tạo nên nhiễm sắc thể ở người. Vi mảng DNA đo mức độ biểu hiện của một gen trong tế bào bằng cách đo lượng mRNA (axit ribonucleic thông tin) có mặt. Vi mảng được coi là một bước đột phá trong công nghệ sinh học, tạo điều kiện cho việc nghiên cứu định lượng về hàng nghìn gen đồng thời từ một mẫu tế bào đơn lẻ.

Cách thức hoạt động của một vi mảng DNA như sau. Trình tự nucleotide của vài nghìn gen được in trên một phiên kính. Một mẫu mục tiêu và một mẫu tham chiếu được đánh dấu bằng thuốc nhuộm đỏ và xanh lục, sau đó được lai hóa (hybridized) với DNA trên phiên kính. Thông qua phép đo huỳnh quang, logarit của cường độ RNA lai hóa (màu đỏ/xanh lục) tại mỗi vị trí được đo. Kết quả thường dao động từ -6 đến 6 , đo mức độ biểu hiện của mỗi gen tương đối so với mẫu tham chiếu. Các giá trị dương cho thấy biểu hiện cao hơn ở mẫu mục tiêu so với mẫu tham chiếu, và ngược lại đối với các giá trị âm.

Tập dữ liệu biểu hiện gene tập hợp các giá trị biểu hiện từ một loạt các thí nghiệm vi mảng DNA, với mỗi cột đại diện cho một thí nghiệm. Do đó, có vài nghìn hàng đại diện cho các gene cá nhân và hàng chục cột đại diện cho các mẫu thí nghiệm; cụ thể trong ví dụ của Hình 5.1 có 6830 gen (cột) và 64 mẫu (hàng), mặc dù để làm rõ chỉ hiển thị một bản đồ nhiệt của 100 cột. Hình này hiển thị dữ liệu dưới dạng bản đồ nhiệt, dao động từ màu xanh lá cây (âm tính) đến màu đỏ (dương tính). Các mẫu là 64 khối u ung thư từ các bệnh nhân khác nhau.

Ví dụ 5.2 (Phân cụm khách hàng). Phân khúc khách hàng (Customer Segmentation) là quá trình phân chia khách hàng thành các nhóm dựa trên các đặc điểm khách hàng mục tiêu chung để các công ty có thể tiếp thị cho từng nhóm một cách hiệu quả và phù hợp. Đây là bước khá quan trọng, là yếu tố giúp tăng tỉ lệ chuyển đổi cho doanh nghiệp. Nếu doanh nghiệp thực hiện tốt bước này có thể giúp doanh nghiệp phân chia ngân sách quảng cáo tốt hơn và tiết kiệm được nhiều hơn.

Đối với người làm marketing, việc xác định đúng phân khúc khách hàng giúp cho họ hiểu khách hàng mình hơn. Biết khách hàng đang suy nghĩ gì? Khách hàng đang quan



Hình 5.1: Dữ liệu DNA: Ma trận biểu hiện 6830 genes (cột) và 64 mẫu (hàng).

tâm đến vấn đề gì? Nhu cầu của khách hàng là gì? Khách hàng đang cần gì?

Dựa vào những câu hỏi trên mà các tiếp thị viên tập trung triển khai các chiến dịch đáp ứng được nhu cầu mong muốn của họ, từ đó sẽ đem lại hiệu quả và doanh thu cao cho doanh nghiệp. Đây còn là một công cụ quản trị được khách hàng của mình. Tùy theo phân khúc khách hàng mà các marketer sẽ chia ngân sách phù hợp. Sẽ có những nhóm khách hàng phân bố rộng rãi hơn nên cần chi phí ngân sách cao hơn.

5.1.2 Biến đổi dữ liệu

Trong phần này chúng ta nghiên cứu một số phương pháp phổ biến được sử dụng để biến đổi dữ liệu trước khi thực hiện các phương pháp phân cụm dữ liệu.

Chuẩn hoá dữ liệu Khi các thuộc tính dữ liệu có nhiều thang đo khác nhau ta thường chuẩn hoá dữ liệu.

Chuẩn hoá dữ liệu đối với biến thứ j , $1 \leq j \leq p$ theo công thức

$$x_{ij}^* = \frac{x_{ij} - L_j}{M_j}, 1 \leq i \leq n. \quad (5.1)$$

Trong đó x_{ij} là giá trị được chuẩn hoá, L_j là độ đo vị trí, M_j là độ đo tỷ lệ.

Phép chuẩn hoá z-score là phép biến đổi biến có trung bình bằng 0 và phương sai bằng 1, trong đó

$$\bar{x}_j = L_j = \frac{1}{n} \sum_{i=1}^n x_{ij}, \sigma_j = M_j = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_{ij} - \bar{x}_j)^2}.$$

Khi đó phép biến đổi z-score có dạng

$$x_{ij}^* = \frac{x_{ij} - \bar{x}_j}{\sigma_j}, \quad (5.2)$$

với $\bar{x}_j$ là giá trị trung bình, và σ_j là độ lệch tiêu chuẩn.

Một phép chuẩn hoá dữ liệu về đoạn $[0, 1]$ thường được sử dụng trong nhiều dữ liệu với

$$L_j = \min_{1 \leq i \leq n} x_{ij}, M_j = \max_{1 \leq i \leq n} x_{ij} - \min_{1 \leq i \leq n} x_{ij}.$$

Khi đó phép chuẩn hoá dữ liệu về đoạn $[0, 1]$ được cho bởi công thức

$$x_{ij}^* = \frac{x_{ij} - \min_{1 \leq i \leq n} x_{ij}}{\max_{1 \leq i \leq n} x_{ij} - \min_{1 \leq i \leq n} x_{ij}}. \quad (5.3)$$

Một số phương pháp chuẩn hoá dữ liệu khác được cho bởi bảng dưới đây.

Phép biến đổi	L_j	M_j
z-score	$\bar{x}_j$	σ_j
USTD	0	σ_j
Maximum	0	$\max_{1 \leq i \leq n} x_{ij}$
Mean	$\bar{x}_j$	1
Median	$m_j = \text{median}\{x_{ij}, 1 \leq i \leq n\}$	1
Sum	0	$\sum_{i=1}^n x_{ij}$
Range	$\min_{1 \leq i \leq n} x_{ij}$	$R_j = \max_{1 \leq i \leq n} x_{ij} - \min_{1 \leq i \leq n} x_{ij}$

Bảng 5.1: Các phép biến đổi dữ liệu

Biến đổi dữ liệu Phép biến đổi dữ liệu khác với phép chuẩn hoá dữ liệu. Phép chuẩn hoá dữ liệu chỉ tập trung vào từng biến của dữ liệu. Phép biến đổi dữ liệu xét cho toàn bộ dữ liệu. Nói chung phép chuẩn hoá dữ liệu là một trường hợp đặc biệt của biến đổi dữ liệu. Tuy nhiên các phép biến đổi dữ liệu thường phức tạp hơn phép chuẩn hoá dữ liệu. Trong phần này chúng ta làm quen với phương pháp phân tích thành phần chính (PCA).

Giả sử ta có n quan sát $x_1, x_2, \dots, x_n$ với p biến $X_1, X_2, \dots, X_p$. Phương pháp phân tích thành phần chính (PCA-Principal Component Analysis) là phương pháp giảm số chiều của dữ liệu thông qua các thành phần chính (Principal Components).

Thành phần chính thứ nhất là tổ hợp tuyến tính của các thuộc tính $X_1, X_2, \dots, X_p$ được biểu diễn bởi biểu thức

$$Z_1 = \Phi_{11}X_1 + \Phi_{12}X_2 + \dots + \Phi_{1p}X_p \quad (5.4)$$

có phương sai lớn nhất. Để chuẩn hoá ta coi véc-tơ $\Phi_1 = (\Phi_{11}, \Phi_{12}, \dots, \Phi_{1p})^T$ có độ dài đơn vị, nghĩa là

$$\Phi_{11}^2 + \Phi_{12}^2 + \dots + \Phi_{1p}^2 = 1. \quad (5.5)$$

Giả sử ma trận dữ liệu X là ma trận cỡ $n \times p$ và được quy tâm (có trung bình bằng 0 theo cột). Nếu không thì ta tính giá trị $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$ và thay các giá trị mới $x_i \leftarrow x_i - \bar{x}$.

Mỗi điểm x_i được chiếu xuống thành phần chính thứ nhất Φ_1 có giá trị là z_{i1} được tính bởi công thức

$$z_{i1} = x_i \Phi_1 = \Phi_{11}x_{i1} + \Phi_{12}x_{i2} + \dots + \Phi_{1p}x_{ip}, \quad (5.6)$$

và giá trị phương sai của nó là σ_1^2 được tính theo công thức

$$\begin{aligned} \sigma_1^2 &= \frac{1}{n} \sum_{i=1}^n z_{i1}^2 = \frac{1}{n} \sum_{i=1}^n z_{i1}^T \cdot z_{i1} \\ &= \frac{1}{n} \sum_{i=1}^n \Phi_1^T x_i^T \cdot x_i \Phi_1 \\ &= \Phi_1^T \left(\frac{1}{n} \sum_{i=1}^n x_i^T \cdot x_i \right) \Phi_1 \\ &= \Phi_1^T \left(\frac{1}{n} \sum_{i=1}^n X^T X \right) \Phi_1 \\ &= \Phi_1^T S \Phi_1. \end{aligned}$$

Ma trận $S = \frac{1}{n} \sum_{i=1}^n X^T X$ gọi là ma trận hiệp phương sai (nó là ma trận đối xứng và xác định không âm). Giá trị phương sai lớn nhất $\Phi_1^T S \Phi_1$ với $\|\Phi_1\| = 1$ chính là giá trị riêng lớn nhất của ma trận S và Φ_1 chính là véc-tơ riêng đơn vị tương ứng với giá trị riêng lớn nhất của S .

Thành phần chính thứ hai là tổ hợp tuyến tính của các thuộc tính $X_1, X_2, \dots, X_p$ được biểu diễn bởi biểu thức

$$Z_2 = \Phi_{21}X_1 + \Phi_{22}X_2 + \dots + \Phi_{2p}X_p \quad (5.7)$$

là véc-tơ vuông góc với thành phần chính thứ nhất Z_1 và cũng có phương sai lớn nhất. Do đó véc-tơ thành phần chính thứ hai là $\Phi_2 = (\Phi_{21}, \Phi_{22}, \dots, \Phi_{2p})^T$ là véc-tơ vuông góc với

Φ_1 đó chính là véc-tơ riêng đơn vị của ma trận hiệp phương sai S ứng với giá trị riêng lớn thứ hai của S .

Tương tự như vậy véc-tơ thành phần chính thứ k , Φ_k là véc-tơ riêng đơn vị của ma trận S tương ứng với giá trị riêng lớn thứ k của ma trận S .

Ta ký hiệu không gian ma trận $U = [\Phi_1, \Phi_2, \dots, \Phi_k]$ là ma trận cỡ $p \times k$ có các cột trực giao gồm k véc-tơ trực chuẩn. Dữ liệu được chiếu xuống không gian con sinh bởi k véc-tơ trong U là không gian con k -chiều của R^p và được tính theo công thức

$$Z = X.U. \quad (5.8)$$

Tập các điểm trong Z gọi là tập các điểm biểu diễn của dữ liệu ban đầu X xuống không gian k -chiều. Khi ta lấy $k = 2$ hoặc $k = 3$ để dễ dàng trực quan hoá dữ liệu.

Thuật toán PCA

Bước 1 Tính giá trị trung bình

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i,$$

Bước 2 Quy tâm dữ liệu

$$\hat{x}_i \leftarrow x_i - \bar{x}$$

Bước 3 Tính ma trận hiệp phương sai

$$S = \frac{1}{n} \sum_{i=1}^n \hat{X}^T \hat{X}$$

Bước 4 Tính k giá trị riêng lớn nhất của S , và k véc-tơ riêng đơn vị tương ứng $\Phi_1, \Phi_2, \dots, \Phi_k$

Bước 5 Chiếu dữ liệu xuống không gian k chiều, $U = [\Phi_1, \Phi_2, \dots, \Phi_k]$ và

$$Z = \hat{X}U$$

Bước 6 Xấp xỉ dữ liệu $\tilde{x}_i \approx \bar{x} + \hat{x}_i U U^T$.

5.1.3 Khoảng cách

Để xác định các đối tượng gần nhau hay tương tự nhau ta cần xác định một độ đo tương ứng để đo. Khoảng cách Euclid (khoảng cách L_2) được sử dụng phổ biến nhất.

Khoảng cách Euclid giữa x_i và x_j xác định bởi công thức

$$D(x_i, x_j) = \sqrt{\sum_{k=1}^p (x_{ik} - x_{jk})^2}. \quad (5.9)$$

Tổng quát hoá khoảng cách Euclid, ta gọi là khoảng cách Minkowski (khoảng cách L_p , $p \geq 1$) xác định bởi công thức.

$$D(x_i, x_j) = \sum_{k=1}^p \left((x_{ik} - x_{jk})^{\frac{1}{p}} \right)^p. \quad (5.10)$$

Với $p = 1$ ta gọi là khoảng cách Manhattan.

Khoảng cách Mahalanobis được xác định bởi công thức

$$D(x_i, x_j) = (x_i - x_j)^T S^{-1} (x_i - x_j), \quad (5.11)$$

trong đó S là ma trận hiệp phương sai của dữ liệu xác định bởi công thức

$$S = \frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})(x_i - \bar{x})^T,$$

với $\bar{x}$ là giá trị trung bình của dữ liệu $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$.

Khoảng cách dựa trên hệ số tương quan Pearson giữa x_i và x_j xác định bởi công thức

$$D(x_i, x_j) = \frac{1 - r_{ij}}{2}, \quad (5.12)$$

trong đó

$$r_{ij} = \frac{\sum_{k=1}^p (x_{ik} - \bar{x}_i)(x_{jk} - \bar{x}_j)}{\sqrt{\sum_{k=1}^p (x_{ik} - \bar{x}_i)^2 \sum_{k=1}^p (x_{jk} - \bar{x}_j)^2}}.$$

Một khoảng cách khác được xác định dựa vào độ đo góc giữa hai véc-tơ và định nghĩa bởi công thức

$$D(x_i, x_j) = 1 - S(x_i, x_j) = 1 - \frac{x_i^T x_j}{\|x_i\| \cdot \|x_j\|}. \quad (5.13)$$

5.1.4 Đánh giá cụm dữ liệu

Trong các phương pháp phân cụm dữ liệu thì việc xác định số cụm dữ liệu rất quan trọng và để xác định được số cụm dữ liệu cần phân hoạch ta thường thông quan các hệ số đánh giá chất lượng của phân cụm dữ liệu. Trong phần này chúng ta làm quen với một số hệ số đánh giá chất lượng phân cụm dữ liệu.

Hệ số Davies-Bouldin Giả sử dữ liệu được phân thành K cụm, ký hiệu $C_1, C_2, \dots, C_K$ và gọi n_k là số lượng điểm dữ liệu trong cụm C_k , và $m_k = \frac{1}{n_k} \sum_{x \in C_k} x$ là tâm của cụm C_k .

Độ phân tán nội bộ của mỗi cụm C_k xác định bởi công thức

$$S_k = \frac{1}{n_k} \sum_{x \in C_k} D(x, m_k).$$

Khoảng cách giữa hai cụm C_i và C_j (với $i \neq j$) là

$$M_{ij} = D(m_i, m_j).$$

Với mỗi cụm C_i ta tính hệ số R_i xác định cụm gần nhất theo công thức

$$R_i = \max_{j \neq i} \left(\frac{S_i + S_j}{M_{ij}} \right).$$

Hệ số này càng nhỏ thì cụm C_i được phân tách tốt với các cụm khác và các điểm trong cụm có độ gắn kết cao.

Hệ số Davies-Bouldin (DBI) là giá trị trung bình của các giá trị R_i và xác định bởi công thức

$$DBI = \frac{1}{K} \sum_{i=1}^K R_i = \frac{1}{K} \sum_{i=1}^K \left(\max_{j \neq i} \left\{ \frac{S_i + S_j}{M_{ij}} \right\} \right). \quad (5.14)$$

Giá trị của hệ số DBI càng thấp thì chất lượng phân cụm dữ liệu tốt nghĩa là khoảng cách giữa các tâm của nhóm lớn và độ gắn kết nội bộ của các nhóm cao.

Hệ số Dunn Các ký hiệu xác định tương tự như hệ số Davies-Bouldin. Ta ký hiệu khoảng cách giữa hai cụm dữ liệu C_i và C_j (với $i \neq j$) là khoảng cách nhỏ nhất giữa hai điểm thuộc hai cụm khác nhau theo công thức

$$D(C_i, C_j) = \min_{x \in C_i, y \in C_j} D(x, y).$$

Đường kính của mỗi cụm dữ liệu C_i là khoảng cách lớn nhất giữa hai điểm trong cụm và được xác định bởi công thức

$$D(C_i) = \max_{x, y \in C_i} D(x, y).$$

Hệ số Dunn là tỷ số khoảng cách tối thiểu giữa hai cụm bất kỳ và đường kính tối đa của các cụm và công thức hệ số Dunn như sau:

$$D = \min_{i \neq j} \left(\frac{D(C_i, C_j)}{\max_{1 \leq i \leq K} D(C_i)} \right). \quad (5.15)$$

Đối với hệ số Dunn thì có giá trị càng cao càng tốt.

Hệ số Silhouette Hệ số Silhouette là một chỉ số phổ biến và trực quan để đánh giá chất lượng của các thuật toán phân cụm dữ liệu. Hệ số Silhouette là hệ số vô hướng nằm trong khoảng $(-1, 1)$ xác định mức độ của một điểm dữ liệu phù hợp với cụm được so sánh với cụm gần nó nhất. Hệ số Silhouette là trung bình của các hệ số Silhouette cho mỗi điểm dữ liệu.

Đại lượng a_i là khoảng cách trung bình của điểm x_i đến các điểm trong cụm chứa nó. Giả sử điểm x_i thuộc cụm dữ liệu C_k nào đó, khi đó

$$a_i = \frac{1}{n_k - 1} \sum_{x_j \in C_k} D(x_i, x_j).$$

Đại lượng b_i là khoảng cách trung bình của điểm x_i đến các điểm trong cụm lân cận gần với cụm chứa điểm x_i nhất. Đại lượng b_i xác định bởi công thức

$$b_i = \min_{i \neq k} \left(\frac{1}{n_i} \sum_{x_j \in C_i} d(x_i, x_j) \right).$$

Ta ký hiệu s_i là hệ số Silhouette của điểm dữ liệu x_i và được tính bởi công thức

$$s_i = \frac{b_i - a_i}{\max\{a_i, b_i\}}. \quad (5.16)$$

Hệ số Silhouette cho toàn bộ dữ liệu là hệ số S xác định bởi công thức

$$S = \frac{1}{n} \sum_{i=1}^n s_i. \quad (5.17)$$

Giá trị hệ số Silhouette càng cao thì chất lượng phân cụm dữ liệu càng tốt.

5.2 Phân cụm dữ liệu bằng phân vùng

5.2.1 Thuật toán k -means

Thuật toán k -means là thuật toán phân cụm dữ liệu đơn giản nhất trong học máy không có giám sát để giải bài toán phân cụm dữ liệu.

Giả sử ta có n quan sát $x_1, x_2, \dots, x_n$ trong không gian p -chiều và ta muốn chia tập dữ liệu thành K cụm $C_1, C_2, \dots, C_K$ với các cụm này rời nhau và mỗi điểm dữ liệu chỉ phân vào đúng 1 trong các nhóm trên. Ta mô tả thuật toán như sau:

Đầu tiên ta xuất phát với K điểm tùy ý được gọi là tâm của mỗi nhóm dữ liệu. Sau đó ta gán các điểm của dữ liệu vào các nhóm mà gần tâm của nhóm đó nhất. Tiếp theo

ta cập nhật tâm của mỗi nhóm bằng trung bình cộng của các điểm dữ liệu trong nhóm đó. Ta tiếp tục lặp lại quá trình trên đến khi các tâm nhóm không thay đổi thì dừng. Các điểm được phân loại theo các nhóm mà chúng gần tâm nhất. Mỗi nhóm được biểu diễn bằng các tâm của mỗi nhóm.

Thuật toán Ta ký hiệu khoảng cách giữa hai véc-tơ trong không gian $\mathbb{R}^p$ là khoảng cách Euclid

$$d(x, y) = \|x - y\| = \sqrt{\sum_{j=1}^p (x_j - y_j)^2}.$$

Thuật toán k -means là chia tập dữ liệu thành các K tập con rời nhau $C_1, C_2, \dots, C_K$ và một tập các tâm nhóm $m_1, m_2, \dots, m_K$ để cực tiểu hoá tổng sai số trong mỗi nhóm, nghĩa là cực tiểu hoá hàm mục tiêu

$$\sum_{k=1}^K \sum_{x \in C_k} \|x - m_k\|^2 \rightarrow \min. \quad (5.18)$$

Bước 1: Chọn ngẫu nhiên K điểm tâm $m_1, m_2, \dots, m_K$.

Bước 2: Tính khoảng cách $d(x_i, m_k)$ từ các điểm x_i đến các tâm của mỗi nhóm m_k .

Bước 3: Với mỗi điểm dữ liệu $x_i \in C_k$ ta gán điểm vào nhóm k mà đạt giá trị nhỏ nhất $\|x_i - m_k\| = \min_{\ell} \{\|x_i - m_{\ell}\|\}$.

Bước 4: Tính lại tâm của mỗi nhóm

$$m_k = \frac{1}{|C_k|} \sum_{x \in C_k} x.$$

Bước 5: Lặp lại Bước 2 đối với các tâm mới.

Bước 6: Nếu không có điểm nào được gán lại nhãn thì dừng. Nếu trái lại thì lặp lại Bước 3.

Thuật toán k -means dựa vào phương pháp tối ưu cực tiểu hoá hàm mục tiêu với các biến là các tâm $m_1, m_2, \dots, m_K$.

$$E = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - m_k\|^2. \quad (5.19)$$

Để cực tiểu hoá hàm mục tiêu trên ta tính đạo hàm đối với m_k cho bởi công thức

$$\begin{aligned}
\frac{\partial E}{\partial m_k} &= \frac{\partial}{\partial m_k} \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - m_k\|^2 \\
&= \sum_{k=1}^K \frac{\partial}{\partial m_k} \sum_{x_i \in C_k} \|x_i - m_k\|^2 \\
&= \sum_{k=1}^K \sum_{x_i \in C_k} \frac{\partial}{\partial m_k} \|x_i - m_k\|^2 \\
&= \sum_{x_i \in C_k} -2(x_i - m_k).
\end{aligned}$$

Giải phương trình $\frac{\partial E}{\partial m_k} = 0$ ta được $m_k = \frac{1}{n_k} \sum_{x_i \in C_k} x_i$. Đây chính là công thức cập nhật tâm của cụm dữ liệu C_k do đó cách cập nhật điểm tâm này là giảm giá trị của hàm mục tiêu E .

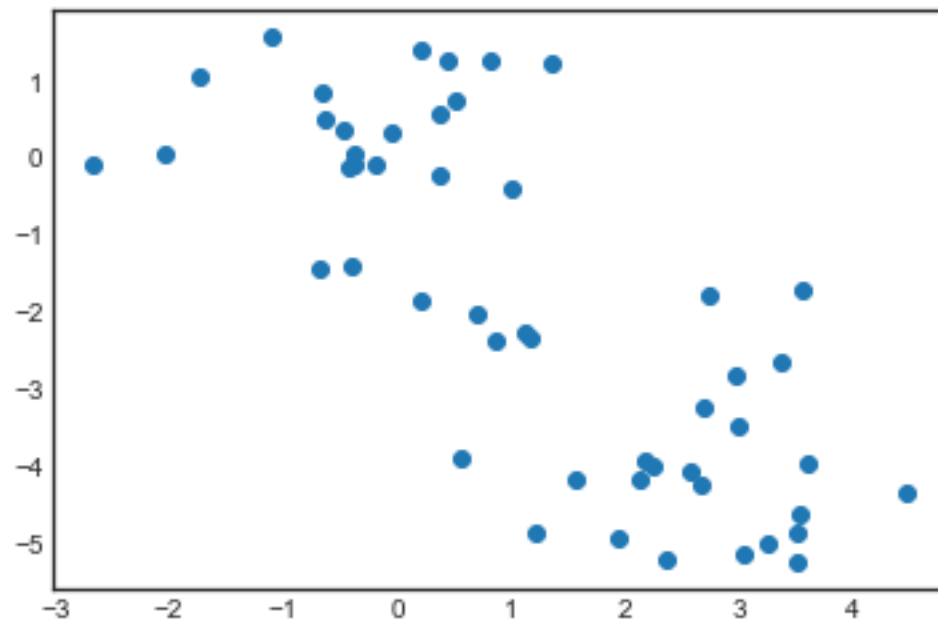
5.2.2 Thực hành với thư viện `sklearn.cluster.KMeans`

Ví dụ 5.3 (k -means). Minh họa phương pháp phân cụm dữ liệu bằng thuật toán k -means. Sử dụng thuật toán `KMeans` với thư viện `sklearn` trong Python.

```
[1]: import pandas as pd
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.cluster import KMeans
%matplotlib inline
plt.style.use('seaborn-white')
```

Ta sinh một bộ dữ liệu gồm 50 quan sát từ phân phối chuẩn và tạo ra hai nhóm dữ liệu: 25 quan sát đầu một nhóm, 25 quan sát sau một nhóm bằng cách dịch chuyển giá trị của các nhóm.

```
[2]: # Generate data
np.random.seed(2)
X = np.random.standard_normal((50, 2))
X[:25, 0] = X[:25, 0] + 3
X[:25, 1] = X[:25, 1] - 4
```



```
[3]: plt.scatter(X[:,0],X[:,1])
      plt.show()
```

Thực hiện thuật toán phân cụm k-means với số cụm $K = 2$. Kết quả phân cụm là nhãn của mỗi quan sát được gắn với nhãn của cụm.

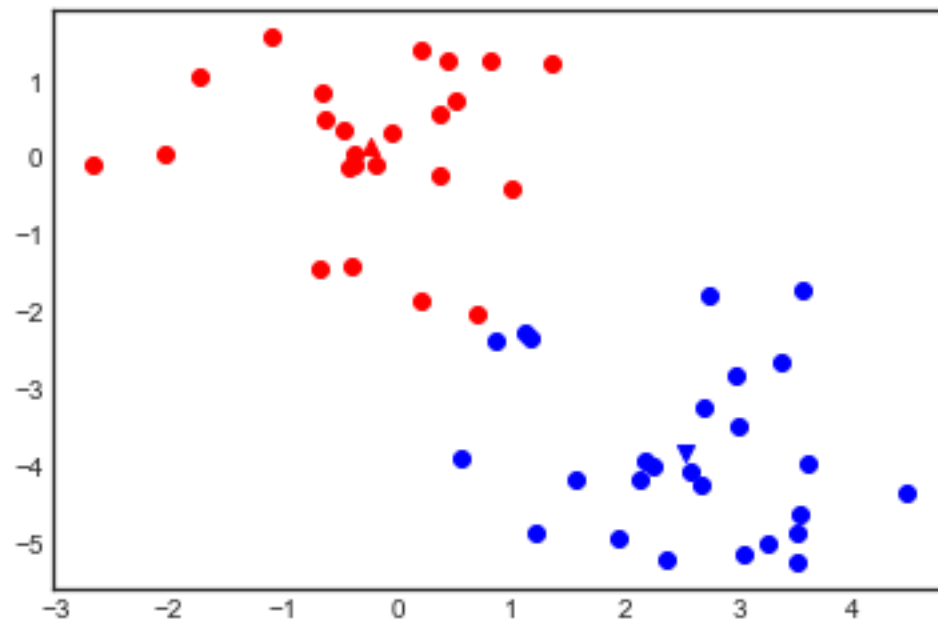
```
[4]: km = KMeans(n_clusters=2, n_init=20)
      km.fit(X)
      km.labels_
```

```
[4]: array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
          -1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
          -0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1])
```

Tâm (centers) của mỗi cụm được thể hiện bởi hình tam giác như trong hình vẽ dưới đây.

```
[5]: m=km.cluster_centers_
      print(m)
```

```
[[-0.23940153  0.14703307]
 [ 2.53287159 -3.80815122]]
```



```
[6]: C1=km.labels_==0
      C2=km.labels_==1
      plt.scatter(X[C1,0],X[C1,1],c='red')
      plt.scatter(X[C2,0],X[C2,1],c='blue')

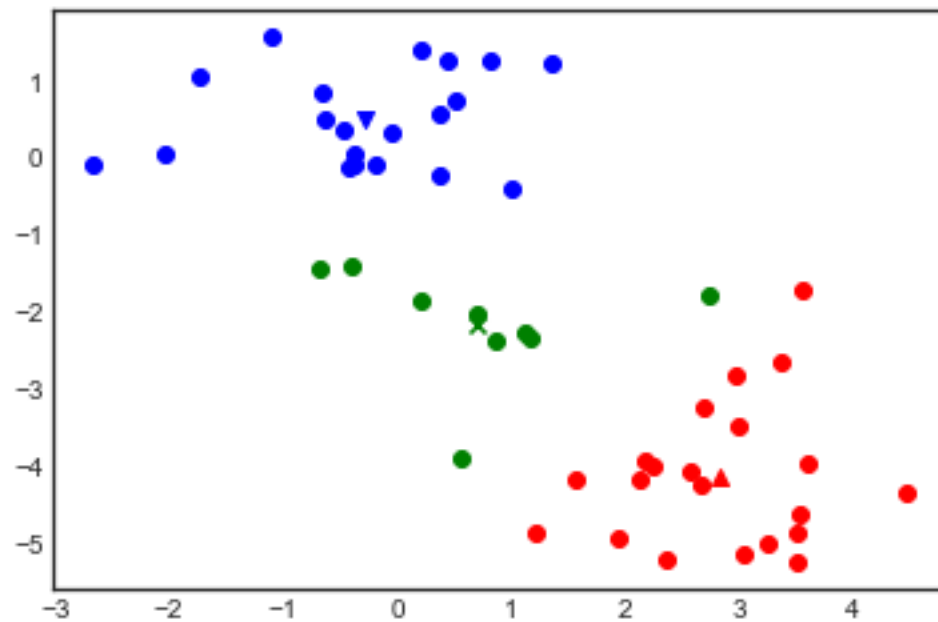
      plt.scatter(m[0][0],m[0,1],c='red', marker="^")
      plt.scatter(m[1][0],m[1,1],c='blue', marker="v")
      plt.show()
```

Đối với dữ liệu thực tế, ta chưa xác định được số cụm K , do đó ta thực hiện thuật toán phân cụm với nhiều số cụm khác nhau. Ví dụ dưới đây thực hiện thuật toán k-means với số cụm $K = 3$.

```
[7]: km3 = KMeans(n_clusters=3, n_init=20)
      km3.fit(X)
      km3.labels_
```

```
[7]: array([0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 2, 0, 0,
           0, 0, 2, 0, 0, 0, 1, 1, 1, 1, 2, 1, 1, 1, 2, 1, 1, 1, 1, 1, 2,
           1, 1, 2, 1, 1, 1, 1, 1, 1, 2])
```

```
[8]: m=km3.cluster_centers_
      print(m)
```



```
[[ 2.82805911 -4.11351797]
 [-0.27876523  0.51224152]
 [ 0.69945422 -2.14934345]]
```

```
[9]: C1=km3.labels_==0
      C2=km3.labels_==1
      C3=km3.labels_==2
      plt.scatter(X[C1,0],X[C1,1],c='red')
      plt.scatter(X[C2,0],X[C2,1],c='blue')
      plt.scatter(X[C3,0],X[C3,1],c='green')

      plt.scatter(m[0][0],m[0,1],c='red', marker="^")
      plt.scatter(m[1][0],m[1,1],c='blue', marker="v")
      plt.scatter(m[2][0],m[2,1],c='green', marker="x")

      plt.show()
```

5.2.3 Một số cải tiến thuật toán k -means

Fuzzy C-Means (FCM) Thuật toán k -means với mỗi điểm ta gán cứng vào một cụm dữ liệu. Đối với nhiều bài toán thì các điểm có thể gán vào nhiều cụm dữ liệu khác nhau

do sự chồng lấn của các cụm dữ liệu. Thuật toán FCM là một cải tiến của thuật toán k -means cho phép gán mỗi điểm dữ liệu theo một hàm thuộc (membership).

Giá trị hàm thuộc của điểm x_i đối với cụm C_j ký hiệu là u_{ij} có giá trị nằm trong khoảng $[0, 1]$ và có tổng bằng 1, nghĩa là

$$\sum_{j=1}^K u_{ij} = 1.$$

Thuật toán FCM cực tiểu hoá hàm mục tiêu

$$J = \sum_{j=1}^K \sum_{i=1}^n u_{ij}^m \|x_i - m_j\|^2, \quad (5.20)$$

trong đó tham số m là độ mờ hoá điều chỉnh độ mềm của phân cụm.

Tương tự như thuật toán K -means, tâm cụm dữ liệu được cập nhật theo công thức

$$m_j = \frac{\sum_{i=1}^n (u_{ij})^m x_i}{\sum_{i=1}^n (u_{ij})^m},$$

và giá trị hàm thuộc được cập nhật theo công thức dưới đây

$$u_{ij} = \frac{1}{\sum_{k=1}^K \left(\frac{\|x_i - m_j\|}{\|x_i - m_k\|} \right)^{\frac{2}{m-1}}}.$$

Thuật toán FCM dừng khi đạt các điều kiện về tiêu chuẩn dừng, tức là khi các tâm không thay đổi, các giá trị hàm thuộc không thay đổi, hoặc số vòng lặp đạt tối đa.

Ngoài thuật toán FCM phải kể đến một số thuật toán phân cụm dữ liệu như: K -Medoids, K -Medians, K -Modes, X -Means, Intelligent K -Means, Bisecting K -Means, Kernel K -Means, Mean Shift Clustering, Weighted K -Means, Genetic K -Means, Making K -Means Faster.

5.3 Phân cụm bằng phân tầng

5.3.1 Phương pháp gộp phân tầng

Phân cụm bằng phân tầng (Hierarchical clustering, hoặc hierarchical cluster analysis HCA) là phương pháp phân cụm mà các nhóm có cấu trúc phân tầng của dữ liệu đã cho. Cấu trúc phân tầng của cụm tạo ra các nhóm mà trong đó mỗi nhóm ở một mức (level)

là hợp của các nhóm với mức thấp hơn tiếp theo. Ở mức thấp nhất là các nhóm đơn gồm có 1 quan sát và mức cao nhất bao gồm tập hợp tất cả dữ liệu. Trong phương pháp phân cụm bằng phân tầng ta cần xác định độ tương đồng giữa hai nhóm để xác định xem hai nhóm đó có được hợp nhất hay không.

Biểu đồ phân cấp (Dendrogram) Phân cụm bằng phân tầng được biểu diễn bằng cây nhị phân. Các nhóm của dữ liệu tương ứng với các nốt của cây. Nút gốc chứa toàn bộ dữ liệu, nút lá của cây chỉ chứa đúng 1 điểm dữ liệu, và các nút không phải nút lá có hai nút con. Khoảng cách giữa các nhóm được hợp nhất là hàm đơn điệu đối với mức của sự hợp nhất. Biểu đồ phân cấp là một sơ đồ biểu diễn cây để sắp xếp các nhóm dữ liệu theo cấu trúc phân tầng.

Khoảng cách giữa hai nhóm Cho A và B là hai nhóm dữ liệu, với $x \in A$ và $y \in B$, ký hiệu $d(x, y)$ là khoảng cách giữa hai điểm x, y . Ta ký hiệu $d(A, B)$ là khoảng cách giữa hai tập A và B và thường được sử dụng các khoảng cách sau trong phân cụm bằng phân tầng

1. **complete-linkage** là khoảng cách lớn nhất giữa các điểm của A và B

$$d(A, B) = \max\{d(x, y) : x \in A, y \in B\}.$$

2. **single-linkage** là khoảng cách nhỏ nhất giữa các điểm của A và B .

$$d(A, B) = \min\{d(x, y) : x \in A, y \in B\}.$$

3. **average-linkage** là khoảng cách trung bình tất cả các khoảng cách giữa các điểm của A và B

$$d(A, B) = \frac{1}{|A||B|} \sum_{x \in A, y \in B} d(x, y).$$

4. **centroid-linkage** là khoảng cách giữa hai tâm của nhóm A và nhóm B

$$d(A, B) = d\left(\frac{1}{|A|} \sum_{x \in A} x, \frac{1}{|B|} \sum_{y \in B} y\right).$$

Có hai chiến lược xây dựng phân cụm bằng phân tầng:

1. Phương pháp gộp (agglomerative hoặc bottom-up),
2. Phương pháp chia (divisive hoặc top-down).

Giả sử ta có tập dữ liệu gồm n quan sát $x_1, x_2, \dots, x_n$.

Phương pháp gộp Phương pháp gộp xuất phát từ n nhóm, mỗi nhóm gồm 1 quan sát. Mức tiếp theo là ta gộp hai nhóm có khoảng cách nhỏ nhất hợp thành một nhóm, như vậy ta có $n - 1$ nhóm. Tiếp theo ta lại gộp hai nhóm ở mức cao hơn có khoảng cách nhỏ nhất và ta có $n - 2$ nhóm. Thuật toán dừng khi ta thu được 1 nhóm gồm toàn bộ dữ liệu.

Bước 1 Coi mỗi điểm là 1 cụm dữ liệu. Ta có $C_i = x_i$ cụm dữ liệu với $1 \leq i \leq n$.

Bước 2 Tính khoảng cách giữa các cụm dữ liệu.

Bước 3 Tìm hai cụm có khoảng cách nhỏ nhất C_i và C_j .

Bước 4 Hợp nhất hai cụm gần nhất thành cụm mới $C_{new} = C_i \cup C_j$.

Bước 5 Cập nhật khoảng cách giữa các cụm dữ liệu.

Bước 6 Kiểm tra điều kiện dừng phù hợp.

Bước 7 Nếu không dừng, quay lại Bước 3.

Clustering algorithms	α_i	α_j	β	γ
Single linkage (nearest neighbor)	1/2	1/2	0	-1/2
Complete linkage (farthest neighbor)	1/2	1/2	0	1/2
Group average linkage (UPGMA)	$\frac{n_i}{n_i+n_j}$	$\frac{n_j}{n_i+n_j}$	0	0
Weighted average linkage (WPGMA)	1/2	1/2	0	0
Median linkage (WPGMC)	1/2	1/2	-1/4	0
Centroid linkage (UPGMC)	$\frac{n_i}{n_i+n_j}$	$\frac{n_j}{n_i+n_j}$	$\frac{-n_i n_j}{(n_i+n_j)^2}$	0
Ward's method	$\frac{n_i+n_l}{n_i+n_j+n_l}$	$\frac{n_j+n_l}{n_i+n_j+n_l}$	$\frac{-n_l}{n_i+n_j+n_l}$	0

Trong Bước 5 cập nhật khoảng cách giữa các cụm dữ liệu ta có một cụm mới $C_{ij} = C_i \cup C_j$ và ta cần tính khoảng cách từ các cụm khác đến cụm dữ liệu mới. Khoảng cách từ cụm C_k đến cụm mới C_{ij} được xác định theo công thức dưới đây với các hệ số được cho trong bảng trên.

$$D(C_k, C_{ij}) = \alpha_i D(C_i, C_k) + \alpha_j D(C_j, C_k) + \beta D(C_i, C_j) + \gamma |D(C_i, C_k) - D(C_j, C_k)|. \quad (5.21)$$

single linkage

$$D(C_k, C_{ij}) = \min\{D(C_k, C_i), D(C_k, C_j)\}$$

complete linkage

$$D(C_k, C_{ij}) = \max\{D(C_k, C_i), D(C_k, C_j)\}$$

group average linkage

$$D(C_k, C_{ij}) = \frac{D(C_k, C_i) + D(C_k, C_j)}{2}$$

weighted average linkage

$$D(C_k, C_{ij}) = \frac{n_i}{n_i + n_j} D(C_k, C_i) + \frac{n_j}{n_i + n_j} D(C_k, C_j)$$

centroid linkage

$$D(C_k, C_{ij}) = \frac{n_i}{n_i + n_j} D(C_i, C_k) + \frac{n_j}{n_i + n_j} D(C_j, C_k) - \frac{n_i n_j}{(n_i + n_j)^2} D(C_i, C_j)$$

median linkage

$$D(C_k, C_{ij}) = \frac{1}{2} D(C_i, C_k) + \frac{1}{2} D(C_j, C_k) - \frac{1}{4} D(C_i, C_j)$$

Ward's method

$$D(C_k, C_{ij}) = \frac{n_i + n_k}{n_i + n_j + n_k} D(C_i, C_k) + \frac{n_j + n_k}{n_i + n_j + n_k} D(C_j, C_k) - \frac{n_k}{n_i + n_j + n_k} D(C_i, C_j)$$

Ví dụ 5.4. Cho dữ liệu gồm 5 điểm $x_1 = (1, 2)$, $x_2 = (1, 2.5)$, $x_3 = (3, 1)$, $x_4 = (4, 0.5)$, $x_5 = (4, 2)$. Thực hiện phân cụm dữ liệu trên bằng phương pháp phân tầng với phương pháp gộp với khoảng cách single linkage.

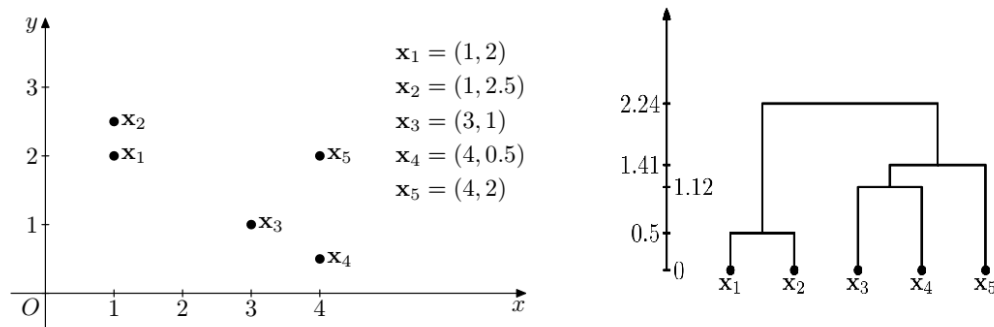
Trước hết ta tính khoảng cách giữa các điểm của dữ liệu:

Distance	x_1	x_2	x_3	x_4	x_5
x_1	0	0.5	2.24	3.35	3
x_2	0.5	0	2.5	3.61	3.04
x_3	2.24	2.5	0	1.12	1.41
x_4	3.35	3.61	1.12	0	1.5
x_5	3	3.04	1.41	1.5	0

Giá trị nhỏ nhất giữa x_1 và x_2 là nhỏ nhất nên ta gộp 2 điểm này thành một cụm $\{x_1, x_2\}$.

Tiếp theo ta tính khoảng cách từ cụm này đến các điểm x_3, x_4, x_5 theo khoảng cách nhỏ nhất đến các điểm và ta có ma trận khoảng cách mới như sau:

Distance	$\{x_1, x_2\}$	$\{x_3\}$	$\{x_4\}$	$\{x_5\}$
$\{x_1, x_2\}$	0	2.24	3.35	3
$\{x_3\}$	2.24	0	1.12	1.41
$\{x_4\}$	3.35	1.12	0	1.5
$\{x_5\}$	3	1.41	1.5	0



Khoảng cách nhỏ nhất là khoảng cách giữa x_3 và x_4 nên ta gộp hai điểm này thành cụm $\{x_3, x_4\}$. Cập nhật khoảng cách của cụm này với các cụm $\{x_1, x_2\}$, x_3 ta được ma trận khoảng cách mới như sau:

Distance	$\{x_1, x_2\}$	$\{x_3, x_4\}$	$\{x_5\}$
$\{x_1, x_2\}$	0	2.24	3
$\{x_3, x_4\}$	2.24	0	1.41
$\{x_5\}$	3	1.41	0

Khoảng cách nhỏ nhất là giữa $\{x_3, x_4\}$ với x_5 nên ta gộp hai cụm này thành cụm mới $\{x_3, x_4, x_5\}$, cập nhật khoảng cách hai cụm này như sau:

Distance	$\{x_1, x_2\}$	$\{x_3, x_4, x_5\}$
$\{x_1, x_2\}$	0	2.24
$\{x_3, x_4, x_5\}$	2.24	0

Bước cuối cùng ta gộp hai cụm còn lại thành cụm $\{x_1, x_2, x_3, x_4, x_5\}$.

5.3.2 Phương pháp chia phân tầng

Phương pháp chia Phương pháp chia xuất phát tập gốc gồm toàn bộ dữ liệu. Mức độ thấp hơn ta chia tập này thành hai tập con sao cho hai tập con này có khoảng cách lớn nhất. Tiếp theo mỗi nút con lại được chia thành hai tập con. Thuật toán dừng khi mỗi tập con có đúng 1 phần tử.

Ta có thể sử dụng thuật toán k -means với $K = 2$ để chia nhóm dữ liệu thành hai nhóm con có sai số bé hơn. Tuy nhiên ta có thể áp dụng thuật toán DIANA (DIVISIVE ANALYSIS) như sau:

Bước 1 Giả sử ta cần chia nhóm hiện tại C thành hai nhóm C_l, C_r .

Bước 2 Ta đặt $C_l = C$ và $C_r = \emptyset$.

Bước 3 Với mỗi điểm $x \in C_l$:

1. Tính khoảng cách trung bình từ x đến tập C_l và C_r là $d_{mean}(x, C_l), d_{mean}(x, C_r)$ tương ứng,
2. Tính khoảng cách $D_x = d_{mean}(x, C_l) - d_{mean}(x, C_r)$.

Bước 4

1. Với bước lặp đầu tiên thì ta chuyển điểm x có D_x lớn nhất sang tập C_r .
2. Các vòng lặp tiếp theo ta tìm điểm x và nếu $D_x > 0$ thì di chuyển x sang tập C_r .

Bước 5 Lặp lại Bước 3.2 và Bước 4.2 đến khi tất cả các giá trị $D_x < 0$. Nhóm C chia thành hai nhóm C_l và C_r .

Bước 6 Chọn nhóm có đường kính lớn nhất (đường kính là khoảng cách lớn nhất giữa các điểm trong nhóm) và chia nhóm đó theo các Bước 1 đến Bước 5.

Bước 7 Lặp lại Bước 6 cho đến khi mỗi nhóm còn lại một phần tử.

Ví dụ 5.5. Cho dữ liệu gồm 5 điểm $x_1 = (1, 2), x_2 = (1, 2.5), x_3 = (3, 1), x_4 = (4, 0.5), x_5 = (4, 2)$. Thực hiện phân cụm dữ liệu trên bằng phương pháp phân tầng với phương pháp chia.

Trước hết ta tính khoảng cách giữa các điểm của dữ liệu:

Distance	x_1	x_2	x_3	x_4	x_5
x_1	0	0.5	2.24	3.35	3
x_2	0.5	0	2.5	3.61	3.04
x_3	2.24	2.5	0	1.12	1.41
x_4	3.35	3.61	1.12	0	1.5
x_5	3	3.04	1.41	1.5	0

Khởi tạo ta có cụm dữ liệu $C = \{x_1, x_2, x_3, x_4, x_5\}$. Do có 1 cụm nên ta tách cụm này bằng cách tìm điểm xa nhất nhất cụm.

$$D_{x_1} = d_{mean}(x_1, C) = (0.5 + 2.24 + 3.35 + 3)/4 = 9.09/4 = 2.2725$$

$$D_{x_2} = d_{mean}(x_2, C) = (0.5 + 2.5 + 3.61 + 3.04)/4 = 9.65/4 = 2.4125$$

$$D_{x_3} = d_{mean}(x_3, C) = (2.24 + 2.5 + 1.12 + 1.41)/4 = 7.27/4 = 1.8175$$

$$D_{x_4} = d_{mean}(x_4, C) = (3.35 + 3.61 + 1.12 + 1.5)/4 = 9.58/4 = 2.395$$

$$D_{x_5} = d_{mean}(x_5, C) = (3 + 3.04 + 1.41 + 1.5)/4 = 8.95/4 = 2.2375$$

Giá trị lớn nhất là D_{x_2} nên ta có 2 cụm là $C_l = \{x_1, x_3, x_4, x_5\}$ và $C_r = \{x_2\}$.

Tiếp theo với mỗi điểm $x_i \in C_l$ ta tính giá trị D_{x_i} tương ứng

$$D_{x_1} = d_{mean}(x_1, C_l) - d_{mean}(x_1, C_r) = (2.24 + 3.35 + 3)/3 - 0.5 = 2.3633 > 0,$$

do đó ta chuyển x_1 sang C_r . Vậy ta cập nhật cụm tiếp theo $C_l = \{x_3, x_4, x_5\}$ và $C_r = \{x_2, x_1\}$.

Xét các điểm x_3, x_4, x_5 . Ta tính các giá trị D_{x_i} tương ứng và ta tính được

$$D_{x_3} = d_{mean}(x_3, C_l) - d_{mean}(x_3, C_r) = 1.265 - 2.37 < 0,$$

$$D_{x_4} = d_{mean}(x_4, C_l) - d_{mean}(x_4, C_r) = 1.31 - 3.48 < 0,$$

$$D_{x_5} = d_{mean}(x_5, C_l) - d_{mean}(x_5, C_r) = 1.455 - 3.02 < 0.$$

Vậy ta được 2 cụm dữ liệu mới là $C_1 = \{x_3, x_4, x_5\}$, $C_2 = \{x_2, x_1\}$.

Chọn cụm $C_1 = \{x_3, x_4, x_5\}$, tiếp tục quá trình chia tách bằng

$$d(x_3, C_1) = (d(x_3, x_4) + d(x_3, x_5))/2 = (1.12 + 1.41)/2 = 1.265$$

$$d(x_4, C_1) = (d(x_4, x_3) + d(x_4, x_5))/2 = (1.12 + 1.5)/2 = 1.31$$

$$d(x_5, C_1) = (d(x_5, x_3) + d(x_5, x_4))/2 = (1.41 + 1.5)/2 = 1.455$$

Điểm xa nhất là x_5 nên ta chia thành 2 cụm $C_l = \{x_3, x_4\}$ và $C_r = \{x_5\}$. Xét các điểm x_3, x_4 và tính D_{x_i} tương ứng

$$D_{x_3} = d_{mean}(x_3, C_l) - d_{mean}(x_3, C_r) = 1.12 - 1.41 < 0,$$

$$D_{x_4} = d_{mean}(x_4, C_l) - d_{mean}(x_4, C_r) = 1.12 - 1.5 < 0,$$

Vậy cụm $C_1 = \{x_3, x_4, x_5\}$ tách thành hai cụm mới $\{x_3, x_4\}$ và $\{x_5\}$. Sau bước này ta có 3 cụm $C_1 = \{x_3, x_4\}$, $C_2 = \{x_5\}$, $C_3 = \{x_1, x_2\}$. Nếu cần tiếp tục ta lại tách các cụm trên thành các cụm mới $\{x_3\}$, $\{x_4\}$, $\{x_5\}$, $\{x_1\}$, $\{x_2\}$.

5.3.3 Một số phương pháp khác phân cụm bằng phân tầng

Những chỉ trích thường thấy về các thuật toán phân cụm phân cấp cổ điển tập trung vào sự thiếu mạnh mẽ (robustness) và độ nhạy cảm của chúng đối với nhiễu và các điểm ngoại lai. Một khi một đối tượng đã được gán cho một cụm, nó sẽ không được xem xét lại, điều này có nghĩa là các thuật toán phân cụm phân cấp không có khả năng sửa chữa các phân loại sai có thể xảy ra trước đó. Một nhược điểm khác của các thuật toán

này là độ phức tạp tính toán của chúng, ít nhất là $O(n^2)$ và không đáp ứng được yêu cầu xử lý các bộ dữ liệu quy mô lớn trong khai thác dữ liệu và các tác vụ khác trong những năm gần đây. Do những yêu cầu như vậy, nhiều phương pháp phân cụm mới, với kết quả phân cụm phân cấp, đã xuất hiện và cải thiện đáng kể hiệu suất phân cụm. Các ví dụ điển hình bao gồm BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies, Zhang và cộng sự, 1996), CURE (Balanced Iterative Reducing and Clustering using Hierarchies, Guha và cộng sự, 1998), ROCK (RObust Clustering using linKS, Guha và cộng sự, 2000) và Chameleon (Karypis và cộng sự, 1999).

5.3.4 Thực hành với thư viện `scipy.cluster.hierarchy`

Ví dụ 5.6. Thực hiện phương pháp gộp cho phân cụm bằng phương pháp phân tầng và vẽ biểu đồ phân cấp cho mỗi trường hợp phân cụm dữ liệu.

```
[1]: import pandas as pd
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from scipy.cluster import hierarchy
from scipy.cluster.hierarchy import linkage
from scipy.cluster.hierarchy import fcluster
%matplotlib inline
plt.style.use('seaborn-white')
```

Dữ liệu **USArrests** được thực hiện biến đổi dữ liệu thông qua phép biến đổi **StandardScaler()** hay còn gọi là phép biến đổi z-score chuyển dữ liệu có trung bình 0 và phương sai bằng 1 của mỗi cột dữ liệu.

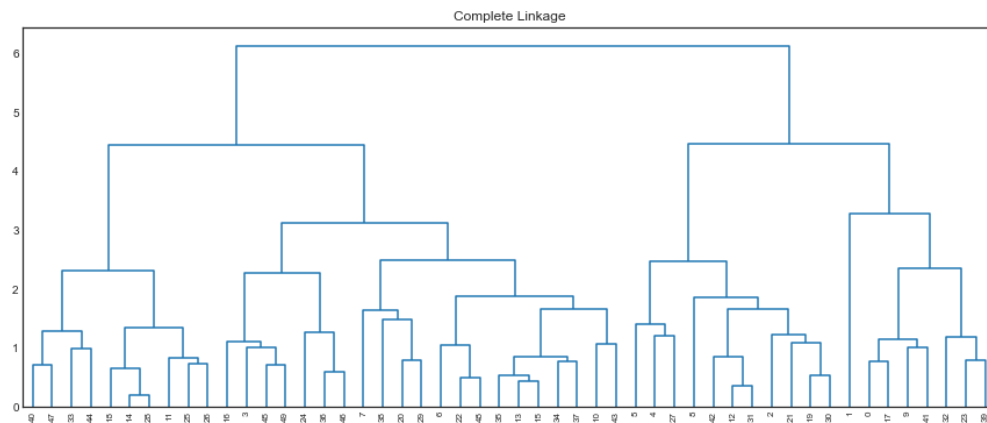
```
[2]: USArrests = pd.read_csv('./data/USArrests.csv', index_col=0)
sc = StandardScaler()
X = pd.DataFrame(sc.fit_transform(USArrests), index=USArrests.
    -index, columns=USArrests.columns)
```

```
[3]: X.shape
```

```
[3]: (50, 4)
```

Thực hiện phương pháp phân cụm phân tầng với phương pháp **complete-link**.

```
[4]: #hierarchy.complete
fig, (ax) = plt.subplots(1,1, figsize=(15,6))
linkage=hierarchy.complete(X)
cluster = hierarchy.dendrogram(linkage, ax=ax,
    -color_threshold=0)
plt.title('Complete Linkage')
plt.show()
```



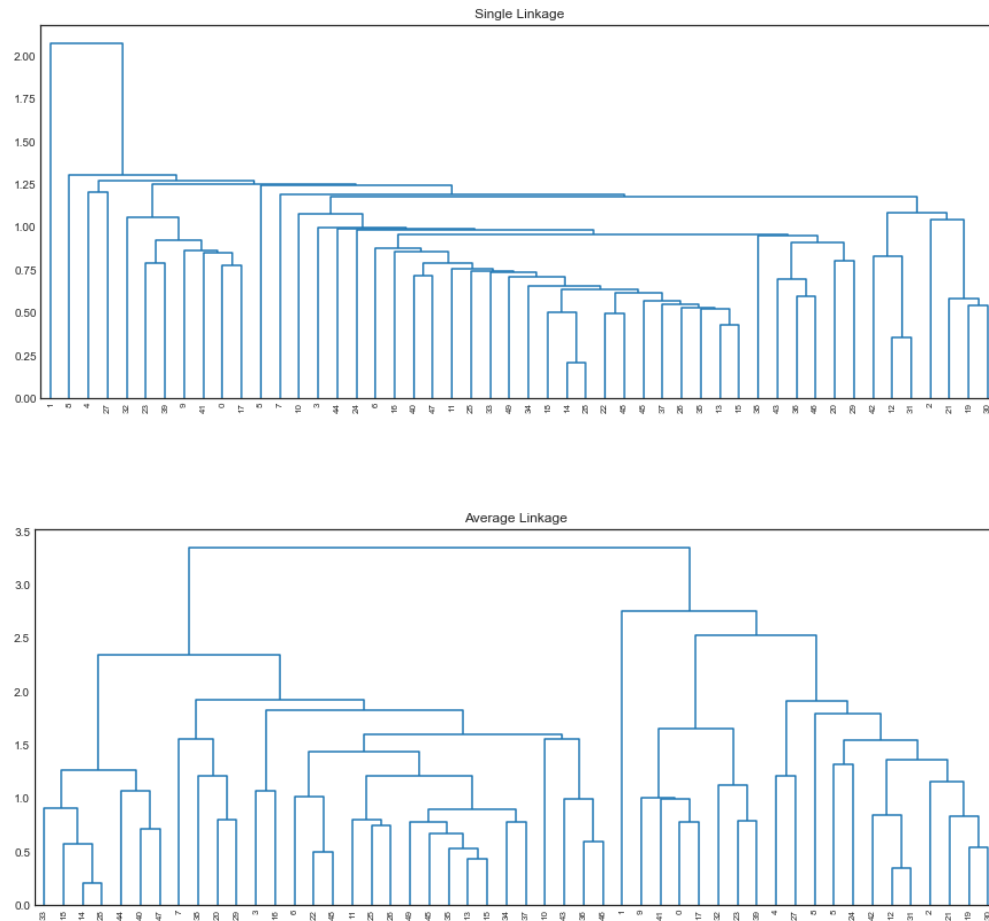
Thực hiện phương pháp phân cụm phân tầng với phương pháp **single-link**.

```
[5]: #hierarchy.single
fig, (ax) = plt.subplots(1,1, figsize=(15,6))
linkage=hierarchy.single(X)
cluster = hierarchy.dendrogram(linkage, ax=ax,
    -color_threshold=0)
plt.title('Single Linkage')
plt.show()
```

Thực hiện phương pháp phân cụm phân tầng với phương pháp **average-link**.

```
[6]: #hierarchy.average
fig, (ax) = plt.subplots(1,1, figsize=(15,6))
linkage=hierarchy.average(X)
cluster = hierarchy.dendrogram(linkage, ax=ax,
    -color_threshold=0)
plt.title('Average Linkage')
plt.show()
```

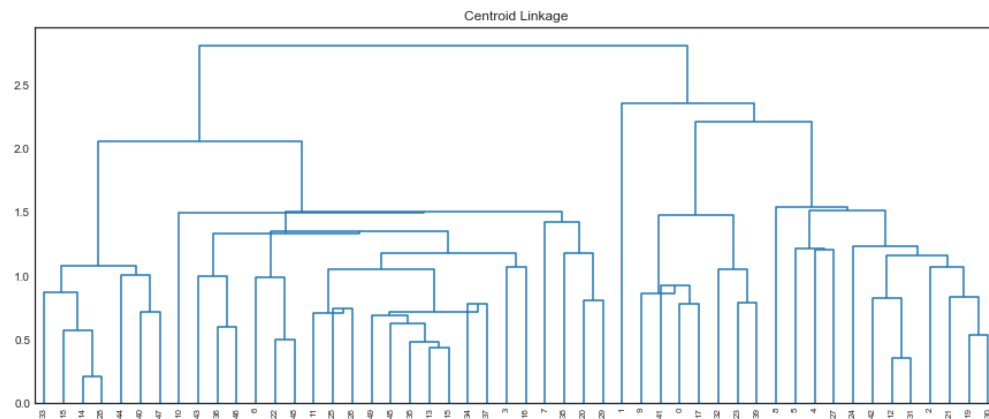
Thực hiện phương pháp phân cụm phân tầng với phương pháp **centroid-link**.



```
[7]: #hierarchy.centroid
fig, (ax) = plt.subplots(1,1, figsize=(15,6))
linkage=hierarchy.centroid(X)
cluster = hierarchy.dendrogram(linkage, ax=ax,
    -color_threshold=0)
plt.title('Centroid Linkage')
plt.show()
```

Thể hiện đường mức cắt ngang biểu đồ phân cấp để xác định số cụm dữ liệu. Với mức khoảng cách bằng 4 xác định được $K = 4$ cụm dữ liệu. Với mức khoảng cách bằng 2.7 ta xác định được $K = 6$ cụm dữ liệu.

```
[8]: plt.figure(figsize=(15,8))
cut = hierarchy.dendrogram(hierarchy.complete(X),
    labels=X.index,
```

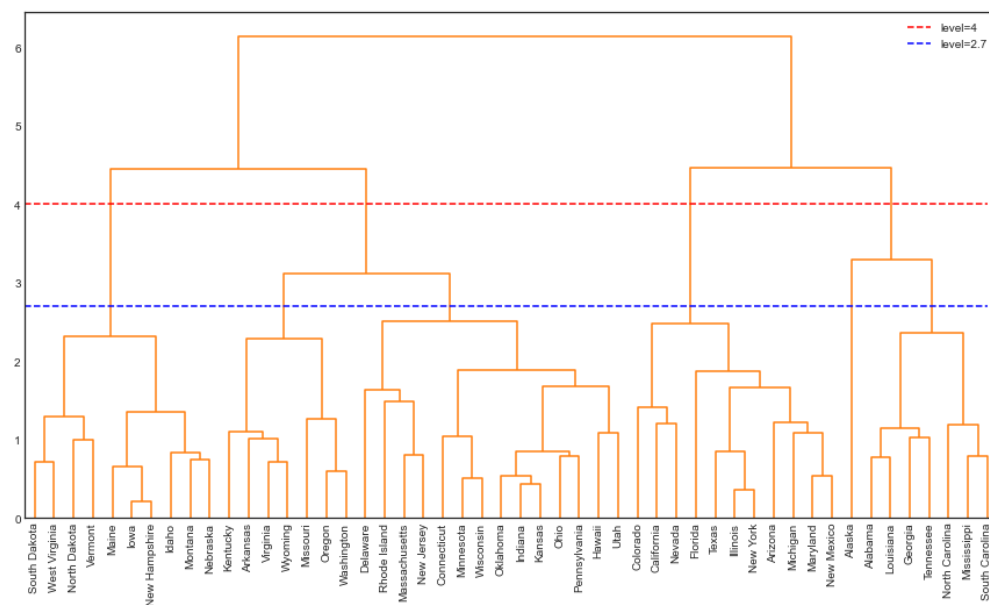


```

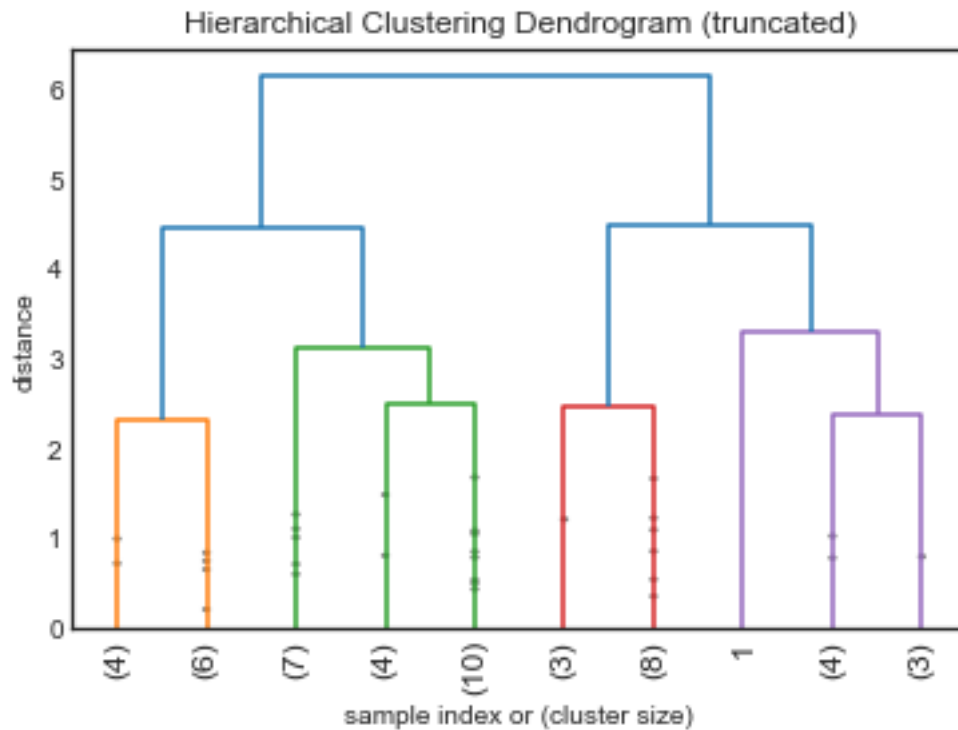
                                color_threshold=140,
    -leaf_font_size=10)
plt.hlines(4.0,0,plt.gca().xaxis.get_data_interval()[1],
          colors='r', linestyle='dashed',label='level=4')

plt.hlines(2.7,0,plt.gca().xaxis.get_data_interval()[1],
          colors='blue', linestyle='dashed',label='level=2.7')
plt.legend()
plt.show()

```



Khi thể hiện biểu đồ phân cấp của cây phân tầng, ta có thể sử dụng kỹ thuật cắt gọn thể hiện số lớp con p theo tham số `truncate_mode`.



```
[9]: plt.title('Hierarchical Clustering Dendrogram (truncated)')
plt.xlabel('sample index or (cluster size)')
plt.ylabel('distance')
hierarchy.dendrogram(
    hierarchy.complete(X),
    truncate_mode='lastp',
    p=10,
    leaf_rotation=90.,
    leaf_font_size=12.,
    show_contracted=True,
)
plt.show()
```

Ma trận Z là ma trận cỡ $(n-1) \times 4$ thể hiện các thông tin về chỉ số cụm thứ nhất trước khi gộp, chỉ số cụm thứ hai trước khi gộp, khoảng cách, và số điểm sau khi gộp. Số các cụm ban đầu của dữ liệu được đánh số từ 0 đến $n-1$, các thứ i gộp mới có chỉ số cụm là $n+i$. Thứ tự lá cho biết thứ tự hiển thị từ trái sang phải (thứ tự các nút là được xác định theo thuật toán duyệt cây nhị phân theo phương pháp duyệt trung tự - In-order traversal).

```
[10]: Z=hierarchy.complete(X)
      Z[:5]
```

```
[10]: array([[14.      , 28.      , 0.2079438 , 2.      ],
          [12.      , 31.      , 0.35377437, 2.      ],
          [13.      , 15.      , 0.43312429, 2.      ],
          [22.      , 48.      , 0.49909939, 2.      ],
          [35.      , 52.      , 0.53571007, 3.      ]])
```

Giá trị khoảng cách nhỏ nhất được gộp 2 điểm dữ liệu gần nhau nhất, giá trị khoảng cách lớn nhất là gộp tất cả dữ liệu vào một cụm.

```
[11]: print(Z[:,2].min(), Z[:,2].max())
```

```
0.2079437976133826 6.138334936847244
```

Để cắt biểu đồ phân cấp thành K cụm, dùng `fcluster(Z, t=K, criterion='distance')`.

```
[12]: fcluster(Z,t=4,criterion='distance')
```

```
[12]: array([4, 4, 3, 2, 3, 3, 2, 2, 3, 4, 2, 1, 3, 2, 1, 2, 2, 4, 1,
          -3, 2, 3, 2, 4, 2, 1, 1, 3, 1, 2, 3, 3, 4, 1, 2, 2, 2, 2, 2, 4,
          -1, 4, 3, 2, 1, 2, 2, 1, 2, 2], dtype=int32)
```

5.4 Bài tập

Bài 5.1 (Fuzzy K-means). a) Thực hiện thuật toán Fuzzy K-means đối với dữ liệu Wine `sklearn.datasets.load_wine()`.

b) So sánh kết quả phân cụm dữ liệu với dữ liệu gốc bằng `pandas.crosstab()`.

Bài 5.2 (DIANA). a) Thực hiện thuật toán DIANA đối với dữ liệu Iris `sklearn.datasets.load_iris()`.

b) So sánh kết quả phân cụm dữ liệu với dữ liệu gốc bằng `pandas.crosstab()`.

Bài 5.3 (Spectral clustering). a) Hãy viết một báo cáo ngắn giải thích các khái niệm cốt lõi của Spectral Clustering.

b) Thực hiện phân tích cụm bằng phương pháp Spectral Clustering với dữ liệu cụ thể.

Bài 5.4 (DBSCAN). a) Hãy viết một báo cáo ngắn giải thích các khái niệm cốt lõi của DBSCAN.

b) Thực hiện phân tích cụm bằng phương pháp DBSCAN với dữ liệu cụ thể.

Bài 5.5 (Gaussian Mixture). a) Hãy viết một báo cáo ngắn giải thích các khái niệm cốt lõi của Gaussian Mixture.

b) Thực hiện phân tích cụm bằng phương pháp Gaussian Mixture với dữ liệu cụ thể.

5.5 Thực hành

Thực hành 5.1. Thực hành phân tích cụm bằng thuật toán K-means với dữ liệu **Wine** từ thư viện `sklearn.datasets.load_wine()`.

a) Thực hiện thuật toán K-means với số cụm $K = 3$.

b) Đánh giá kết quả phân cụm bằng cách so sánh với nhãn lớp thực tế.

c) Tính các hệ số Davis-Bouldin, Dunn, Silhouette với số cụm $K = 3$.

Thực hành 5.2. Thực hành phương pháp phân cụm phân tầng với dữ liệu **NCI60**.

a) Sử dụng hệ số tương quan Pearson để xác định độ tương đồng giữa hai mẫu.

b) Thực hiện thuật toán phương pháp gộp với các độ đo khác nhau.

c) Vẽ sơ đồ biểu đồ phân cấp với mỗi phương pháp.

d) Xác định số cụm K phù hợp.

Thực hành 5.3. a) Sử dụng phương pháp K-means đối với dữ liệu **USArrests** với số nhóm K phù hợp.

b) Thực hiện phương pháp phân tích thành phần chính PCA đối với dữ liệu **USArrests**.

c) Trực quan hoá các nhóm dữ liệu đối với số thành phần chính $K = 2$.

Thực hành 5.4 (Phân cụm khách hàng). Dữ liệu **Online Retail.xlsx** bao gồm các giao dịch của một cửa hàng bán lẻ online từ 1.12.2010 đến 9.12.2011. Thực hiện thuật toán phân cụm dữ liệu để phân tích các nhóm khách hàng khác nhau.

Mô hình **RFM** (Recency, Frequency, Monetary) là một kỹ thuật phân tích hành vi khách hàng được sử dụng rộng rãi trong tiếp thị, đặc biệt là tiếp thị quan hệ và phân khúc khách hàng.

R Recency là khoảng thời gian từ lần mua hàng cuối cùng của khách hàng đến ngày phân tích. Giá trị R càng thấp (mua gần đây) thì khách hàng càng tốt.

F Frequency là tổng số lần khách hàng mua hàng trong một khoảng thời gian xác định. Giá trị F càng cao (mua thường xuyên) thì khách hàng càng tốt.

M Monetary là tổng số tiền khách hàng đã chi tiêu trong khoảng thời gian xác định. Giá trị M càng cao thì khách hàng càng tốt.

- a) Xây dựng bảng dữ liệu bằng cách sử dụng mô hình **RFM**.
- b) Thực hiện chuẩn hoá dữ liệu.
- c) Thực hành phương pháp K-means đối với dữ liệu chuẩn hoá và xác định số cụm K phù hợp.
- d) Thực hành phương pháp phân tầng và xác định số cụm phù hợp.
- e) So sánh hai phương pháp phân cụm trên.

Chương 6

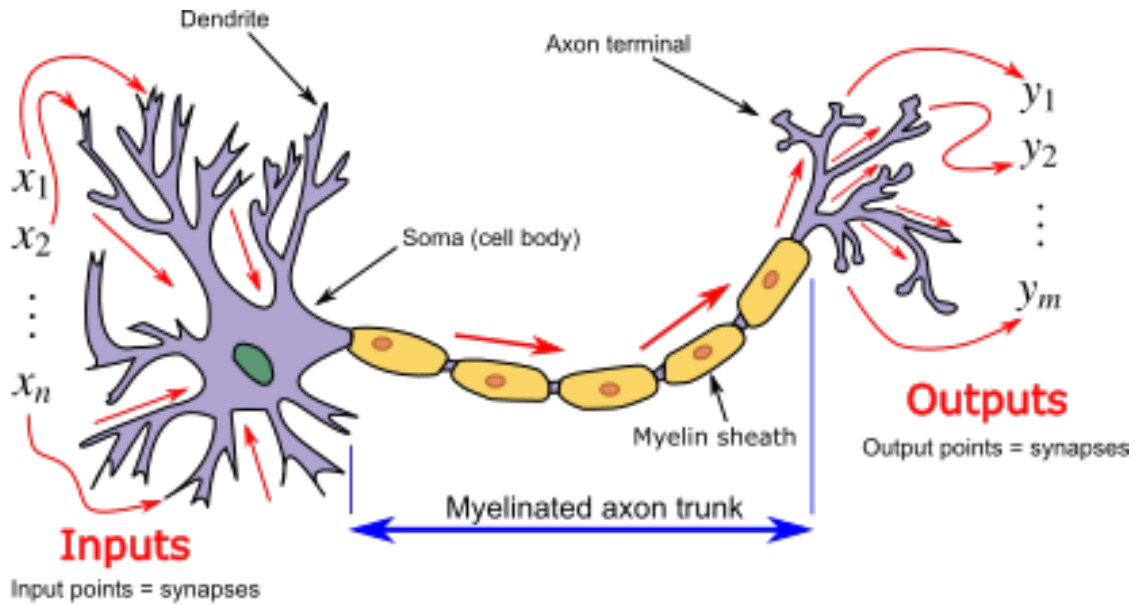
MẠNG NEURON

6.1 Mô hình mạng neuron

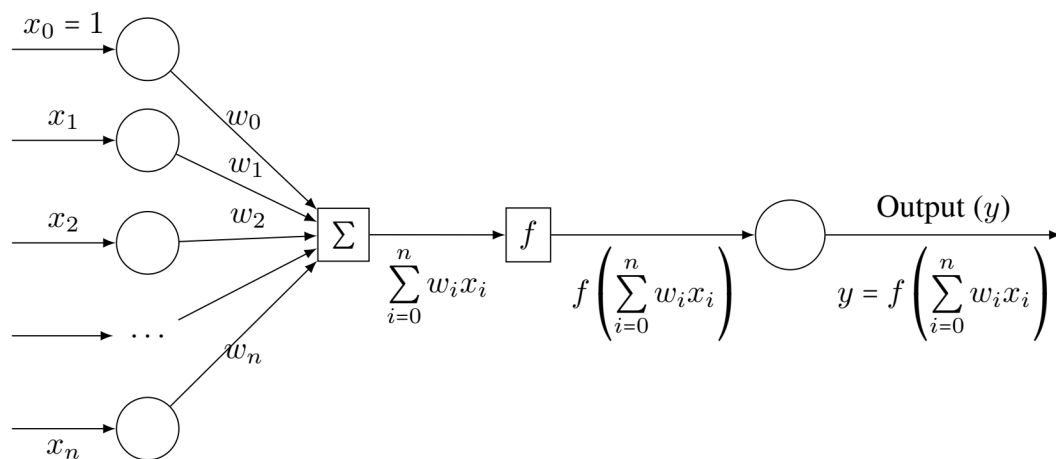
Mạng neuron nhân tạo được mô hình hoá của mạng thần kinh trong sinh học. Trong mỗi tế bào (cell), khi tín hiệu đến được nhận bởi các nhánh (dendrites) thông qua quá trình sinh hoá. Quá trình này cho phép thúc đẩy theo mức độ quan trọng hoặc theo tần suất. Các tín hiệu tích lũy đạt đến một ngưỡng nào đó sẽ kích hoạt và tín hiệu được truyền qua một quá trình điện hoá xuống các sợi trục (axon). Tại các đầu cuối của các sợi trục, tín hiệu điện một lần nữa được xử lý dưới dạng tín hiệu điện hoá để truyền đến các tế bào thần kinh lân cận qua một hệ thống các khớp nối (synapse).

Trong sơ đồ của mạng thần kinh sinh học các tín hiệu vào và các tín hiệu ra được mô tả bằng sơ đồ như trong hình vẽ trên. Mạng thần kinh nhân tạo là mô hình toán học của mạng thần kinh sinh học trong đó:

1. $x_1, x_2, \dots, x_n$ là các tín hiệu đầu vào,
2. $w_1, w_2, \dots, w_n$ là các trọng số ứng với tín hiệu đầu vào,
3. x_0 tín hiệu hằng (luôn coi bằng 1),
4. w_0 trọng số tương ứng với tín hiệu hằng (còn gọi là bias),
5. Σ là ký hiệu tổng để tổng hợp các tín hiệu,
6. f là hàm kích hoạt,
7. y là tín hiệu đầu ra.



Hình 6.1: Mạng thần kinh sinh học



Hình 6.2: Mạng thần kinh nhân tạo

Mạng neuron được đặc trưng bởi ba thành phần chính là: hàm kích hoạt, cấu trúc mạng, và thuật toán huấn luyện.

Các thành phần cơ bản của một mạng neuron:

Nút (Neuron): Đây là đơn vị xử lý cơ bản của mạng. Mỗi neuron nhận đầu vào từ các neuron khác (hoặc từ dữ liệu đầu vào), thực hiện một phép tính (thường là tổng

có trọng số của các đầu vào cộng với một độ lệch - bias), và sau đó áp dụng một hàm kích hoạt (activation function) để tạo ra đầu ra.

Trọng số (Weights): Mỗi kết nối giữa các neuron có một trọng số liên quan. Trọng số này quyết định mức độ ảnh hưởng của đầu vào từ neuron trước đó đến neuron hiện tại. Trong quá trình huấn luyện, các trọng số này sẽ được điều chỉnh để mạng có thể học được các mẫu trong dữ liệu.

Độ lệch (Bias): Đây là một giá trị được cộng vào tổng có trọng số của các đầu vào trong một neuron. Độ lệch cho phép mạng điều chỉnh ngưỡng kích hoạt của neuron, giúp mạng linh hoạt hơn trong việc học các hàm phức tạp.

Hàm kích hoạt (Activation): Hàm này quyết định đầu ra của một neuron dựa trên tổng có trọng số của các đầu vào và độ lệch. Các hàm kích hoạt phổ biến bao gồm sigmoid, ReLU (Rectified Linear Unit), tanh, và softmax. Mỗi hàm có những đặc điểm riêng phù hợp với các loại bài toán khác nhau.

Lớp (Layer): Các neuron được tổ chức thành các lớp. Một mạng neuron điển hình có ba loại lớp chính:

Lớp đầu vào (Input layer): Nhận dữ liệu đầu vào từ thế giới bên ngoài. Số lượng neuron trong lớp này tương ứng với số lượng đặc trưng (features) trong dữ liệu đầu vào.

Lớp ẩn (Hidden layer): Nằm giữa lớp đầu vào và lớp đầu ra. Một mạng có thể có nhiều lớp ẩn. Các lớp này thực hiện các phép biến đổi phức tạp trên dữ liệu đầu vào để trích xuất các đặc trưng quan trọng.

Lớp đầu ra (Output layer): Tạo ra kết quả cuối cùng của mạng. Số lượng neuron và hàm kích hoạt trong lớp này phụ thuộc vào loại bài toán (ví dụ: phân loại nhị phân, phân loại đa lớp, hồi quy).

Quá trình hoạt động của mạng neuron:

Lan truyền xuôi (Forward Propagation): Dữ liệu đầu vào được đưa vào lớp đầu vào, sau đó truyền qua các lớp ẩn. Tại mỗi neuron, dữ liệu được nhân với các trọng số tương ứng, cộng với độ lệch, và sau đó được đưa qua hàm kích hoạt để tạo ra đầu ra của neuron đó. Quá trình này tiếp tục cho đến khi dữ liệu đến được lớp đầu ra.

Tính toán hàm mất mát (Loss Function): Hàm mất mát đo lường sự khác biệt giữa đầu ra dự đoán của mạng và giá trị thực tế. Mục tiêu của quá trình huấn luyện là giảm thiểu giá trị của hàm mất mát này.

Lan truyền ngược (Backpropagation): Thuật toán này tính toán gradient của hàm mất mát đối với các trọng số và độ lệch trong mạng. Gradient cho biết hướng mà các trọng số và độ lệch cần được điều chỉnh để giảm thiểu hàm mất mát.

Cập nhật trọng số và độ lệch: Dựa trên gradient tính được, các trọng số và độ lệch được cập nhật bằng một thuật toán tối ưu hóa (ví dụ: gradient descent) để mạng học được các mẫu trong dữ liệu và cải thiện độ chính xác của dự đoán.

Hàm kích hoạt Một số hàm kích hoạt thường sử dụng trong mạng neuron nhân tạo:

1. Hàm dấu **sgn**

$$\text{sgn}(x) = \begin{cases} 1 & x \geq 0; \\ 0 & x < 0. \end{cases}$$

2. Hàm **ReLU** (Rectified Linear Unit)

$$\text{ReLU}(x) = \begin{cases} x & x \geq 0; \\ 0 & x < 0. \end{cases}$$

3. Hàm **sigmoid**

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}.$$

4. Hàm **tanh**

$$\tanh(x) = \frac{\exp(2x) - 1}{\exp(2x) + 1}.$$

5. Hàm **softmax**

$$\text{softmax}(z) = \text{softmax}([z_1, z_2, \dots, z_K]) = \frac{1}{\sum_{k=1}^K \exp(z_k)} \left(\exp(z_1), \exp(z_2), \dots, \exp(z_K) \right).$$

Cấu trúc mạng Mạng có thể có một lớp hoặc có nhiều lớp ẩn, mạng có thể có một giá trị đầu ra hoặc nhiều giá trị đầu ra.

Thuật toán huấn luyện Trong việc huấn luyện mạng neuron trước hết ta cần xác định hàm tổn thất (loss function). Đối với bài toán hồi quy ta thường sử dụng sai số trung bình bình phương (**MSE**). Đối với bài toán phân loại ta thường sử dụng entropy chéo (**cross entropy**).

Thuật toán Batch GD: Thuật toán huấn luyện Batch GD là thuật toán sử dụng Gradient Descent với toàn bộ tập dữ liệu.

Thuật toán Stochastic GD: Thuật toán Stochastic GD là thuật toán sử dụng Gradient Descent đối với từng mẫu dữ liệu.

Thuật toán Mini-batch GD: Thuật toán Mini-batch GD là thuật toán sử dụng Gradient Descent với một bộ phận của dữ liệu. Thông thường ta chia dữ liệu huấn luyện thành các tập con (mini-batch) và cho từng tập con để huấn luyện mô hình.

Thuật toán huấn luyện mô hình là sử dụng phương pháp lan truyền ngược (**back-propagation**) để tính đạo hàm (gradient) với phương pháp tối ưu theo độ dốc (**gradient descent**).

6.2 Mô hình mạng neuron

6.2.1 Mạng neuron Perceptron

Mạng neuron Perceptron là mô hình mạng đơn giản nhất (không có lớp ẩn). Cấu trúc mạng kết nối trực tiếp từ lớp đầu vào với lớp đầu ra. Một số mô hình học máy có thể được xem như mô hình mạng Perceptron.

Mô hình hồi quy tuyến tính Mô hình mạng có p biến đầu vào $X_1, X_2, \dots, X_p$ và ta xây dựng hàm tuyến tính

$$Y = f(X) = w_0 + w_1 X_1 + \dots + w_p X_p$$

dự báo cho biến đầu ra Y . Đối với mô hình này ta sử dụng hàm kích hoạt là hàm đồng nhất.

Giả sử ta cần huấn luyện (training) mô hình mạng neuron đơn lớp cho bài toán học máy có giám sát $(x_1, y_1^{(t)}), (x_2, y_2^{(t)}), \dots, (x_n, y_n^{(t)})$ gồm có n dữ liệu. Ta cực tiểu hoá hàm tổn thất $L(y, y^{(t)})$

$$E = L(y, y^{(t)}) = \frac{1}{n} \sum_{i=1}^n e_i^2 = \frac{1}{n} \sum_{i=1}^n (y_i - y_i^{(t)})^2. \quad (6.1)$$

Mô hình hồi quy logistic Mô hình mạng có p biến đầu vào $X_1, X_2, \dots, X_p$ với K biến đầu ra tương ứng với mỗi lớp. Đối với mô hình này ta sử dụng hàm kích hoạt là hàm softmax.

Với mỗi node đầu ra $z_k = b_{k0} + w_{k1}X_1 + \dots + w_{kp}X_p, k = 1, 2, \dots, K$. Ta có thể viết thu gọn lại dạng đơn giản $z = Wx + b$ và đầu ra thông qua hàm kích hoạt $\hat{y} = \text{softmax}(z)$.

Giả sử ta cần huấn luyện (training) mô hình mạng neuron đơn lớp cho bài toán học máy có giám sát $(x_1, y_1^{(t)}), (x_2, y_2^{(t)}), \dots, (x_n, y_n^{(t)})$ gồm có n dữ liệu. Ta cực tiểu hoá hàm tổn thất $L(y, y^{(t)})$

$$E = L(y, y^{(t)}) = - \sum_{i=1}^n \left(\sum_{k=1}^K y_{ik}^{(t)} \log(y_{ik}) \right). \quad (6.2)$$

Ví dụ 6.1 (AND). Bảng giá trị chân lý của hàm AND. Ta có thể sử dụng hàm Perceptron

$$y = \text{sgn}(x_1 + x_2 - 1.5).$$

x_1	x_2	$y = \text{AND}(x_1, x_2)$
0	0	0
0	1	0
1	0	0
1	1	1

Bảng 6.1: Bảng chân trị cho hàm AND

Ví dụ 6.2 (OR). Bảng giá trị chân lý của hàm OR. Ta có thể sử dụng hàm Perceptron

$$y = \text{sgn}(x_1 + x_2 - 0.5).$$

x_1	x_2	$y = \text{OR}(x_1, x_2)$
0	0	0
0	1	1
1	0	1
1	1	1

Bảng 6.2: Bảng chân trị cho hàm OR

Ví dụ 6.3 (NOT). Bảng giá trị chân lý của hàm NOT. Ta có thể sử dụng hàm Perceptron

$$y = \text{sgn}(x - 0.5).$$

x	$y = \text{NOT}(x)$
0	1
1	0

Bảng 6.3: Bảng chân trị cho hàm NOT

Ví dụ 6.4 (XOR). Bảng giá trị chân lý của hàm XOR. Với hàm XOR ta không thể sử dụng hàm Perceptron để phân loại do hàm XOR là hàm không tách tuyến tính. Để phân loại hàm XOR ta cần thêm một lớp ẩn giữa lớp đầu vào và lớp đầu ra.

Xét mạng neuron với 1 lớp ẩn a_1, a_2 với

$$a_1 = \text{sgn}(-2x_1 - 2x_2 + 3),$$

$$a_2 = \text{sgn}(2x_1 + 2x_2 - 1),$$

Hàm phân loại

$$y = \text{sgn}(a_1 + a_2 - 1).$$

x_1	x_2	$y = \text{XOR}(x_1, x_2)$
0	0	0
0	1	1
1	0	1
1	1	0

Bảng 6.4: Bảng chân trị cho hàm XOR

6.2.2 Mạng neuron đơn tầng

Mạng neuron đơn tầng là mạng neuron có đúng 1 lớp ẩn. Một mô hình mạng neuron đơn tầng có p biến đầu vào $X_1, X_2, \dots, X_p$ và ta xây dựng hàm phi tuyến $f(X)$ dự báo cho biến đầu ra Y .

Xét một mạng đơn tầng với L nút ẩn, và ta ký hiệu các nút ẩn là $A_1, A_2, \dots, A_L$, gọi $W^{(1)}, b^{(1)}$ lần lượt là ma trận trọng số và độ lệch kết nối giữa lớp đầu vào và lớp ẩn. Ký hiệu $\sigma(\cdot)$ là hàm kích hoạt.

Giá trị tại nút ẩn được tính theo công thức sau:

$$\begin{aligned} A_1 &= \sigma\left(\sum_{j=1}^p W_{1j}^{(1)} x_j + b_1^{(1)}\right), \\ A_2 &= \sigma\left(\sum_{j=1}^p W_{2j}^{(1)} x_j + b_2^{(1)}\right), \\ &\vdots \\ A_L &= \sigma\left(\sum_{j=1}^p W_{Lj}^{(1)} x_j + b_L^{(1)}\right). \end{aligned}$$

Ta viết gọn lớp ẩn thành công thức dạng véc-tơ

$$A = \sigma(W^{(1)}x + b^{(1)}). \quad (6.3)$$

Ta ký hiệu $W^{(2)}, b^{(2)}$ là ma trận trọng số và độ lệch kết nối giữa lớp ẩn và lớp đầu ra. Tương tự như trên ta có giá trị của lớp đầu ra cho bởi công thức

$$y = \sigma(W^{(2)}A + b^{(2)}). \quad (6.4)$$

Mô hình trong Hình 6.3 mô tả một mạng đơn lớp với $p = 4$ biến đầu vào ở lớp Input, và một biến đầu ra ở lớp Output. Giữa lớp Input và lớp Output có 1 lớp ẩn gồm có L nút ẩn (trong ví dụ trên gồm có 5 nút trong lớp ẩn).

Giả sử ta cần huấn luyện (training) mô hình mạng neuron đơn lớp cho bài toán học máy có giám sát $(x_1, y_1^{(t)}), (x_2, y_2^{(t)}), \dots, (x_n, y_n^{(t)})$ gồm có n dữ liệu. Ta sử dụng hàm tổn thất $L(y, y^{(t)})$

$$E = L(y, y^{(t)}), \quad (6.5)$$

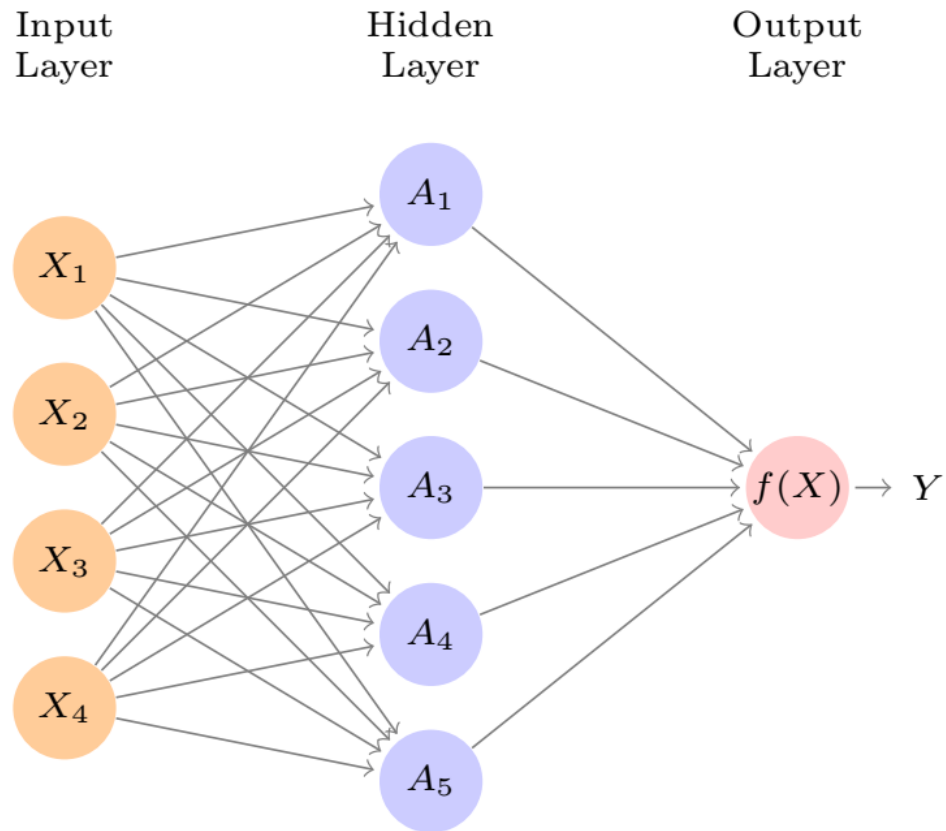
trong đó $y_i = f(x_i)$ là giá trị dự báo của mô hình.

Bài toán hồi quy: Xét bài toán hồi quy ta sử dụng hàm tổn thất là bình phương của sai số (phương pháp bình phương tối thiểu) với hàm tổn thất được cho bởi công thức

$$E = \frac{1}{2} \|y - y^{(t)}\|^2 = \frac{1}{2} \sum_{i=1}^n (y_i - y_i^{(t)})^2. \quad (6.6)$$

Bài toán phân loại: Xét bài toán phân loại ta sử dụng hàm tổn thất là entropy-chéo (cross entropy). Giả sử ta xét bài toán phân làm K lớp khác nhau khi đó giá trị $y^{(t)} \in \{0, 1\}^K$ và giá trị dự báo $y \in [0, 1]^K$. Hàm tổn thất entropy chéo được cho bởi công thức

$$E = - \sum_{i=1}^n \left(\sum_{k=1}^K y_{ik}^{(t)} \log(y_{ik}) \right). \quad (6.7)$$



Hình 6.3: Mạng neuron đơn tầng

Chú ý rằng đối với bài toán phân loại thì ta luôn đảm bảo

$$\sum_{k=1}^K y_{ik} = 1, \forall i = 1, 2, \dots, n.$$

Khi đó, đối với lớp cuối cùng ta thường sử dụng hàm softmax.

Huấn luyện mạng neuron là xác định các tham số $W = [W^{(1)}, W^{(2)}]$, $b = [b^{(1)}, b^{(2)}]$ để hàm tổn thất nhận giá trị cực tiểu.

6.2.3 Mạng neuron đa tầng

Một mô hình mạng neuron có p biến đầu vào $X_1, X_2, \dots, X_p$ và ta xây dựng hàm phi tuyến $f(X)$ dự báo cho biến đầu ra Y . Mạng neuron đa tầng là tổng quát hoá mạng

neuron đơn tầng khi số lớp ẩn tăng lên. Xét mạng neuron đa tầng có L lớp ẩn ở giữa lớp đầu vào và lớp đầu ra.

Ta ký hiệu $\sigma(\cdot)$ là một hàm kích hoạt nào đó. Giả sử ta xây dựng mô hình mạng đa lớp theo công thức sau:

$$\begin{aligned} a^{(0)} &= x, \\ a^{(1)} &= \sigma(W^{(1)} a^{(0)} + b^{(1)}), \\ a^{(2)} &= \sigma(W^{(2)} a^{(1)} + b^{(2)}), \\ &\vdots \\ a^{(L)} &= \sigma(W^{(L)} a^{(L-1)} + b^{(L)}), \\ y &= a^{(L)}. \end{aligned}$$

Trong sơ đồ trên các ma trận $W = [W^{(1)}, \dots, W^{(L)}]$, và các véc-tơ $b = [b^{(1)}, \dots, b^{(L)}]$ có kích thước phù hợp với mô hình mạng.

Mô hình trong Hình 6.4 mô tả một mạng hai lớp với p biến đầu vào ở lớp Input, và nhiều biến đầu ra ở lớp Output. Giữa lớp Input và lớp Output có 2 lớp ẩn gồm có K_1, K_2 nút ẩn. Mạng neuron đa tầng là mô hình mạng có nhiều hơn một lớp ẩn.

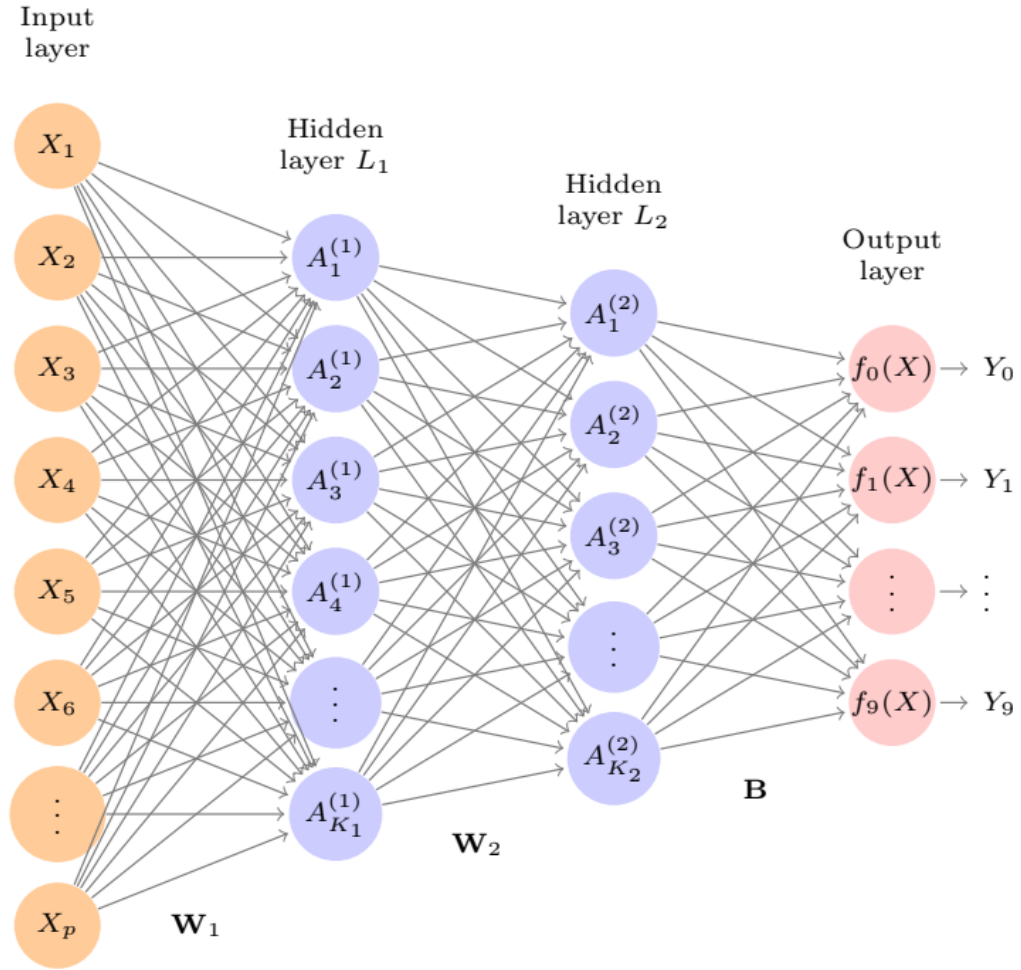
Tương tự như mạng neuron đơn tầng thì hàm tổn thất đối với mạng đa tầng cũng được xác định tương tự giữa dữ liệu huấn luyện $y^{(t)}$ với giá trị của lớp đầu ra y .

Định lý xấp xỉ phổ quát: Cho một hàm kích hoạt $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ là một hàm liên tục, phi hằng và bị chặn. Khi đó, cho bất kỳ hàm liên tục $f : K \rightarrow \mathbb{R}^m$ trên một tập con compact $K \subseteq \mathbb{R}^n$ và bất kỳ $\epsilon > 0$, tồn tại một mạng neuron với một lớp ẩn duy nhất có hàm dạng:

$$g(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

trong đó:

- $x \in \mathbb{R}^n$ là đầu vào.
- $W_1 \in \mathbb{R}^{N \times n}$ là ma trận trọng số kết nối lớp đầu vào với lớp ẩn (với N là số neuron trong lớp ẩn).
- $b_1 \in \mathbb{R}^N$ là vectơ độ lệch của lớp ẩn.
- $W_2 \in \mathbb{R}^{m \times N}$ là ma trận trọng số kết nối lớp ẩn với lớp đầu ra.



Hình 6.4: Mạng neuron đa tầng

- $b_2 \in \mathbb{R}^m$ là vectơ độ lệch của lớp đầu ra.
- σ được áp dụng theo phần tử cho vectơ $W_1 x + b_1$.

sao cho:

$$|f(x) - g(x)| < \epsilon, \quad \forall x \in K$$

Điều này có nghĩa là mạng neuron $g(x)$ có thể xấp xỉ hàm $f(x)$ với độ chính xác bất kỳ trên tập compact K bằng cách chọn đủ số lượng neuron ẩn N và các tham số W_1, b_1, W_2, b_2 phù hợp.

Định lý biểu diễn Kolmogorov–Arnold Định lý biểu diễn chồng chất (superposition hay Kolmogorov–Arnold) đề cập đến việc biểu diễn một hàm nhiều biến liên tục trên

$[0, 1]^n$ bởi sự chõng chất các hàm một biến. Cụ thể, mọi hàm liên tục $f : [0, 1]^n \rightarrow \mathbb{R}$ có biểu diễn dạng

$$f(x_1, x_2, \dots, x_n) = \sum_{q=0}^{2n} \Phi_q \left(\sum_{p=0}^n \Phi_{qp}(x_p) \right),$$

với các hàm một biến $\Phi_{qp} : [0, 1] \rightarrow \mathbb{R}$ và $\Phi_q : \mathbb{R} \rightarrow \mathbb{R}$.

6.3 Thuật toán lan truyền ngược

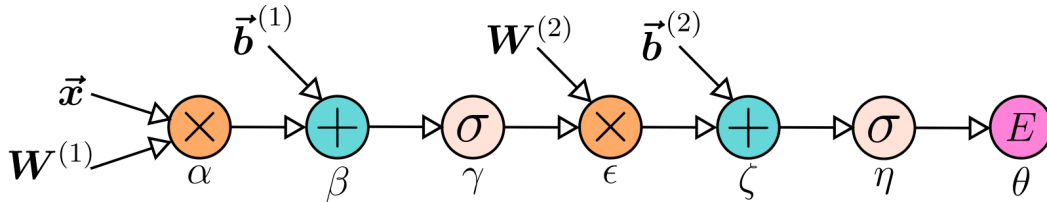
Thuật toán lan truyền ngược (Backpropagation) là một phương pháp cốt lõi trong việc huấn luyện mạng neuron nhân tạo. Nó được sử dụng để tối ưu hóa các trọng số của mạng neuron bằng cách giảm thiểu hàm tổn thất (loss function) thông qua việc tính toán đạo hàm của hàm tổn thất đối với từng tham số và cập nhật chúng theo hướng ngược lại.

Ta cần giải bài toán tối ưu là tìm cực tiểu hàm sai số E . Để cực tiểu hoá hàm tổn thất ta sử dụng phương pháp đạo hàm theo độ dốc, nghĩa là ta cần tính các đạo hàm của hàm sai số E đối với các biến $[W, b]$ và cập nhật theo tốc độ học η theo công thức

$$W_{new} = W_{old} - \eta \frac{\partial E}{\partial W} (W_{old}), \quad (6.8)$$

$$b_{new} = b_{old} - \eta \frac{\partial E}{\partial b} (b_{old}). \quad (6.9)$$

Việc tính đạo hàm của các lớp ta sử dụng quy tắc đạo hàm của hàm hợp. Trước hết ta xét mạng neuron đơn tầng với các tham số $[W, b] = [W^{(1)}, W^{(2)}, b^{(1)}, b^{(2)}]$. Ta có thể mô tả chi tiết sơ đồ tính toán tiến (forward) theo sơ đồ như Hình 6.5 dưới đây.

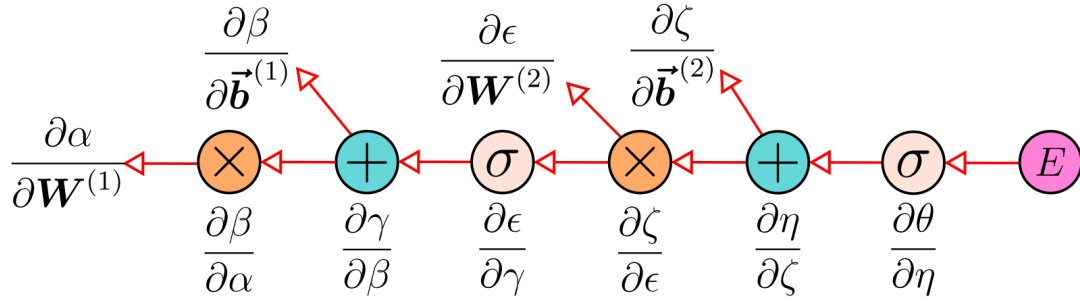


Hình 6.5: Sơ đồ tiến của mạng neuron.

Giá trị tại các nút (node) được tính theo các biến và các phép toán như sau:

$$\begin{aligned}
 \alpha &= W^{(1)}x, & \frac{\partial \alpha}{\partial W^{(1)}} &= x^T, \\
 \beta &= \alpha + b^{(1)}, & \frac{\partial \beta}{\partial \alpha} &= 1, \frac{\partial \beta}{\partial b^{(1)}} = 1, \\
 \gamma &= \sigma(\beta), & \frac{\partial \gamma}{\partial \beta} &= \sigma'(\beta), \\
 \epsilon &= W^{(2)}\gamma, & \frac{\partial \epsilon}{\partial W^{(2)}} &= \gamma^T, \frac{\partial \epsilon}{\partial \gamma} = W^{(2)}, \\
 \zeta &= \epsilon + b^{(2)}, & \frac{\partial \zeta}{\partial \epsilon} &= 1, \frac{\partial \zeta}{\partial b^{(2)}} = 1, \\
 \eta &= \sigma(\zeta), & \frac{\partial \eta}{\partial \zeta} &= \sigma'(\zeta), \\
 \theta &= L(\eta, y^{(t)}), & \frac{\partial \theta}{\partial \eta} &= \frac{\partial E}{\partial y}, \frac{\partial E}{\partial \theta} = 1.
 \end{aligned}$$

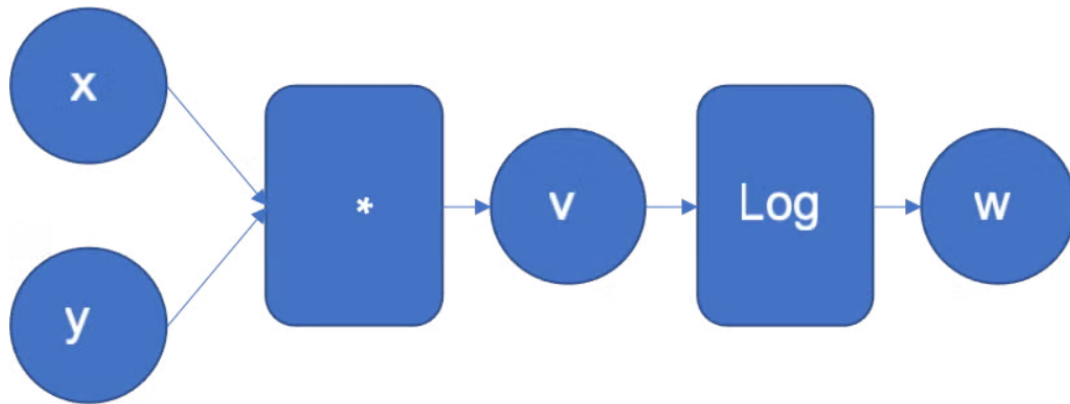
Các đạo hàm trong sơ đồ tính toán trên được dễ dàng tính toán theo các công thức trên. Các đạo hàm đối với các biến được tính theo quy tắc xích đối với hàm hợp và ta gọi quá trình tính toán đạo hàm là quá trình lan truyền ngược (backpropagation) như mô tả trong hình vẽ.



Hình 6.6: Sơ đồ lan truyền ngược tính đạo hàm.

Đối với sơ đồ mạng neuron trên ta có thể tính các đạo hàm như sau:

$$\begin{aligned}
 \frac{\partial E}{\partial b^{(2)}} &= \frac{\partial E}{\partial \theta} \cdot \frac{\partial \theta}{\partial \eta} \cdot \frac{\partial \eta}{\partial \zeta} \cdot \frac{\partial \zeta}{\partial b^{(2)}}, \\
 \frac{\partial E}{\partial W^{(2)}} &= \frac{\partial E}{\partial \theta} \cdot \frac{\partial \theta}{\partial \eta} \cdot \frac{\partial \eta}{\partial \zeta} \cdot \frac{\partial \zeta}{\partial \epsilon} \cdot \frac{\partial \epsilon}{\partial W^{(2)}}, \\
 \frac{\partial E}{\partial b^{(1)}} &= \frac{\partial E}{\partial \theta} \cdot \frac{\partial \theta}{\partial \eta} \cdot \frac{\partial \eta}{\partial \zeta} \cdot \frac{\partial \zeta}{\partial \epsilon} \cdot \frac{\partial \epsilon}{\partial \gamma} \cdot \frac{\partial \gamma}{\partial \beta} \cdot \frac{\partial \beta}{\partial b^{(1)}}, \\
 \frac{\partial E}{\partial W^{(1)}} &= \frac{\partial E}{\partial \theta} \cdot \frac{\partial \theta}{\partial \eta} \cdot \frac{\partial \eta}{\partial \zeta} \cdot \frac{\partial \zeta}{\partial \epsilon} \cdot \frac{\partial \epsilon}{\partial \gamma} \cdot \frac{\partial \gamma}{\partial \beta} \cdot \frac{\partial \beta}{\partial \alpha} \cdot \frac{\partial \alpha}{\partial W^{(1)}}.
 \end{aligned}$$



Hình 6.7: Đồ thị tính hàm nhiều biến

Ví dụ 6.5 (Thuật toán lan truyền ngược với hàm nhiều biến). Tính đạo hàm của hàm nhiều biến bằng sơ đồ đồ thị $w = \ln(x * y)$.

Sử dụng thư viện `pytorch` tính các đạo hàm riêng $\frac{\partial w}{\partial x}, \frac{\partial w}{\partial y}$ tại điểm $(x, y) = (1, 1)$.

Ta sử dụng thư viện `torch` và khai báo các biến x, y cần tính đạo hàm.

```
import torch
x = torch.tensor(1.0, requires_grad=True)
y = torch.tensor(1.0, requires_grad=True)
```

Khai báo các biến $v = x * y$ và $w = \text{torch.log}(v)$.

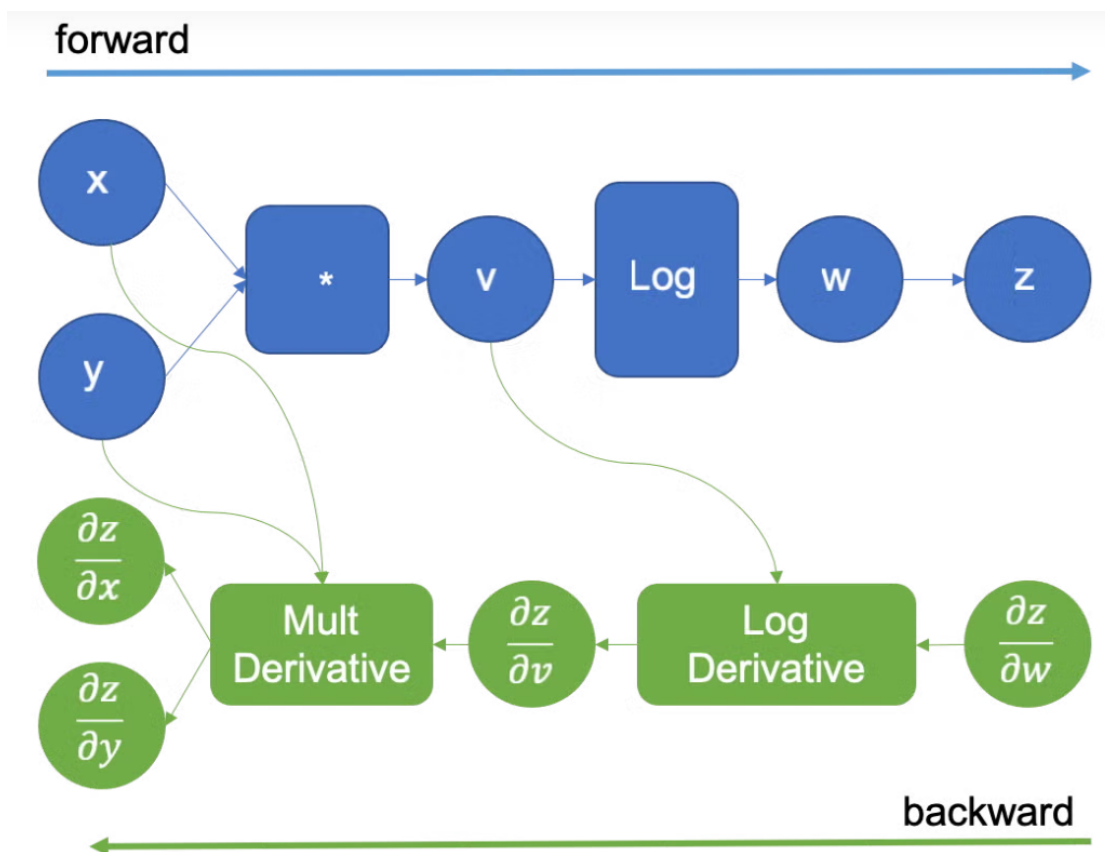
Để tính đạo hàm bằng quá trình lan truyền ngược ta cần thực hiện lệnh `w.backward()`.

Các giá trị đạo hàm cần tính là `x.grad.item(), y.grad.item()`.

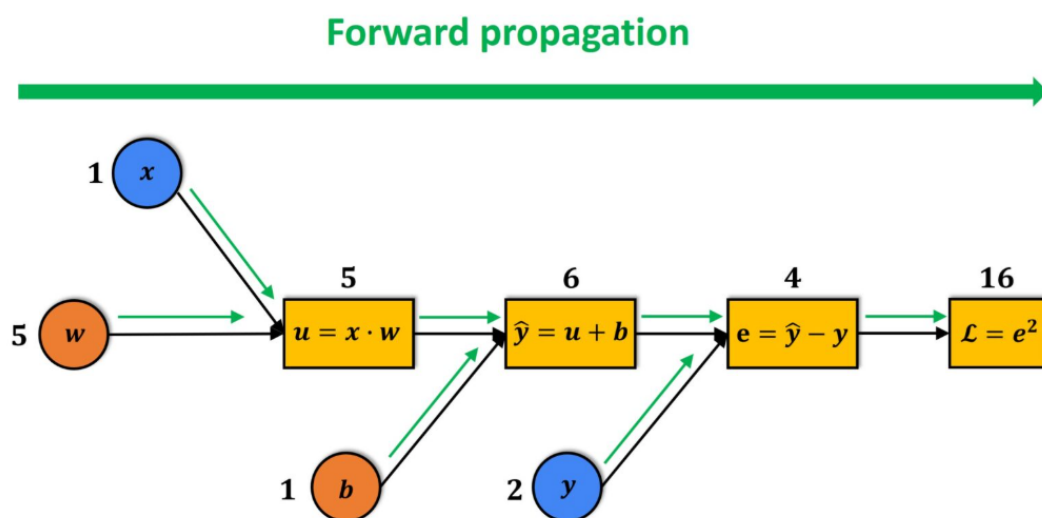
Cụ thể coi $z = w$ nên ta có

$$\begin{aligned}\frac{\partial z}{\partial x} &= \frac{\partial z}{\partial w} \cdot \frac{\partial w}{\partial v} \cdot \frac{\partial v}{\partial x} = 1 \cdot \frac{1}{v} \cdot y = 1, \\ \frac{\partial z}{\partial y} &= \frac{\partial z}{\partial w} \cdot \frac{\partial w}{\partial v} \cdot \frac{\partial v}{\partial y} = 1 \cdot \frac{1}{v} \cdot x = 1.\end{aligned}$$

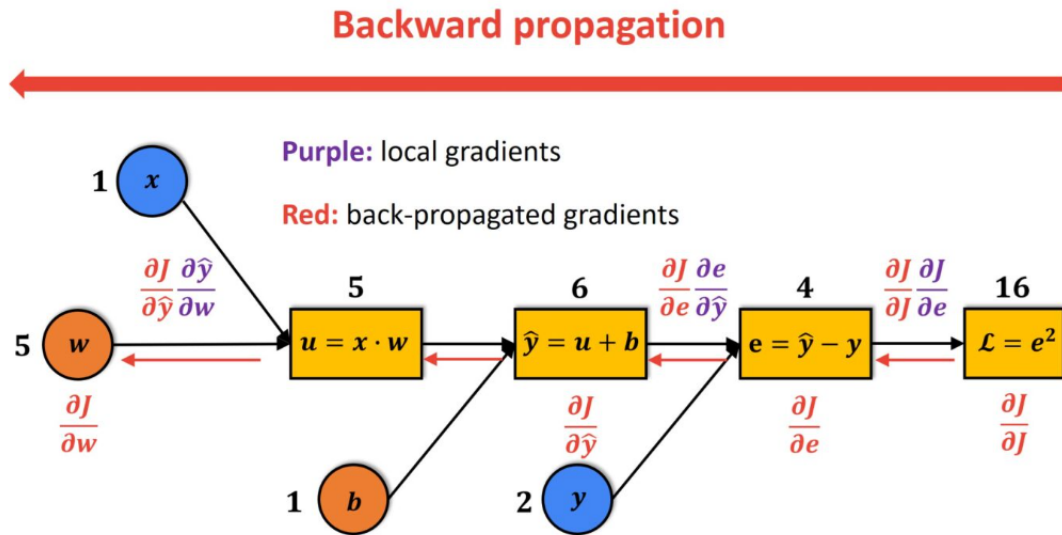
Ví dụ 6.6 (Thuật toán lan truyền ngược với hàm hồi quy). Cho mô hình tính toán như trên Hình 6.9. Với giá trị tham số ban đầu $w = 5, b = 1$. Tính các giá trị mới của các tham số w và b bằng thuật toán gradient descent với tốc độ học $\eta = 0.01$ khi mẫu huấn luyện $(x, y) = (1, 2)$.



Hình 6.8: Đồ thị tính đạo hàm của hàm nhiều biến



Hình 6.9: Đồ thị tính mô hình hồi quy



Hình 6.10: Đồ thị tính mô hình hồi quy

Phần tính thuận (forward): Với $(x, y) = (1, 2)$, theo sơ đồ tính toán ta tính được

$$\begin{aligned} u &= x \cdot w = 1 \cdot 5 = 5, \\ \hat{y} &= u + b = 5 + 1 = 6, \\ e &= \hat{y} - y = 6 - 2 = 4, \\ L &= e^2 = r^2 = 16. \end{aligned}$$

Phần tính ngược (backward): Đặt $J = L$ là biến ban đầu

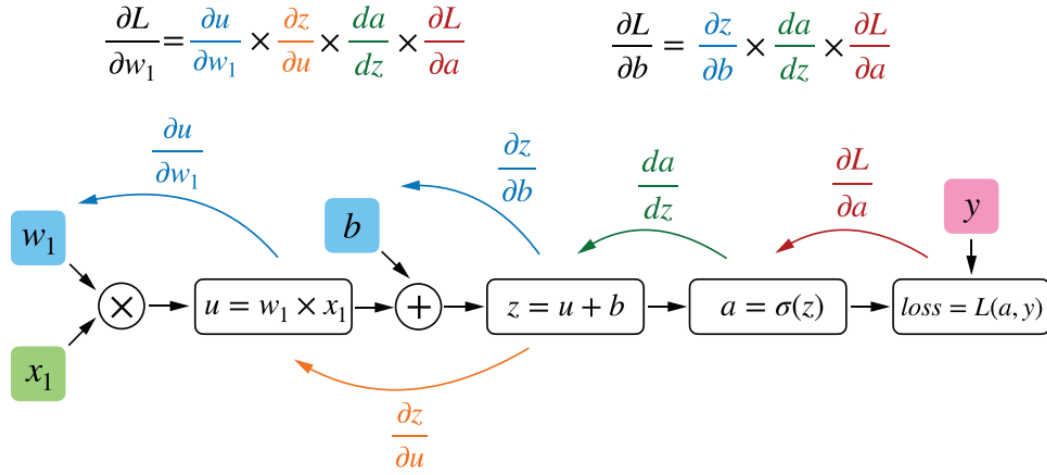
$$\begin{aligned} \frac{\partial J}{\partial L} &= 1, \frac{\partial L}{\partial e} = 2e = 2 \cdot 4 = 8, \frac{\partial e}{\partial \hat{y}} = 1, \\ \frac{\partial \hat{y}}{\partial u} &= 1, \frac{\partial \hat{y}}{\partial b} = 1, \frac{\partial u}{\partial w} = x = 1. \end{aligned}$$

Tính các đạo hàm:

$$\begin{aligned} \frac{\partial J}{\partial b} &= \frac{\partial J}{\partial L} \cdot \frac{\partial L}{\partial e} \cdot \frac{\partial e}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial b} = 8, \\ \frac{\partial J}{\partial w} &= \frac{\partial J}{\partial L} \cdot \frac{\partial L}{\partial e} \cdot \frac{\partial e}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial u} \cdot \frac{\partial u}{\partial w} = 8. \end{aligned}$$

Cập nhật các tham số

$$\begin{aligned} w &= w - \eta \frac{\partial J}{\partial w} = 5 - 0.01 \cdot 8 = 4.92 \\ b &= b - \eta \frac{\partial J}{\partial b} = 1 - 0.01 \cdot 8 = 0.92. \end{aligned}$$



Hình 6.11: Đồ thị tính mô hình phân loại

Với các tham số mới cập nhật giá trị hàm tổn thất là $L = 14.7456$ giảm so với trước khi cập nhật tham số với $L = 16$.

Ví dụ 6.7 (Thuật toán lan truyền ngược với hàm phân loại). Cho mô hình mạng phân loại như Hình 6.11 với hàm kích hoạt $\sigma(z) = \frac{1}{1 + \exp(-z)}$, hàm tổn thất là cross-entropy $L(a, y) = -[y \log(a) + (1 - y) \log(1 - a)]$.

Tính các tham số w_1, b bằng thuật toán gradient descent với tốc độ học $\eta = 0.01$. Cho biết các tham số ban đầu $w_1 = 2, b = 1$. Thực hiện huấn luyện lần lượt từng mẫu một. Mẫu 1 $(x_1, y) = (-1.0, 1)$, mẫu 2 $(x_1, y) = (1.0, 0)$

Ta huấn luyện mô hình với dữ liệu $(x_1, y) = (-1.0, 1)$

Trước hết ta tính phần forward:

$$u = w_1 * x_1 = -2,$$

$$z = u + b = -2 + 1 = -1,$$

$$a = \sigma(z) = \frac{1}{1 + e^{-1}} = 0.2689,$$

$$L = -[y \log(a) + (1 - y) * \log(1 - a)] = 1.3133$$

Tính đạo hàm trung gian:

$$\begin{aligned}\frac{\partial L}{\partial a} &= -\left[\frac{y}{a} - \frac{1-y}{1-a}\right] = \frac{a-y}{a(1-a)} = -1.0498, \\ \frac{\partial a}{\partial z} &= \sigma(z)(1-\sigma(z)) = 0.0472, \\ \frac{\partial z}{\partial u} &= 1, \frac{\partial z}{\partial b} = 1, \frac{\partial u}{\partial w_1} = x_1 = -1.\end{aligned}$$

Tính đạo hàm theo lan truyền ngược:

$$\begin{aligned}\frac{\partial L}{\partial b} &= \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial b} = -0.7311, \\ \frac{\partial L}{\partial w_1} &= \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial u} \cdot \frac{\partial u}{\partial w_1} = 0.7311.\end{aligned}$$

Cập nhật tham số mới:

$$\begin{aligned}w_1 &= w_1 - \eta * \frac{\partial L}{\partial w_1} = 1.992689, \\ b &= b - \eta * \frac{\partial L}{\partial b} = 1.007311\end{aligned}$$

Hàm tổn thất sau khi cập nhật tham số mới với mẫu 1 là $L = 1.3026$.

Tương tự như vậy ta tiếp tục huấn luyện mô hình với mẫu 2 $(x_1, y) = (1.0, 0)$

6.4 Thực hành với thư viện `sklearn` và `pytorch`

6.4.1 Mạng hồi quy đơn tầng với `sklearn`

Ta sử dụng thư viện `sklearn.neural_network.MLPClassifier` và `sklearn.neural_network.MLPRegressor`.

```
[1]: import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
plt.style.use('seaborn-white')
```

```
[2]: Hitters = pd.read_csv('data/Hitters.csv', header=0,
    _na_values='NA').dropna()
Hitters=Hitters.reset_index(drop=True).drop(['Unnamed:
    _0'],axis=1)
dummies = pd.get_dummies(Hitters[['League', 'Division',
    _'NewLeague']])
y = Hitters.Salary.values # the response variable
X_prep = Hitters.drop(['Salary', 'League', 'Division',
    _'NewLeague'], axis = 1)
X = pd.concat([X_prep, dummies[['League_A', 'Division_E',
    _'NewLeague_A']]], axis=1).values
```

```
[3]: X.shape,y.shape
```

```
[3]: ((263, 19), (263,))
```

```
[4]: X_train, X_test , y_train, y_test = train_test_split(X, y,
    _test_size=0.3)
```

```
[5]: from sklearn.neural_network import MLPRegressor
```

```
[6]: # MLP Training
mlp = MLPRegressor(hidden_layer_sizes=(50),
    activation='relu',
    learning_rate_init=0.008,
    learning_rate='adaptive')
```

```
[7]: model = mlp.fit(X_train,y_train)
print("Training set score: %f" % model.score(X_train, y_train))
print("Test set score: %f" % model.score(X_test, y_test))
```

Training set score: 0.792756

Test set score: 0.625721

6.4.2 Mạng phân loại đơn tầng với sklearn

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
plt.style.use('ggplot')
```



```
[2]: from sklearn.model_selection import train_test_split
      from sklearn.utils import shuffle
      from sklearn.preprocessing import StandardScaler
      from sklearn.neural_network import MLPClassifier
      from sklearn.datasets import load_iris
```

```
[3]: iris = load_iris()
      X = iris['data']
      y = iris['target']
      names = iris['target_names']
      feature_names = iris['feature_names']
      # Scale data to have mean 0 and variance 1
      # which is importance for convergence of the neural network
      scaler = StandardScaler()
      X_scaled = scaler.fit_transform(X)
      # Split the data set into training and testing
      X_train, X_test, y_train, y_test = train_test_split(
          X_scaled, y, test_size=0.2, random_state=2)
```

```
[4]: print("X_train shape= ",X_train.shape)
      print("y_train.shape= ",y_train.shape)
      print("X_test shape= ",X_test.shape)
      print("y_test shape=",y_test.shape)
```

```
X_train shape= (120, 4)
y_train.shape= (120,)
X_test shape= (30, 4)
y_test shape= (30,)
```

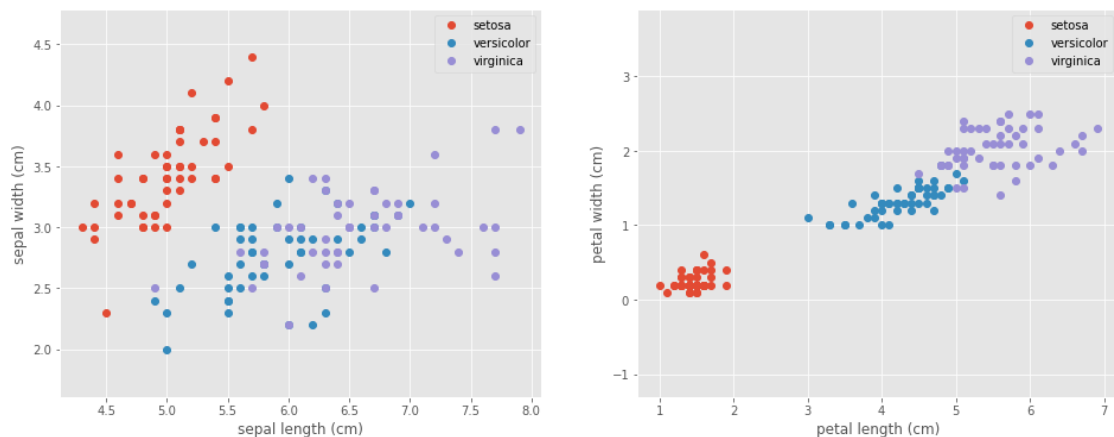
```
[5]: # MLP Training
      mlp = MLPClassifier(hidden_layer_sizes=(32),
                          activation='relu',
                          learning_rate_init=0.01,
                          learning_rate='adaptive')
      model = mlp.fit(X_train,y_train)
      print("Training set score: %f" % model.score(X_train, y_train))
      print("Test set score: %f" % model.score(X_test, y_test))
```

```
Training set score: 0.983333
Test set score: 0.933333
```

```
[6]: # Cross validation
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
print('Mean of Cross validation score: ', scores.mean())
```

Mean of Cross validation score: 0.98

```
[7]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
for target, target_name in enumerate(names):
    X_plot = X[y == target]
    ax1.plot(X_plot[:, 0], X_plot[:, 1],
             linestyle='none',
             marker='o',
             label=target_name)
ax1.set_xlabel(feature_names[0])
ax1.set_ylabel(feature_names[1])
ax1.axis('equal')
ax1.legend();
for target, target_name in enumerate(names):
    X_plot = X[y == target]
    ax2.plot(X_plot[:, 2], X_plot[:, 3],
             linestyle='none',
             marker='o',
             label=target_name)
ax2.set_xlabel(feature_names[2])
ax2.set_ylabel(feature_names[3])
ax2.axis('equal')
ax2.legend();
```



6.4.3 Mạng hồi quy đa tầng với sklearn

```
[1]: import numpy as np
import pandas as pd
from sklearn.neural_network import MLPRegressor
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
plt.style.use('seaborn-white')
```

```
[2]: Hitters = pd.read_csv('data/Hitters.csv', header=0,
    -na_values='NA').dropna()
Hitters=Hitters.reset_index(drop=True).drop(['Unnamed:
    -0'],axis=1)
dummies = pd.get_dummies(Hitters[['League', 'Division',
    -'NewLeague']])
y = Hitters.Salary.values # the response variable
X_prep = Hitters.drop(['Salary', 'League', 'Division',
    -'NewLeague'], axis = 1)
X = pd.concat([X_prep, dummies[['League_A', 'Division_E',
    -'NewLeague_A']]], axis=1).values
```

```
[3]: X.shape,y.shape
```

```
[3]: ((263, 19), (263,))
```

```
[4]: X_train, X_test , y_train, y_test = train_test_split(X, y,
    -test_size=0.3)
```

```
[5]: # MLP Training
mlp = MLPRegressor(hidden_layer_sizes=(50,30,20),
    activation='relu',
    learning_rate_init=0.008,
    learning_rate='adaptive')
```

```
[6]: model = mlp.fit(X_train,y_train)
print("Training set score: %f" % model.score(X_train, y_train))
print("Test set score: %f" % model.score(X_test, y_test))
```

Training set score: 0.883232

Test set score: 0.186252

6.4.4 Mạng phân loại đa tầng với sklearn

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
plt.style.use('ggplot')
```

```
[2]: from sklearn.model_selection import train_test_split
from sklearn.utils import shuffle
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_iris
```

```
[3]: iris = load_iris()
X = iris['data']
y = iris['target']
names = iris['target_names']
feature_names = iris['feature_names']
# Scale data to have mean 0 and variance 1
# which is importance for convergence of the neural network
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Split the data set into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=2)
```

```
[4]: print("X_train shape= ",X_train.shape)
print("y_train.shape= ",y_train.shape)
print("X_test shape= ",X_test.shape)
print("y_test shape=",y_test.shape)
```

```
X_train shape= (120, 4)
y_train.shape= (120,)
X_test shape= (30, 4)
y_test shape= (30,)
```

```
[5]: # MLP Training
mlp = MLPClassifier(hidden_layer_sizes=(8,18,8),
                    activation='relu',
                    learning_rate_init=0.01,
                    learning_rate='adaptive')
model = mlp.fit(X_train,y_train)
```

```
print("Training set score: %f" % model.score(X_train, y_train))
print("Test set score: %f" % model.score(X_test, y_test))
```

Training set score: 0.983333

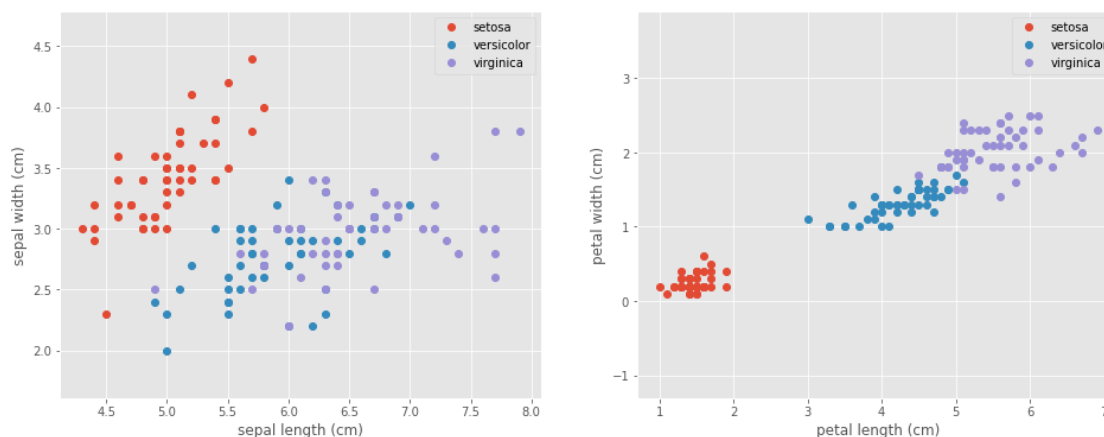
Test set score: 0.933333

```
[6]: # Cross validation
from sklearn.model_selection import cross_val_score
scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
print('Mean of Cross validation score: ', scores.mean())
```

Mean of Cross validation score: 0.98

```
[7]: fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
for target, target_name in enumerate(names):
    X_plot = X[y == target]
    ax1.plot(X_plot[:, 0], X_plot[:, 1],
             linestyle='none',
             marker='o',
             label=target_name)
    ax1.set_xlabel(feature_names[0])
    ax1.set_ylabel(feature_names[1])
    ax1.axis('equal')
    ax1.legend();

for target, target_name in enumerate(names):
    X_plot = X[y == target]
    ax2.plot(X_plot[:, 2], X_plot[:, 3],
             linestyle='none',
             marker='o',
             label=target_name)
    ax2.set_xlabel(feature_names[2])
    ax2.set_ylabel(feature_names[3])
    ax2.axis('equal')
    ax2.legend();
```



6.4.5 Mạng hồi quy đơn tầng với pytorch

Trong phần thực hành với thư viện pytorch chúng ta xây dựng mạng neuron tuần tự từ thư viện `torch.nn.Sequential` hoặc xây dựng mô hình mạng từ thư viện `torch.nn.Module`.

```
[1]: import copy
import numpy as np
import torch
import tqdm
import torch.nn as nn
import torch.optim as optim
```

```
[2]: # Define the model
model = nn.Sequential(
    nn.Linear(19, 50),
    nn.ReLU(),
    nn.Linear(50, 1)
)
```

```
[3]: # loss function and optimizer
loss_fn = nn.MSELoss() # mean square error
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```
[4]: # train-test split of the dataset
X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1,
    -1)
X_test = torch.tensor(X_test, dtype=torch.float32)
```

```

y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1,
-1)

# training parameters
n_epochs = 100    # number of epochs to run
batch_size = 10   # size of each batch
batch_start = torch.arange(0, len(X_train), batch_size)

# Hold the best model
best_mse = np.inf    # init to infinity
best_weights = None
history = []

# training loop
for epoch in range(n_epochs):
    model.train()
    with tqdm.tqdm(batch_start, unit="batch", mininterval=0,
-disable=True) as bar:
        bar.set_description(f"Epoch {epoch}")
        for start in bar:
            # take a batch
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass
            optimizer.zero_grad()
            loss.backward()
            # update weights
            optimizer.step()
            # print progress
            bar.set_postfix(mse=float(loss))
        # evaluate accuracy at end of each epoch
        model.eval()
        y_pred = model(X_test)
        mse = loss_fn(y_pred, y_test)
        mse = float(mse)
        history.append(mse)
        if mse < best_mse:
            best_mse = mse
            best_weights = copy.deepcopy(model.state_dict())

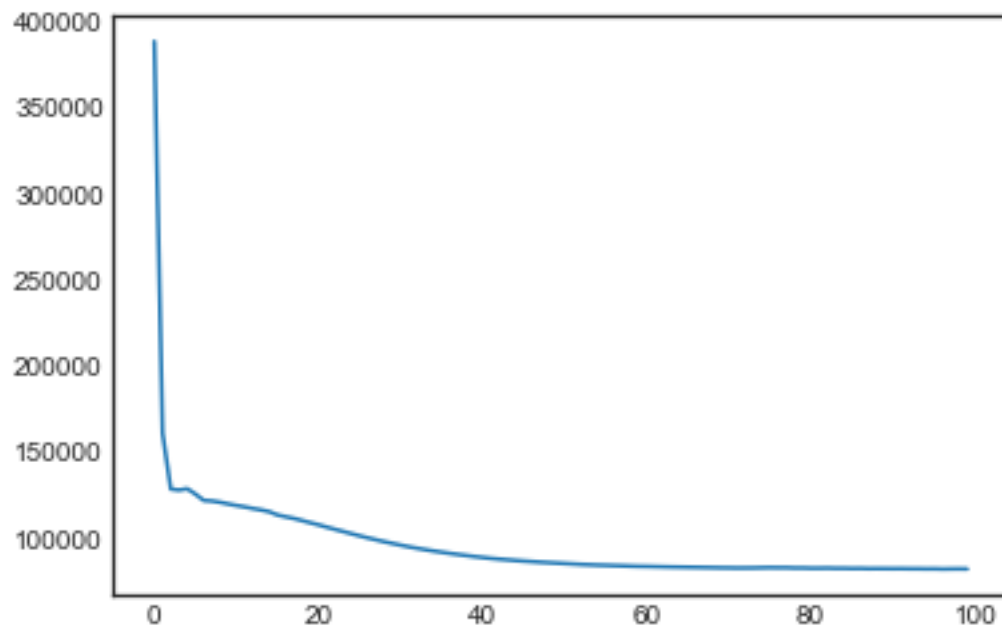
```

```
# restore model and return best accuracy
model.load_state_dict(best_weights)
```

```
[5]: print("MSE: %.2f" % best_mse)
      print("RMSE: %.2f" % np.sqrt(best_mse))
      plt.plot(history)
      plt.show()
```

MSE: 82412.31

RMSE: 287.08



6.4.6 Mạng phân loại đơn tầng với pytorch

```
[1]: import torch
      import torch.nn.functional as F
      import torch.nn as nn
      from torch.autograd import Variable
```



```
[2]: class Model(nn.Module):
    def __init__(self, input_dim):
        super(Model, self).__init__()
        self.layer1 = nn.Linear(input_dim, 50)
        self.layer2 = nn.Linear(50, 3)
        #self.layer3 = nn.Linear(50, 3)
    def forward(self, x):
        x = F.relu(self.layer1(x))
        #x = F.relu(self.layer2(x))
        x = F.softmax(self.layer2(x), dim=1)
        return x
```

```
[3]: model      = Model(X_train.shape[1])
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
loss_fn    = nn.CrossEntropyLoss()
model
```

```
[3]: Model(
  (layer1): Linear(in_features=4, out_features=50, bias=True)
  (layer2): Linear(in_features=50, out_features=3, bias=True)
)
```

```
[4]: import tqdm

EPOCHS = 100
X_train = Variable(torch.from_numpy(X_train)).float()
y_train = Variable(torch.from_numpy(y_train)).long()
X_test  = Variable(torch.from_numpy(X_test)).float()
y_test  = Variable(torch.from_numpy(y_test)).long()

loss_list      = np.zeros((EPOCHS,))
accuracy_list  = np.zeros((EPOCHS,))

for epoch in tqdm.trange(EPOCHS):
    y_pred = model(X_train)
    loss = loss_fn(y_pred, y_train)
    loss_list[epoch] = loss.item()

    # Zero gradients
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

```

with torch.no_grad():
    y_pred = model(X_test)
    correct = (torch.argmax(y_pred, dim=1) == y_test).
    .type(torch.FloatTensor)
    accuracy_list[epoch] = correct.mean()

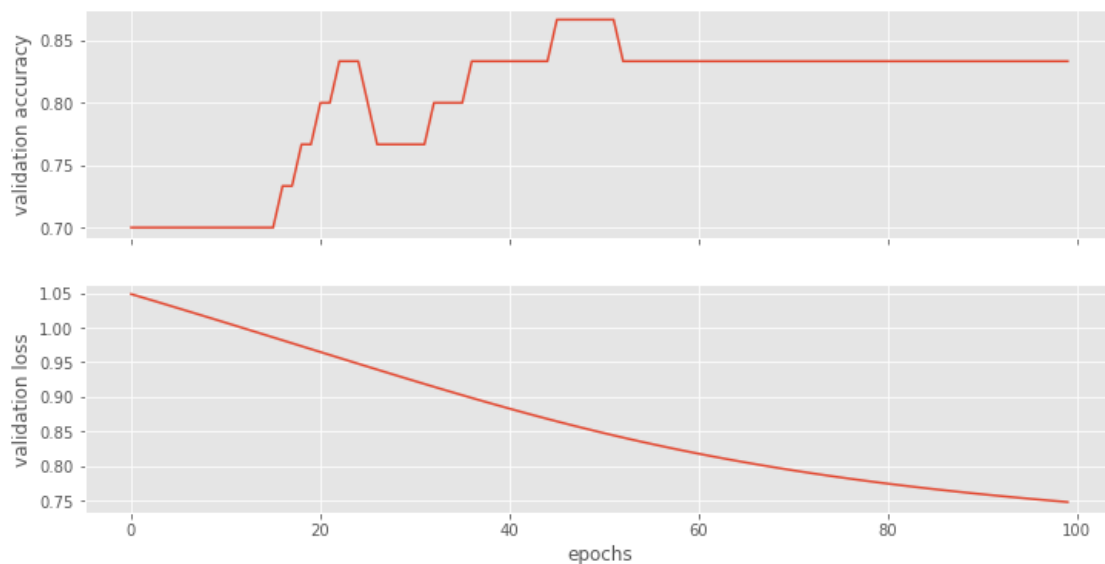
```

```

[5]: fig, (ax1, ax2) = plt.subplots(2, figsize=(12, 6), sharex=True)

ax1.plot(accuracy_list)
ax1.set_ylabel("validation accuracy")
ax2.plot(loss_list)
ax2.set_ylabel("validation loss")
ax2.set_xlabel("epochs");

```



```

[6]: from sklearn.metrics import roc_curve, auc
     from sklearn.preprocessing import OneHotEncoder

plt.figure(figsize=(10, 10))
plt.plot([0, 1], [0, 1], 'k--')

# One hot encoding
enc = OneHotEncoder()
Y_onehot = enc.fit_transform(y_test[:, np.newaxis]).toarray()

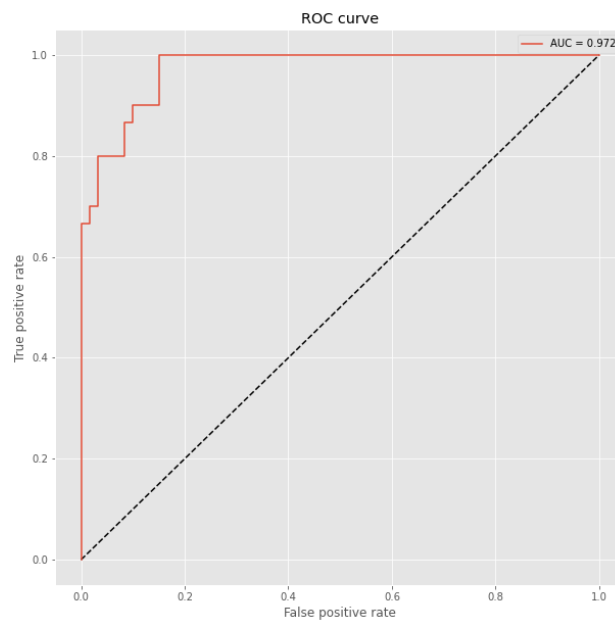
```

```

with torch.no_grad():
    y_pred = model(X_test).numpy()
    fpr, tpr, threshold = roc_curve(Y_onehot.ravel(), y_pred.
    .ravel())

plt.plot(fpr, tpr, label='AUC = {:.3f}'.format(auc(fpr, tpr)))
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.title('ROC curve')
plt.legend();

```



6.4.7 Mạng hồi quy đa tầng với pytorch

```

[1]: import copy
import numpy as np
import torch
import tqdm
import torch.nn as nn
import torch.optim as optim

```

```

[2]: # Define the model
model = nn.Sequential(
    nn.Linear(19, 50),
    nn.ReLU(),

```

```

nn.Linear(50, 20),
nn.ReLU(),
nn.Linear(20, 1),
nn.ReLU()
)

```

```

[3]: # loss function and optimizer
loss_fn = nn.MSELoss() # mean square error
optimizer = optim.Adam(model.parameters(), lr=0.001)

```

```

[4]: # train-test split of the dataset
X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1,
-1)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1,
-1)

# training parameters
n_epochs = 100 # number of epochs to run
batch_size = 10 # size of each batch
batch_start = torch.arange(0, len(X_train), batch_size)

# Hold the best model
best_mse = np.inf # init to infinity
best_weights = None
history = []

# training loop
for epoch in range(n_epochs):
    model.train()
    with tqdm.tqdm(batch_start, unit="batch", mininterval=0,
-disable=True) as bar:
        bar.set_description(f"Epoch {epoch}")
        for start in bar:
            # take a batch
            X_batch = X_train[start:start+batch_size]
            y_batch = y_train[start:start+batch_size]
            # forward pass
            y_pred = model(X_batch)
            loss = loss_fn(y_pred, y_batch)
            # backward pass

```

```

        optimizer.zero_grad()
        loss.backward()
        # update weights
        optimizer.step()
        # print progress
        bar.set_postfix(mse=float(loss))
    # evaluate accuracy at end of each epoch
    model.eval()
    y_pred = model(X_test)
    mse = loss_fn(y_pred, y_test)
    mse = float(mse)
    history.append(mse)
    if mse < best_mse:
        best_mse = mse
        best_weights = copy.deepcopy(model.state_dict())

# restore model and return best accuracy
model.load_state_dict(best_weights)

```

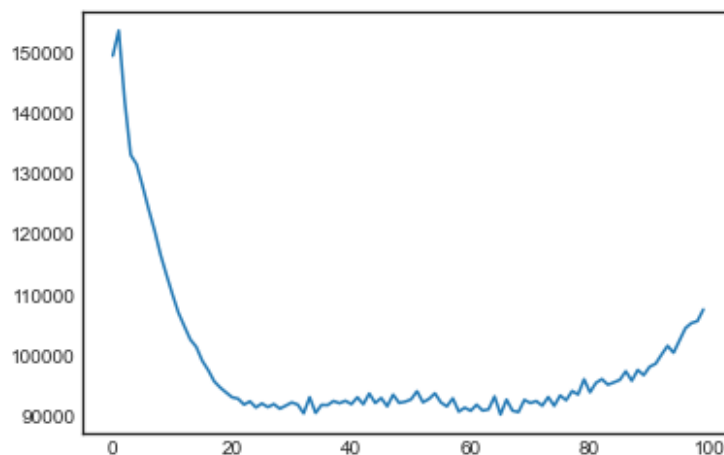
```

[5]: print("MSE: %.2f" % best_mse)
     print("RMSE: %.2f" % np.sqrt(best_mse))
     plt.plot(history)
     plt.show()

```

MSE: 90172.88

RMSE: 300.29



6.4.8 Mạng phân loại đa tầng với pytorch

```
[1]: import torch
import torch.nn.functional as F
import torch.nn as nn
from torch.autograd import Variable
```

```
[2]: class Model(nn.Module):
    def __init__(self, input_dim):
        super(Model, self).__init__()
        self.layer1 = nn.Linear(input_dim, 32)
        self.layer2 = nn.Linear(32, 16)
        self.layer3 = nn.Linear(16, 3)

    def forward(self, x):
        x = F.relu(self.layer1(x))
        x = F.relu(self.layer2(x))
        x = F.softmax(self.layer3(x), dim=1)
        return x
```

```
[3]: model = Model(X_train.shape[1])
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
loss_fn = nn.CrossEntropyLoss()
model
```

```
[3]: Model(
  (layer1): Linear(in_features=4, out_features=32, bias=True)
  (layer2): Linear(in_features=32, out_features=16, bias=True)
  (layer3): Linear(in_features=16, out_features=3, bias=True)
)
```

```
[4]: import tqdm

EPOCHS = 100
X_train = Variable(torch.from_numpy(X_train)).float()
y_train = Variable(torch.from_numpy(y_train)).long()
X_test = Variable(torch.from_numpy(X_test)).float()
y_test = Variable(torch.from_numpy(y_test)).long()

loss_list = np.zeros((EPOCHS,))
accuracy_list = np.zeros((EPOCHS,))
```

```

for epoch in tqdm.trange(EPOCHS):
    y_pred = model(X_train)
    loss = loss_fn(y_pred, y_train)
    loss_list[epoch] = loss.item()

    # Zero gradients
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    with torch.no_grad():
        y_pred = model(X_test)
        correct = (torch.argmax(y_pred, dim=1) == y_test).
        .type(torch.FloatTensor)
        accuracy_list[epoch] = correct.mean()

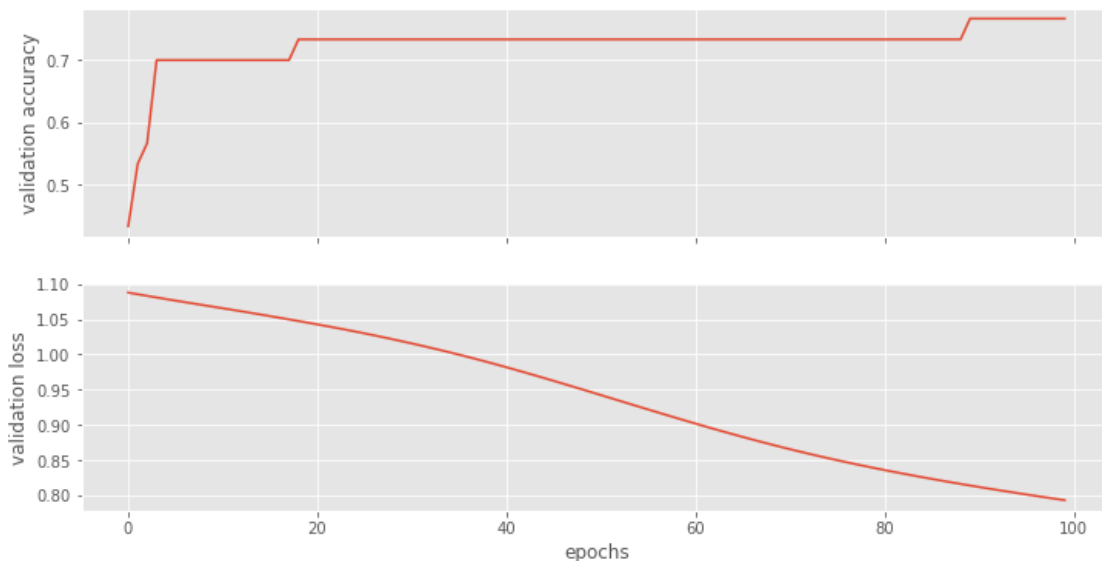
```

```

[5]: fig, (ax1, ax2) = plt.subplots(2, figsize=(12, 6), sharex=True)

ax1.plot(accuracy_list)
ax1.set_ylabel("validation accuracy")
ax2.plot(loss_list)
ax2.set_ylabel("validation loss")
ax2.set_xlabel("epochs");

```



```

[6]: from sklearn.metrics import roc_curve, auc
      from sklearn.preprocessing import OneHotEncoder

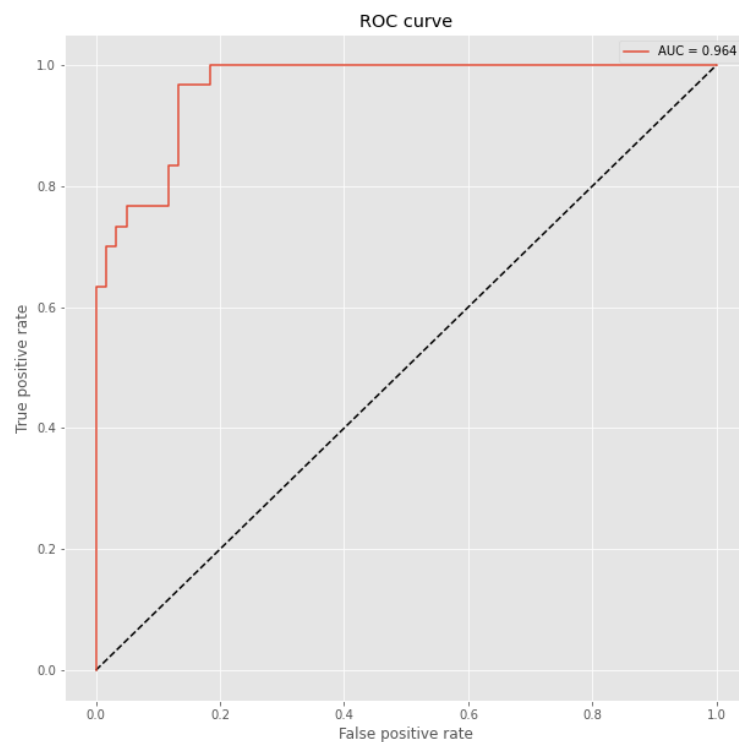
```

```
plt.figure(figsize=(10, 10))
plt.plot([0, 1], [0, 1], 'k--')

# One hot encoding
enc = OneHotEncoder()
Y_onehot = enc.fit_transform(y_test[:, np.newaxis]).toarray()

with torch.no_grad():
    y_pred = model(X_test).numpy()
    fpr, tpr, threshold = roc_curve(Y_onehot.ravel(), y_pred.
    -ravel())

plt.plot(fpr, tpr, label='AUC = {:.3f}'.format(auc(fpr, tpr)))
plt.xlabel('False positive rate')
plt.ylabel('True positive rate')
plt.title('ROC curve')
plt.legend();
```



6.5 Bài tập

Bài 6.1 (Tính đạo hàm của hàm nhiều biến sử dụng Autograd trong PyTorch). Sử dụng thư viện PyTorch để tính gradient (đạo hàm riêng) của một hàm nhiều biến bằng cơ chế tự động tính đạo hàm (autograd). Xem xét hàm

$$f(x, y, z) = x^2y + \sin(y) + e^z,$$

với các biến đầu vào là tensor yêu cầu gradient.

- Tạo các tensor đầu vào x, y, z với giá trị ban đầu (ví dụ: $x = 2.0, y = 3.0, z = 1.0$) và thiết lập `requires_grad=True` để theo dõi gradient.
- Định nghĩa hàm $f(x, y, z)$ sử dụng các phép toán PyTorch.
- Thực hiện lan truyền thuận bằng cách tính giá trị f .
- Sử dụng `f.backward()` để tính gradient lan truyền ngược.
- In ra các gradient: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$ (truy cập qua `x.grad, y.grad, z.grad`).
- Kiểm tra thủ công bằng cách tính đạo hàm riêng theo công thức. So sánh kết quả từ PyTorch với tính toán thủ công tại điểm đã cho.

Bài 6.2 (Mạng Perceptron đơn giản). Mô hình: $\hat{y} = \sigma(w_1x_1 + w_2x_2 + b)$ với σ là hàm kích hoạt ReLU.

Dữ liệu: Một mẫu đầu vào $x = [5, 3]$, nhãn thực $y = 10$. Tốc độ học $\eta = 0.01$. Hàm tổn thất $L = \frac{1}{2}(y - \hat{y})^2$.

Trọng số ban đầu: $w_1 = 1, w_2 = 3$, bias $b = -3$.

- Tính toán lan truyền thuận (forward) hàm tổn thất L .
- Tính toán lan truyền ngược (backpropagation): Tính $\frac{\partial L}{\partial w_1}, \frac{\partial L}{\partial w_2}, \frac{\partial L}{\partial b}$.
- Cập nhật các tham số theo thuật toán gradient descent.
- Tính hàm tổn thất L với tham số mới sau cập nhật.

Bài 6.3 (Mạng neuron với 1 lớp ẩn). Xem xét mạng với 2 đầu vào, 1 lớp ẩn gồm 2 neuron, và 1 đầu ra. Ký hiệu:

Trọng số lớp ẩn: $W^{(1)} = \begin{bmatrix} 0.1 & 0.3 \\ -0.2 & 0.4 \end{bmatrix}$, bias ẩn $b^{(1)} = [0.1, -0.1]$.

Trọng số lớp đầu ra: $W^{(2)} = [0.5, -0.5]$, bias đầu ra $b^{(2)} = 0.2$.

Dữ liệu: Một mẫu $x = [0.5, 0.3]$, $y = 1$. Tốc độ học $\eta = 0.01$. Hàm kích hoạt $\sigma = \text{ReLU}$.

- Lan truyền thuận (forward):
 - Tính $z^{(1)} = W^{(1)}x + b^{(1)}$, $a^{(1)} = \sigma(z^{(1)})$.
 - Tính $z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$, $\hat{y} = \sigma(z^{(2)})$.
- Tính mất mát $L = \frac{1}{2}(y - \hat{y})^2$.
- Lan truyền ngược (backpropagation):
 - $\frac{\partial L}{\partial W^{(2)}}, \frac{\partial L}{\partial b^{(2)}}, \frac{\partial L}{\partial W^{(1)}}, \frac{\partial L}{\partial b^{(1)}}$.
 - Cập nhật $W^{(1)}, W^{(2)}, b^{(1)}, b^{(2)}$.
- Lặp lại lan truyền thuận với tham số mới và tính mất mát mới.

Bài 6.4. Sử dụng hàm kích hoạt sigmoid, hàm mất mát là Binary Cross Entropy:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})],$$

với learning rate $\eta = 0.1$. Thực hiện cập nhật trọng số theo kiểu stochastic gradient descent (từng mẫu một). Ký hiệu mạng (2 đầu vào, lớp ẩn với 2 neuron, 1 đầu ra.)

Trọng số lớp ẩn: $W^{(1)} = \begin{bmatrix} 0.1 & 0.3 \\ -0.2 & 0.4 \end{bmatrix}$, bias ẩn $b^{(1)} = [0.1, -0.1]$.

Trọng số lớp đầu ra: $W^{(2)} = [0.5, -0.5]$, bias đầu ra $b^{(2)} = 0.2$.

Dữ liệu: mẫu 1: $x_1 = [0.5, 0.3]$, $y_1 = 1$, mẫu 2: $x_2 = [0.4, 0.6]$, $y_2 = 0$.

- **Cho mẫu 1** ($x_1 = [0.5, 0.3]$, $y_1 = 1$):
 - Tính lan truyền thuận
 - Tính mất mát L_1 .
 - Tính lan truyền ngược (backpropagation) cập nhật các tham số.
- **Cho mẫu 2** ($x_2 = [0.4, 0.6]$, $y_2 = 0$), sử dụng trọng số đã cập nhật từ mẫu 1:
 - Tính lan truyền thuận
 - Tính mất mát L_2 .
 - Tính lan truyền ngược (backpropagation) và cập nhật các tham số.

6.6 Thực hành

Thực hành 6.1 (Hồi quy trên tập dữ liệu California Housing bằng mạng neuron với 1 lớp ẩn). Sử dụng tập dữ liệu California Housing từ `sklearn.datasets.fetch_california_housing()`.

- Tải dữ liệu và chia thành tập huấn luyện (80%) và tập kiểm tra (20%).
- Xây dựng mô hình `MLPRegressor` với 1 lớp ẩn gồm 50 neuron, hàm kích hoạt 'relu', và tối đa 500 vòng lặp.
- Huấn luyện mô hình để dự đoán giá nhà (target là 'MedHouseVal').
- Đánh giá mô hình bằng mean squared error (MSE) trên tập kiểm tra bằng `mean_squared_error` từ `sklearn.metrics`.
- Thử chuẩn hóa dữ liệu đầu vào bằng `StandardScaler` từ `sklearn.preprocessing` và so sánh MSE trước/sau chuẩn hóa.

Thực hành 6.2 (Hồi quy trên tập dữ liệu Boston Housing bằng mạng neuron với 2 lớp ẩn). Sử dụng tập dữ liệu Boston Housing từ `sklearn.datasets.load_boston()`.

- Tải dữ liệu và chia thành tập huấn luyện (80%) và tập kiểm tra (20%) bằng `train_test_split` từ `sklearn.model_selection`.
- Xây dựng mô hình `MLPRegressor` từ `sklearn.neural_network` với 2 lớp ẩn: lớp ẩn đầu tiên 64 neuron, lớp ẩn thứ hai 32 neuron, hàm kích hoạt `relu`, thuật toán tối ưu `adam`, và tối đa 1000 vòng lặp.
- Huấn luyện mô hình để dự đoán giá nhà (target là `MEDV`).
- Đánh giá mô hình bằng mean squared error (MSE) và R-squared trên tập kiểm tra bằng `mean_squared_error` và `r2_score` từ `sklearn.metrics`.
- Thử chuẩn hóa dữ liệu đầu vào bằng `StandardScaler` từ `sklearn.preprocessing` và so sánh kết quả MSE trước/sau chuẩn hóa.
- Thử thay đổi kích thước lớp ẩn (100 và 50 neuron) và quan sát sự thay đổi trong hiệu suất mô hình.

Thực hành 6.3 (Phân loại tập dữ liệu Iris bằng mạng neuron với 1 lớp ẩn). Sử dụng tập dữ liệu Iris từ `sklearn.datasets.load_iris()`.

- Tải dữ liệu và chia thành tập huấn luyện (80%) và tập kiểm tra (20%) bằng `train_test_split` từ `sklearn.model_selection`.
- Xây dựng mô hình `MLPClassifier` từ `sklearn.neural_network` với cấu hình: 1 lớp ẩn gồm 10 neuron, hàm kích hoạt `relu`, và thuật toán tối ưu `adam`.
- Huấn luyện mô hình trên tập huấn luyện.
- Đánh giá độ chính xác trên tập kiểm tra bằng `accuracy_score` từ `sklearn.metrics`.
- Thử thay đổi kích thước lớp ẩn (20 neuron) và so sánh kết quả.

Thực hành 6.4 (Phân loại tập dữ liệu Digits bằng mạng neuron với 2 lớp ẩn). Sử dụng tập dữ liệu Digits từ `sklearn.datasets.load_digits()`.

- Tải dữ liệu và chia thành tập huấn luyện (70%) và tập kiểm tra (30%).
- Xây dựng mô hình `MLPClassifier` với 2 lớp ẩn: lớp ẩn đầu tiên 64 neuron, lớp ẩn thứ hai 32 neuron, hàm kích hoạt 'logistic', và learning rate ban đầu 0.001.
- Huấn luyện mô hình và tính toán ma trận nhầm lẫn trên tập kiểm tra bằng `confusion_matrix` từ `sklearn.metrics`.
- Vẽ biểu đồ ma trận nhầm lẫn bằng `matplotlib` để trực quan hóa lỗi phân loại.
- Thử thêm regularization ($\alpha=0.1$) và quan sát sự thay đổi trong độ chính xác.

Thực hành 6.5 (Phân loại tập dữ liệu Wine bằng mạng neuron với PyTorch). Sử dụng tập dữ liệu Wine từ `sklearn.datasets.load_wine()`. Mục tiêu là xây dựng và huấn luyện một mạng neuron đơn giản bằng PyTorch để phân loại các loại rượu dựa trên các đặc trưng hóa học.

- Tải dữ liệu Wine bằng `from sklearn.datasets import load_wine`, sau đó chia thành tập huấn luyện (80%) và tập kiểm tra (20%) bằng `train_test_split` từ `sklearn.model_selection`.
- Chuẩn hóa dữ liệu đầu vào bằng `StandardScaler` từ `sklearn.preprocessing` để cải thiện hiệu suất huấn luyện.
- Chuyển đổi dữ liệu thành `Tensor` PyTorch và tạo `DataLoader` cho tập huấn luyện và kiểm tra (`batch_size = 32`) bằng `torch.utils.data.DataLoader`.

- Định nghĩa một lớp mô hình mạng neuron bằng PyTorch (`torch.nn.Module`) với: lớp đầu vào (13 đặc trưng), 2 lớp ẩn (ví dụ: 64 neuron và 32 neuron với hàm kích hoạt `ReLU`), và lớp đầu ra (3 lớp cho phân loại) với `Softmax`.
- Sử dụng hàm mất mát `CrossEntropyLoss` và optimizer `Adam` với learning rate 0.001.
- Huấn luyện mô hình trong 100 epochs, tính toán loss trên tập huấn luyện và độ chính xác trên tập kiểm tra sau mỗi epoch.
- Đánh giá mô hình cuối cùng bằng độ chính xác (`accuracy_score` từ `sklearn.metrics`) và ma trận nhầm lẫn trên tập kiểm tra.
- Thử thay đổi số lượng neuron trong lớp ẩn (ví dụ: 128 và 64) hoặc thêm dropout để tránh `overfitting`, sau đó so sánh kết quả.

Tài liệu tham khảo

- [1] James, G., Witten, D., Hastie, T., Tibshirani, R., Taylor, J. , An Introduction to Statistical Learning with Applications in Python, Springer, 2023.
- [2] James, G., Witten, D., Hastie, T., Tibshirani, R., An Introduction to Statistical Learning with Applications in R, Springer, 2015.
- [3] Trevor H., Robert T.i, Joreome F., The Elements of Statistical Learning: Data Mining, Inference, and Prediction, Second Edition, Springer, 2009.
- [4] Bishop, C.M., Pattern Recognition and Machine Learning, Springer, 2006.
- [5] Murphy, K.P., Machine Learning: A Probabilistic Perspective, MIT Press, 2012.
- [6] McKinney, W., Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython, 2012.
- [7] Jake VanderPlas, Python Data Science Handbook: Essential Tools for Working with Data, O'Reilly Media, 2016.

